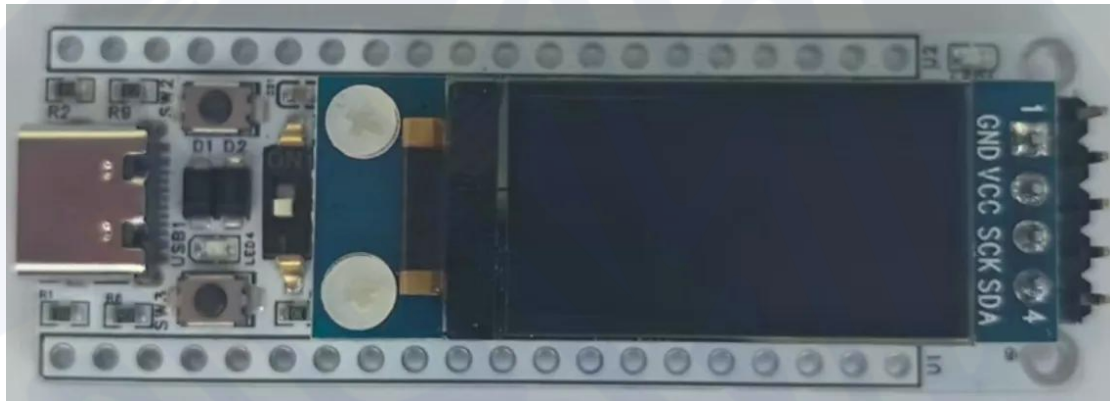


project10: 定时器 AGT 输出 PWM

概述

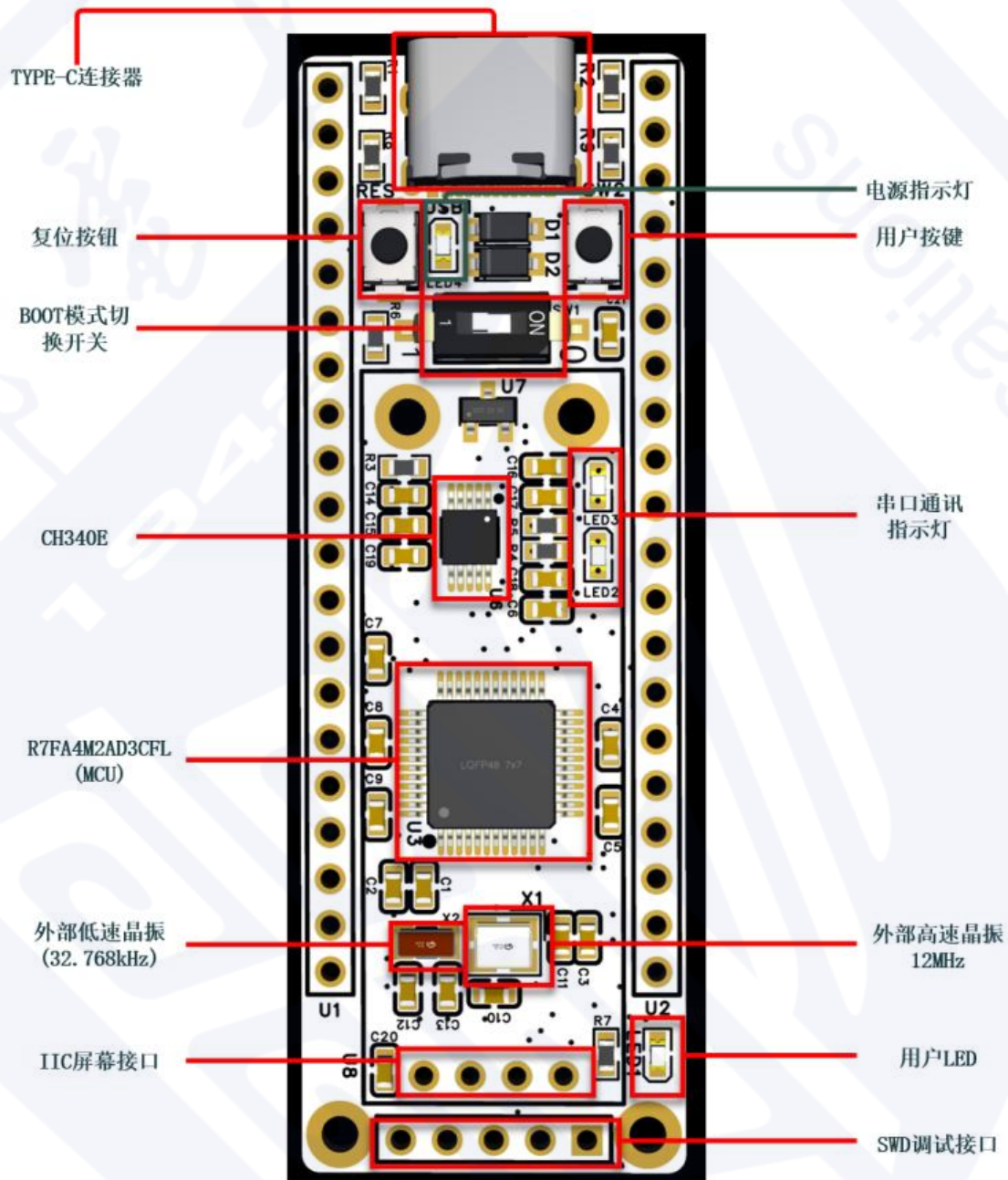
AGT 模块是 RA MCU（微控制单元）系列中的一个定时器外设，提供灵活的计时和事件捕获功能，能够满足多种应用场景中的定时和控制需求。它支持 16 位和 32 位定时器，并具有多种工作模式，适用于需要高精度时间控制和事件捕获的应用。它广泛应用于定时控制、信号测量、PWM 生成等应用，尤其适用于电机控制、LED 调光、PWM 伺服驱动和 ADC 触发等场景。



硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL



参考程序

https://github.com/CoreMaker-lab/RA4M2_MINI

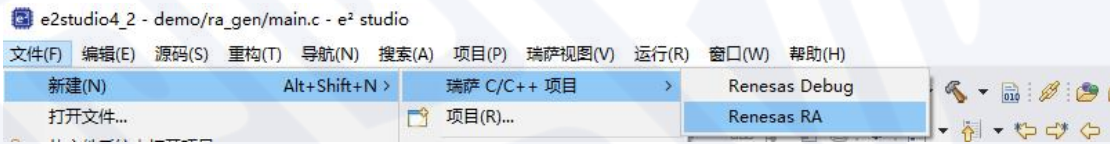
https://gitee.com/CoreMaker/RA4M2_MINI

选择计时器

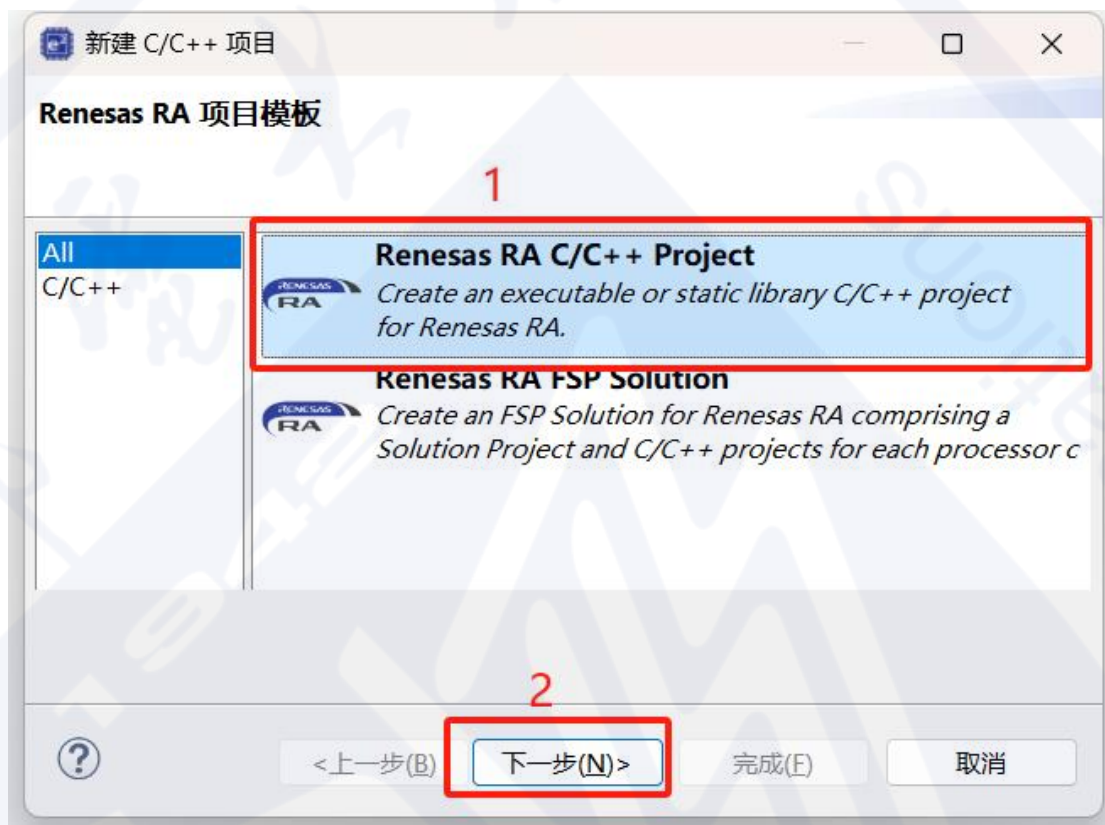
RA MCU 有两个定时器外设：通用 PWM 定时器 (GPT) 和异步通用定时器 (AGT)。

	GPT	AGT
低功耗模式	GPT 可以在睡眠模式下运行。	AGT 可以在所有低功耗模式下运行。
可用频道	GPT 通道的数量因设备而异。目前所有支持的 MCU 至少都有 7 个 GPT 通道。	所有 MCU 都有 2 个 AGT 通道。
定时器分辨率	所有 MCU 均至少有一个 32 位 GPT 计时器。	AGT 计时器是 16 位计时器。
时钟源	GPT 通过可配置分频器（最高可达 1024）的 PCLKD 运行。它还可以配置为计数 ELC 事件或外部脉冲。	AGT 通过 PCLKB、LOCO 或子时钟运行。

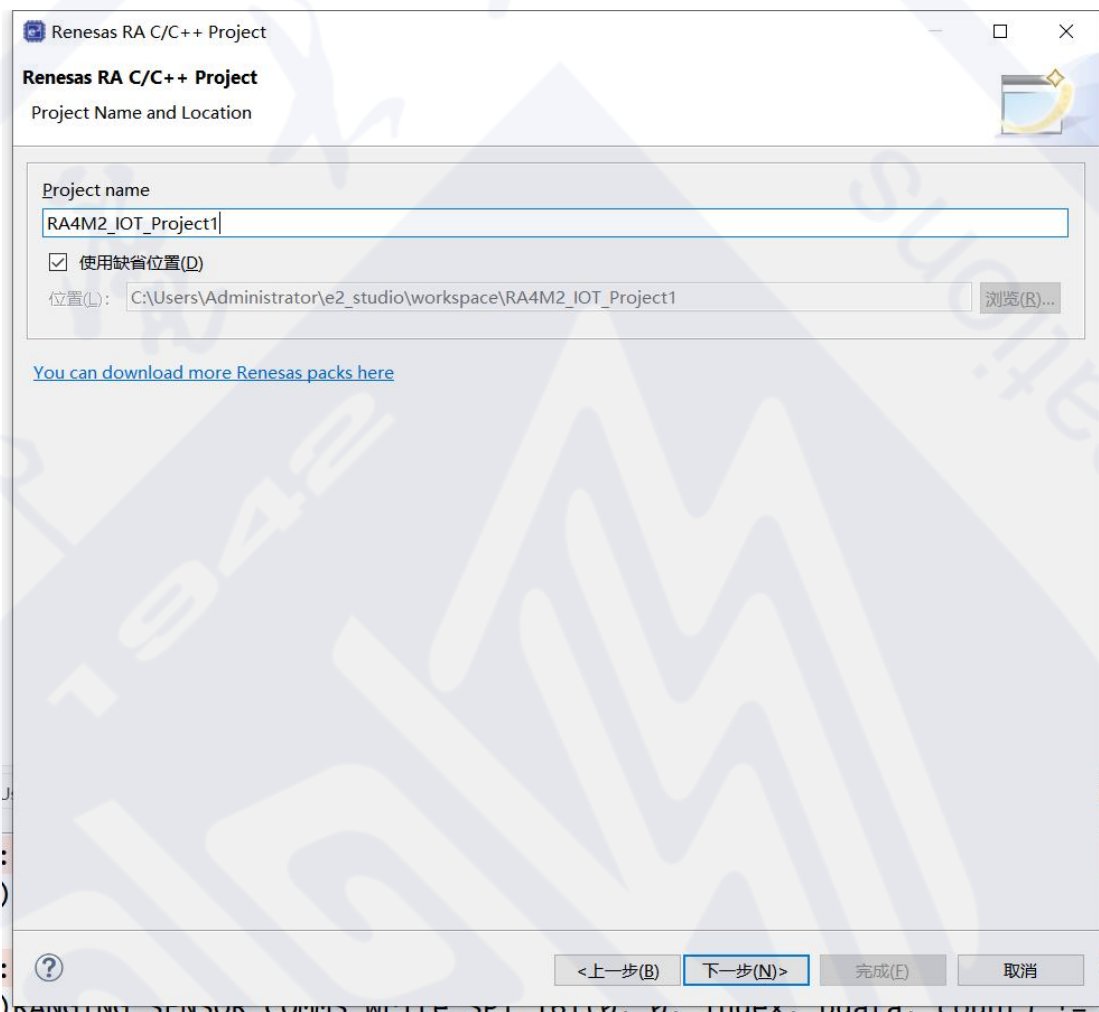
新建工程



工程模板



保存工程路径



芯片配置

本文中使用 R7FA4M2AD3CFL 来进行演示。

Renesas RA C/C++ Project

Renesas RA C/C++ Project

Device and Tools Selection

Device Selection

FSP Version: 5.9.0 **版本**

Board: Custom User Board (Any Device) ...

Device: R7FA4M2AD3CFL **芯片型号**

Core: CM33

Language: ☒ C ☐ C++

Board Description

Device Details

TrustZone	Yes
Pins	48
Processor	Cortex-M33

IDE Project Type

e2 studio managed build

Toolchains

LLVM Embedded Toolchain for Arm

GNU ARM Embedded **工具链**

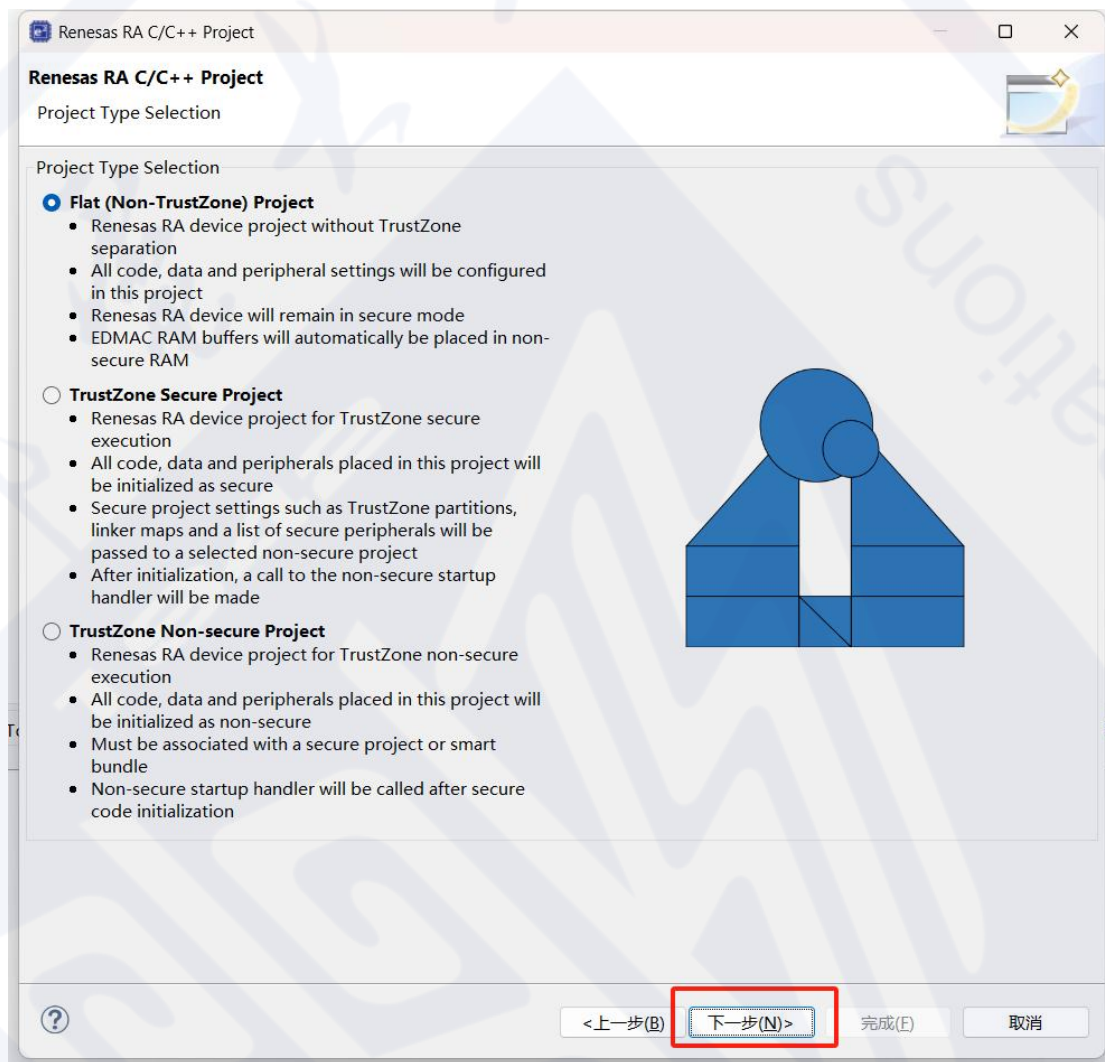
13.2.1.arm-13-7 [Manage Toolchains...](#)

Debugger

J-Link ARM **使用JLink进行调试**

[?](#) <上一步(B) 下一步(N)> 完成(F) 取消

工程模板选择



Renesas RA C/C++ Project

Renesas RA C/C++ Project

Preceding Project or Smart Bundle Selection

☒ None

Choose this option when creating a project for the primary processor core (no preceding project or Smart Bundle).

☐ Preceding Project:

Choose this option if you have access to the preceding bootloader project or TrustZone secure project source code. A debug configuration for Secure Software Debug (SSD) state will be generated where applicable and both secure and non-secure images will be downloaded to the target device. Should you wish to test Non-Secure Software Debug (NSECSO) state, use the Renesas Device Partition Manager to change the Device Lifecycle state as needed.

☐ Smart Bundle:

Resolved location:

Workspace...

File System...

Variables...

Choose this option if you only have access to a pre-programmed device containing TrustZone secure code. The Smart Bundle file (*.sbd) should be obtained from the TrustZone Secure Project developer. A debug configuration for Non-Secure Software Debug (NSECSO) state will be generated and only the non-secure code image will be downloaded to the target device.

Preceding Project/Smart Bundle Details

FSP version

Toolchain

Toolchain version

Board

Device

Core

Zones

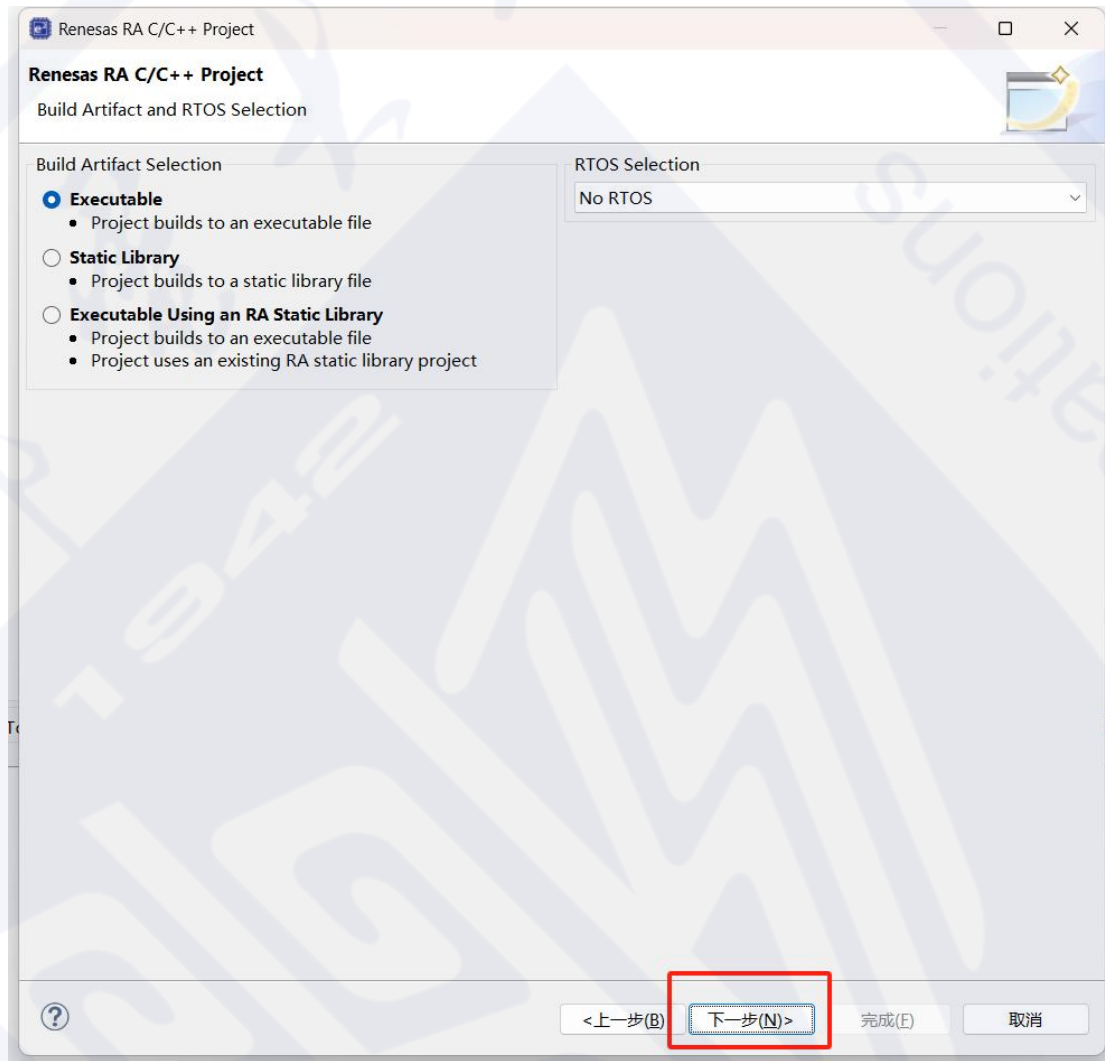
?

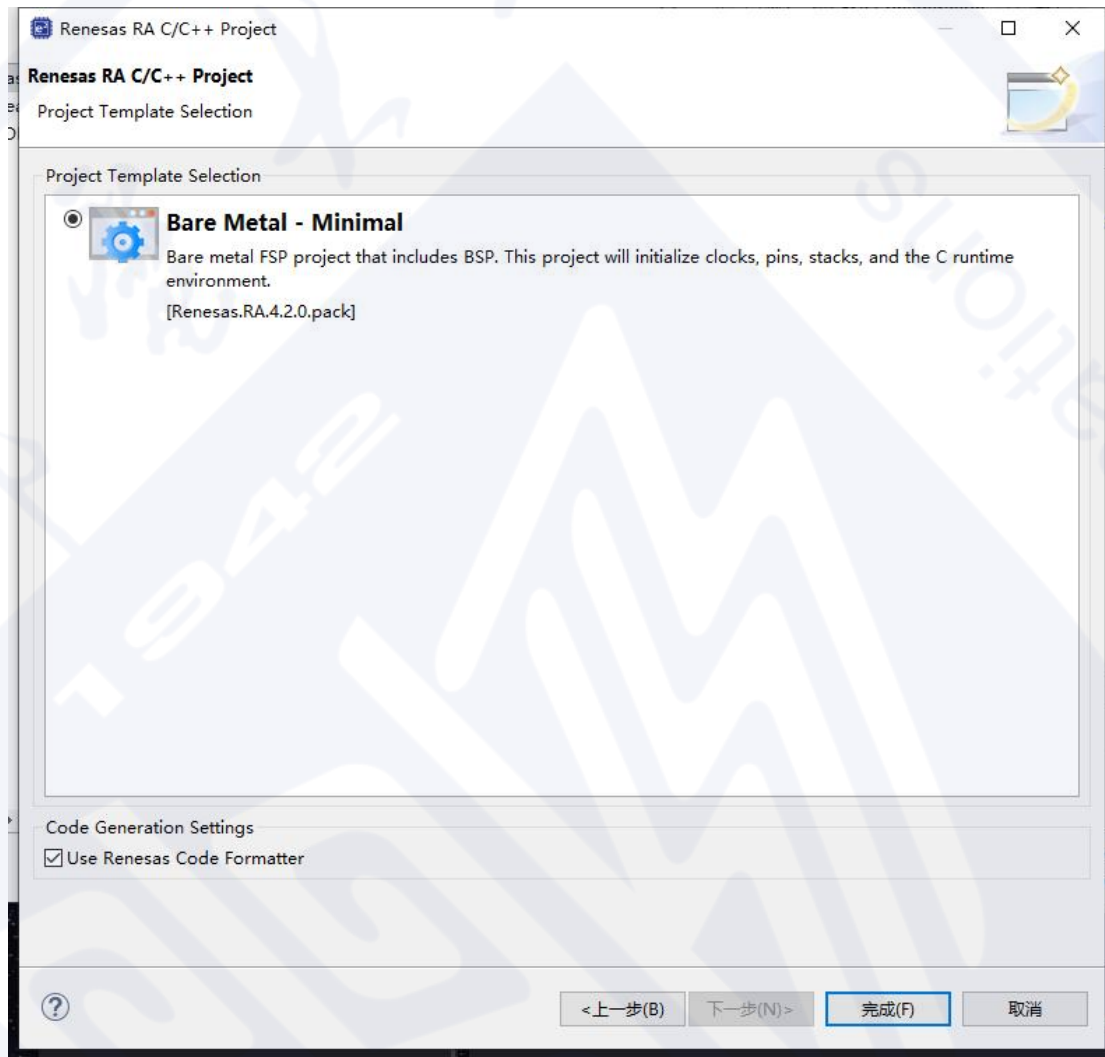
< 上一步(B)

下一步(N) >

完成(E)

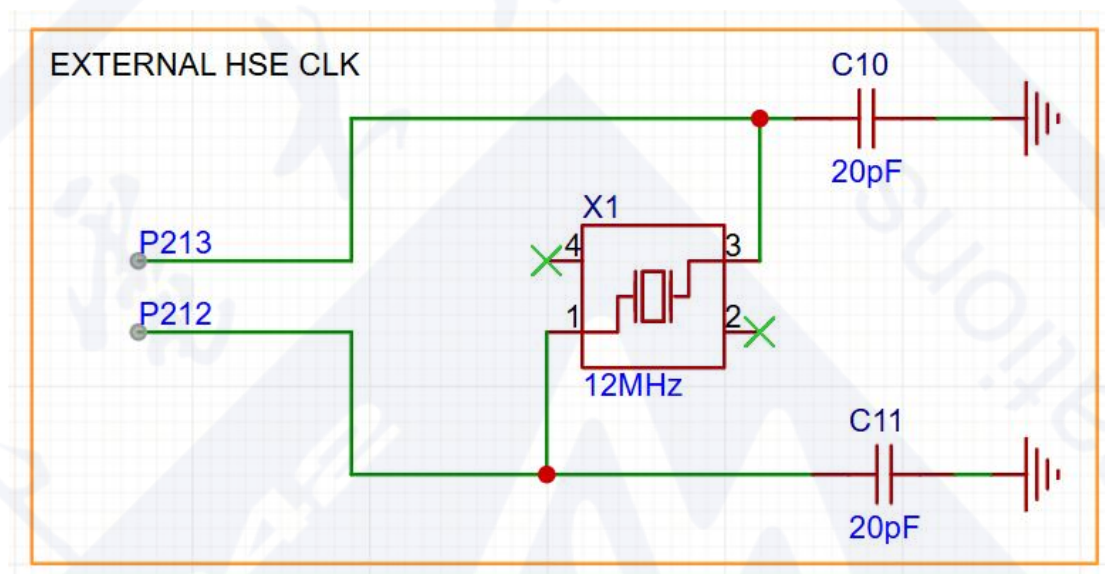
取消



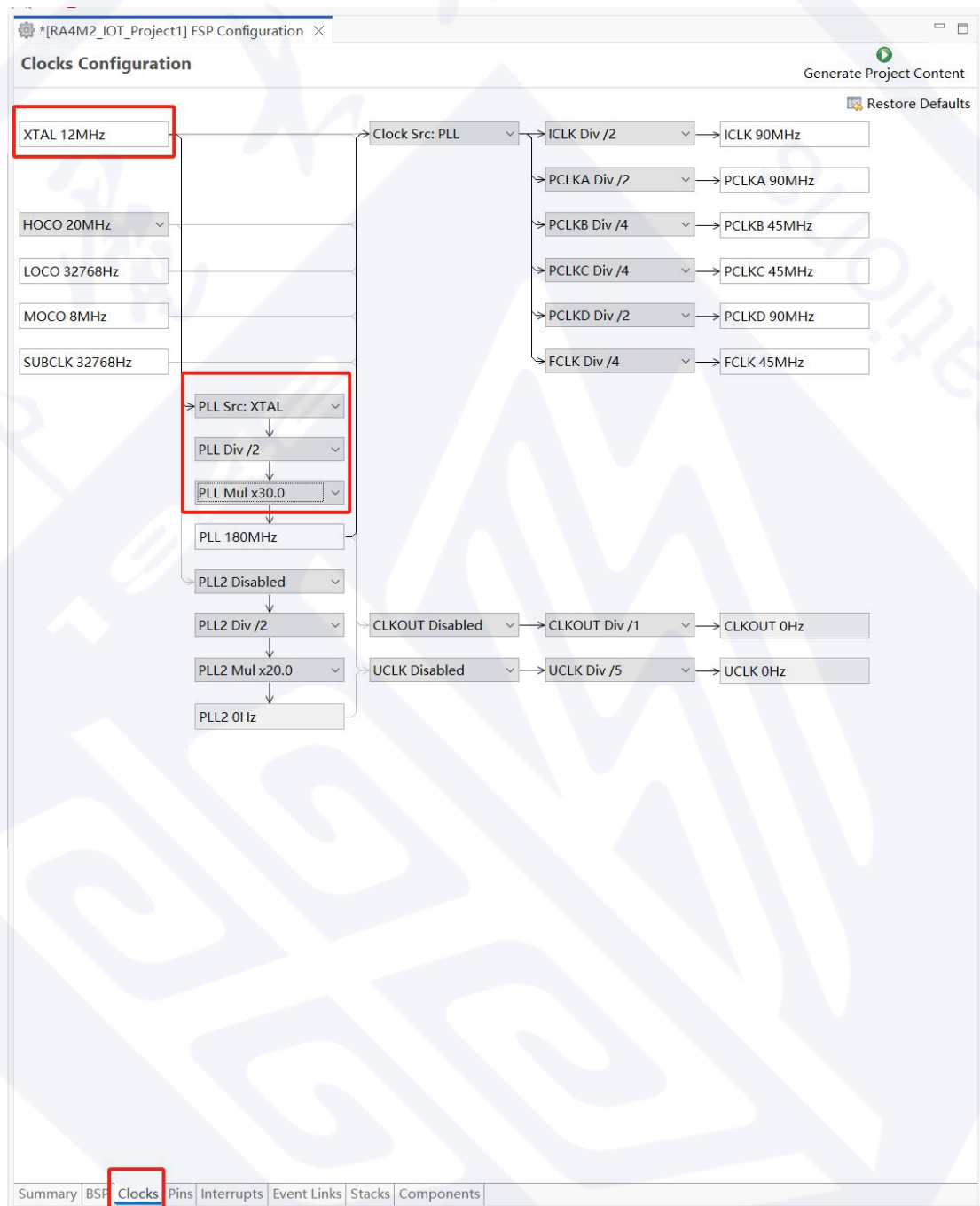


时钟设置

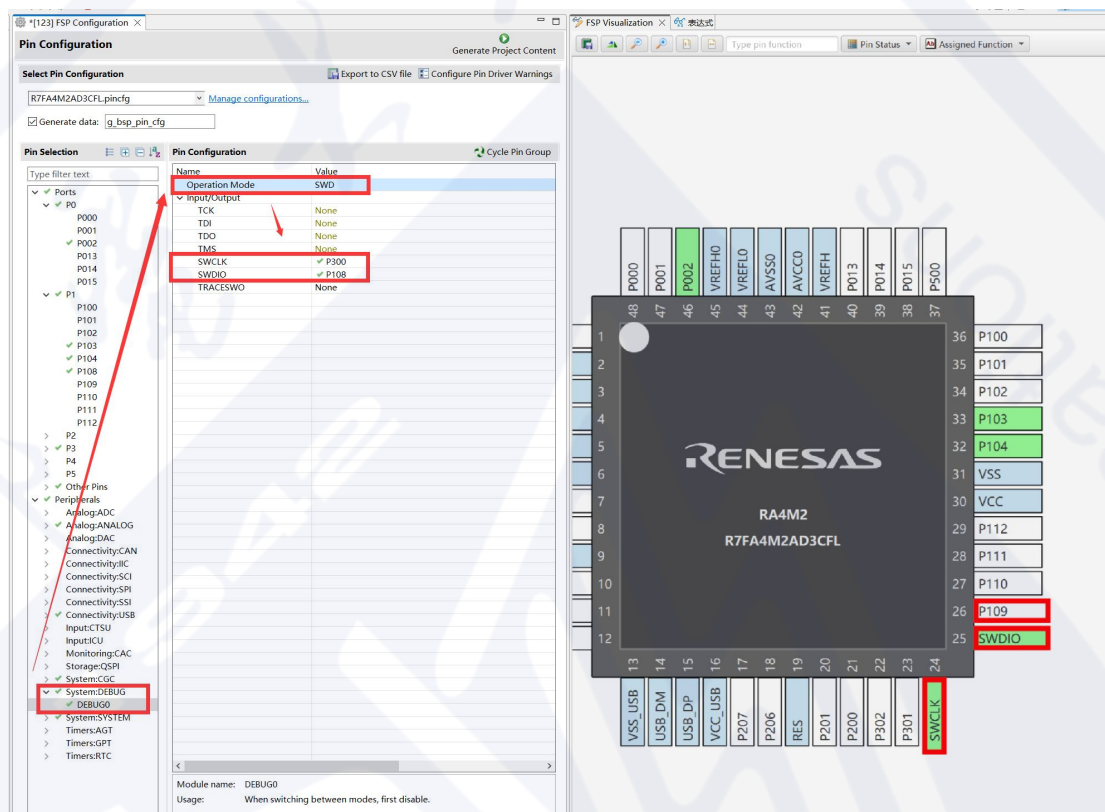
开发板上的外部高速晶振为 12M.



需要修改 XTAL 为 12M。

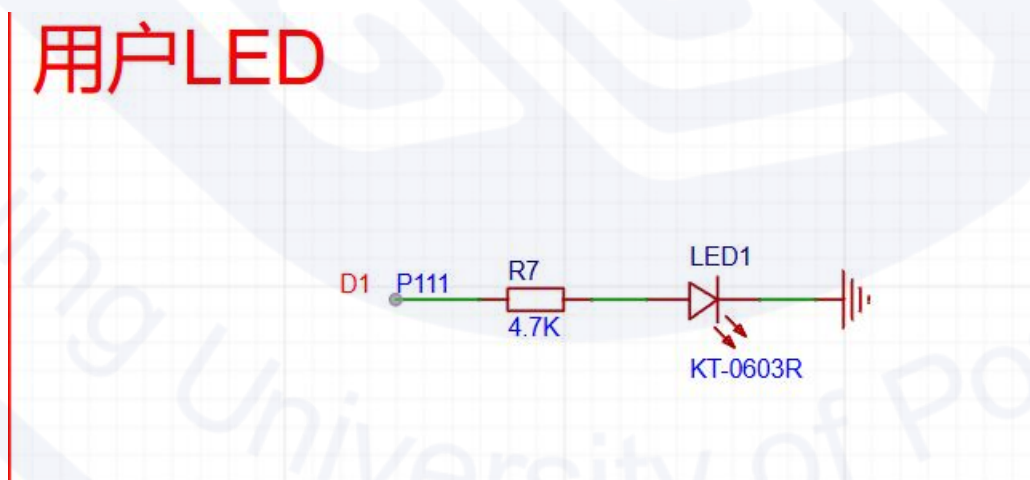


SWD 调试口设置

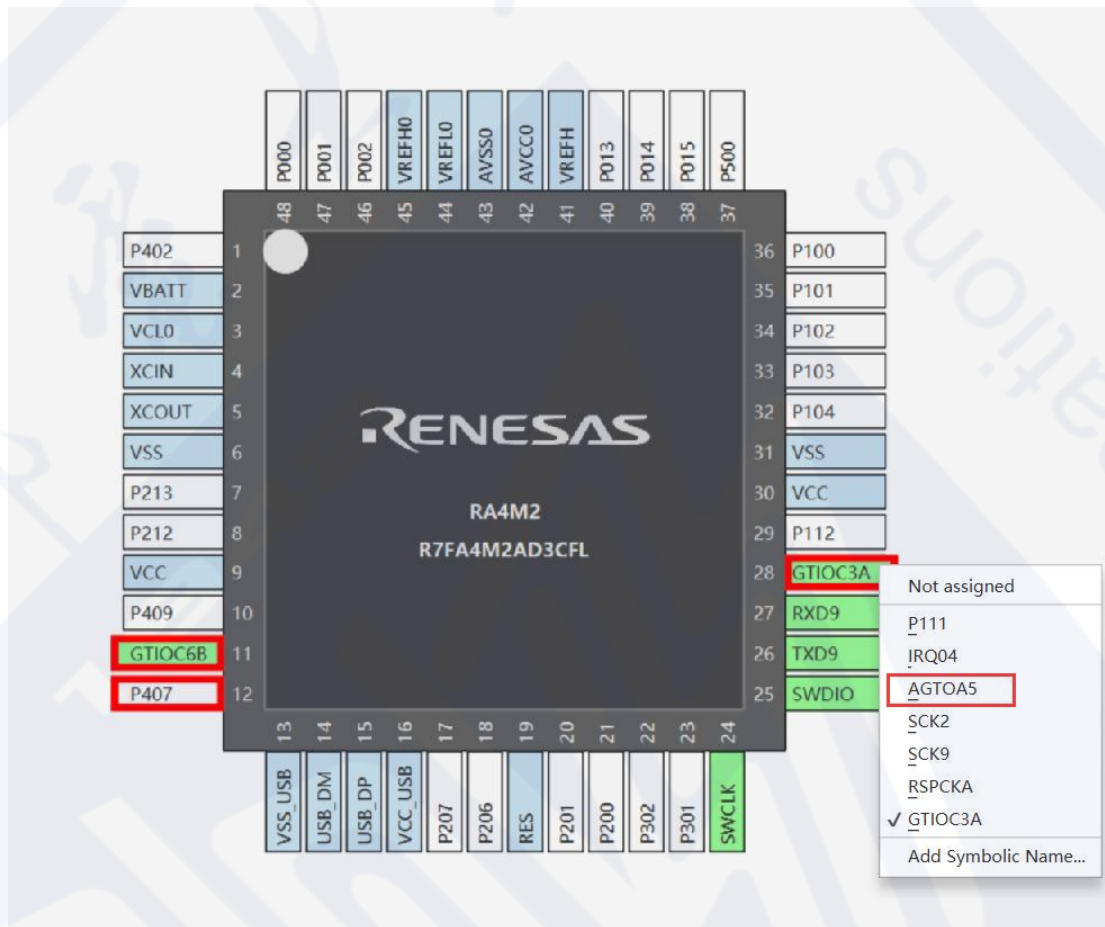


GPIO 口配置

由下图我们可以得知，板子上有 1 个 LED 灯，进行呼吸灯演示，故以 P111 管脚为例，配置方式如下所示。

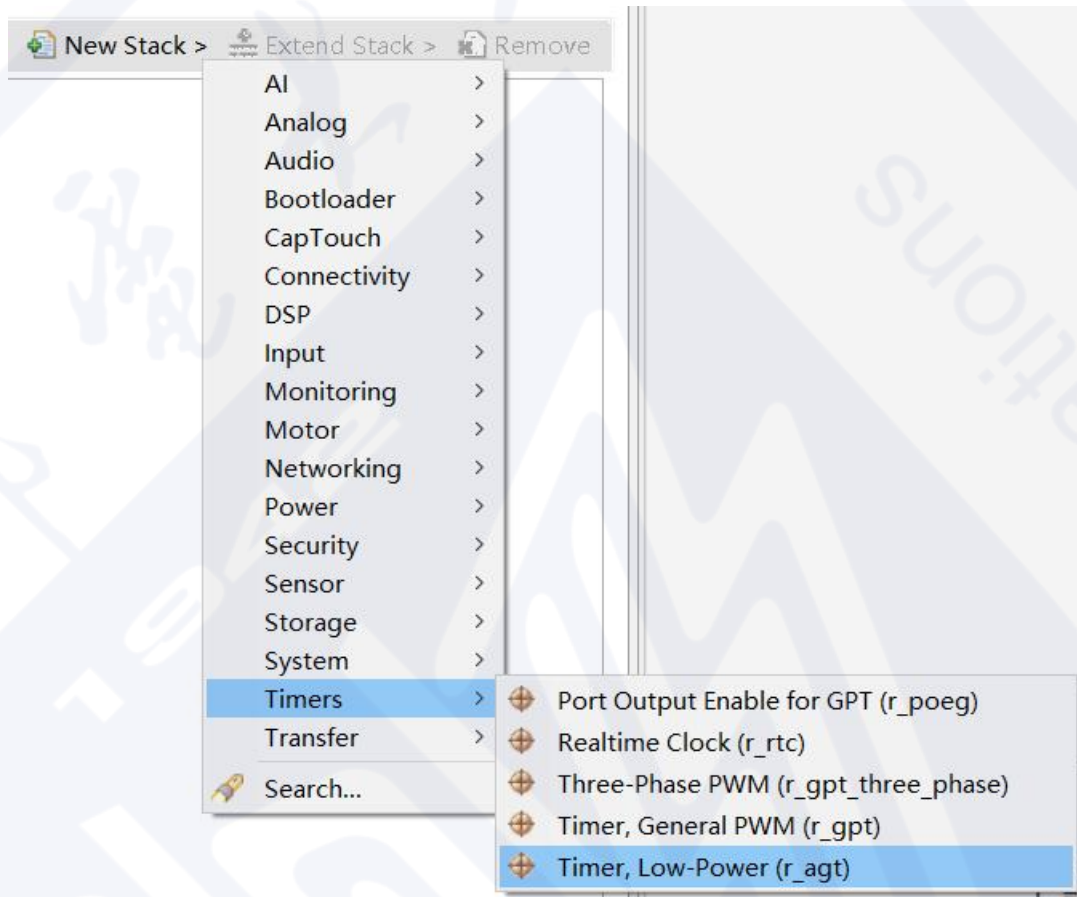


查看可视化界面可以得知，对应的是 AGT5。



AGT 定时器

操作 “New Stack > Timers > Timer, Low-Power (r_agt)” 在项目中添加 APT 定时器的多通道操作功能，特别是用于生成 PWM 信号或单次脉冲信号。



AGT 定时器属性配置

- **Pin Output Support:** 设为 Enabled，允许 AGT 将 PWM 信号输出到外部引脚。
- **Name:** 设置为 g_timer5，这是该 AGT 定时器的唯一标识符，用于在代码中引用此定时器实例。
- **Counter Bit Width:** 设为 AGT 16-bit，表示该定时器为 16 位定时器，可进行 16 位计数。
- **Channel:** 设为 5，表示使用定时器通道 5。
- **Mode:** 设为 Periodic，表示定时器工作在周期模式，会周期性地重复计数。
- **Period:** 周期值设置为 1，表示每个周期的时间间隔为 1。
- **Period Unit:** 单位选择为 Kilohertz（千赫兹），表示周期单位为千赫兹。
- **Count Source:** 设为 PCLKB，表示定时器使用 PCLKB 时钟源进行计数。
- **Duty Cycle Percent (only applicable in PWM mode):** 设为 50，表示在 PWM 模式下，输出的占空比为 50%。
- **AGTOA Output:** 设为 Start Level Low，表示 AGTOA 引脚的初始电平为低。
- **AGTOB Output:** 设为 Disabled，表示禁用 AGTOB 输出。
- **AGTO Output:** 设为 Start Level Low，表示 AGTO 引脚的初始电平为低。
- **AGTIOA5:** 连接到 P111 引脚，表示定时器的 PWM 信号输出连接到该引脚。

属性 × 控制台 调试 问题 Smart Browser Pin Conflicts 错误日志		
g_timer5 Timer, Low-Power (r_agt)		
Settings	属性	值
API Info	▼ Common	
	Parameter Checking	Default (BSP)
	Pin Output Support	Enabled 启动AGT输出
	Pin Input Support	Disabled
	▼ Module g_timer5 Timer, Low-Power (r_agt)	
	▼ General	
	Name	g_timer5 定时器名称
	Counter Bit Width	AGT 16-bit 16位计数
	Channel	5 通道
	Mode	Periodic 周期模式
	Period	1
	Period Unit	Kilohertz 1kHz
	Count Source	PCLKB 使用PCLKB时钟源
	▼ Output	
	Duty Cycle Percent (only applicable in PWM mode)	50 占空比为50%
	AGTOA Output	Start Level Low AGTOA引脚的初始电平为低
	AGTOB Output	Disabled
	AGTO Output	Disabled
	> Input	
	> Interrupts	
	▼ Pins	
	AGTOA5	P111 对应管脚
	AGTOB5	None

初始化 AGT

R_AGT_Open() 函数用于初始化 AGT 模块实例，它实现了 timer_api_t::open 接口。这个函数为 AGT 定时器配置了参数并启动定时器。

◆ R_AGT_Open()

```
fsp_err_t R_AGT_Open ( timer_ctrl_t *const p_ctrl,
                      timer_cfg_t const *const p_cfg
                      )
```

Initializes the AGT module instance. Implements timer_api_t::open.

The AGT hardware does not support one-shot functionality natively. The one-shot feature is therefore implemented in the AGT HAL layer. For a timer configured as a one-shot timer, the timer is stopped upon the first timer expiration.

The AGT implementation of the general timer can accept an optional agt_extended_cfg_t extension parameter. For AGT, the extension specifies the clock to be used as timer source and the output pin configurations. If the extension parameter is not specified (NULL), the default clock PCLKB is used and the output pins are disabled.

Example:

```
/* Initializes the module. */
err = R_AGT_Open(&g_timer0_ctrl, &g_timer0_cfg);
```

Return values

FSP_SUCCESS	Initialization was successful and timer has started.
FSP_ERR_ASSERTION	A required input pointer is NULL or the period is not in the valid range of 1 to 0xFFFF.
FSP_ERR_ALREADY_OPEN	R_AGT_Open has already been called for this p_ctrl.
FSP_ERR_IRQ_BSP_DISABLED	A required interrupt has not been enabled in the vector table.
FSP_ERR_IP_CHANNEL_NOT_PRESENT	Requested channel number is not available on AGT.

```
fsp_err_t err = FSP_SUCCESS;
```

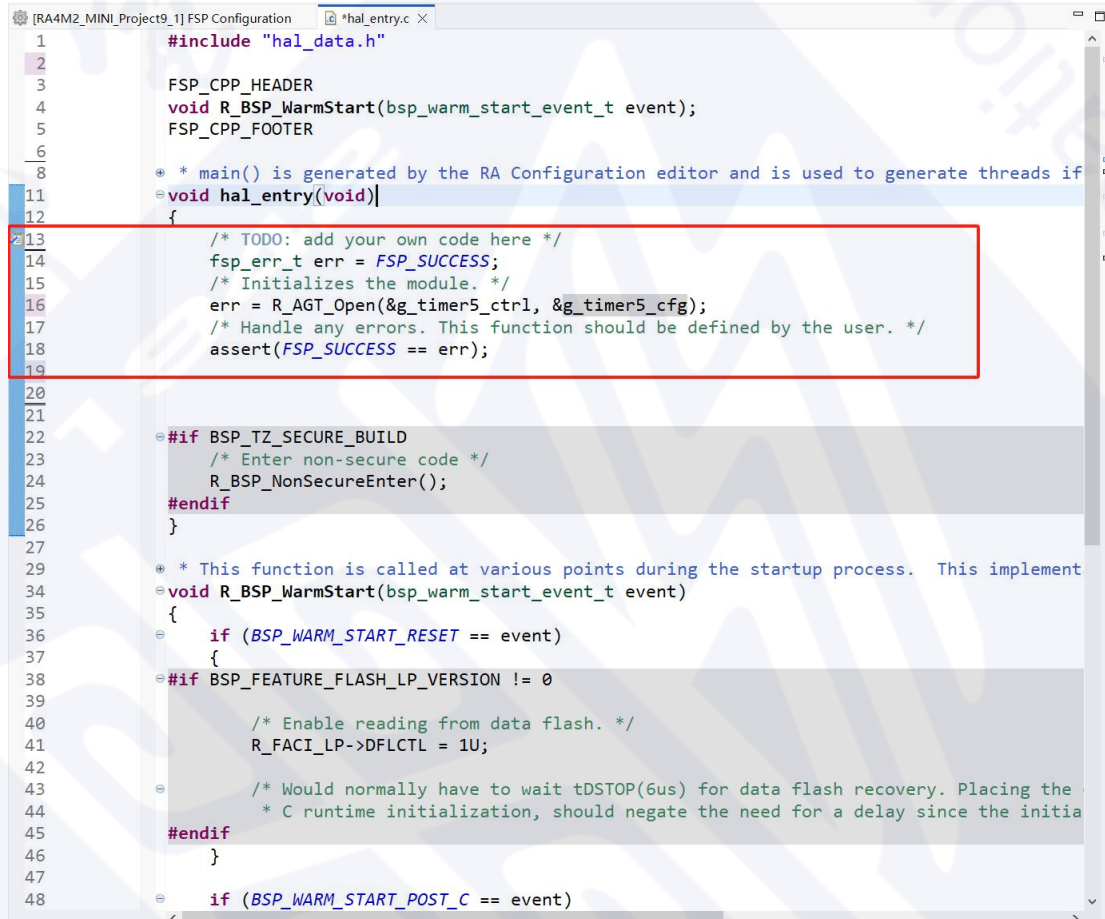
```

/* Initializes the module. */

err = R_AGT_Open(&g_timer5_ctrl, &g_timer5_cfg);

/* Handle any errors. This function should be defined by the user. */
assert(FSP_SUCCESS == err);

```



```

1  #include "hal_data.h"
2
3  FSP_CPP_HEADER
4  void R_BSP_WarmStart(bsp_warm_start_event_t event);
5  FSP_CPP_FOOTER
6
7  /* * main() is generated by the RA Configuration editor and is used to generate threads if
8  *
9  * void hal_entry(void)
10 {
11
12     /* TODO: add your own code here */
13     fsp_err_t err = FSP_SUCCESS;
14     /* Initializes the module. */
15     err = R_AGT_Open(&g_timer5_ctrl, &g_timer5_cfg);
16     /* Handle any errors. This function should be defined by the user. */
17     assert(FSP_SUCCESS == err);
18
19
20
21
22 #ifdef BSP_TZ_SECURE_BUILD
23     /* Enter non-secure code */
24     R_BSP_NonSecureEnter();
25 #endif
26 }
27
28 /* * This function is called at various points during the startup process. This implement
29 *
30 * void R_BSP_WarmStart(bsp_warm_start_event_t event)
31 {
32     if (BSP_WARM_START_RESET == event)
33     {
34         #ifdef BSP_FEATURE_FLASH_LP_VERSION != 0
35
36             /* Enable reading from data flash. */
37             R_FACI_LP->DFLCTL = 1U;
38
39             /* Would normally have to wait tDSTOP(6us) for data flash recovery. Placing the
40              * C runtime initialization, should negate the need for a delay since the initia
41             */
42         #endif
43     }
44
45     if (BSP_WARM_START_POST_C == event)
46
47
48

```

启动 AGT PWM 模块

`R_AGT_Start()` 函数用于启动 AGT 模块定时器。它实现了 `timer_api_t::start` 接口。调用该函数后，定时器开始计时。

◆ R_AGT_Start()

fsp_err_t R_AGT_Start (timer_ctrl_t *const p_ctrl)

Starts timer. Implements `timer_api_t::start`.

Example:

```
/* Start the timer. */
(void) R_AGT_Start(&g_timer0_ctrl);
```

Return values

FSP_SUCCESS Timer started.
FSP_ERR_ASSERTION p_ctrl is null.
FSP_ERR_NOT_OPEN The instance control structure is not opened.

```
/* Start the timer. */
(void) R_AGT_Start(&g_timer5_ctrl);
```

```
1  #include "hal_data.h"
2
3  FSP_CPP_HEADER
4  void R_BSP_WarmStart(bsp_warm_start_event_t event);
5  FSP_CPP_FOOTER
6
7  /* * main() is generated by the RA Configuration editor and is used to generate threads if
8  * void hal_entry(void)
9  {
10     /* TODO: add your own code here */
11     fsp_err_t err = FSP_SUCCESS;
12     /* Initializes the module. */
13     err = R_AGT_Open(&g_timer5_ctrl, &g_timer5_cfg);
14     /* Handle any errors. This function should be defined by the user. */
15     assert(FSP_SUCCESS == err);
16
17     /* Start the timer. */
18     (void) R_AGT_Start(&g_timer5_ctrl);
19
20     #if BSP_TZ_SECURE_BUILD
21     /* Enter non-secure code */
22     R_BSP_NonSecureEnter();
23     #endif
24 }
25
26 /* * This function is called at various points during the startup process. This implement
27 * void R_BSP_WarmStart(bsp_warm_start_event_t event)
28 {
29     if (BSP_WARM_START_RESET == event)
30     {
31         #if BSP_FEATURE_FLASH_LP_VERSION != 0
32
33         /* Enable reading from data flash. */
34         R_FACI_LP->DFLCTL = 1U;
35
36         /* Would normally have to wait tDSTOP(6us) for data flash recovery. Placing the
37          * C runtime initialization, should negate the need for a delay since the initia
38     }
39     #endif
40 }
```

AGTIO 和 AGTO

AGTIO 和 AGTO 是与 AGT（异步通用定时器）相关的输出引脚，主要用于在定时器工作时输出与定时器计数器状态相关的信号。

AGTIO 和 AGTO 的输出引脚每次周期结束时会翻转一次。在一次性模式下，输出是通过翻转产生脉冲的。

21.3.4 Pulse Output Mode

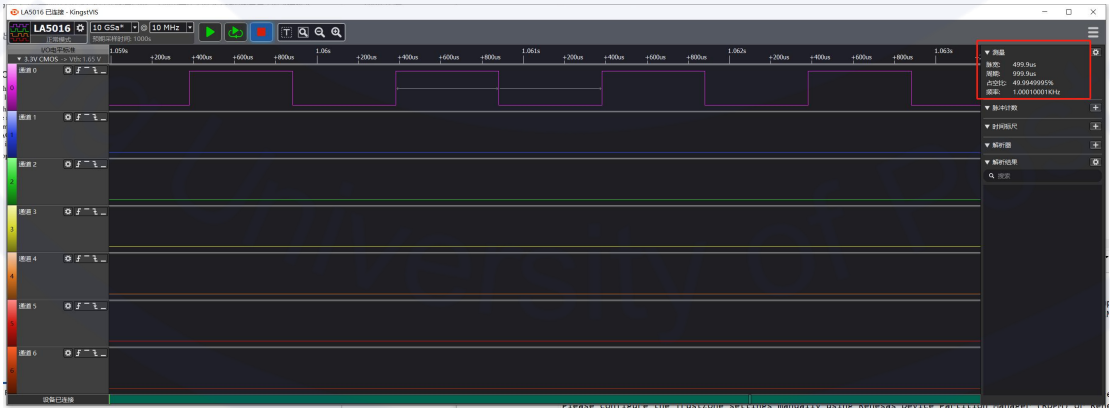
In pulse output mode, the counter is decremented by the count source selected with the TCK[2:0] bits in the AGTMR1 register, and the output level of the AGTIO pin and AGTO pin inverted each time an underflow occurs.

In pulse output mode, the count value is decremented by 1 on each rising edge of the count source. When the count value reaches 0x0000 and the next count source is input, an underflow occurs and an interrupt request is generated. In addition, a pulse can be output from the AGTIO pin and AGTO pin. The output level is inverted each time an underflow occurs. The pulse output from the AGTO pin can be stopped with the TOE bit in the AGTIOC register. The output level can be selected with the TEDGSEL bit in the AGTIOC register.

Figure 21.7 shows the operation example in pulse output mode.

演示

AGTOA5 输出 1k 占空比 50%的 PWM。

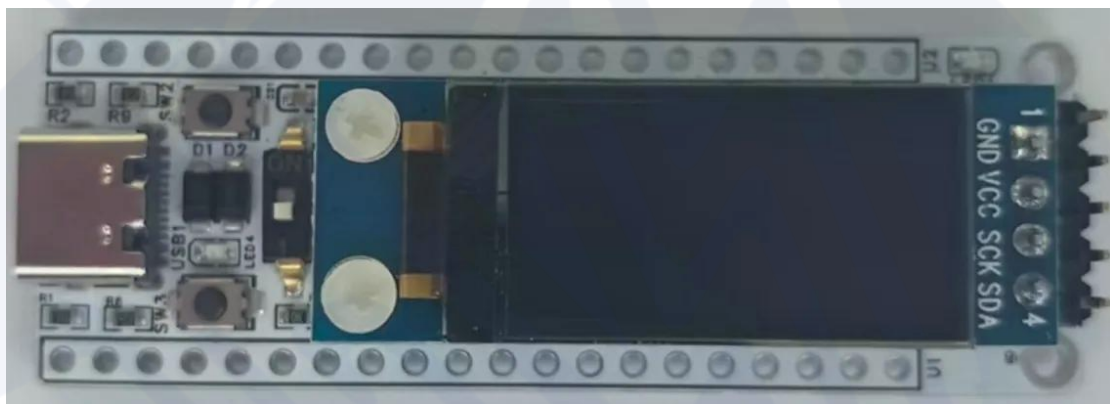


project11: AGT 定时器频率与占空比的设置

概述

AGT 模块是一个高精度定时器，用于计数事件、测量外部输入信号、生成周期性中断，或输出周期性信号/PWM 信号到 GTIOC 引脚。它广泛应用于定时控制、信号测量、PWM 生成等应用，尤其适用于电机控制、LED 调光、PWM 伺服驱动和 ADC 触发等场景。

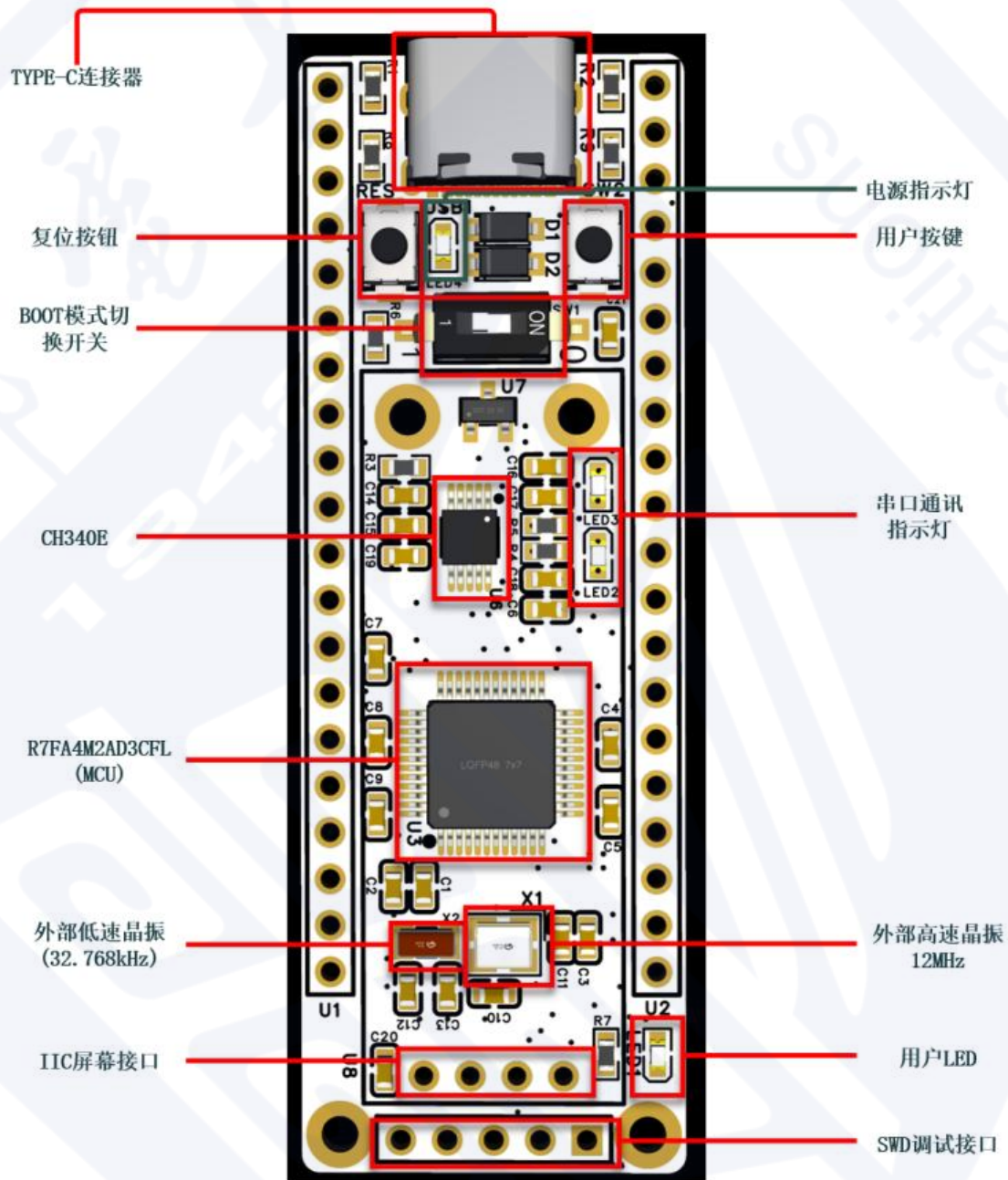
本文将详细介绍如何上一章基础上通过 AGT 定时器的 API 调整频率和占空比，并解释相关的关键概念和设置方法。



硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL



参考程序

https://github.com/CoreMaker-lab/RA4M2_MINI

https://gitee.com/CoreMaker/RA4M2_MINI

选择计时器

RA MCU 有两个定时器外设：通用 PWM 定时器 (GPT) 和异步通用定时器 (AGT)。

	GPT	AGT
低功耗模式	GPT 可以在睡眠模式下运行。	AGT 可以在所有低功耗模式下运行。
可用频道	GPT 通道的数量因设备而异。目前所有支持的 MCU 至少都有 7 个 GPT 通道。	所有 MCU 都有 2 个 AGT 通道。
定时器分辨率	所有 MCU 均至少有一个 32 位 GPT 计时器。	AGT 计时器是 16 位计时器。
时钟源	GPT 通过可配置分频器（最高可达 1024）的 PCLKD 运行。它还可以配置为计数 ELC 事件或外部脉冲。	AGT 通过 PCLKB、LOCO 或子时钟运行。

时钟源

AGT（异步通用定时器）可以选择不同的时钟源进行运行，具体有三种时钟源可供选择：

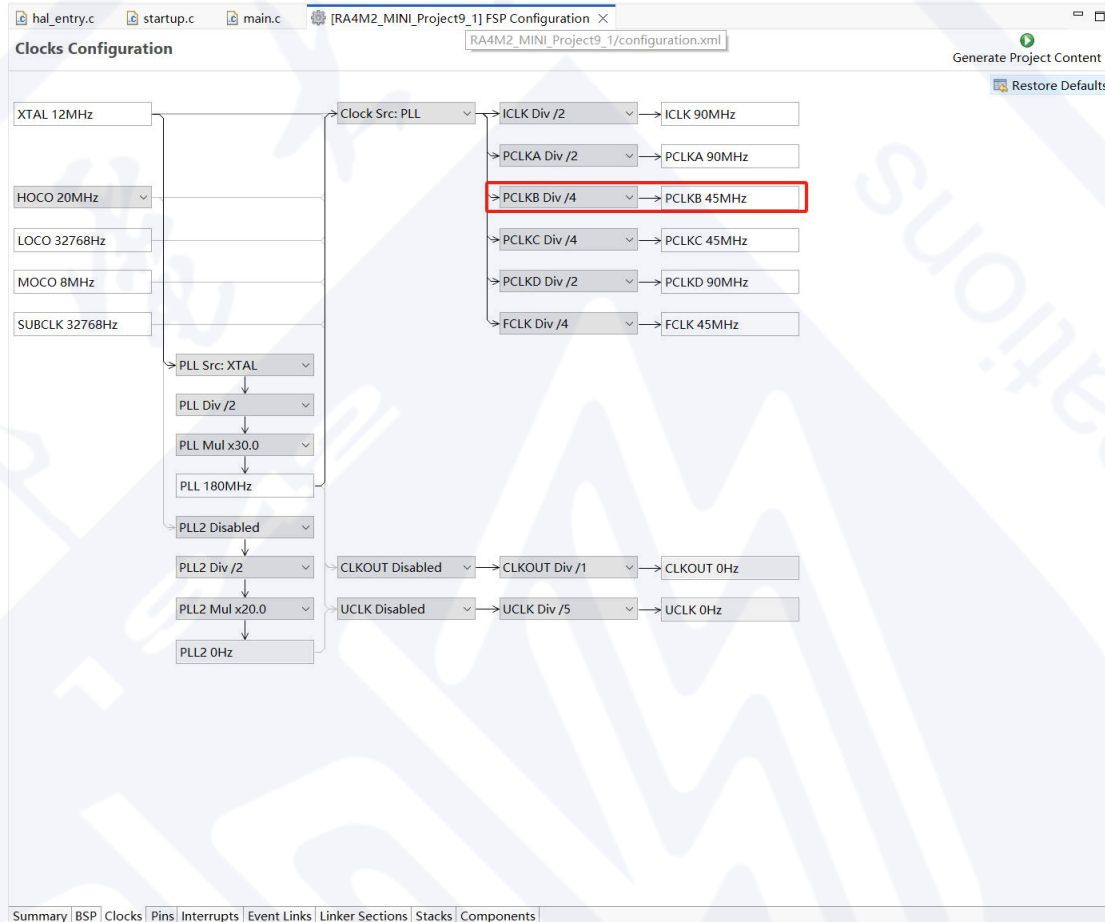
1. PCLKB：系统时钟 PCLKB。
2. LOCO：低速内部时钟 LOCO（Low-Speed On-Chip Oscillator）。
3. Subclock：外部子时钟。

	GPT	AGT
Low Power Modes	The GPT can operate in sleep mode.	The AGT can operate in all low power modes (when count source is LOCO or subclock).
Available Channels	The number of GPT channels is device specific. All currently supported MCUs have at least 7 GPT channels.	All MCUs have at least 2 AGT channels.
Timer Resolution	All MCUs have at least one 32-bit GPT timer.	The AGT timers can be 16-bit or 32-bit timers.
Clock Source	The GPT runs off PCLKD with a configurable divider up to 1024. It can also be configured to count ELC events or external pulses.	The AGT runs off PCLKB, LOCO, or subclock with a configurable divider up to 8 for PCLKB or up to 128 for LOCO or subclock.

在这使用的是 PCLKB。

属性 ×	控制台	调试	问题	Smart Browser	Pin Conflicts	错误日志
g_timer5 Timer, Low-Power (r_agt)						
Settings	属性					
API Info	值					
	▼ Common					
	Parameter Checking					
	Default (BSP)					
	Pin Output Support					
	Enabled					
	Pin Input Support					
	Disabled					
	▼ Module g_timer5 Timer, Low-Power (r_agt)					
	▼ General					
	Name					
	g_timer5					
	Counter Bit Width					
	AGT 16-bit					
	Channel					
	5					
	Mode					
	Periodic					
	Period					
	1					
	Period Unit					
	Kilohertz					
	Count Source					
	PCLKB					
	▼ Output					
	Duty Cycle Percent (only applicable in PWM mode)					
	50					
	AGTOA Output					
	Start Level Low					
	AGTOB Output					
	Disabled					
	AGTO Output					
	Disabled					
	> Input					
	> Interrupts					
	▼ Pins					
	AGTOA5					
	P111					
	AGTOB5					
	None					

选择 XTAL 12MHz 作为时钟源 (Clock Src: XTAL)。PCLKB 时钟的分频器设置为 Div /4，意味着 PCLKB 直接运行在 45MHz。



PWM(脉冲宽度调制)

脉冲宽度调制是一种模拟控制方式，根据相应载荷的变化来调制晶体管基极或 MOS 管栅极的偏置，来实现晶体管或 MOS 管导通时间的改变，从而实现开关稳压电源输出的改变。这种方式能使电源的输出电压在工作条件变化时保持恒定，是利用微处理器的数字信号对模拟电路进行控制的一种非常有效的技术。脉冲宽度调制是利用微处理器的数字输出来对模拟电路进行控制的一种非常有效的技术，广泛应用在从测量、通信到功率控制与变换的许多领域中。

在瑞萨 RA 系列 MCU 中有两种定时器，一种是通用 PWM 定时器 GPT，另外一种异步通用定时器 AGT。

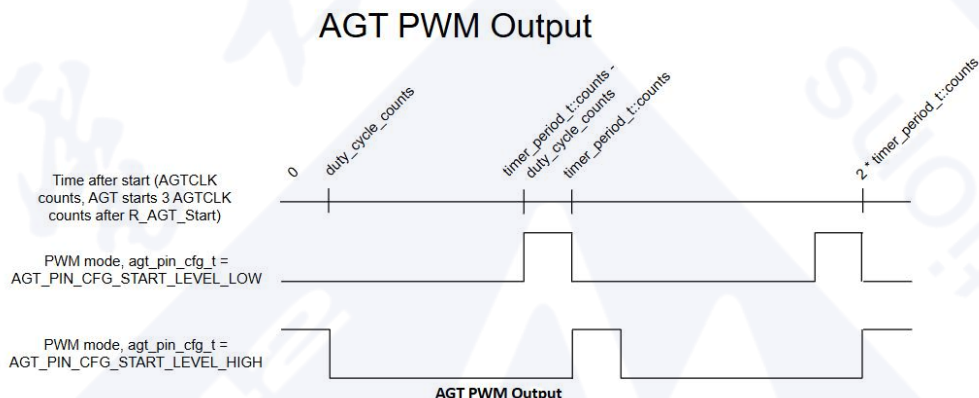
频率=主频/period

+占空比=cycle/period

下图中展示的 PWM 输出信号，在周期开始时为高电平，周期结束时为低电平。通过调整占空比（duty cycle）和周期（timer period），可以调节 PWM 信号的行为。

- **duty_cycle_counts:** 表示占空比部分的持续时间。占空比决定了信号高电平的时间比例。

- **timer_period_counts**: 这是一个周期的时间，单位是 PCLKD 计数（即时钟频率的周期数）。



R_AGT_PeriodSet() 函数说明

R_AGT_PeriodSet 函数它的功能是设置 AGT（异步通用定时器）的周期值，并在设置时将计数器重置为最大值。该函数实现了定时器周期的更新。

◆ R_AGT_PeriodSet()

```
fsp_err_t R_AGT_PeriodSet ( timer_ctrl_t *const p_ctrl,
                           uint32_t const   period_counts
                           )
```

Updates period. The new period is updated immediately and the counter is reset to the maximum value. Implements `timer_api_t::periodSet`.

Warning

If periodic output is used, the duty cycle buffer registers are updated after the period buffer register. If this function is called while the timer is running and an AGT underflow occurs during processing, the duty cycle will not be the desired 50% duty cycle until the counter underflow after processing completes.

Stop the timer before calling this function if one-shot output is used.

Example:

```
/* Get the source clock frequency (in Hz). There are several ways to do this in FSP:
 * - If L000 or subclock is chosen in agt_extended_cfg.t::clock_source
 * - The source clock frequency is BSP_L000_HZ >> timer_cfg.t::source_div
 * - If PCLKB is chosen in agt_extended_cfg.t::clock_source and the PCLKB frequency has not changed since reset,
 * - The source clock frequency is BSP_STARTUP_PCLKB_HZ >> timer_cfg.t::source_div
 * - Use the R_AGT_InfoGet function (it accounts for the clock source and divider).
 * - Calculate the current PCLKB frequency using R_FSP_SystemClockHzGet(FSP_PRIV_CLOCK_PCLKB) and right shift
 *   by timer_cfg.t::source_div.
 * This example uses the last option (R_FSP_SystemClockHzGet).
 */
uint32_t timer_freq_hz = R_FSP_SystemClockHzGet(FSP_PRIV_CLOCK_PCLKB) >> g_timer0_cfg.source_div;

/* Calculate the desired period based on the current clock. Note that this calculation could overflow if the
 * desired period is larger than UINT32_MAX / pclkb_freq_hz. A cast to uint64_t is used to prevent this. */
uint32_t period_counts =
    (uint32_t) (((uint64_t) timer_freq_hz * AGT_EXAMPLE_DESIRED_PERIOD_MSEC) / AGT_EXAMPLE_MSEC_PER_SEC);

/* Set the calculated period. This will return an error if parameter checking is enabled and the calculated
 * period is larger than UINT16_MAX. */
err = R_AGT_PeriodSet(&g_timer0_ctrl, period_counts);
assert(FSP_SUCCESS == err);
```

Return values

FSP_SUCCESS Period value updated.

FSP_ERR_ASSERTION A required pointer was NULL, or the period was not in the valid range of 1 to 0xFFFF.

FSP_ERR_NOT_OPEN The instance control structure is not opened.

频率=时钟源/period，若设置频率为 10K，则 period=45M/10K=4500

```
err = R_AGT_PeriodSet(&g_timer5_ctrl, 4500); //频率
assert(FSP_SUCCESS == err);

R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS); //不加延时可能会设置不成功
```

```
*hal_entry.c × startup.c main.c [RA4M2_MINI_Project9_1] FSP Configuration
1 #include "hal_data.h"
2
3 FSP_CPP_HEADER
4 void R_BSP_WarmStart(bsp_warm_start_event_t event);
5 FSP_CPP_FOOTER
6
7
8 * * main() is generated by the RA Configuration editor and is used to generate threads if
9
10 void hal_entry(void)
11 {
12     /* TODO: add your own code here */
13     fsp_err_t err = FSP_SUCCESS;
14     /* Initializes the module. */
15     err = R_AGT_Open(&g_timer5_ctrl, &g_timer5_cfg);
16     /* Handle any errors. This function should be defined by the user. */
17     assert(FSP_SUCCESS == err);
18
19     /* Start the timer. */
20     (void) R_AGT_Start(&g_timer5_ctrl);
21     R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS); //不加延时可能会设置不成功
22
23     err = R_AGT_PeriodSet(&g_timer5_ctrl, 4500); //频率
24     assert(FSP_SUCCESS == err);
25     R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS); //不加延时可能会设置不成功
26
27
28
29
30
31
32
33
34
35 #if BSP_TZ_SECURE_BUILD
36     /* Enter non-secure code */
37     R_BSP_NonSecureEnter();
38 #endif
39 }
40
41 * * This function is called at various points during the startup process. This implement
42 void R_BSP_WarmStart(bsp_warm_start_event_t event)
43 {
44     <
```

R_AGT_DutyCycleSet() 函数说明

R_AGT_DutyCycleSet() 函数用于设置 AGT 定时器的占空比。

◆ R_AGT_DutyCycleSet()

```
fsp_err_t R_AGT_DutyCycleSet ( timer_ctrl_t *const p_ctrl,
                               uint32_t const duty_cycle_counts,
                               uint32_t const pin
                               )
```

Updates duty cycle. If the timer is counting, the new duty cycle is reflected after the next counter underflow. Implements `timer_api_t::dutyCycleSet`.

Example:

```
/* Get the current period setting. */
timer_info_t info;
(void) R_AGT_InfoGet(&g_timer0_ctrl, &info);
uint32_t current_period_counts = info.period_counts;

/* Calculate the desired duty cycle based on the current period. */
uint32_t duty_cycle_counts = (current_period_counts * AGT_EXAMPLE_DESIRED_DUTY_CYCLE_PERCENT) /
                              AGT_EXAMPLE_MAX_PERCENT;

/* Set the calculated duty cycle. */
err = R_AGT_DutyCycleSet(&g_timer0_ctrl, duty_cycle_counts, AGT_OUTPUT_PIN_AGTOA);
assert(FSP_SUCCESS == err);
```

Return values

FSP_SUCCESS	Duty cycle updated.
FSP_ERR_ASSERTION	A required pointer was NULL, or the pin was not AGT_AGTO_AGTOA or AGT_AGTO_AGTOB.
FSP_ERR_INVALID_ARGUMENT	Duty cycle was not in the valid range of 0 to period (counts) - 1
FSP_ERR_NOT_OPEN	The instance control structure is not opened.
FSP_ERR_UNSUPPORTED	AGT_CFG_OUTPUT_SUPPORT_ENABLE is 0.

+ 占空比=cycle/period, 若设置 GTIOCA+ 占空比为 25%, 则 cycle= + 占空比* period=25%*4500=1125

```
err = R_AGT_DutyCycleSet(&g_timer5_ctrl, 1125, AGT_OUTPUT_PIN_AGTOA); // 占空比
assert(FSP_SUCCESS == err);

R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS);
```



```
1 #include "hal_data.h"
2
3 FSP_CPP_HEADER
4 void R_BSP_WarmStart(bsp_warm_start_event_t event);
5 FSP_CPP_FOOTER
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
```

R_AGT_Reset() 函数说明

R_AGT_Reset() 函数用于将 AGT 定时器的计数器重置为周期值减一。

◆ R_AGT_Reset()

fsp_err_t R_AGT_Reset (timer_ctrl_t *const p_ctrl)

Resets the counter value to the period minus one. Implements `timer_api_t::reset`.

Return values

FSP_SUCCESS	Counter reset.
FSP_ERR_ASSERTION	p_ctrl is NULL
FSP_ERR_NOT_OPEN	The instance control structure is not opened.

R_AGT_Close() 函数说明

R_AGT_Close() 函数用于停止 AGT 定时器、禁用中断、禁用输出引脚，并清除内部驱动程序数据。

◆ R_AGT_Close()

fsp_err_t R_AGT_Close (timer_ctrl_t *const p_ctrl)

Stops counter, disables interrupts, disables output pins, and clears internal driver data. Implements timer_api_t::close.

Return values

FSP_SUCCESS Timer closed.
FSP_ERR_ASSERTION p_ctrl is NULL.
FSP_ERR_NOT_OPEN The instance control structure is not opened.

关闭定时器 GPT。

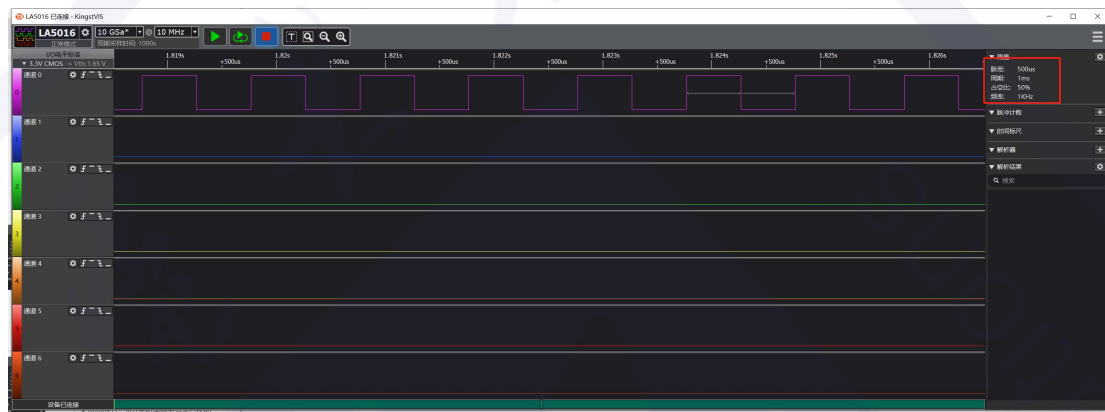
```
err = R_AGT_Close(&g_timer5_ctrl);  
  
assert(FSP_SUCCESS == err);  
  
R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS);
```



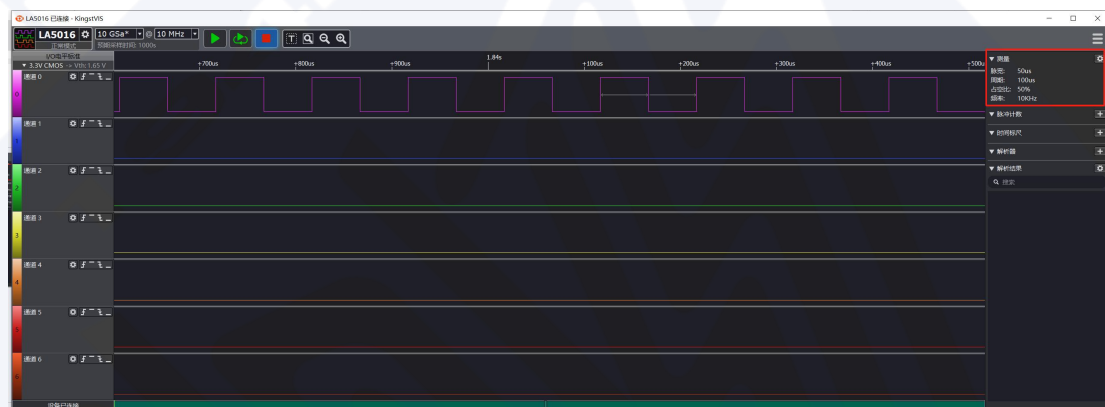
```
1  #include "hal_data.h"  
2  
3  FSP_CPP_HEADER  
4  void R_BSP_WarmStart(bsp_warm_start_event_t event);  
5  FSP_CPP_FOOTER  
6  
7  /* * main() is generated by the RA Configuration editor and is used to generate threads if  
8  * void hal_entry(void)  
9  {  
10     /* TODO: add your own code here */  
11     fsp_err_t err = FSP_SUCCESS;  
12     /* Initializes the module. */  
13     err = R_AGT_Open(&g_timer5_ctrl, &g_timer5_cfg);  
14     /* Handle any errors. This function should be defined by the user. */  
15     assert(FSP_SUCCESS == err);  
16  
17     /* Start the timer. */  
18     (void) R_AGT_Start(&g_timer5_ctrl);  
19     R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS); // 不加延时可能会设置不成功  
20  
21     err = R_AGT_PeriodSet(&g_timer5_ctrl, 4500); // 频率  
22     assert(FSP_SUCCESS == err);  
23     R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS); // 不加延时可能会设置不成功  
24  
25     err = R_AGT_DutyCycleSet(&g_timer5_ctrl, 1125, AGT_OUTPUT_PIN_AGTOA); // 占空比  
26     assert(FSP_SUCCESS == err);  
27     R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS);  
28  
29     err = R_AGT_Close(&g_timer5_ctrl);  
30     assert(FSP_SUCCESS == err);  
31     R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS);  
32  
33     #if BSP_TZ_SECURE_BUILD  
34     /* Enter non-secure code */  
35     R_BSP_NonSecureEnter();  
36     #endif  
37 }  
38  
39  
40  
41  
42  
43
```

波形情况

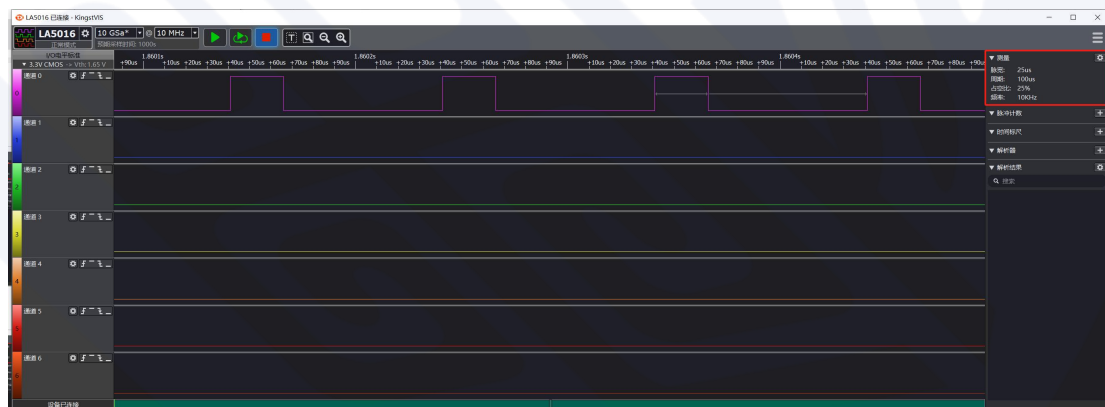
最开始 AGTOA5 输出 1k 占空比 50% 的 PWM。



AGTOA5 频率设置成功后频率直接修改为 10k，占空比为 50%。



最后 AGTOA5 正占空比成功修改为 25%。



呼吸灯示例

LED 亮度随占空比线性升降，呈现柔和的 Breathing 灯效果。

```
void hal_entry(void)
{
    /* TODO: add your own code here */

    fsp_err_t err = FSP_SUCCESS;

    /* Initializes the module. */
}
```

```

err = R_AGT_Open(&g_timer5_ctrl, &g_timer5_cfg);
/* Handle any errors. This function should be defined by the user. */
assert(FSP_SUCCESS == err);

/* Start the timer. */
(void) R_AGT_Start(&g_timer5_ctrl);
R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS); //不加延时可能会设置不成功

err = R_AGT_PeriodSet(&g_timer5_ctrl, 4500); //频率
assert(FSP_SUCCESS == err);
R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS); //不加延时可能会设置不成功

err = R_AGT_DutyCycleSet(&g_timer5_ctrl, 1125, AGT_OUTPUT_PIN_AGTOA); //占空比
assert(FSP_SUCCESS == err);
R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS);

// err = R_AGT_Close(&g_timer5_ctrl);
// assert(FSP_SUCCESS == err);
// R_BSP_SoftwareDelay (20, BSP_DELAY_UNITS_MILLISECONDS);

/* -----
 * PWM “呼吸”效果核心循环
 * duty   : 0-1000, 对应 0%-100% 占空比 (后面 ×45 映射到 APT 计数)
 * rising : true 表示占空比递增, false 表示递减
 * -----*/
int duty = 1;      /* 当前占空比计数 (1/100 分辨率) */
bool rising = true; /* true=递增, false=递减 */
while(1)
{
    /* ----- 1. 计算下一步 duty ----- */
    if (rising) /* 处于递增阶段 */
    {
        duty++; /* 占空比 +1 */
        if (duty >= 100) /* 到达上限时 */
        {
            duty = 100; /* 饱和 */
            rising = false; /* 方向改为递减 */
        }
    }
    else /* 递减阶段 */
    {
        duty--; /* 占空比 -1 */
        if (duty <= 0) /* 到达下限时 */
        {

```

```

        duty = 0;          /* 饱和 */
        rising = true;     /* 方向改为递增 */
    }
}

/* ----- 2. 将新 duty 写入 APT ----- */
/* 45 × duty 把 0-100 映射到 0-4500 的 APT 计数值，
   具体比例由 PWM 频率 / 时钟设置决定。 */
err = R_AGT_DutyCycleSet(&g_timer5_ctrl,
                        45 * duty,
                        AGT_OUTPUT_PIN_AGTOA);
assert(FSP_SUCCESS == err); /* 确认写入成功 */

/* ----- 3. 节拍延时 ----- */
/* 10ms 更新一次占空比，控制呼吸速度。
   若运行在 RTOS，可改为 vTaskDelay(1)。 */
R_BSP_SoftwareDelay(10, BSP_DELAY_UNITS_MILLISECONDS);
}

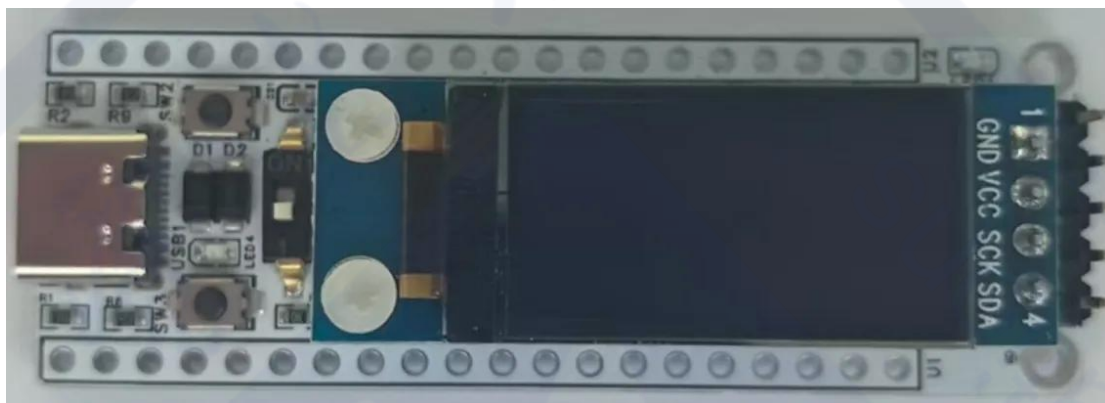
#ifdef BSP_TZ_SECURE_BUILD
    /* Enter non-secure code */
    R_BSP_NonSecureEnter();
#endif
}

```

project12: 打印函数(sprintf、printf)的实现

概述

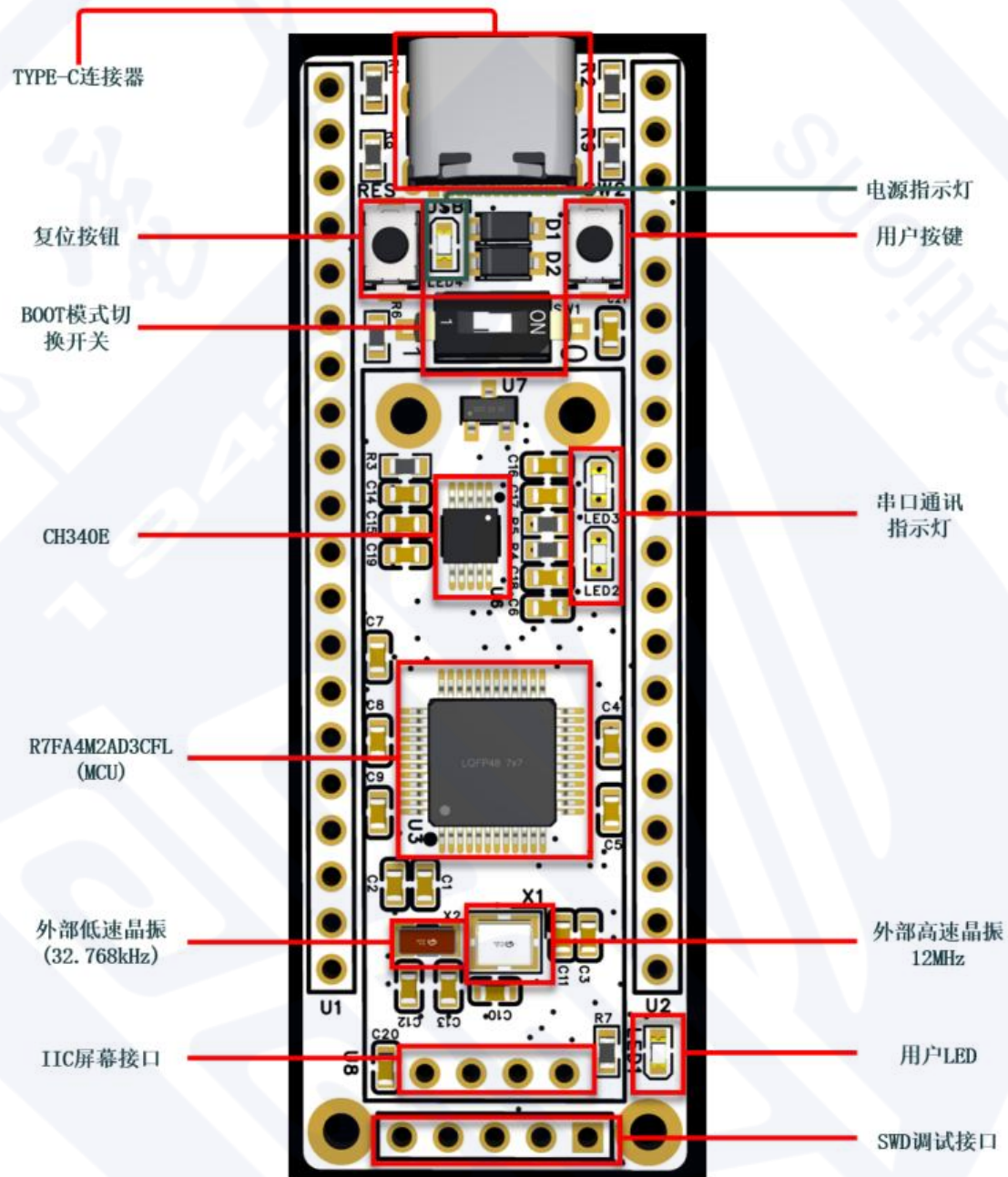
`printf` 和 `sprintf` 是 C 语言中常用的输出函数，广泛应用于各种嵌入式、桌面应用程序和调试过程中。这些函数可以将格式化的数据输出到标准输出（如控制台）或存储到字符串中。在系统开发中，了解它们的底层实现不仅能够帮助优化性能，还能提高代码的可移植性和灵活性。



硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL

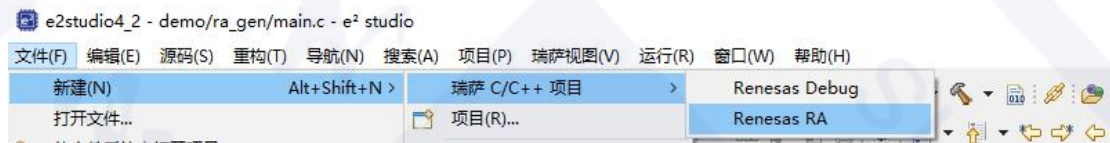


参考程序

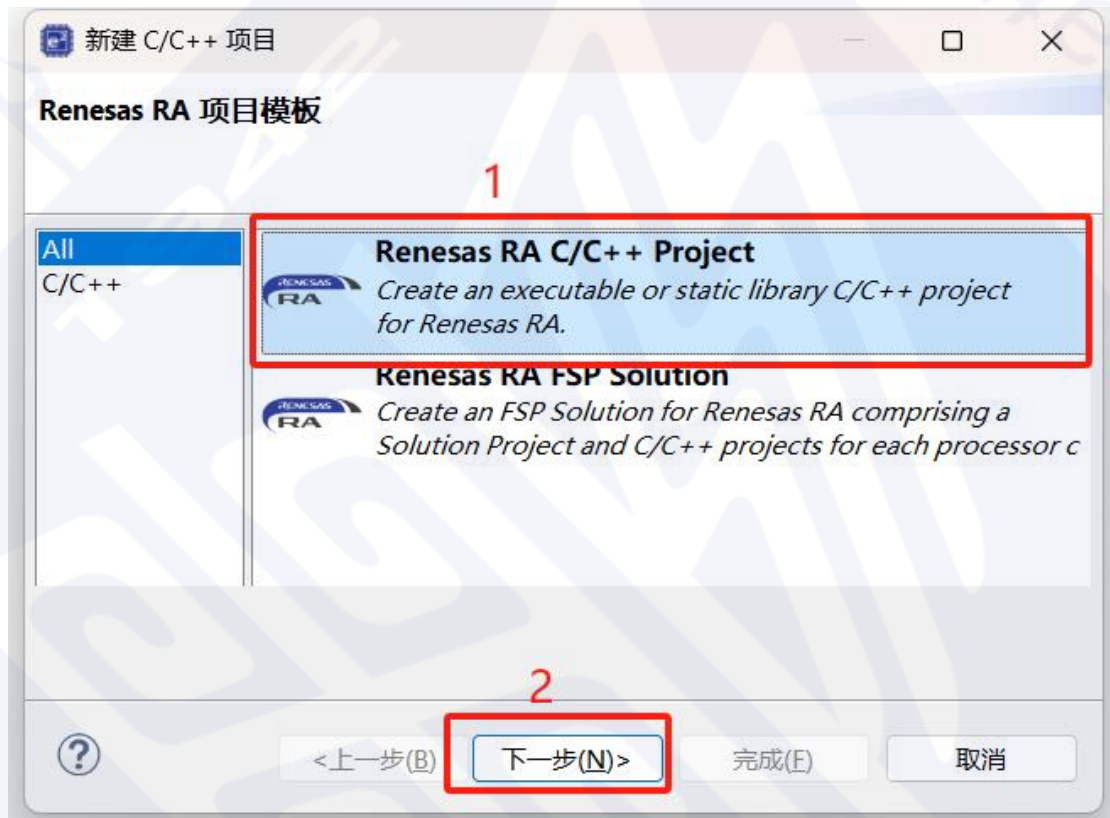
https://github.com/CoreMaker-lab/RA4M2_MINI

https://gitee.com/CoreMaker/RA4M2_MINI

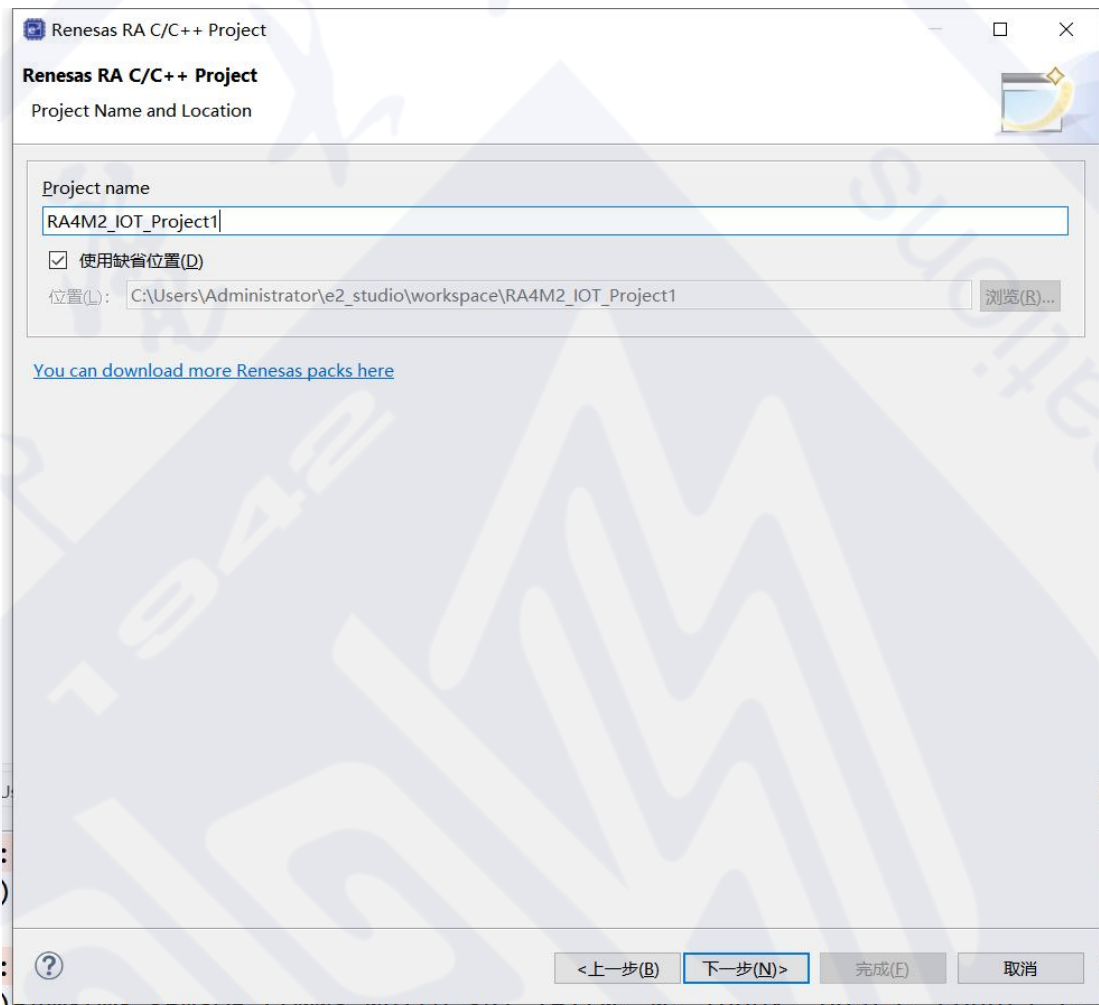
新建工程



工程模板



保存工程路径



芯片配置

本文中使用 R7FA4M2AD3CFL 来进行演示。

Renesas RA C/C++ Project

Renesas RA C/C++ Project

Device and Tools Selection

Device Selection

FSP Version: 5.9.0 **版本**

Board: Custom User Board (Any Device) ...

Device: R7FA4M2AD3CFL **芯片型号**

Core: CM33

Language: ☒ C ☐ C++

Board Description

Device Details

TrustZone	Yes
Pins	48
Processor	Cortex-M33

IDE Project Type

e2 studio managed build

Toolchains

LLVM Embedded Toolchain for Arm

GNU ARM Embedded **工具链**

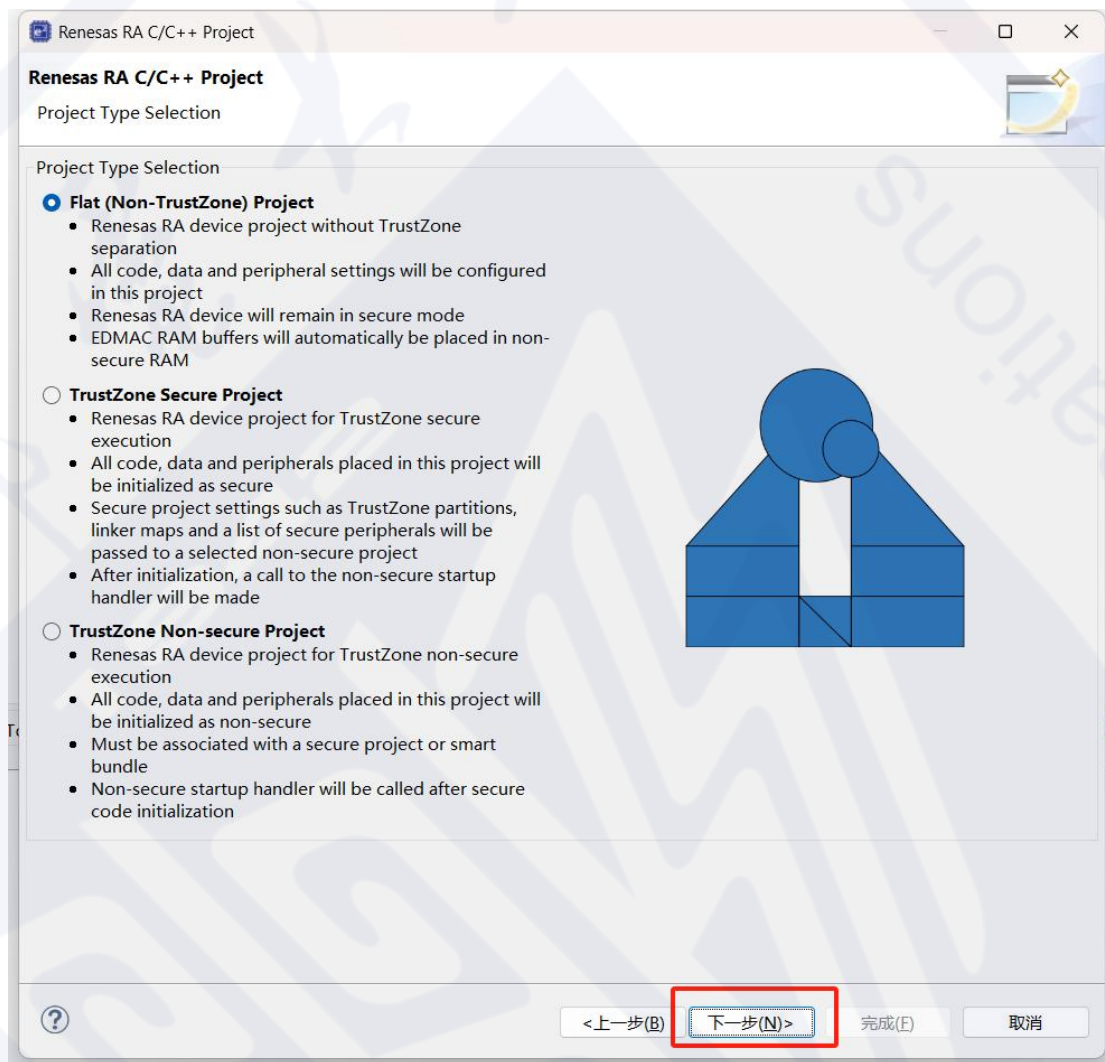
13.2.1.arm-13-7 [Manage Toolchains...](#)

Debugger

J-Link ARM **使用JLink进行调试**

[?](#) <上一步(B) 下一步(N)> 完成(F) 取消

工程模板选择



Renesas RA C/C++ Project

Renesas RA C/C++ Project

Preceding Project or Smart Bundle Selection

☒ None

Choose this option when creating a project for the primary processor core (no preceding project or Smart Bundle).

☐ Preceding Project:

Choose this option if you have access to the preceding bootloader project or TrustZone secure project source code. A debug configuration for Secure Software Debug (SSD) state will be generated where applicable and both secure and non-secure images will be downloaded to the target device. Should you wish to test Non-Secure Software Debug (NSECSO) state, use the Renesas Device Partition Manager to change the Device Lifecycle state as needed.

☐ Smart Bundle:

Resolved location:

Workspace...

File System...

Variables...

Choose this option if you only have access to a pre-programmed device containing TrustZone secure code. The Smart Bundle file (*.sbd) should be obtained from the TrustZone Secure Project developer. A debug configuration for Non-Secure Software Debug (NSECSO) state will be generated and only the non-secure code image will be downloaded to the target device.

Preceding Project/Smart Bundle Details

FSP version

Toolchain

Toolchain version

Board

Device

Core

Zones

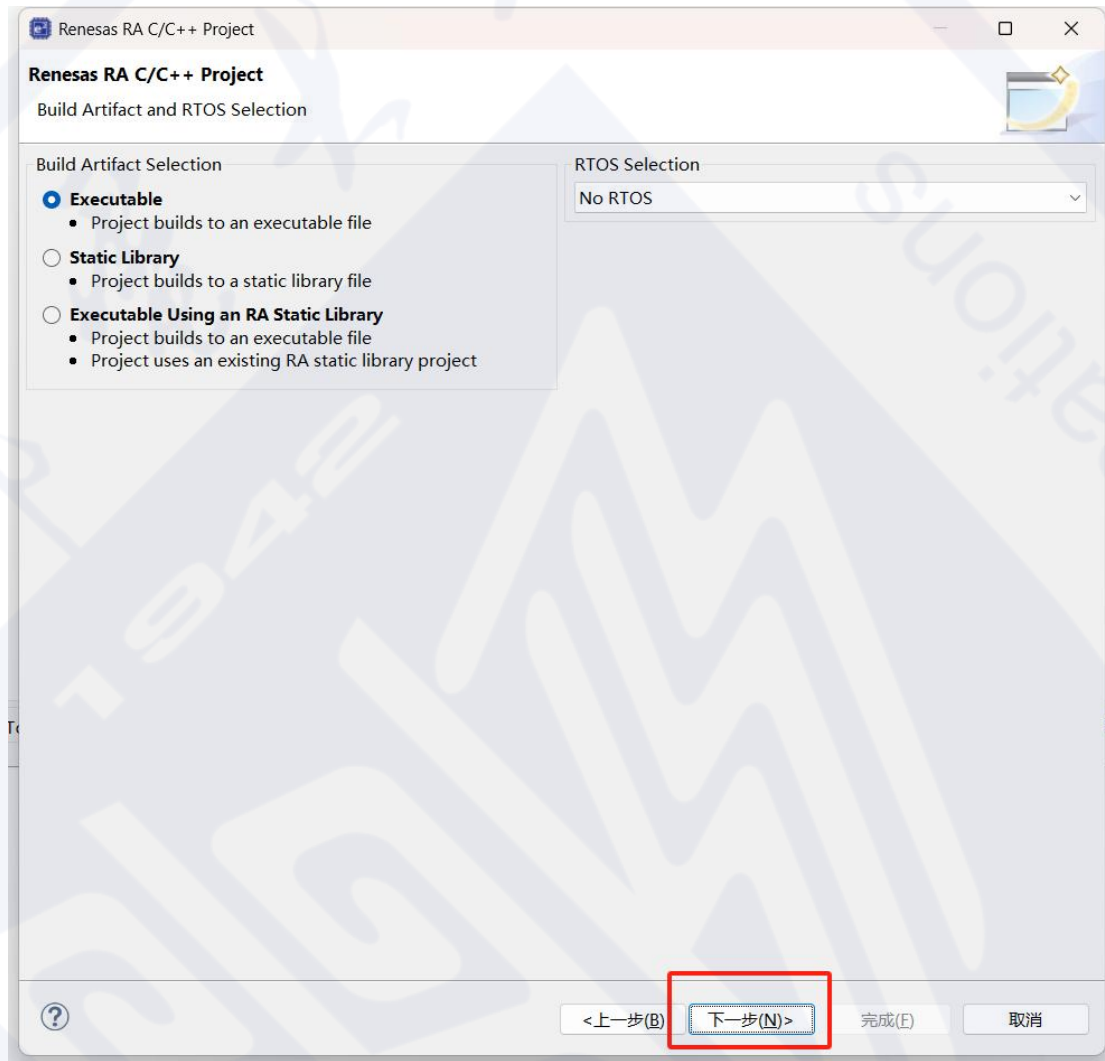
?

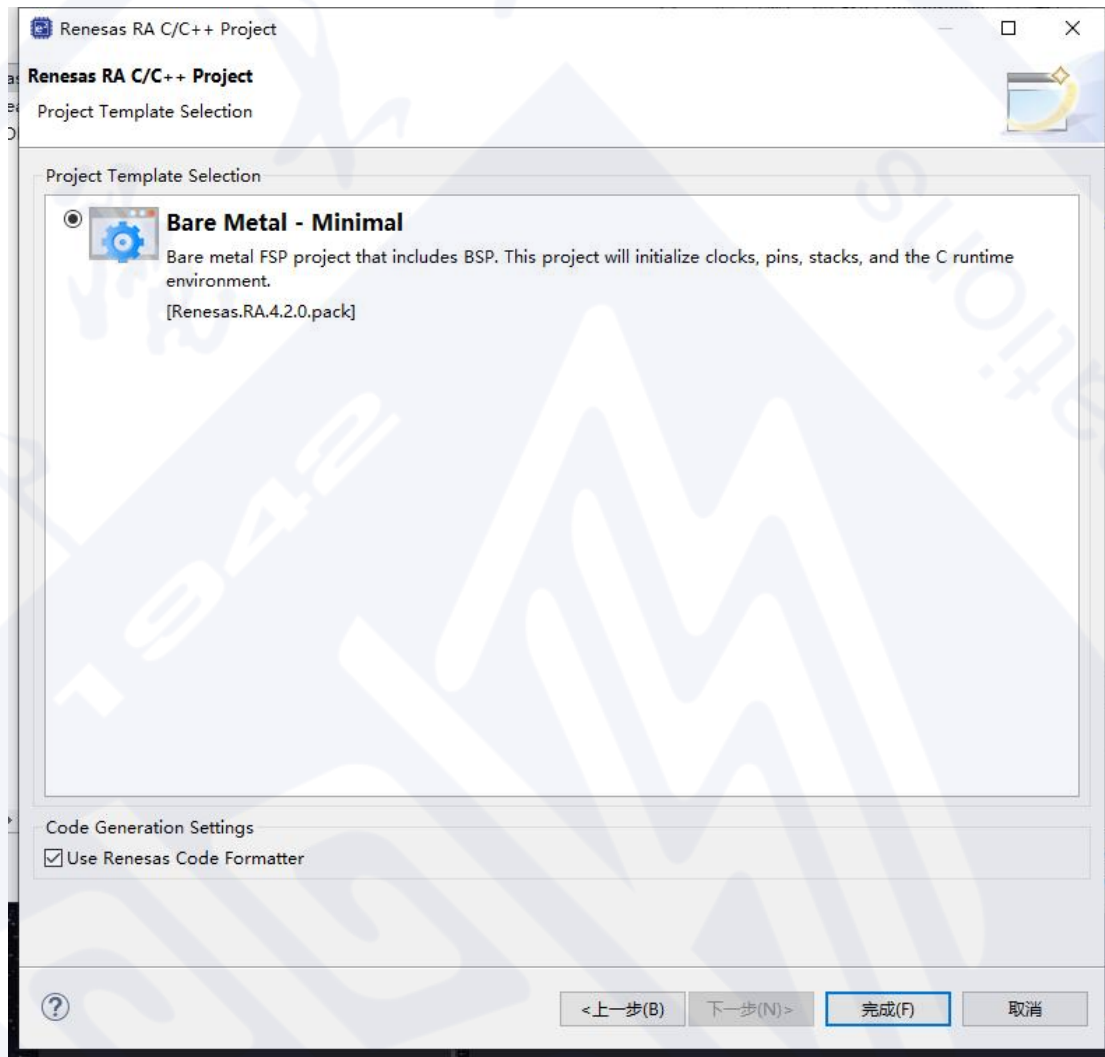
< 上一步(B)

下一步(N) >

完成(E)

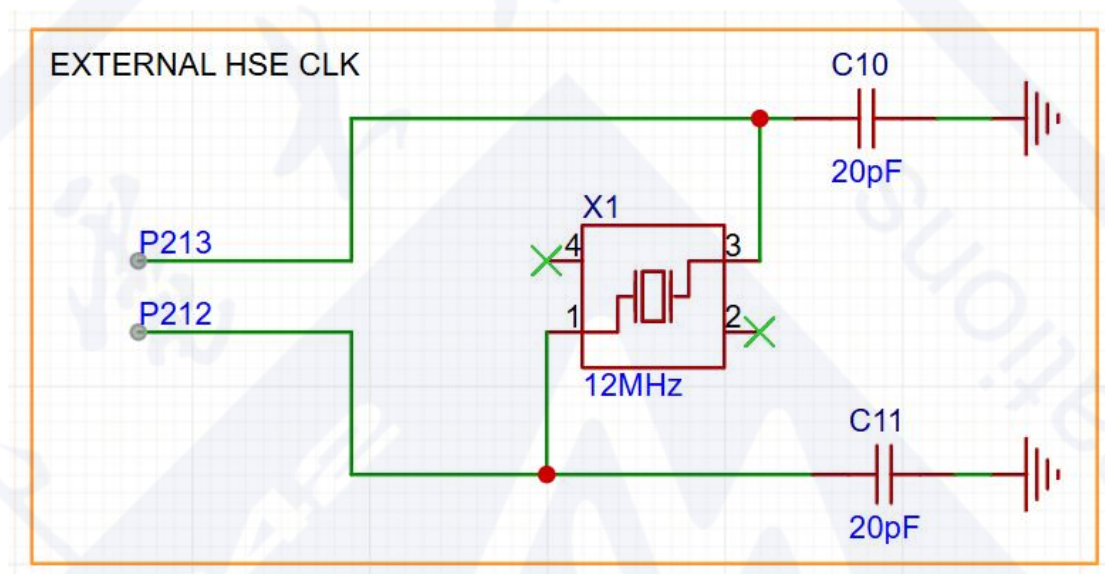
取消



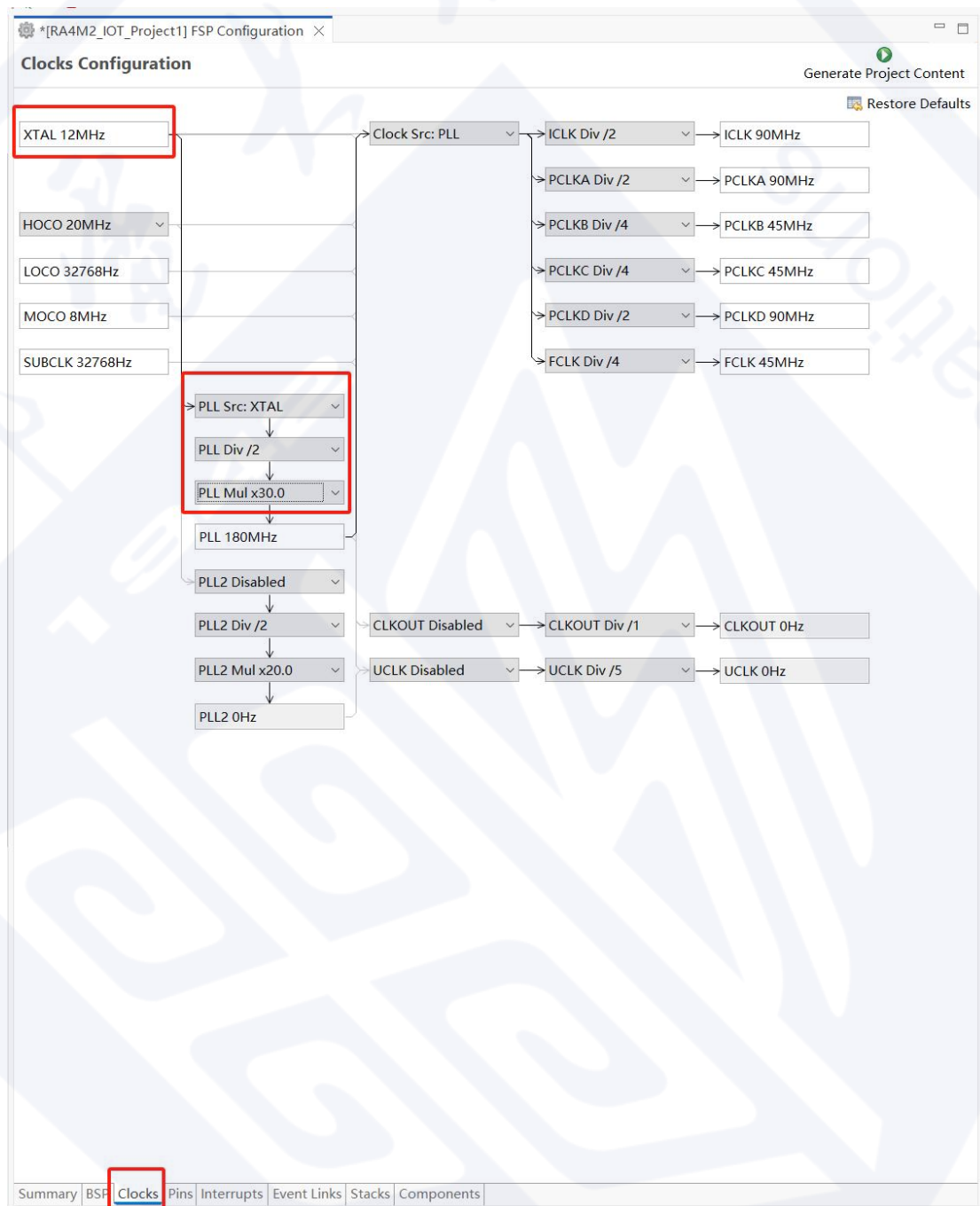


时钟设置

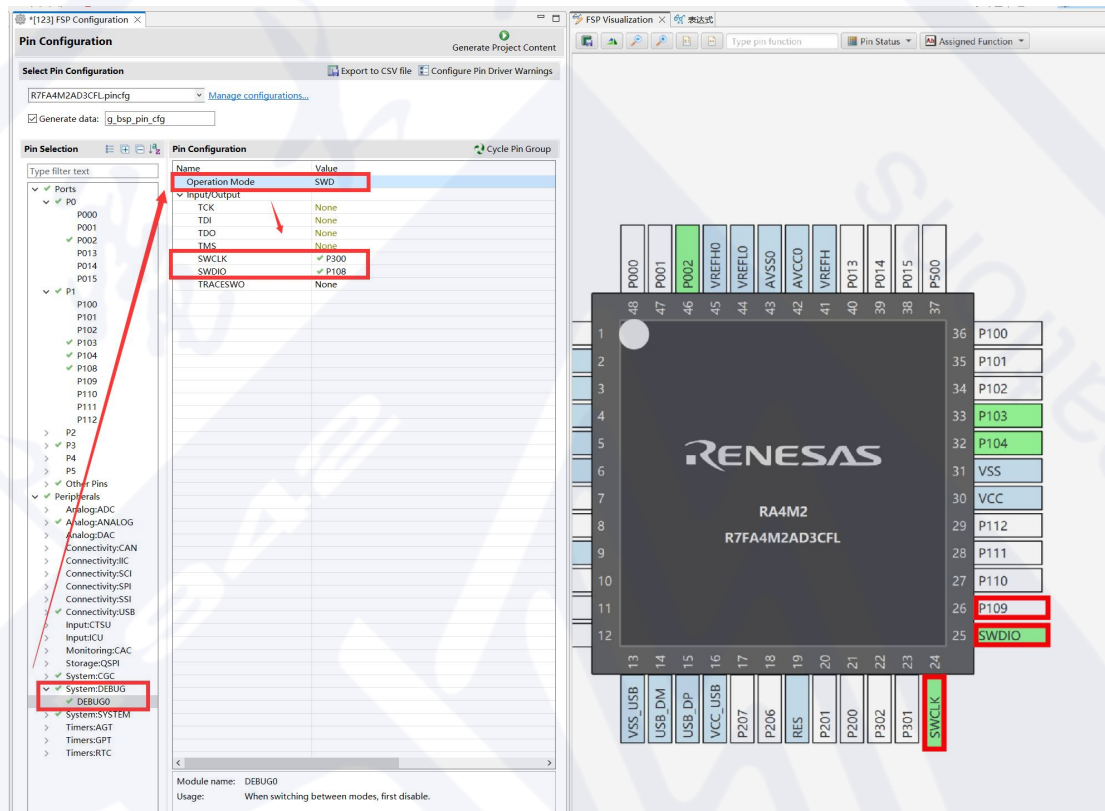
开发板上的外部高速晶振为 12M.



需要修改 XTAL 为 12M。

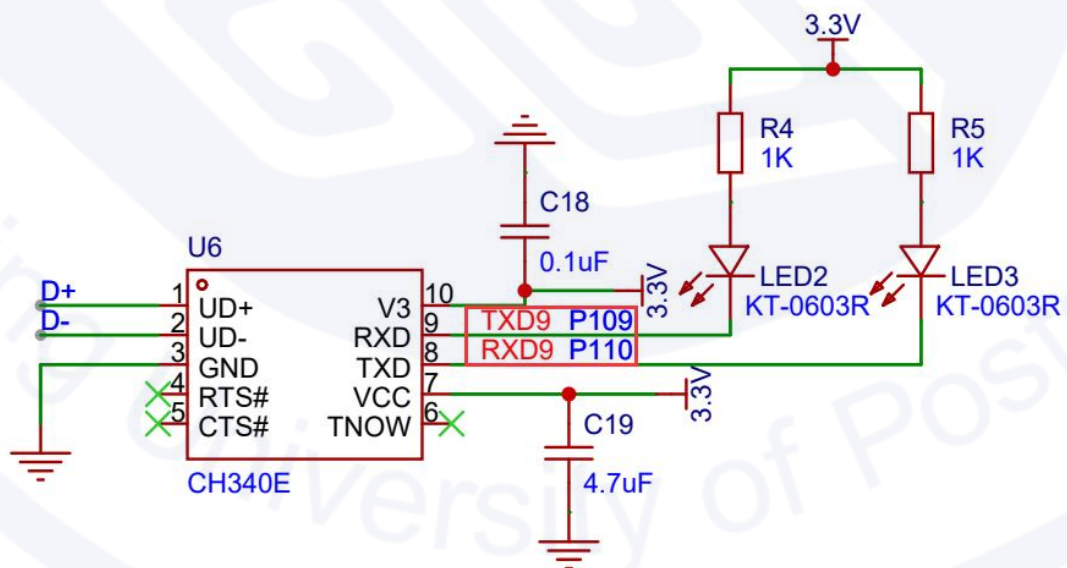


SWD 调试口设置

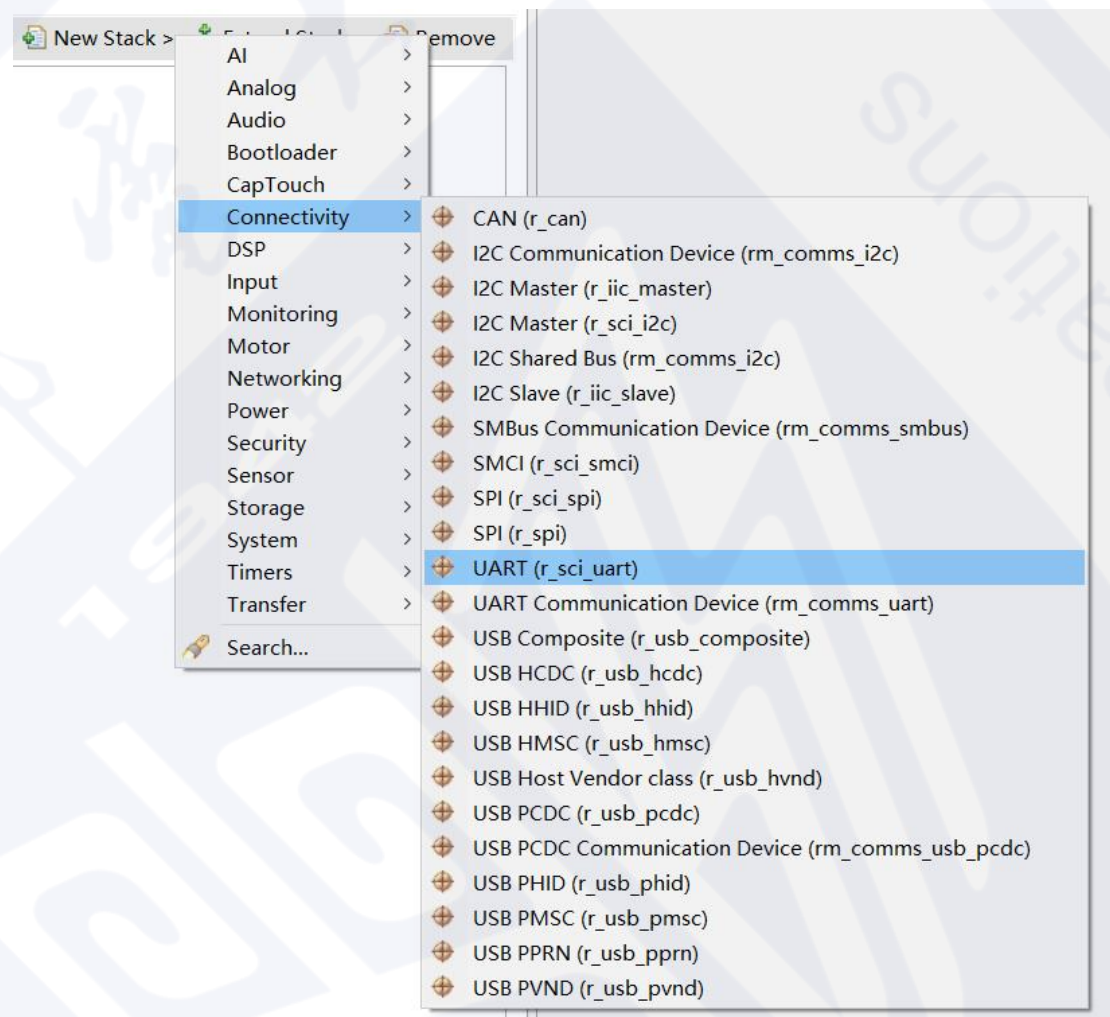


UART 配置

串口CH340E



点击 Stacks->New Stack->Connectivity -> UART(r_sci_uart)。



UART 属性配置

属性 ×	问题	Smart Browser	控制台	调试
g_uart9 UART (r_sci_uart)				
Settings	属性		值	
API Info	▼ Common			
	Parameter Checking		Default (BSP)	
	FIFO Support		Disable	
	DTC Support		Disable	
	Flow Control Support		Disable	
	RS-485 Support		Disable	
	IrDA Support		Disabled	
	▼ Module g_uart9 UART (r_sci_uart)			
	▼ General			
	Name		g_uart9	
	Channel		9	
	Data Bits		8bits	
	Parity		None	
	Stop Bits		1bit	
	▼ Baud			
	Baud Rate		115200	
	Baud Rate Modulation		Enabled	
	Max Error (%)		5	
	► Flow Control			
	► Extra			
	▼ Interrupts			
	Callback		user_uart_callback	
	Receive Interrupt Priority		Priority 2	
	Transmit Data Empty Interrupt Priority		Priority 2	
	Transmit End Interrupt Priority		Priority 2	
	Error Interrupt Priority		Priority 2	
	▼ Pins			
	TXD9		P109	
	RXD9		P110	
	CTS9		<unavailable>	

名称
通道
数据位
无校验位
停止位

波特率
启用波特率调制

回调函数名称

串口发送端口
串口接收端口

设置 e2studio 堆栈

printf 函数通常需要设置堆栈大小。这是因为 printf 函数在运行时需要使用栈空间来存储临时变量和函数调用信息。如果堆栈大小不足，可能会导致程序崩溃或不可预期的行为。

printf 函数使用了可变参数列表，它会在调用时使用栈来存储参数，在函数调用结束时再清除参数，这需要足够的栈空间。另外 printf 也会使用一些临时变量，如果栈空间不足，会导致程序崩溃。

因此，为了避免这类问题，应该根据程序的需求来合理设置堆栈大小。

项目资源管理器 | [RA2M4_print] FSP Configuration

Board Support Package Configuration

Generate Project Content | Restore Defaults

Device Selection

FSP version: 4.2.0

Board: Custom User Board (Any Device)

Device: R7FA4M2AD3CFP

Core: CM33

RTOS: No RTOS

Board Details

Summary **BSP** Clocks Pins Interrupts Event Links Stacks Components

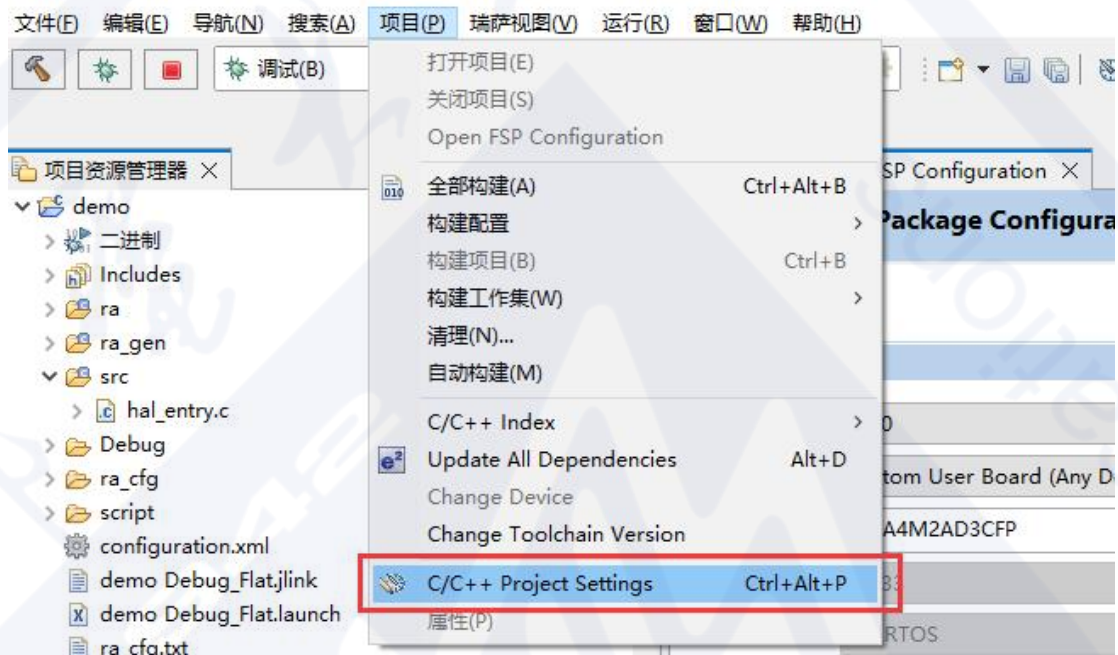
Custom User Board (Any Device)

Settings | 属性 | 问题 | Smart Browser | 控制台

属性	值
√ R7FA4M2AD3CFP	
part_number	R7FA4M2AD3CFP
rom_size_bytes	524288
ram_size_bytes	131072
data_flash_size_bytes	8192
package_style	LQFP
package_pins	100
√ RA4M2	
series	4
√ RA4M2 Family	
> Security	
> OFS0 register settings	
> OFS1 register settings	
> Block Protection Settings (BPS)	
> Permanent Block Protection Settings (PBPS)	
> Clocks	
Startup C-Cache Line Size	32 Bytes
Main Oscillator Wait Time	8163 cycles
√ RA Common	
Main stack size (bytes)	0x400
Heap size (bytes)	0x2000
MCU Vcc (mV)	3300
Parameter checking	Disabled
Assert Failures	Return FSP_ERR_ASSERTION
Error Log	No Error Log
Clock Registers not Reset Values during Startup	Disabled
Main Oscillator Populated	Populated
PFS Protect	Enabled
C Runtime Initialization	Enabled
Early BSP Initialization	Disabled
Main Oscillator Clock Source	Crystal or Resonator
Subclock Populated	Populated
Subclock Drive (Drive capacitance availability varies by MCU)	Standard/Normal mode
Subclock Stabilization Time (ms)	1000

3.设置堆栈大小

e2studio 的重定向 printf 设置



在嵌入式系统的开发中，尤其是在使用 GNU 编译器集合（GCC）时，`--specs` 参数用于指定链接时使用的系统规格（specs）文件。这些规格文件控制了编译器和链接器的行为，尤其是关于系统库和启动代码的链接。`--specs=rdimon.specs` 和 `--specs=nosys.specs` 是两种常见的规格文件，它们用于不同的场景。

`--specs=rdimon.specs`

用途：这个选项用于链接“Redlib”库，这是为裸机（bare-metal）和半主机（semihosting）环境设计的 C 库的一个变体。半主机环境是一种特殊的运行模式，允许嵌入式程序通过宿主机（如开发 PC）的调试器进行输入输出操作。

应用场景：当你需要在没有完整操作系统的环境中运行程序，但同时需要使用调试器来处理输入输出（例如打印到宿主机的终端），这个选项非常有用。

特点：它提供了一些基本的系统调用，通过调试接口与宿主机通信。

`--specs=nosys.specs`

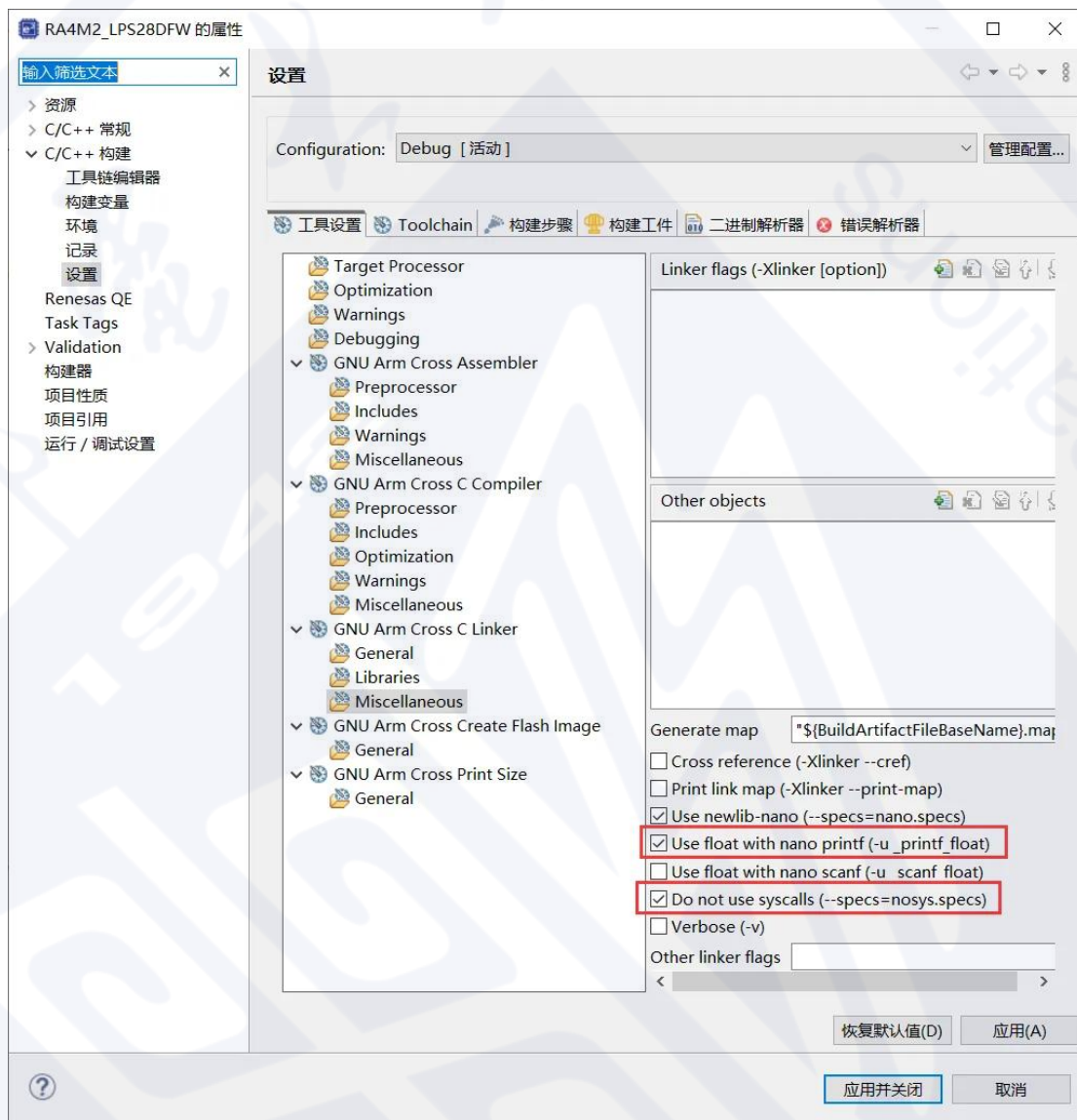
用途：这个选项链接了一个非常基本的系统库，这个库不提供任何系统服务的实现。

应用场景：适用于完全的裸机程序，其中程序不执行任何操作系统调用，比如不进行文件操作或者系统级输入输出。

特点：这是一个更“裸”的环境，没有任何操作系统支持。使用这个规格文件，程序不期望有操作系统层面的任何支持。

如果你的程序需要与宿主机进行交互（如在开发期间的调试），并且通过调试器进行基本的输入输出操作，则使用 `--specs=rdimon.specs`。

如果你的程序是完全独立的，不需要任何形式的操作系统服务，包括不进行任何系统级的输入输出，则使用 `--specs=nosys.specs`。



R_SCI_UART_Open() 函数原型

◆ R_SCI_UART_Open()

```
fsp_err_t R_SCI_UART_Open ( uart_ctrl_t *const p_api_ctrl,  
                           uart_cfg_t const *const p_cfg  
                           )
```

Configures the UART driver based on the input configurations. If reception is enabled at compile time, reception is enabled at the end of this function. Implements `uart_api_t::open`

Return values

FSP_SUCCESS	Channel opened successfully.
FSP_ERR_ASSERTION	Pointer to UART control block or configuration structure is NULL.
FSP_ERR_IP_CHANNEL_NOT_PRESENT	The requested channel does not exist on this MCU.
FSP_ERR_INVALID_ARGUMENT	Flow control is enabled but flow control pin is not defined or selected channel does not support "Hardware CTS and Hardware RTS" flow control. (or) restricted channel is selected.
FSP_ERR_ALREADY_OPEN	Control block has already been opened or channel is being used by another instance. Call <code>close()</code> then <code>open()</code> to reconfigure.
FSP_ERR_INVALID_CHANNEL	IrDA is requested for a channel that does not support IrDA.

Returns

See [Common Error Codes](#) or functions called by this function for other possible return codes. This function calls:

- `transfer_api_t::open`

故可以用 `R_SCI_UART_Open()`函数进行配置，开启和初始化 UART。

```
/* Open the transfer instance with initial configuration. */  
err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);  
assert(FSP_SUCCESS == err);
```

```
[RA4M2_MINI_Project11_1] FSP Configuration  *hal_entry.c x startup.c main.c
2
3 FSP_CPP_HEADER
4 void R_BSP_WarmStart(bsp_warm_start_event_t event);
5 FSP_CPP_FOOTER
6 /* unused parameter 'fd' [-Wunused-parameter] | no previous declaration for '_write' [-W
7
8
9
10
11
12
13
14
15 /* Open the transfer instance with initial configuration. */
16 err = R_SCI_UART_Open(&g_uart9_ctrl1, &g_uart9_cfg);
17 assert(FSP_SUCCESS == err);
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
```

回调函数 user_uart_callback ()

当数据发送的时候，可以查看 UART_EVENT_TX_COMPLETE 来判断是否发送完毕。

◆ write

```
fsp_err_t(*uart_api_t::write)(uart_ctrl_t *const p_ctrl, uint8_t const *const p_src, uint32_t const bytes)
```

Write to UART device. The write buffer is used until write is complete. Do not overwrite write buffer contents until the write is finished. When the write is complete (all bytes are fully transmitted on the wire), the callback called with event **UART_EVENT_TX_COMPLETE**. The maximum transfer size is reported by **infoGet[]**.

Implemented as

- R_SCI_UART_Write[]
- R_SCI_B_UART_Write[]

Parameters

[in] p_ctrl Pointer to the UART control block.

[in] p_src Source address to write data to.

[in] bytes Write data length.

Enumeration Type Documentation

◆ uart_event_t

enum `uart_event_t`

UART Event codes

Enumerator	
UART_EVENT_RX_COMPLETE	Receive complete event.
UART_EVENT_TX_COMPLETE	Transmit complete event.
UART_EVENT_RX_CHAR	Character received.
UART_EVENT_ERR_PARITY	Parity error event.
UART_EVENT_ERR_FRAMING	Mode fault error event.
UART_EVENT_ERR_OVERFLOW	FIFO Overflow error event.
UART_EVENT_BREAK_DETECT	Break detect error event.
UART_EVENT_TX_DATA_EMPTY	Last byte is transmitting, ready for more data.

可以检查检查 "p_args" 结构体中的 "event" 字段的值是否等于 "UART_EVENT_TX_COMPLETE"。如果条件为真，那么 if 语句后面的代码块将会执行。

```
fsp_err_t err = FSP_SUCCESS;
volatile bool uart_send_complete_flag = false;
void user_uart_callback (uart_callback_args_t * p_args)
{
    if(p_args->event == UART_EVENT_TX_COMPLETE)
    {
        uart_send_complete_flag = true;
    }
}
```

```
[RA4M2_MINI_Project11_1] FSP Configuration  *hal_entry.c  startup.c  main.c
1  #include "hal_data.h"
2
3  FSP_CPP_HEADER
4  void R_BSP_WarmStart(bsp_warm_start_event_t event);
5  FSP_CPP_FOOTER
6
7  fsp_err_t err = FSP_SUCCESS;
8  volatile bool uart_send_complete_flag = false;
9  void user_uart_callback (uart_callback_args_t * p_args)
10 {
11     if(p_args->event == UART_EVENT_TX_COMPLETE)
12     {
13         uart_send_complete_flag = true;
14     }
15 }
16
17 unused parameter 'fd' [-Wunused-parameter] | no previous declaration for '_write' [-W
18
19 * main() is generated by the RA Configuration editor and is used to generate threads if
20 void hal_entry(void)
21 {
22     /* TODO: add your own code here */
23
24     /* Open the transfer instance with initial configuration. */
25     err = R_SCI_UART_Open(&g_uart9_ctrl1, &g_uart9_cfg);
26     assert(FSP_SUCCESS == err);
27
28     #if BSP_TZ_SECURE_BUILD
29     /* Enter non-secure code */
30     R_BSP_NonSecureEnter();
31     #endif
32
33     * This function is called at various points during the startup process. This implement
34     void R_BSP_WarmStart(bsp_warm_start_event_t event)
35     {
36         if (BSP_WARM_START_RESET == event)
37         {
38             #if BSP_FEATURE_FLASH_LP_VERSION != 0
39
```

R_SCI_UART_Write() 函数原型

◆ R_SCI_UART_Write()

```
fsp_err_t R_SCI_UART_Write ( uart_ctrl_t *const    p_api_ctrl,
                             uint8_t const *const p_src,
                             uint32_t const    bytes
                             )
```

Transmits user specified number of bytes from the source buffer pointer. Implements `uart_api_t::write`

Return values

FSP_SUCCESS	Data transmission finished successfully.
FSP_ERR_ASSERTION	Pointer to UART control block is NULL. Number of transfers outside the max or min boundary when transfer instance used
FSP_ERR_INVALID_ARGUMENT	Source address or data size is not valid for 9-bit mode.
FSP_ERR_NOT_OPEN	The control block has not been opened
FSP_ERR_IN_USE	A UART transmission is in progress
FSP_ERR_UNSUPPORTED	SCI_UART_CFG_TX_ENABLE is set to 0

Returns

See [Common Error Codes](#) or functions called by this function for other possible return codes. This function calls:

- `transfer_api_t::reset`

Note

If 9-bit data length is specified at `R_SCI_UART_Open` call, `p_src` must be aligned on a 16-bit boundary.

故可以用 `R_UARTA_Write()` 函数进行串口数据输出。

```

unsigned char buff[]="RA E2STUDIO";

uint8_t buff_len = strlen(buff);

err = R_SCI_UART_Write(&g_uart9_ctrl, buff, buff_len);

if(FSP_SUCCESS != err) __BKPT();

    while(uart_send_complete_flag == false){}

uart_send_complete_flag = false;

```

```

1  #include "hal_data.h"
2
3  FSP_CPP_HEADER
4  void R_BSP_WarmStart(bsp_warm_start_event_t event);
5  FSP_CPP_FOOTER
6
7  fsp_err_t err = FSP_SUCCESS;
8  volatile bool uart_send_complete_flag = false;
9  void user_uart_callback (uart_callback_args_t * p_args)
10 {
11     if(p_args->event == UART_EVENT_TX_COMPLETE)
12     {
13         uart_send_complete_flag = true;
14     }
15 }
16
17 // unused parameter 'fd' [-Wunused-parameter] | no previous declaration for '_write' [-W
18
19 // * main() is generated by the RA Configuration editor and is used to generate threads if
20 void hal_entry(void)
21 {
22     /* TODO: add your own code here */
23
24     /* Open the transfer instance with initial configuration. */
25     err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);
26     assert(FSP_SUCCESS == err);
27
28     unsigned char buff[]="RA E2STUDIO";
29     uint8_t buff_len = strlen(buff);
30     err = R_SCI_UART_Write(&g_uart9_ctrl, buff, buff_len);
31     if(FSP_SUCCESS != err) __BKPT();
32     while(uart_send_complete_flag == false){}
33     uart_send_complete_flag = false;
34
35     #if BSP_TZ_SECURE_BUILD
36     /* Enter non-secure code */
37     R_BSP_NonSecureEnter();
38     #endif
39 }
40
41
42

```

[22:40:56.313] 收 ← ◆ RA E2STUDIO

sprintf() 函数

printf 指的是字符串格式化命令, 函数声明为 `int sprintf(char *string, char *format [argument,...]);`, 主要功能是把格式化的数据写入某个字符串中, 即发送格式化输出到 `string` 所指向的字符串。printf 是个变参函数。使用 printf 对于写入 buffer 的字符数是没有限制的, 这就存在了 buffer 溢出的可能性。解决这个问题, 可以考虑使用 `snprintf` 函数, 该函数可对写入字符数做出限制。

sprintf 函数声明

`int sprintf(char *string, char *format [,argument,...]);`

参数列表

- **string**-- 这是指向一个字符数组的指针，该数组存储了 C 字符串。
- **format**-- 这是字符串，包含了要被写入到字符串 **str** 的文本。它可以包含嵌入的 **format** 标签，**format** 标签可被随后的附加参数中指定的值替换，并按需求进行格式化。**format** 标签属性是 `%[flags][width][.precision][length]specifier`
- **[argument]...**: 根据不同的 **format** 字符串，函数可能需要一系列的附加参数，每个参数包含了一个要被插入的值，替换了 **format** 参数中指定的每个 **%** 标签。参数的个数应与 **%** 标签的个数相同。

功能

- 把格式化的数据写入某个字符串缓冲区。

```
unsigned char send_buff[100];
sprintf(send_buff, "\nHello World!\n");
uint8_t len = strlen(send_buff);
err = R_SCI_UART_Write(&g_uart9_ctrl, send_buff, len);
if(FSP_SUCCESS != err) __BKPT();
while(uart_send_complete_flag == false){}
uart_send_complete_flag = false;
memset(send_buff, '\0', sizeof(100));
```

```
[RA4M2_MINI_Project11_1] FSP Configuration  *hal_entry.c  startup.c  main.c
1      #include "hal_data.h"
2
3      FSP_CPP_HEADER
4      void R_BSP_WarmStart(bsp_warm_start_event_t event);
5      FSP_CPP_FOOTER
6
7      fsp_err_t err = FSP_SUCCESS;
8      volatile bool uart_send_complete_flag = false;
9      void user_uart_callback (uart_callback_args_t * p_args)
10     {
11         if(p_args->event == UART_EVENT_TX_COMPLETE)
12         {
13             uart_send_complete_flag = true;
14         }
15     }
16
17     unused parameter 'fd' [-Wunused-parameter] | no previous declaration for '_write' [-Werror=unused-parameter]
18
19     * * main() is generated by the RA Configuration editor and is used to generate threads in the application.
20     void hal_entry(void)
21     {
22         /* TODO: add your own code here */
23
24         /* Open the transfer instance with initial configuration. */
25         err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);
26         assert(FSP_SUCCESS == err);
27
28         unsigned char buff[]="RA E2STUDIO";
29         uint8_t buff_len = strlen(buff);
30         err = R_SCI_UART_Write(&g_uart9_ctrl, buff, buff_len);
31         if(FSP_SUCCESS != err) __BKPT();
32         while(uart_send_complete_flag == false){}
33         uart_send_complete_flag = false;
34
35         unsigned char send_buff[100];
36         sprintf(send_buff, "\nHello World!\n");
37         uint8_t len = strlen(send_buff);
38         err = R_SCI_UART_Write(&g_uart9_ctrl, send_buff, len);
39         if(FSP_SUCCESS != err) __BKPT();
40         while(uart_send_complete_flag == false){}
41         uart_send_complete_flag = false;
42         memset(send_buff, '\0', sizeof(100));
43
44     }
45
46     #if BSP_TZ_SECURE_BUILD
47         /* Enter non-secure code */
48         R_BSP_NonSecureEnter();
49     #endif
50
51     * * This function is called at various points during the startup process. This implements the warm start event.
52     void R_BSP_WarmStart(bsp_warm_start_event_t event)
53     {
54         if (BSP_WARM_START_RESET == event)
55         {
56             #if BSP_FEATURE_FLASH_LP_VERSION != 0
57                 /* Enable reading from data flash. */
58             #endif
59         }
60     }
```

printf 输出重定向到串口

打印最常用的方法是 printf，所以要解决的问题是将 printf 的输出重定向到串口，然后通过串口将数据发送出去。

注意一定要加上头文件#include <stdio.h>

```
#ifndef __GNUC__ //串口重定向
#define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
#else
#define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
#endif
```

PUTCHAR_PROTOTYPE

```
{  
    err = R_SCI_UART_Write(&g_uart9_ctrl, (uint8_t *)&ch, 1);  
    if(FSP_SUCCESS != err) __BKPT();  
    while(uart_send_complete_flag == false){}  
    uart_send_complete_flag = false;  
    return ch;  
}
```

int _write(int fd, char *pBuffer, int size)

```
{  
    for(int i=0; i<size; i++)  
    {  
        __io_putchar(*pBuffer++);  
    }  
    return size;  
}
```

```
[RA4M2_MINI_Project11_1] FSP Configuration  *hal_entry.c x startup.c main.c
1      #include "hal_data.h"
2      #include <stdio.h>
3      FSP_CPP_HEADER
4      void R_BSP_WarmStart(bsp_warm_start_event_t event);
5      FSP_CPP_FOOTER
6
7      fsp_err_t err = FSP_SUCCESS;
8      volatile bool uart_send_complete_flag = false;
9      void user_uart_callback (uart_callback_args_t * p_args)
10     {
11         if(p_args->event == UART_EVENT_TX_COMPLETE)
12         {
13             uart_send_complete_flag = true;
14         }
15     }
16
17     #ifdef __GNUC__ //串口重定向
18         #define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
19     #else
20         #define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
21     #endif
22
23
24     PUTCHAR_PROTOTYPE
25     {
26         err = R_SCI_UART_Write(&g_uart9_ctrl, (uint8_t *)&ch, 1);
27         if(FSP_SUCCESS != err) __BKPT();
28         while(uart_send_complete_flag == false){}
29         uart_send_complete_flag = false;
30         return ch;
31     }
32
33     int _write(int fd, char *pBuffer, int size)
34     {
35         for(int i=0; i<size; i++)
36         {
37             __io_putchar(*pBuffer++);
38         }
39         return size;
40     }
41
42     /* unused parameter 'fd' [-Wunused-parameter] | no previous declaration for '_write' [-Werror=implicit-function-declaration] */
43
44     /* * main() is generated by the RA Configuration editor and is used to generate threads in the application. */
45     void hal_entry(void)
46     {
47         /* TODO: add your own code here */
48
49         /* Open the transfer instance with initial configuration. */
50         err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);
51         assert(FSP_SUCCESS == err);
52
53         unsigned char buff[]="RA E2STUDIO";
54         uint8_t buff_len = strlen(buff);
55         err = R_SCI_UART_Write(&g_uart9_ctrl, buff, buff_len);
56         if(FSP_SUCCESS != err) __BKPT();
57         while(uart_send_complete_flag == false){}
58         uart_send_complete_flag = false;
59     }
60
```

printf 输出

```
int int_i=0;
float float_i=66.20f;
char char_i[]="hello e2studio";
while(1)
{
    printf("int_i=%d\n",int_i);
    printf("float_i=%.2f\n",float_i);
    printf("char_i='%s'\n",char_i);
}
```

```

int_i++;

if(int_i>100)

    int_i=0;

R_BSP_SoftwareDelay(1000, BSP_DELAY_UNITS_MILLISECONDS);

}

```

```

40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105

```

演示

[22:45:34.601]收←◆RA E2STUDIO
Hello World!.

int_i=0
float_i=66.20
char_i='hello e2studio'

[22:45:35.612]收←◆int_i=1
float_i=66.20
char_i='hello e2studio'

[22:45:36.620]收←◆int_i=2
float_i=66.20
char_i='hello e2studio'

[22:45:37.628]收←◆int_i=3
float_i=66.20
char_i='hello e2studio'

[22:45:38.632]收←◆int_i=4
float_i=66.20
char_i='hello e2studio'

[22:45:39.639]收←◆int_i=5
float_i=66.20
char_i='hello e2studio'

主程序

```
#include "hal_data.h"
#include <stdio.h>

FSP_CPP_HEADER

void R_BSP_WarmStart(bsp_warm_start_event_t event);

FSP_CPP_FOOTER
```

```

fsp_err_t err = FSP_SUCCESS;
volatile bool uart_send_complete_flag = false;
void user_uart_callback (uart_callback_args_t * p_args)
{
    if(p_args->event == UART_EVENT_TX_COMPLETE)
    {
        uart_send_complete_flag = true;
    }
}

#ifdef __GNUC__
//串口重定向
#define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
#else
#define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
#endif

PUTCHAR_PROTOTYPE
{
    err = R_SCI_UART_Write(&g_uart9_ctrl, (uint8_t *)&ch, 1);
    if(FSP_SUCCESS != err) __BKPT();
    while(uart_send_complete_flag == false){}
    uart_send_complete_flag = false;
    return ch;
}

int _write(int fd, char *pBuffer, int size)
{
    for(int i=0; i<size; i++)
    {
        __io_putchar(*pBuffer++);
    }
    return size;
}

/*****
*****//**

* main() is generated by the RA Configuration editor and is used to generate threads if an RTOS
is used. This function
* is called by main() when no RTOS is used.

*****
*****/

```

```

void hal_entry(void)
{
    /* TODO: add your own code here */

    /* Open the transfer instance with initial configuration. */
    err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);
    assert(FSP_SUCCESS == err);

    unsigned char buff[]="RA E2STUDIO";
    uint8_t buff_len = strlen(buff);
    err = R_SCI_UART_Write(&g_uart9_ctrl, buff, buff_len);
    if(FSP_SUCCESS != err) __BKPT();
    while(uart_send_complete_flag == false){}
    uart_send_complete_flag = false;

    unsigned char send_buff[100];
    sprintf(send_buff, "\nHello World!\n");
    uint8_t len = strlen(send_buff);
    err = R_SCI_UART_Write(&g_uart9_ctrl, send_buff, len);
    if(FSP_SUCCESS != err) __BKPT();
    while(uart_send_complete_flag == false){}
    uart_send_complete_flag = false;
    memset(send_buff, '\0', sizeof(100));

    int int_i=0;
    float float_i=66.20f;
    char char_i[]="hello e2studio";
    while(1)
    {
        printf("int_i=%d\n",int_i);
        printf("float_i=%.2f\n",float_i);
        printf("char_i='%s'\n",char_i);
        int_i++;
        if(int_i>100)
            int_i=0;
        R_BSP_SoftwareDelay(1000, BSP_DELAY_UNITS_MILLISECONDS);
    }

#ifdef BSP_TZ_SECURE_BUILD
    /* Enter non-secure code */
    R_BSP_NonSecureEnter();
#endif
}

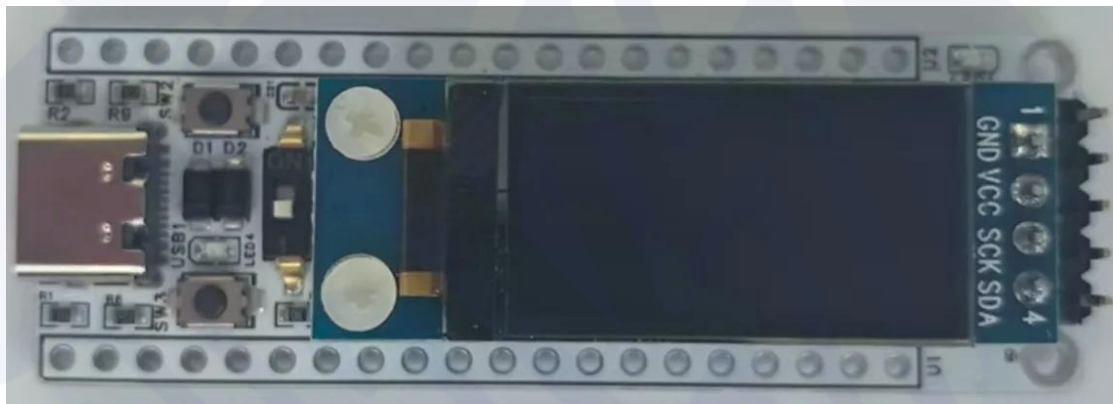
```

```
}
```

project13: 串口接收数据

概述

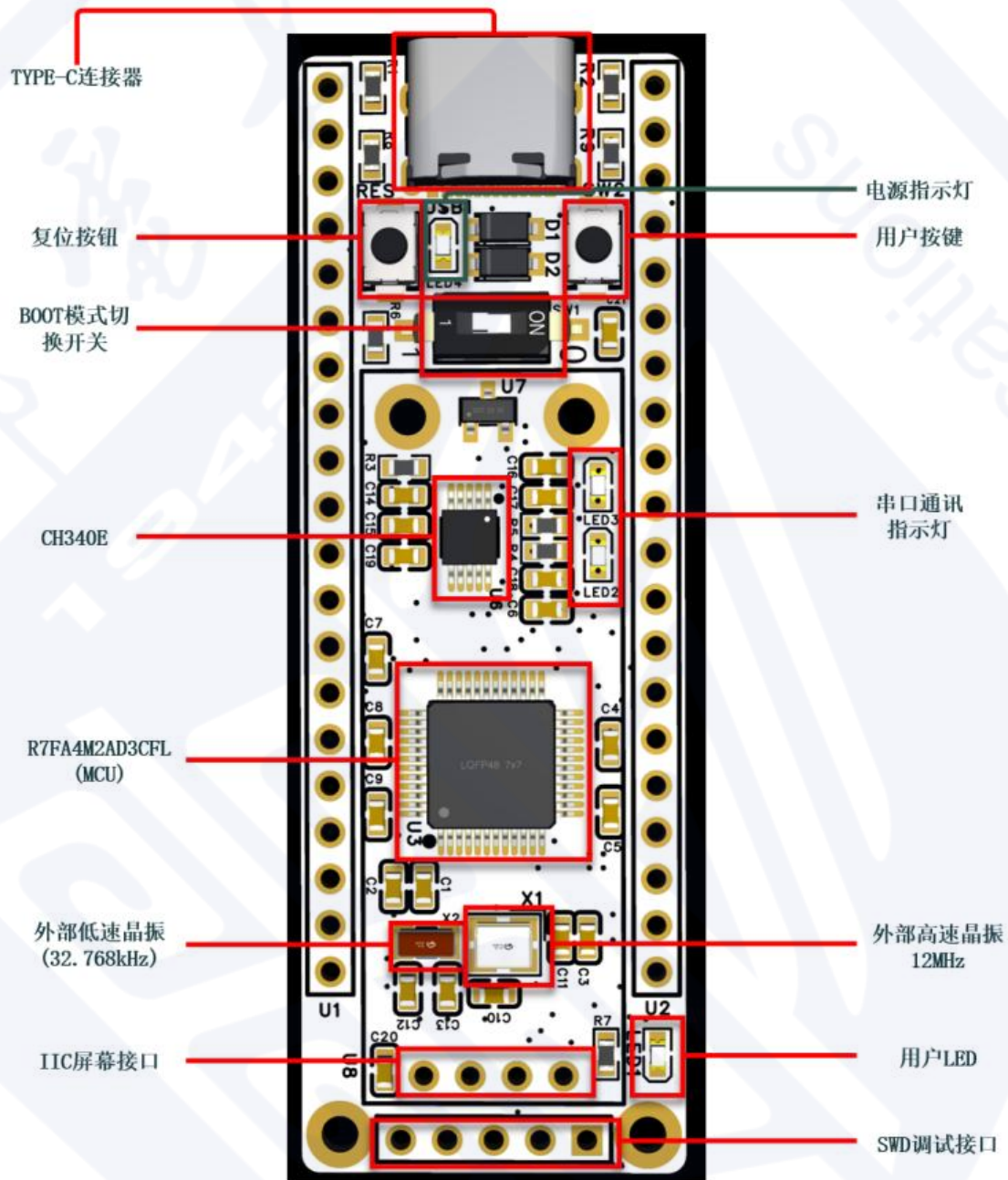
串口通信（UART）是一种常用的异步数据传输方式，在嵌入式系统中广泛应用于设备之间的数据交互。本文主要探讨如何通过串口接收数据，包括初始化配置、数据接收流程、常见问题处理以及实现串口数据接收的实际代码示例。本博客旨在帮助开发者快速掌握串口接收的基础知识与技巧，从而在实际项目中灵活应用。



硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL



参考程序

https://github.com/CoreMaker-lab/RA4M2_MINI

https://gitee.com/CoreMaker/RA4M2_MINI

R_SCI_UART_Read() 函数

R_SCI_UART_Read() 的功能是从指定的 UART 控制块中读取用户指定数量的字节到目标缓冲区中。

◆ R_SCI_UART_Read()

```
fsp_err_t R_SCI_UART_Read ( uart_ctrl_t *const p_api_ctrl,  
                           uint8_t *const p_dest,  
                           uint32_t const bytes  
                           )
```

Receives user specified number of bytes into destination buffer pointer. Implements [uart_api_t::read](#)

Return values

FSP_SUCCESS	Data reception successfully ends.
FSP_ERR_ASSERTION	Pointer to UART control block is NULL. Number of transfers outside the max or min boundary when transfer instance used
FSP_ERR_INVALID_ARGUMENT	Destination address or data size is not valid for 9-bit mode.
FSP_ERR_NOT_OPEN	The control block has not been opened
FSP_ERR_IN_USE	A previous read operation is still in progress.
FSP_ERR_UNSUPPORTED	SCI_UART_CFG_RX_ENABLE is set to 0

Returns

See [Common Error Codes](#) or functions called by this function for other possible return codes. This function calls:

- [transfer_api_t::reset](#)

Note

If 9-bit data length is specified at R_SCI_UART_Open call, p_dest must be aligned 16-bit boundary.

接收用户指定数量的字节并存储到目标缓冲区中，实现 UART 数据的读取操作。故可以用 R_UARTA_Read()函数进行获取固定长度的数据。

```
uint8_t TRANSFER_LENGTH=10;  
uint8_t g_dest[TRANSFER_LENGTH];  
R_SCI_UART_Read(&g_uart9_ctrl, g_dest, TRANSFER_LENGTH);
```

```

41
42
44  /* * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. This function[
47  void hal_entry(void)
48  {
49      /* TODO: add your own code here */
50
51      /* Open the transfer instance with initial configuration. */
52      00000148 err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);
53      0000015a assert(FSP_SUCCESS == err);
54
55      00000168 unsigned char buff[]="RA E2STUDIO";
56      /* pointer targets in passing argument 1 of 'strlen' differ in signedness [-Wpointer-sign] | conversion from 'size_t'
57      00000172 uint8_t buff_len = strlen(buff);
58      0000017e err = R_SCI_UART_Write(&g_uart9_ctrl, buff, buff_len);
59      00000184 if(FSP_SUCCESS != err) __BKPT();
60      0000018a while(uart_send_complete_flag == false){}
61      uart_send_complete_flag = false;
62
63      00000192 unsigned char send_buff[100];
64      /* pointer targets in passing argument 1 of 'sprintf' differ in signedness [-Wpointer-sign]
65      000001a2 sprintf(send_buff, "\nHello World!\n");
66      000001aa /* pointer targets in passing argument 1 of 'strlen' differ in signedness [-Wpointer-sign] | conversion from 'size_t'
67      000001ae uint8_t len = strlen(send_buff);
68      000001b4 err = R_SCI_UART_Write(&g_uart9_ctrl, send_buff, len);
69      000001c0 if(FSP_SUCCESS != err) __BKPT();
70      000001c8 while(uart_send_complete_flag == false){}
71      000001d0 uart_send_complete_flag = false;
72      000001d6 memset(send_buff, '\0', sizeof(100));
73
74      int int_i=0;
75      float float_i=66.20f;
76      char char_i[]="hello e2studio";
77
78      000001dc uint8_t TRANSFER_LENGTH=10;
79      000001e4 uint8_t g_dest[TRANSFER_LENGTH];
80      000001f6 R_SCI_UART_Read(&g_uart9_ctrl, g_dest, TRANSFER_LENGTH);
81
82      000001fc while(1)
83      {
84          printf("int_i=%d\n",int_i);
85          printf("float_i=%.2f\n",float_i);
86          printf("char_i='%s'\n",char_i);
87          int_i++;
88          if(int_i>100)
89              int_i=0;
90          R_BSP_SoftwareDelay(1000, BSP_DELAY_UNITS_MILLISECONDS);
91      }
92
93      #if BSP_TZ_SECURE_BUILD
94      /* Enter non-secure code */
95      R_BSP_NonSecureEnter();
96      #endif
97
98      /* * This function is called at various points during the startup process. This implementation uses the event that is[
99      void R_BSP_WarmStart(bsp_warm_start_event_t event)
100      {

```

UART_EVENT_RX_COMPLETE

UART_EVENT_RX_COMPLETE: 接收完成事件，当一帧数据接收完整后触发。

Enumeration Type Documentation

◆ uart_event_t

enum uart_event_t	
UART Event codes	
Enumerator	
UART_EVENT_RX_COMPLETE	Receive complete event.
UART_EVENT_TX_COMPLETE	Transmit complete event.
UART_EVENT_RX_CHAR	Character received.
UART_EVENT_ERR_PARITY	Parity error event.
UART_EVENT_ERR_FRAMING	Mode fault error event.
UART_EVENT_ERR_OVERFLOW	FIFO Overflow error event.
UART_EVENT_BREAK_DETECT	Break detect error event.
UART_EVENT_TX_DATA_EMPTY	Last byte is transmitting, ready for more data.

UART 回调函数，用于处理 UART 事件，包括发送完成事件 (UART_EVENT_TX_COMPLETE) 和接收完成事件 (UART_EVENT_RX_COMPLETE)。

```
fsp_err_t err = FSP_SUCCESS;
volatile bool uart_send_complete_flag = false;
volatile bool g_receive_complete = false;
void user_uart_callback (uart_callback_args_t * p_args)
{
    if(p_args->event == UART_EVENT_TX_COMPLETE)
    {
        uart_send_complete_flag = true;
    }
    else if(p_args->event == UART_EVENT_RX_COMPLETE)
    {
        g_receive_complete = true;
    }
}
```

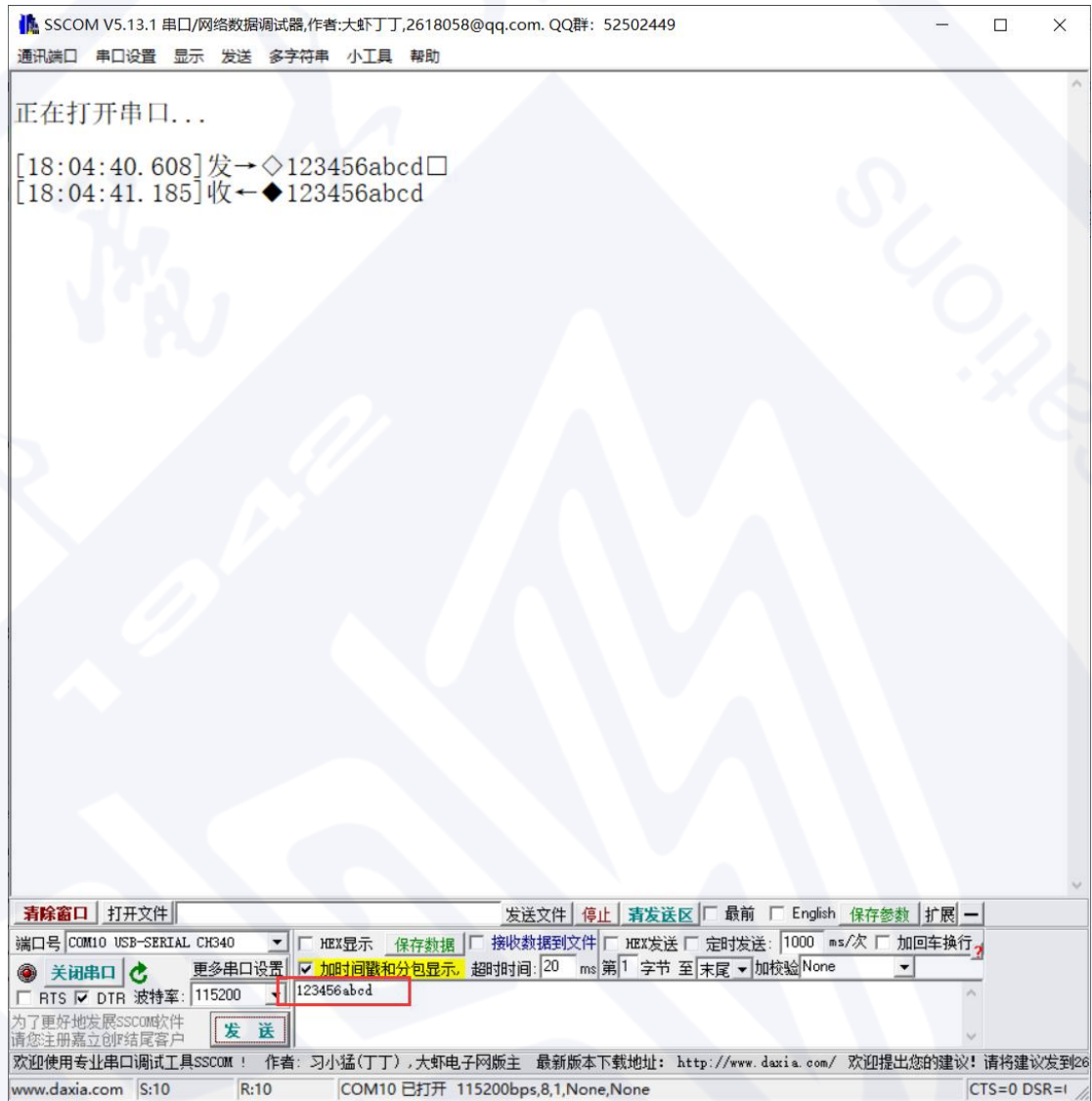
```
1 #include "hal_data.h"
2 #include <stdio.h>
3 FSP_CPP_HEADER
4 void R_BSP_WarmStart(bsp_warm_start_event_t event);
5 FSP_CPP_FOOTER
6
7 fsp_err_t err = FSP_SUCCESS;
8 volatile bool uart_send_complete_flag = false;
9 volatile bool g_receive_complete = false;
10 void user_uart_callback (uart_callback_args_t * p_args)
11 {
12     if(p_args->event == UART_EVENT_TX_COMPLETE)
13     {
14         uart_send_complete_flag = true;
15     }
16     else if(p_args->event == UART_EVENT_RX_COMPLETE)
17     {
18         g_receive_complete = true;
19     }
20 }
21
22 #ifdef __GNUC__ //串口重定向
23 #no previous declaration for '__io_putchar' [-Wmissing-declarations]
24 #define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
25 #else
26 #define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
27 #endif
28
29 PUTCHAR_PROTOTYPE
30 {
31     err = R_SCI_UART_Write(&g_uart9_ctrl, (uint8_t *)&ch, 1);
32     if(FSP_SUCCESS != err) __BKPT();
33     while(uart_send_complete_flag == false){}
34     uart_send_complete_flag = false;
35     return ch;
36 }
37
38 #unused parameter 'fd' [-Wunused-parameter] | #no previous declaration for '_write' [-Wmissing-declarations]
39 int _write(int fd, char *pBuffer, int size)
40 {
41     for(int i=0; i<size; i++)
42     {
43         __io_putchar(*pBuffer++);
44     }
45     return size;
46 }
47
48 /* * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. This function[
49 void hal_entry(void)
50 {
51     /* TODO: add your own code here */
52
53     /* Open the transfer instance with initial configuration. */
54     err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);
55     assert(FSP_SUCCESS == err);
56 }
```

主函数添加如下。

```
while(1)
{
    if(g_receive_complete==1)
    {
        g_receive_complete=0;
        err = R_SCI_UART_Write(&g_uart9_ctrl, g_dest, TRANSFER_LENGTH);
        if(FSP_SUCCESS != err) __BKPT();
        while(uart_send_complete_flag == false){}
    }
    R_BSP_SoftwareDelay(1000, BSP_DELAY_UNITS_MILLISECONDS); //
}
```

```
[RA4M2_MINI_Project11_1] FSP Configuration  [hal_entry.c] startup.c main.c
49  * * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. This function
52 00000148 = void hal_entry(void)
53 0000015a {
54
55 00000168 /* TODO: add your own code here */
56
57 00000172 /* Open the transfer instance with initial configuration. */
58 0000017e err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);
59 00000184 assert(FSP_SUCCESS == err);
60 0000018a
61 unsigned char buff[]="RA E2STUDIO";
62 00000192  *pointer targets in passing argument 1 of 'strlen' differ in signedness [-Wpointer-sign] |  *conversion from 'size_t'
63 00000198 uint8_t buff_len = strlen(buff);
64 000001a2 err = R_SCI_UART_Write(&g_uart9_ctrl, buff, buff_len);
65 000001aa if(FSP_SUCCESS != err) __BKPT();
66 000001ae while(uart_send_complete_flag == false){}
67 000001b4 uart_send_complete_flag = false;
68
69 unsigned char send_buff[100];
70 000001dc  *pointer targets in passing argument 1 of 'sprintf' differ in signedness [-Wpointer-sign]
71 000001e4 sprintf(send_buff, "\nHello World!\n");
72 000001ee  *pointer targets in passing argument 1 of 'strlen' differ in signedness [-Wpointer-sign] |  *conversion from 'size_t'
73 000001fc uint8_t len = strlen(send_buff);
74 00000204 err = R_SCI_UART_Write(&g_uart9_ctrl, send_buff, len);
75 0000020a if(FSP_SUCCESS != err) __BKPT();
76 0000020e while(uart_send_complete_flag == false){}
77 00000212 uart_send_complete_flag = false;
78 00000216 memset(send_buff, '\0', sizeof(100));
79
80 int i=0;
81 0000021c float float_i=66.20f;
82 00000220 char char_i[]="hello e2studio";
83
84 uint8_t TRANSFER_LENGTH=10;
85 00000224 uint8_t g_dest[TRANSFER_LENGTH];
86 00000228 R_SCI_UART_Read(&g_uart9_ctrl, g_dest, TRANSFER_LENGTH);
87
88 while(1)
89 {
90     if(g_receive_complete==1)
91     {
92         g_receive_complete=0;
93         err = R_SCI_UART_Write(&g_uart9_ctrl, g_dest, TRANSFER_LENGTH);
94         if(FSP_SUCCESS != err) __BKPT();
95         while(uart_send_complete_flag == false){}
96     }
97     R_BSP_SoftwareDelay(1000, BSP_DELAY_UNITS_MILLISECONDS); //
98 }
99
100 #if BSP_TZ_SECURE_BUILD
101 /* Enter non-secure code */
102 R_BSP_NonSecureEnter();
103 #endif
104
105 * * This function is called at various points during the startup process. This implementation uses the event that is
106 void R_BSP_WarmStart(bsp_warm_start_event_t event)
107 {
108     if (BSP_WARM_START_RESET == event)
109     {
```

演示结果如下所示。



UART_EVENT_RX_CHAR

UART_EVENT_RX_CHAR: 字符接收事件, 当接收到一个字符时触发。

Enumeration Type Documentation

◆ uart_event_t

enum uart_event_t	
UART Event codes	
Enumerator	
UART_EVENT_RX_COMPLETE	Receive complete event.
UART_EVENT_TX_COMPLETE	Transmit complete event.
UART_EVENT_RX_CHAR	Character received.
UART_EVENT_ERR_PARITY	Parity error event.
UART_EVENT_ERR_FRAMING	Mode fault error event.
UART_EVENT_ERR_OVERFLOW	FIFO Overflow error event.
UART_EVENT_BREAK_DETECT	Break detect error event.
UART_EVENT_TX_DATA_EMPTY	Last byte is transmitting, ready for more data.

UART_EVENT_RX_CHAR 是 **UART** 模块中定义的一个事件，它的作用是用于处理单个字符接收完成的情况。

```
fsp_err_t err = FSP_SUCCESS;
volatile bool uart_send_complete_flag = false;
volatile bool g_receive_complete = false;
#define TRANSFER_LENGTH_1 10
uint8_t g_out_of_band_received[TRANSFER_LENGTH_1];
uint32_t g_out_of_band_index = 0;
void user_uart_callback (uart_callback_args_t * p_args)
{
    if(p_args->event == UART_EVENT_TX_COMPLETE)
    {
        uart_send_complete_flag = true;
    }
    else if(p_args->event == UART_EVENT_RX_COMPLETE)
    {
        g_receive_complete = true;
    }
    else if(p_args->event == UART_EVENT_RX_CHAR)
    {
        /* Only put the next character in the receive buffer if there is space for it */
```

```

        if (sizeof(g_out_of_band_received) > g_out_of_band_index)
        {
            /* Write either the next one or two bytes depending on the receive data size */
            if (UART_DATA_BITS_8 >= g_uart9_cfg.data_bits)
            {
                g_out_of_band_received[g_out_of_band_index++] = (uint8_t) p_args->data;
            }
            else
            {
                uint16_t * p_dest = (uint16_t *) &g_out_of_band_received[g_out_of_band_index];
                *p_dest = (uint16_t) p_args->data;
                g_out_of_band_index += 2;
            }
        }
    }
}

```

```

1  #include "hal_data.h"
2  #include <stdio.h>
3  FSP_CPP_HEADER
4  void R_BSP_WarmStart(bsp_warm_start_event_t event);
5  FSP_CPP_FOOTER
6
7  fsp_err_t err = FSP_SUCCESS;
8  volatile bool uart_send_complete_flag = false;
9  volatile bool g_receive_complete = false;
10 #define TRANSFER_LENGTH_1 10
11 uint8_t g_out_of_band_received[TRANSFER_LENGTH_1];
12 uint32_t g_out_of_band_index = 0;
13 void user_uart_callback (uart_callback_args_t * p_args)
14 {
15     if(p_args->event == UART_EVENT_TX_COMPLETE)
16     {
17         uart_send_complete_flag = true;
18     }
19     else if(p_args->event == UART_EVENT_RX_COMPLETE)
20     {
21         g_receive_complete = true;
22     }
23     else if(p_args->event == UART_EVENT_RX_CHAR)
24     {
25         /* Only put the next character in the receive buffer if there is space for it */
26         if (sizeof(g_out_of_band_received) > g_out_of_band_index)
27         {
28             /* Write either the next one or two bytes depending on the receive data size */
29             if (UART_DATA_BITS_8 >= g_uart9_cfg.data_bits)
30             {
31                 g_out_of_band_received[g_out_of_band_index++] = (uint8_t) p_args->data;
32             }
33             else
34             {
35                 uint16_t * p_dest = (uint16_t *) &g_out_of_band_received[g_out_of_band_index];
36                 *p_dest = (uint16_t) p_args->data;
37                 g_out_of_band_index += 2;
38             }
39         }
40     }
41 }
42
43
44
45
46 #ifdef __GNUC__
47     /* no previous declaration for '__io_putchar' [-Wmissing-declarations]
48     #define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
49 #else
50     #define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
51 #endif
52
53 PUTCHAR_PROTOTYPE
54 {
55     err = R_SCI_UART_Write(&g_uart9_ctrl1, (uint8_t *) &ch, 1);
56     if(FSP_SUCCESS != err) __BKPT();
57     while(!uart_send_complete_flag == false){

```

主程序做如下修改，串口读取关闭。

```

// uint8_t TRANSFER_LENGTH=10;

// uint8_t g_dest[TRANSFER_LENGTH];

// R_UARTA_Read(&g_uart0_ctrl, g_dest, TRANSFER_LENGTH);

```

```

71 00000162
73 0000016c * * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. This function[
76 void hal_entry(void)
77 {
78     /* TODO: add your own code here */
79
80     /* Open the transfer instance with initial configuration. */
81     err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);
82     00000178 assert(FSP_SUCCESS == err);
83
84     unsigned char buff[]="RA E2STUDIO";
85     /*pointer targets in passing argument 1 of 'strlen' differ in signedness [-Wpointer-sign] | /*conversion from 'size_t'
86     uint8_t buff_len = strlen(buff);
87     00000182 err = R_SCI_UART_Write(&g_uart9_ctrl, buff, buff_len);
88     00000186 if(FSP_SUCCESS != err) __BKPT();
89     0000018e while(uart_send_complete_flag == false){}
90     00000194 uart_send_complete_flag = false;
91     00000198
92     unsigned char send_buff[100];
93     /*pointer targets in passing argument 1 of 'sprintf' differ in signedness [-Wpointer-sign]
94     sprintf(send_buff, "\nHello World!\n");
95     /*conversion from 'size_t' {aka 'unsigned int'} to 'uint8_t' {aka 'unsigned char'} may change value [-Wconversion] |
96     uint8_t len = strlen(send_buff);
97     0000019e err = R_SCI_UART_Write(&g_uart9_ctrl, send_buff, len);
98     000001a2 if(FSP_SUCCESS != err) __BKPT();
99     000001a6 while(uart_send_complete_flag == false){}
100     000001b0 uart_send_complete_flag = false;
101     000001b4 memset(send_buff, '\0', sizeof(100));
102
103     /*unused variable 'int_i' [-Wunused-variable]
104     int int_i=0;
105     /*unused variable 'float_i' [-Wunused-variable]
106     float float_i=66.20f;
107     /*unused variable 'char_i' [-Wunused-variable]
108     char char_i[]="hello e2studio";
109
110     // uint8_t TRANSFER_LENGTH=10;
111     // uint8_t g_dest[TRANSFER_LENGTH];
112     // R_SCI_UART_Read(&g_uart0_ctrl, g_dest, TRANSFER_LENGTH);
113
114     while(1)
115     {
116         if(g_receive_complete==1)
117         {
118             g_receive_complete=0;
119             000001c2 err = R_SCI_UART_Write(&g_uart9_ctrl, g_dest, TRANSFER_LENGTH);
120             000001c6 if(FSP_SUCCESS != err) __BKPT();
121             000001d0 while(uart_send_complete_flag == false){}
122         }
123         R_BSP_SoftwareDelay(1000, BSP_DELAY_UNITS_MILLISECONDS); //
124     }
125
126     #if BSP_TZ_SECURE_BUILD
127     /* Enter non-secure code */
128     R_BSP_NonSecureEnter();
129     #endif
130 }

```

while 中添加读取 10 个长度数据进行打印。

```

while(1)
{
    // if(g_receive_complete==1)
    // {
    //     g_receive_complete=0;
    //     err = R_SCI_UART_Write(&g_uart9_ctrl, g_dest, TRANSFER_LENGTH);
    //     if(FSP_SUCCESS != err) __BKPT();
    //     while(uart_send_complete_flag == false){}
    // }

    if(g_out_of_band_index==10)
    {
        g_out_of_band_index=0;
        err = R_SCI_UART_Write(&g_uart9_ctrl, g_out_of_band_received, TRANSFER_LENGTH_1);
        if(FSP_SUCCESS != err) __BKPT();
        while(uart_send_complete_flag == false){}
    }
}

```

```
}
```

```
R_BSP_SoftwareDelay(1000, BSP_DELAY_UNITS_MILLISECONDS); //
```

```
}
```

```
[RA4M2 MINI Project11_1] FSP Configuration  hal_entry.c  startup.c  main.c  bsp_delay.c
70 0000015a
71 00000162
73 0000016c
75
76
77
78
79
80
81
82 00000178
83
84
85
86 00000182
87
88 00000186
89 0000018a
90 00000194
91 00000198
92
93 0000019e
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122 000001d8
123
124
125
126
127 000001de
128
129
130
131
132
133
134
135
136
141
142
143
144
145
146
147

/* * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. This function */
void hal_entry(void)
{
    /* TODO: add your own code here */

    /* Open the transfer instance with initial configuration. */
    err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);
    assert(FSP_SUCCESS == err);

    unsigned char buff[]="RA E2STUDIO";
    /* pointer targets in passing argument 1 of 'strlen' differ in signedness [-Wpointer-sign] / conversion from 'size_t' {aka 'unsigned int'} to
    uint8_t buff_len = strlen(buff);
    err = R_SCI_UART_Write(&g_uart9_ctrl, buff, buff_len);
    if(FSP_SUCCESS != err) __BKPT();
    while(uart_send_complete_flag == false){}
    uart_send_complete_flag = false;

    unsigned char send_buff[100];
    /* pointer targets in passing argument 1 of 'sprintf' differ in signedness [-Wpointer-sign]
    sprintf(send_buff, "\nHello World!\n");
    /* conversion from 'size_t' {aka 'unsigned int'} to 'uint8_t' {aka 'unsigned char'} may change value [-Wconversion] / pointer targets in pas
    uint8_t len = strlen(send_buff);
    err = R_SCI_UART_Write(&g_uart9_ctrl, send_buff, len);
    if(FSP_SUCCESS != err) __BKPT();
    while(uart_send_complete_flag == false){}
    uart_send_complete_flag = false;
    memset(send_buff, '\0', sizeof(100));

    /* unused variable 'int_i' [-Wunused-variable]
    int int_i=0;
    /* unused variable 'float_i' [-Wunused-variable]
    float float_i=66.20f;
    /* unused variable 'char_i' [-Wunused-variable]
    char char_i[]="hello e2studio";

    /* uint8_t TRANSFER_LENGTH=10;
    /* uint8_t g_dest[TRANSFER_LENGTH];
    /* R_SCI_UART_Read(&g_uart9_ctrl, g_dest, TRANSFER_LENGTH);

    while(1)
    {
        if(g_receive_complete==1)
        {
            g_receive_complete=0;
            err = R_SCI_UART_Write(&g_uart9_ctrl, g_dest, TRANSFER_LENGTH);
            if(FSP_SUCCESS != err) __BKPT();
            while(uart_send_complete_flag == false){}
        }
        if(g_out_of_band_index==10)
        {
            g_out_of_band_index=0;
            err = R_SCI_UART_Write(&g_uart9_ctrl, g_out_of_band_received, TRANSFER_LENGTH_1);
            if(FSP_SUCCESS != err) __BKPT();
            while(uart_send_complete_flag == false){}
        }

        R_BSP_SoftwareDelay(1000, BSP_DELAY_UNITS_MILLISECONDS); //
    }

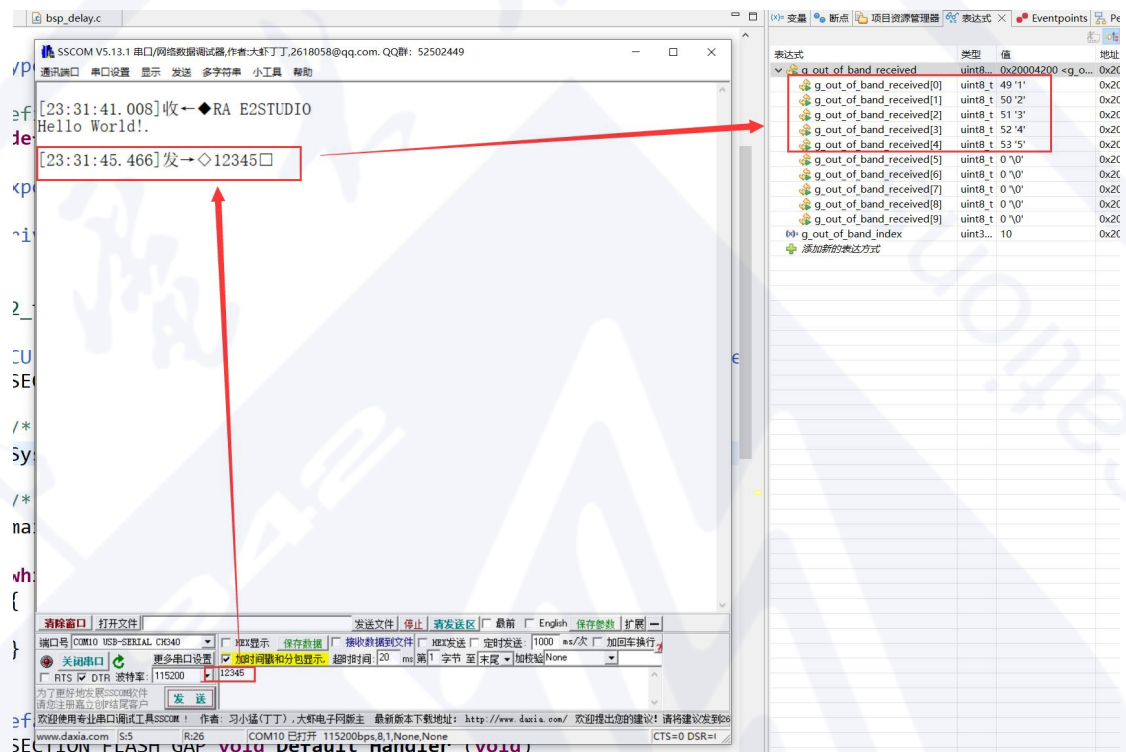
    #if BSP_TZ_SECURE_BUILD
    /* Enter non-secure code */
    R_BSP_NonSecureEnter();
    #endif
}

/* * This function is called at various points during the startup process. This implementation uses the event that is */
void R_BSP_WarmStart(bsp_warm_start_event_t event)
{
    if (BSP_WARM_START_RESET == event)
    {
        #if BSP_FEATURE_FLASH_LP_VERSION != 0

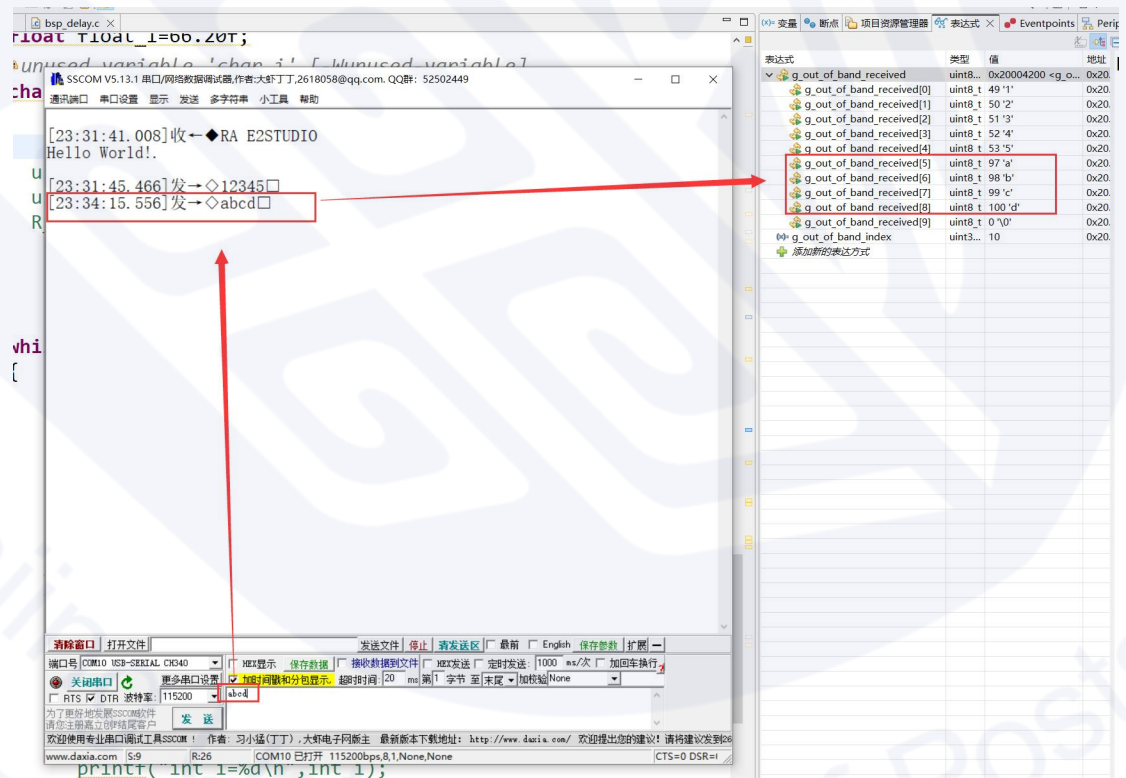
            /* Enable reading from data flash. */

```

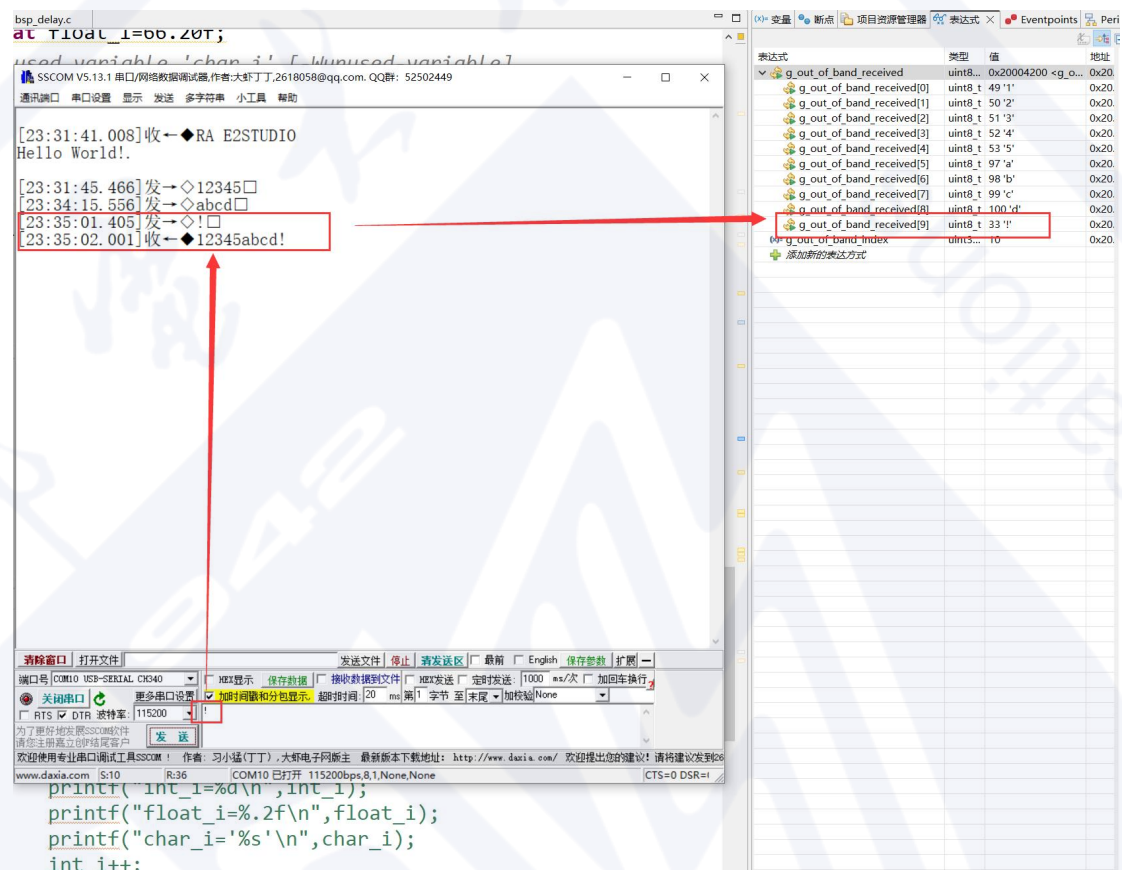
输入 5 个数据，可以实时查看到数据。



输入 4 个字符。



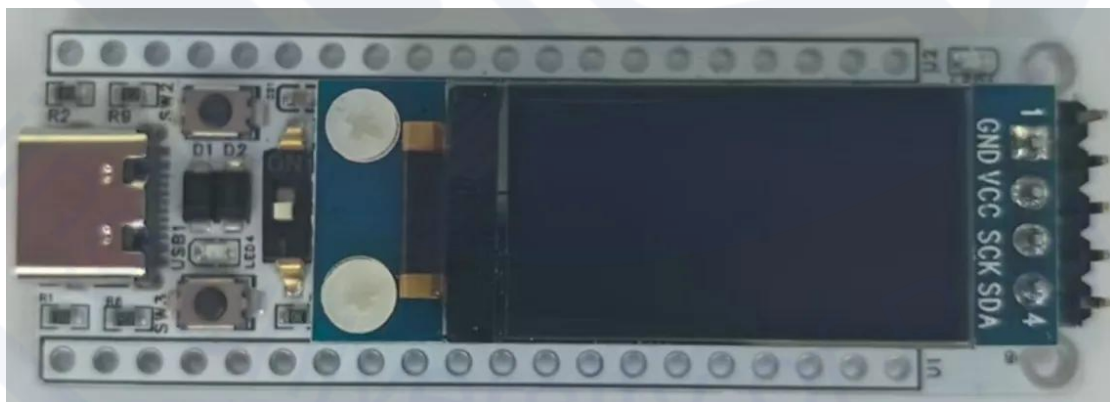
当输入第十个的时候，直接执行打印操作。



project14: IIC 驱动水压计 LPS28DFW

概述

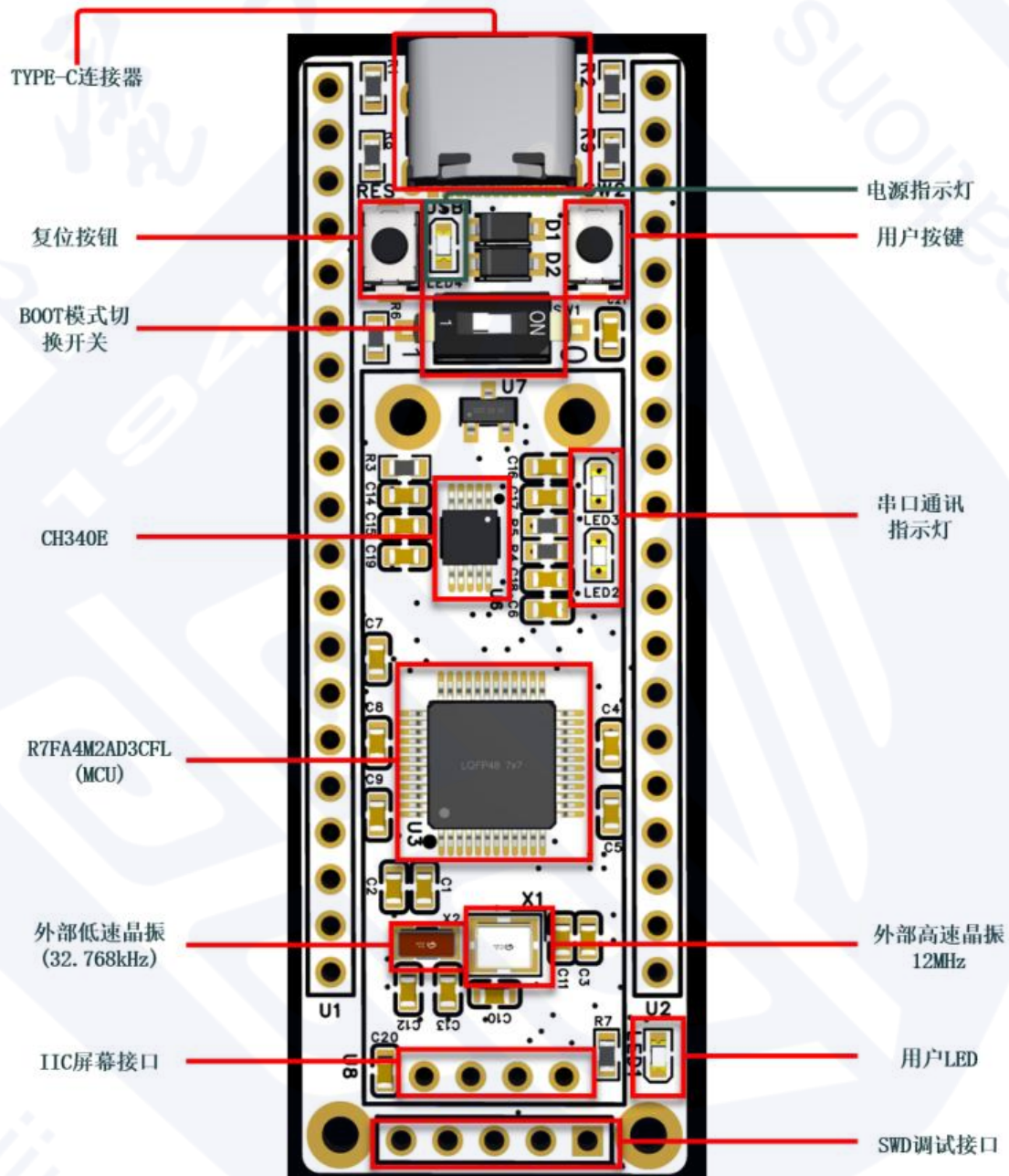
本篇文章主要介绍如何使用 e2studio 对瑞萨进行 IIC 配置, 使用 LPS28DFW 传感器来读取数据, 读取到的数据会被转换为适当的单位并通过串行通信输出。



硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL

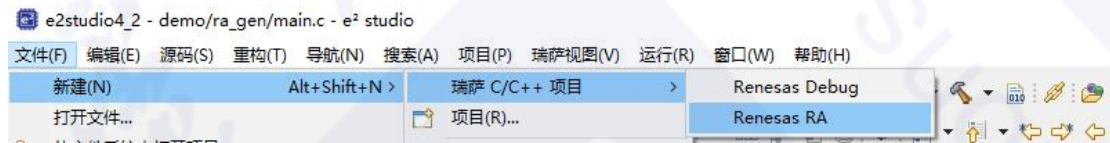


参考程序

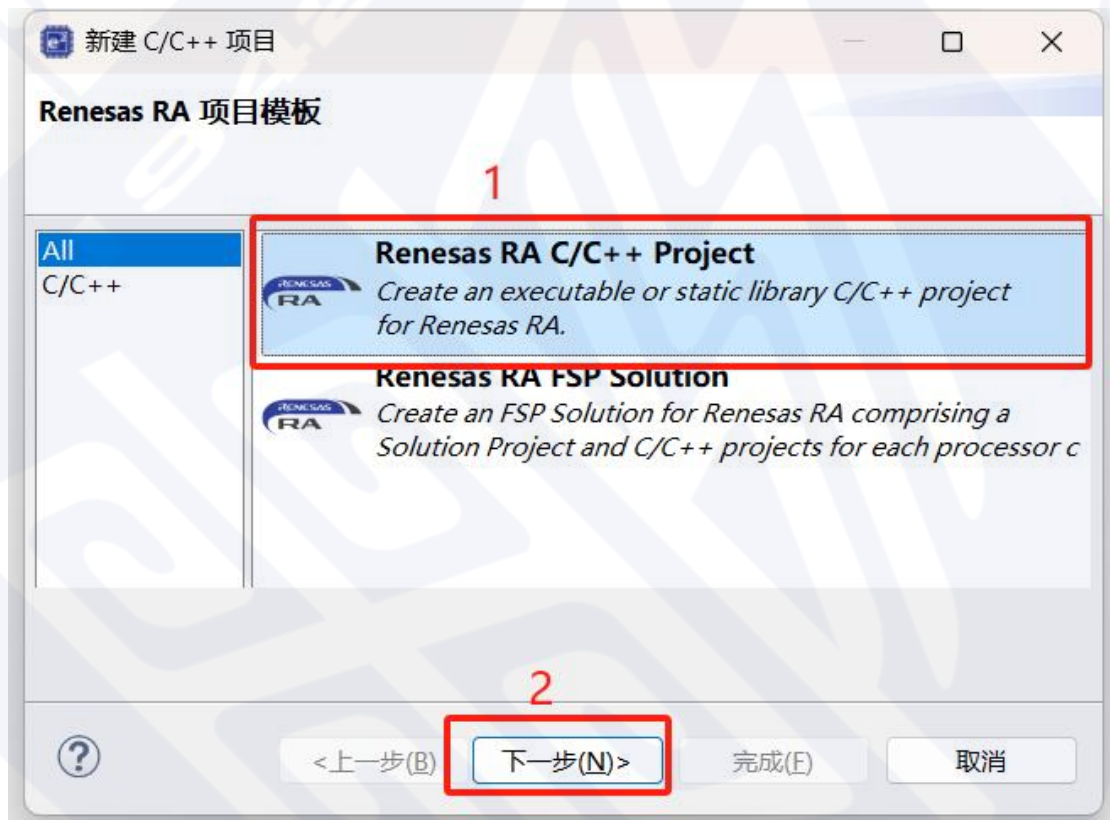
https://github.com/CoreMaker-lab/RA4M2_MINI

https://gitee.com/CoreMaker/RA4M2_MINI

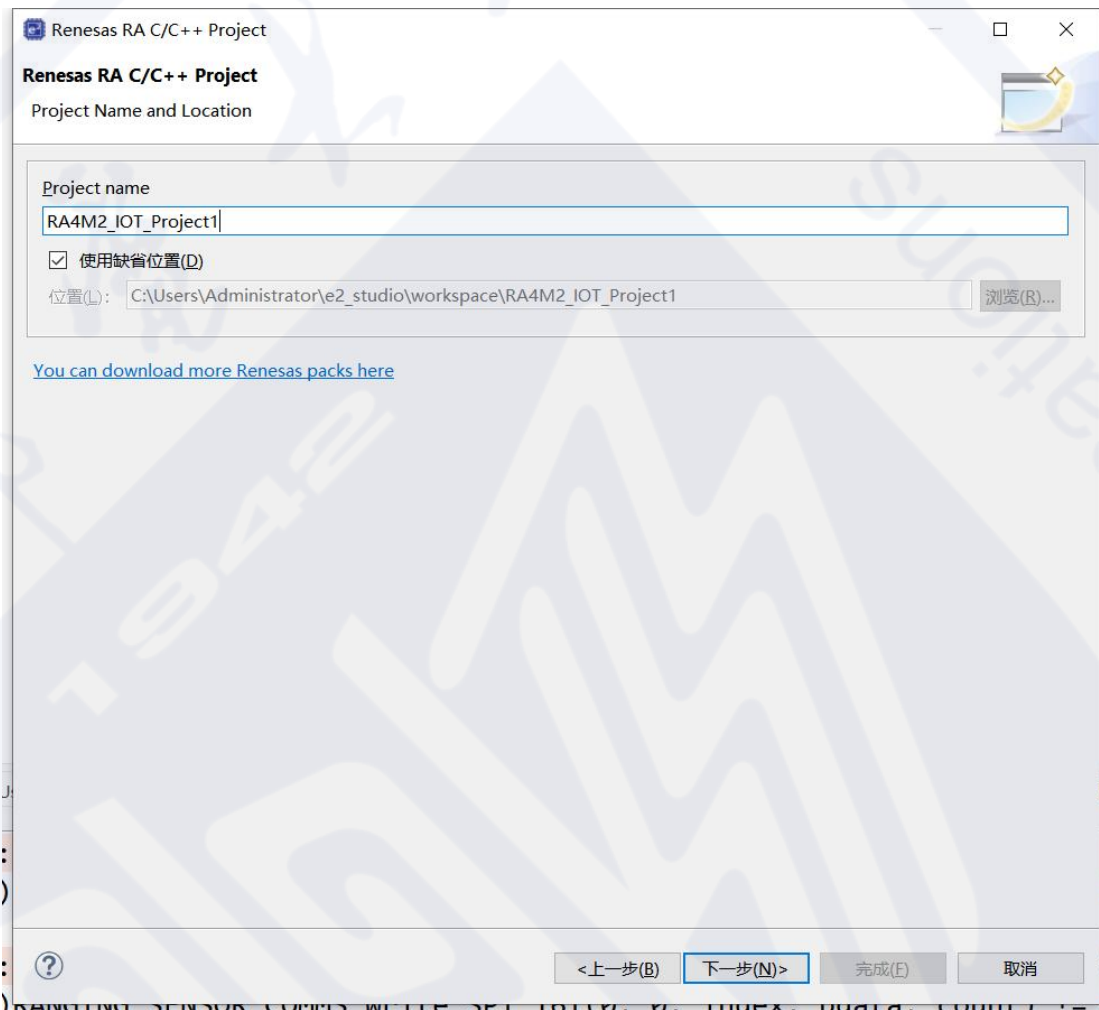
新建工程



工程模板



保存工程路径



芯片配置

本文中使用 R7FA4M2AD3CFL 来进行演示。

Renesas RA C/C++ Project

Renesas RA C/C++ Project

Device and Tools Selection

Device Selection

FSP Version: 5.9.0 **版本**

Board: Custom User Board (Any Device) ...

Device: R7FA4M2AD3CFL **芯片型号**

Core: CM33

Language: ☒ C ☐ C++

Board Description

Device Details

TrustZone	Yes
Pins	48
Processor	Cortex-M33

IDE Project Type

e2 studio managed build

Toolchains

LLVM Embedded Toolchain for Arm

GNU ARM Embedded **工具链**

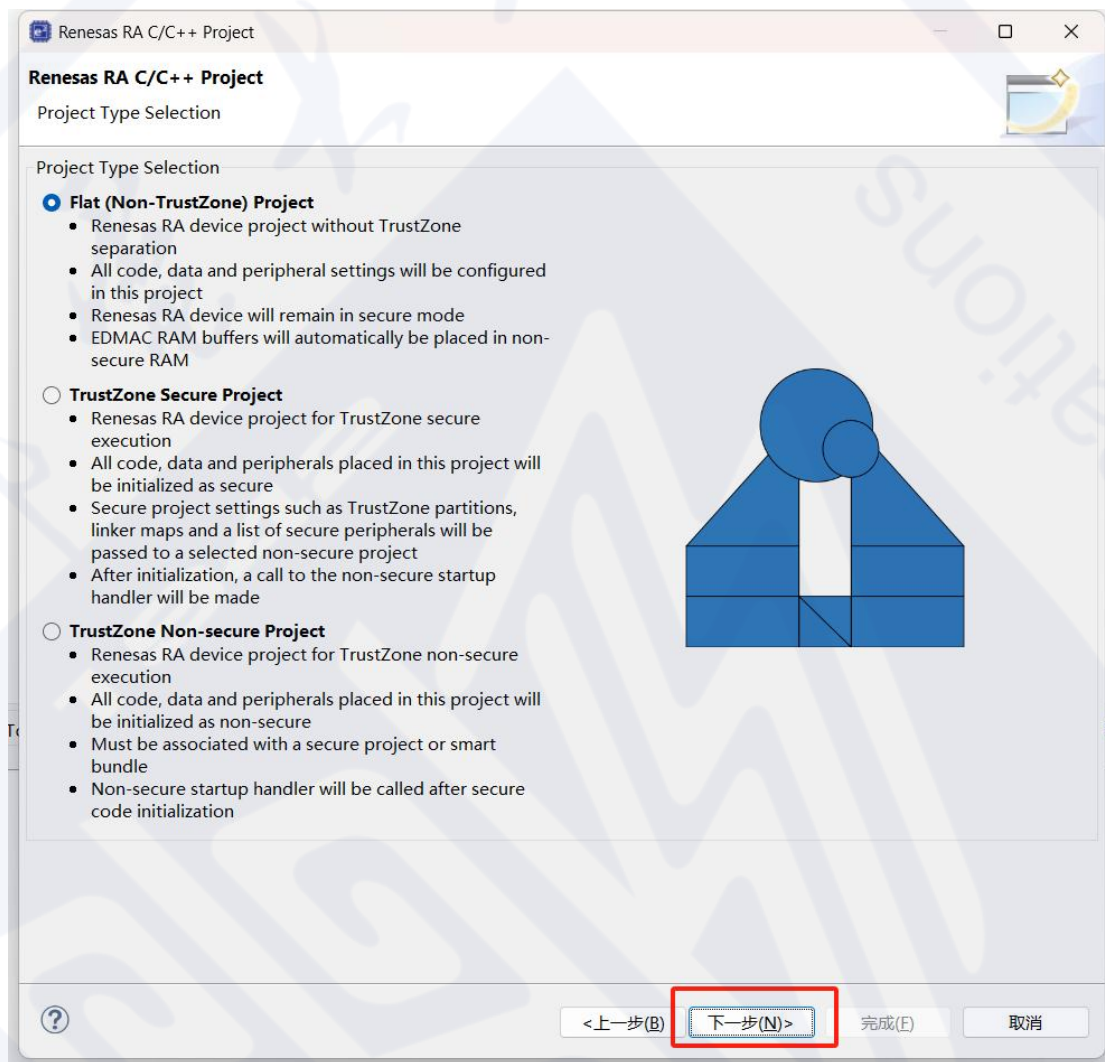
13.2.1.arm-13-7 [Manage Toolchains...](#)

Debugger

J-Link ARM **使用JLink进行调试**

[?](#) <上一步(B) 下一步(N)> 完成(F) 取消

工程模板选择



Renesas RA C/C++ Project

Renesas RA C/C++ Project

Preceding Project or Smart Bundle Selection

☒ None

Choose this option when creating a project for the primary processor core (no preceding project or Smart Bundle).

☐ Preceding Project:

Choose this option if you have access to the preceding bootloader project or TrustZone secure project source code. A debug configuration for Secure Software Debug (SSD) state will be generated where applicable and both secure and non-secure images will be downloaded to the target device. Should you wish to test Non-Secure Software Debug (NSECSO) state, use the Renesas Device Partition Manager to change the Device Lifecycle state as needed.

☐ Smart Bundle:

Resolved location:

Workspace...

File System...

Variables...

Choose this option if you only have access to a pre-programmed device containing TrustZone secure code. The Smart Bundle file (*.sbd) should be obtained from the TrustZone Secure Project developer. A debug configuration for Non-Secure Software Debug (NSECSO) state will be generated and only the non-secure code image will be downloaded to the target device.

Preceding Project/Smart Bundle Details

FSP version

Toolchain

Toolchain version

Board

Device

Core

Zones

?

< 上一步(B)

下一步(N) >

完成(E)

取消

Renesas RA C/C++ Project

Renesas RA C/C++ Project
Build Artifact and RTOS Selection

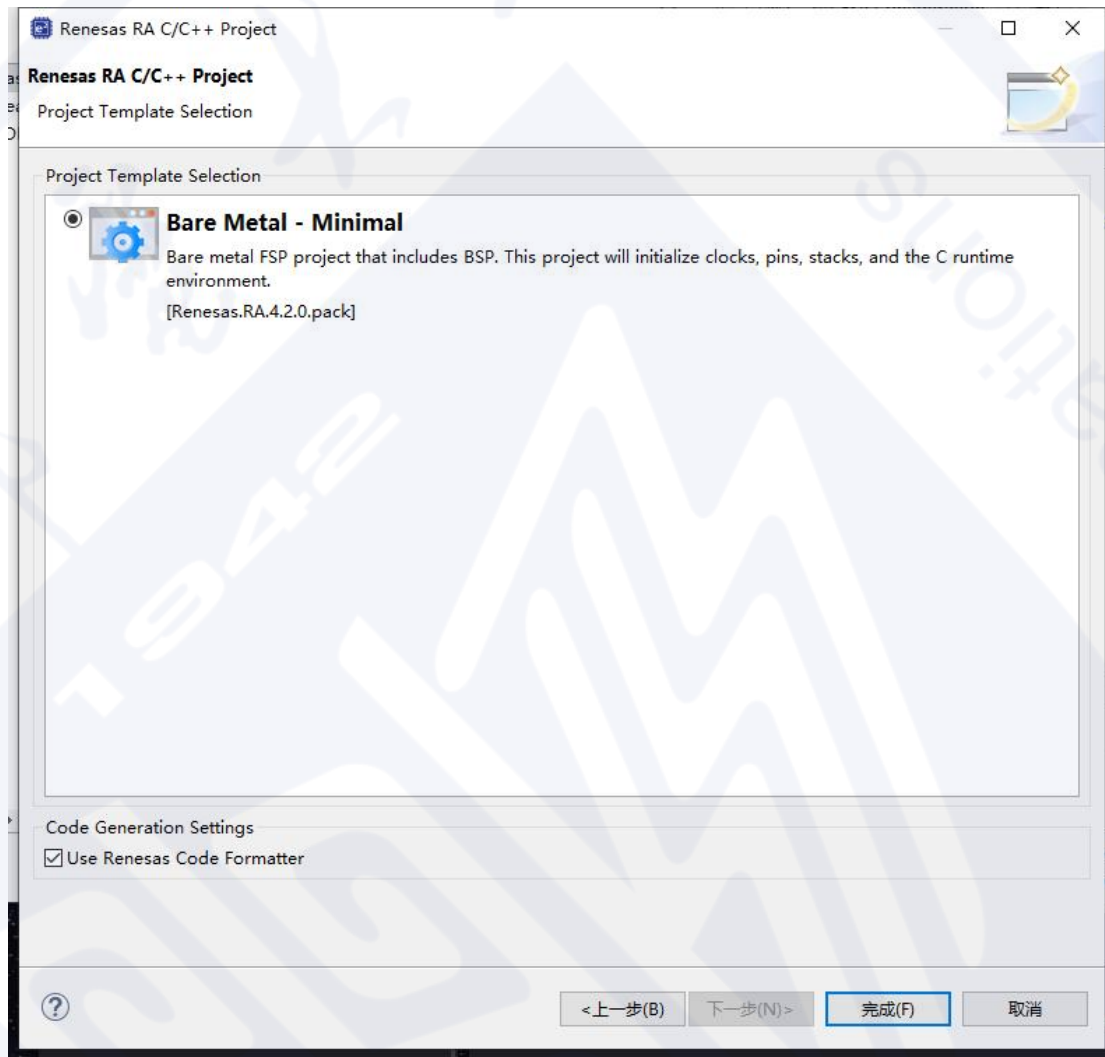
Build Artifact Selection

- ☒ **Executable**
 - Project builds to an executable file
- ☐ **Static Library**
 - Project builds to a static library file
- ☐ **Executable Using an RA Static Library**
 - Project builds to an executable file
 - Project uses an existing RA static library project

RTOS Selection

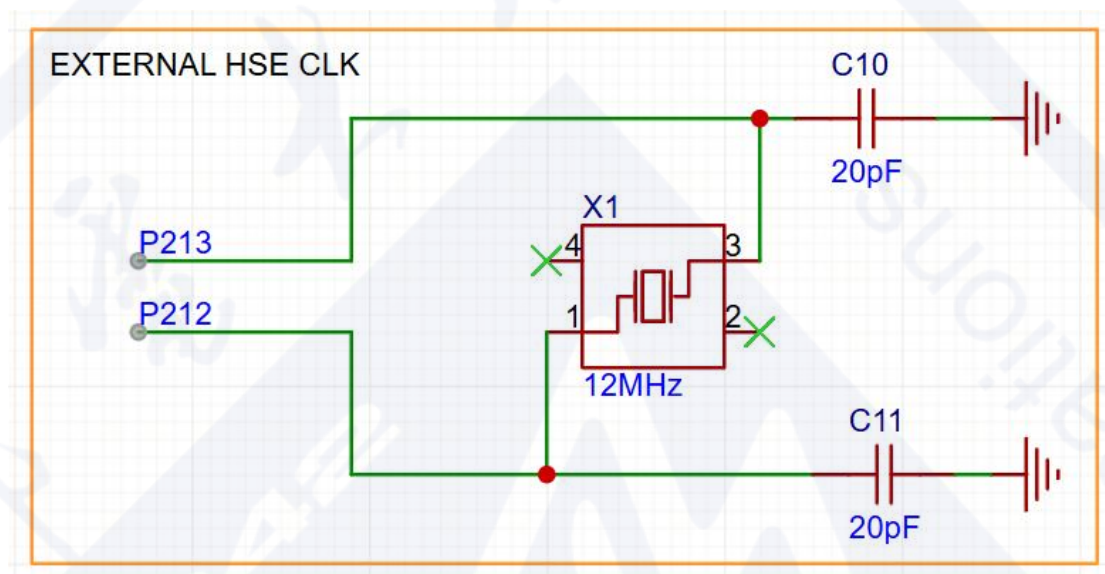
No RTOS

? <上一步(B) **下一步(N)>** 完成(F) 取消

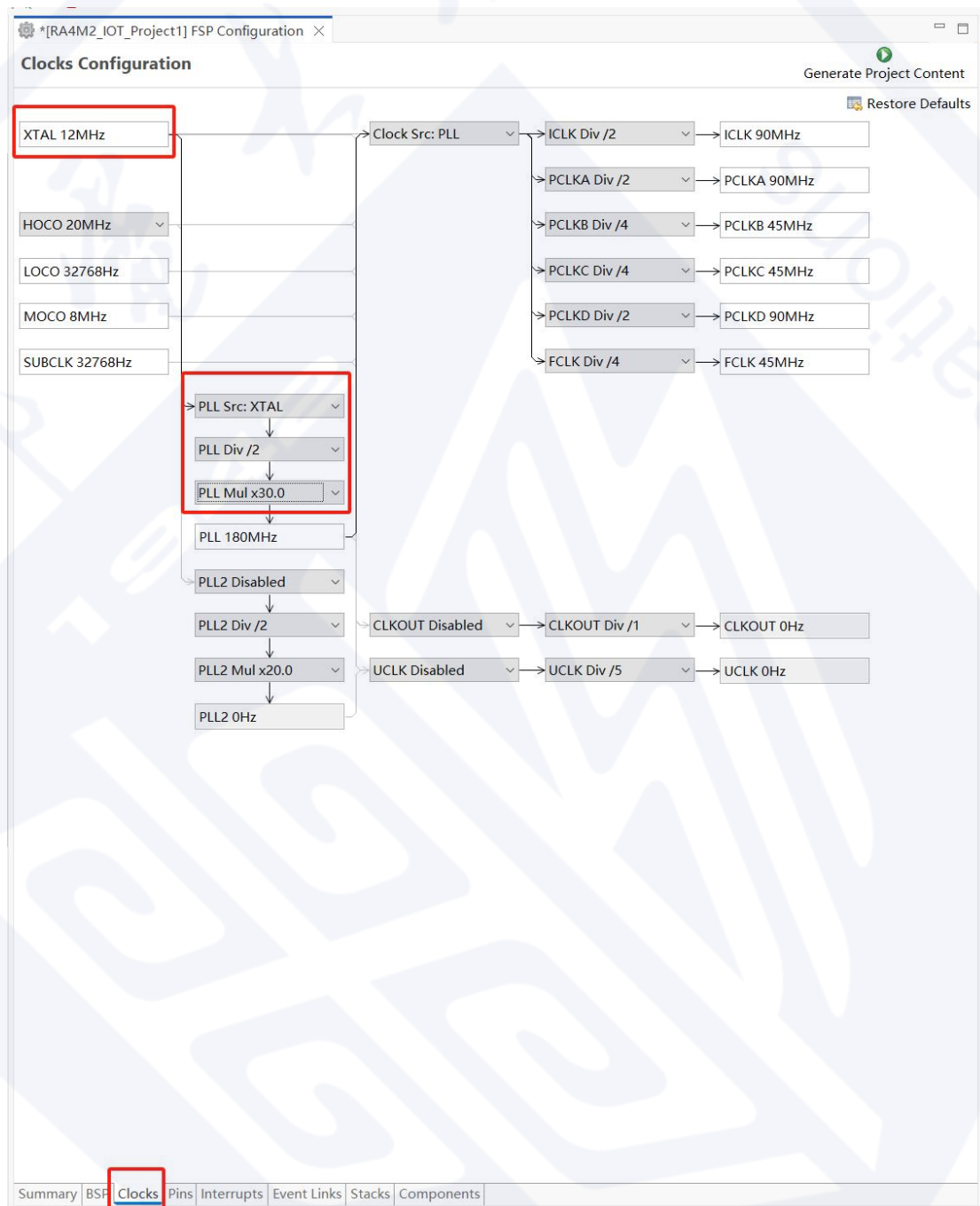


时钟设置

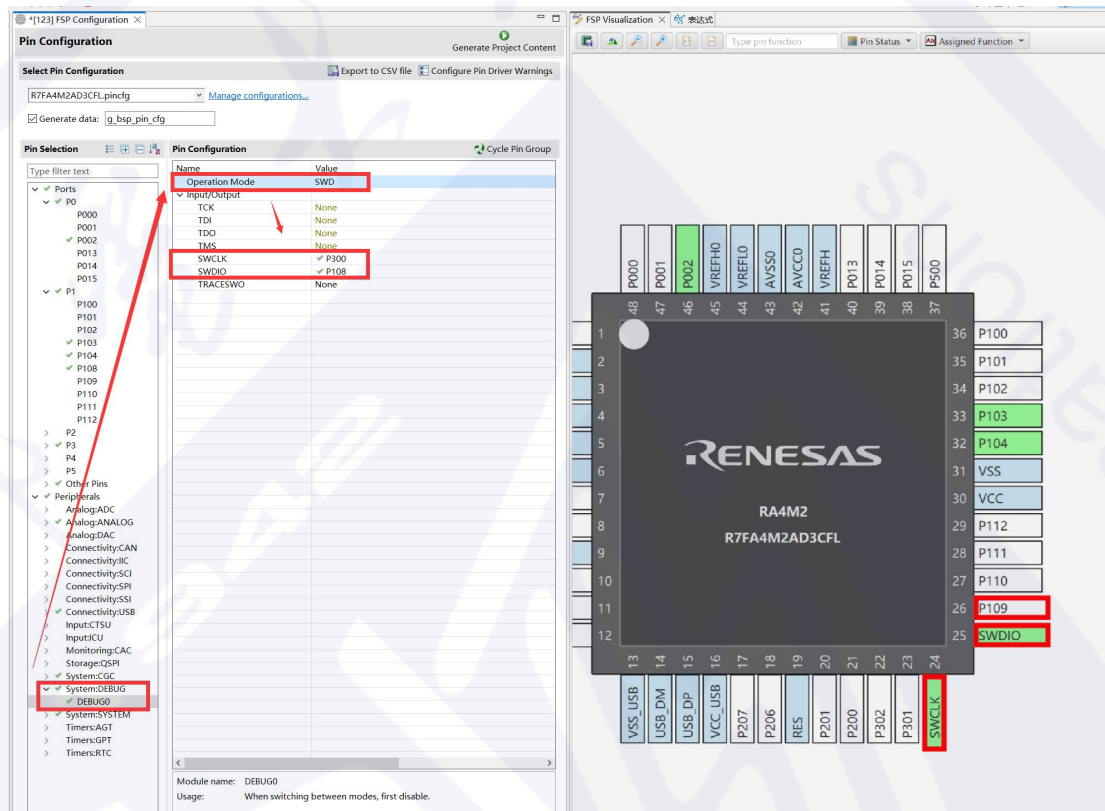
开发板上的外部高速晶振为 12M.



需要修改 XTAL 为 12M。

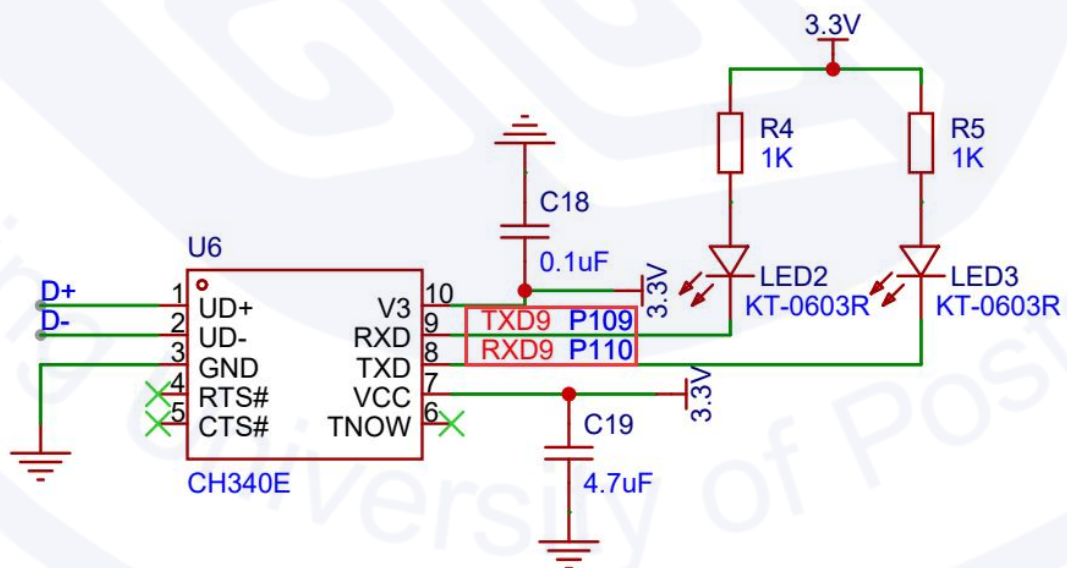


SWD 调试口设置

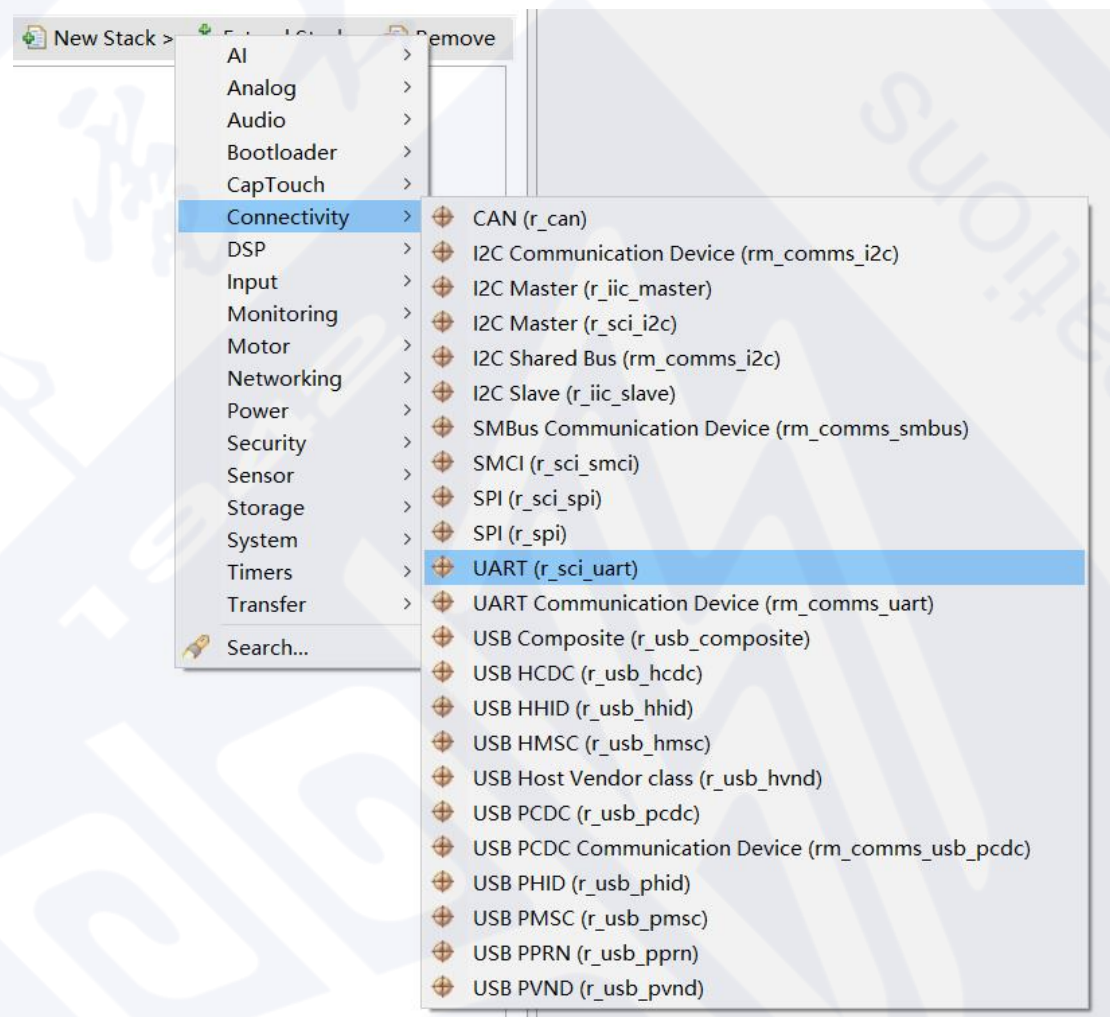


UART 配置

串口CH340E



点击 Stacks->New Stack->Connectivity -> UART(r_sci_uart)。



UART 属性配置

属性 ×	问题	Smart Browser	控制台	调试
g_uart9 UART (r_sci_uart)				
Settings	属性		值	
API Info	▼ Common			
	Parameter Checking		Default (BSP)	
	FIFO Support		Disable	
	DTC Support		Disable	
	Flow Control Support		Disable	
	RS-485 Support		Disable	
	IrDA Support		Disabled	
	▼ Module g_uart9 UART (r_sci_uart)			
	▼ General			
	Name		g_uart9	
	Channel		9	
	Data Bits		8bits	
	Parity		None	
	Stop Bits		1bit	
	▼ Baud			
	Baud Rate		115200	
	Baud Rate Modulation		Enabled	
	Max Error (%)		5	
	► Flow Control			
	► Extra			
	▼ Interrupts			
	Callback		user_uart_callback	
	Receive Interrupt Priority		Priority 2	
	Transmit Data Empty Interrupt Priority		Priority 2	
	Transmit End Interrupt Priority		Priority 2	
	Error Interrupt Priority		Priority 2	
	▼ Pins			
	TXD9		P109	
	RXD9		P110	
	CTS9		<unavailable>	

名称
通道
数据位
无校验位
停止位

波特率
启用波特率调制

回调函数名称

串口发送端口
串口接收端口

设置 e2studio 堆栈

printf 函数通常需要设置堆栈大小。这是因为 printf 函数在运行时需要使用栈空间来存储临时变量和函数调用信息。如果堆栈大小不足，可能会导致程序崩溃或不可预期的行为。

printf 函数使用了可变参数列表，它会在调用时使用栈来存储参数，在函数调用结束时再清除参数，这需要足够的栈空间。另外 printf 也会使用一些临时变量，如果栈空间不足，会导致程序崩溃。

因此，为了避免这类问题，应该根据程序的需求来合理设置堆栈大小。

项目资源管理器

demo

- 二进制
- Includes
- ra
- ra_gen
- src
 - hal_entry.c
 - Debug
 - ra_cfg
 - script
 - configuration.xml
 - demo Debug_FlatJlink
 - demo Debug_FlatLaunch
 - ra_cfg.txt
- Developer Assistance
 - HAL/Common
 - g_ioport I/O Port (r_ioport)

g_ioport I/O Port (r_ioport)

fsp err t R IOPORT Open(ioport ctrl t *const

属性 问题 Smart Browser 控制台

Custom User Board (Any Device)

Settings

属性

属性	值
√ R7FA4M2AD3CFP	
part_number	R7FA4M2AD3CFP
rom_size_bytes	524288
ram_size_bytes	131072
data_flash_size_bytes	8192
package_style	LQFP
package_pins	100
√ RA4M2	
series	4
√ RA4M2 Family	
> Security	
> OFS0 register settings	
> OFS1 register settings	
> Block Protection Settings (BPS)	
> Permanent Block Protection Settings (PBPS)	
> Clocks	
Startup C-Cache Line Size	32 Bytes
Main Oscillator Wait Time	8163 cycles
√ RA Common	
Main stack size (bytes)	0x400
Heap size (bytes)	0x2000
MCU Vcc (mV)	3300
Parameter checking	Disabled
Assert Failures	Return FSP_ERR_ASSERTION
Error Log	No Error Log
Clock Registers not Reset Values during Startup	Disabled
Main Oscillator Populated	Populated
PFS Protect	Enabled
C Runtime Initialization	Enabled
Early BSP Initialization	Disabled
Main Oscillator Clock Source	Crystal or Resonator
Subclock Populated	Populated
Subclock Drive (Drive capacitance availability varies by MCU)	Standard/Normal mode
Subclock Stabilization Time (ms)	1000

Board Support Package Configuration

Generate Project Content

Restore Defaults

Device Selection

FSP version: 4.2.0

Board: Custom User Board (Any Device)

Device: R7FA4M2AD3CFP

Core: CM33

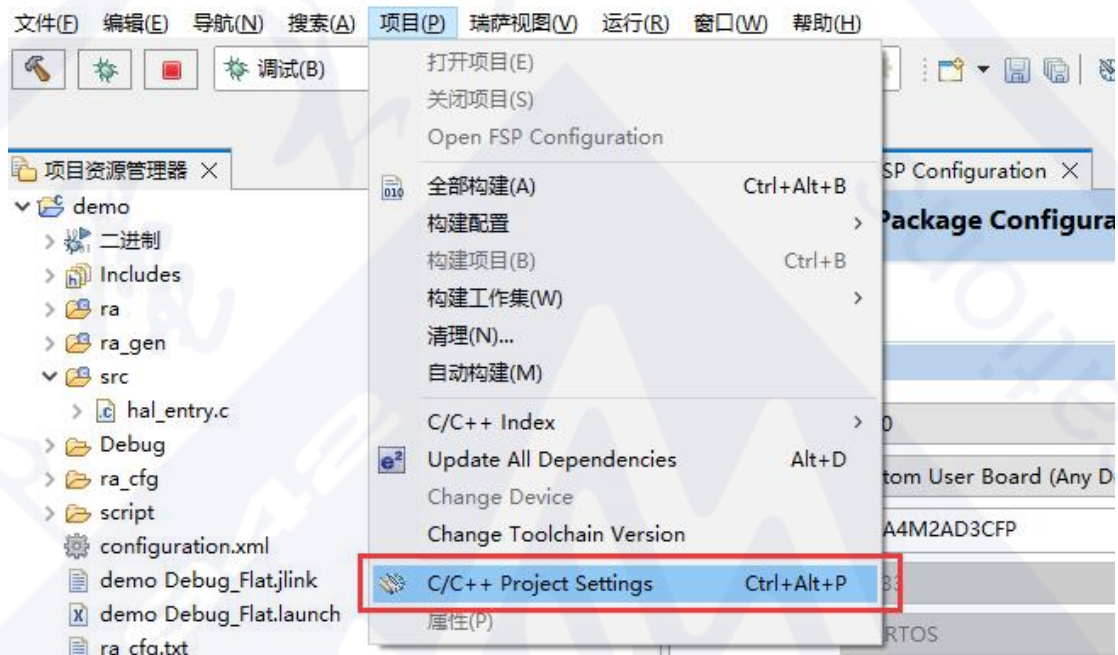
RTOS: No RTOS

Board Details

Summary BSP Clocks Pins Interrupts Event Links Stacks Components

3.设置堆栈大小

e2studio 的重定向 printf 设置



在嵌入式系统的开发中，尤其是在使用 GNU 编译器集合（GCC）时，`--specs` 参数用于指定链接时使用的系统规格（specs）文件。这些规格文件控制了编译器和链接器的行为，尤其是关于系统库和启动代码的链接。`--specs=rdimon.specs` 和 `--specs=nosys.specs` 是两种常见的规格文件，它们用于不同的场景。

`--specs=rdimon.specs`

用途：这个选项用于链接“Redlib”库，这是为裸机（bare-metal）和半主机（semihosting）环境设计的 C 库的一个变体。半主机环境是一种特殊的运行模式，允许嵌入式程序通过宿主机（如开发 PC）的调试器进行输入输出操作。

应用场景：当你需要在没有完整操作系统的环境中运行程序，但同时需要使用调试器来处理输入输出（例如打印到宿主机的终端），这个选项非常有用。

特点：它提供了一些基本的系统调用，通过调试接口与宿主机通信。

`--specs=nosys.specs`

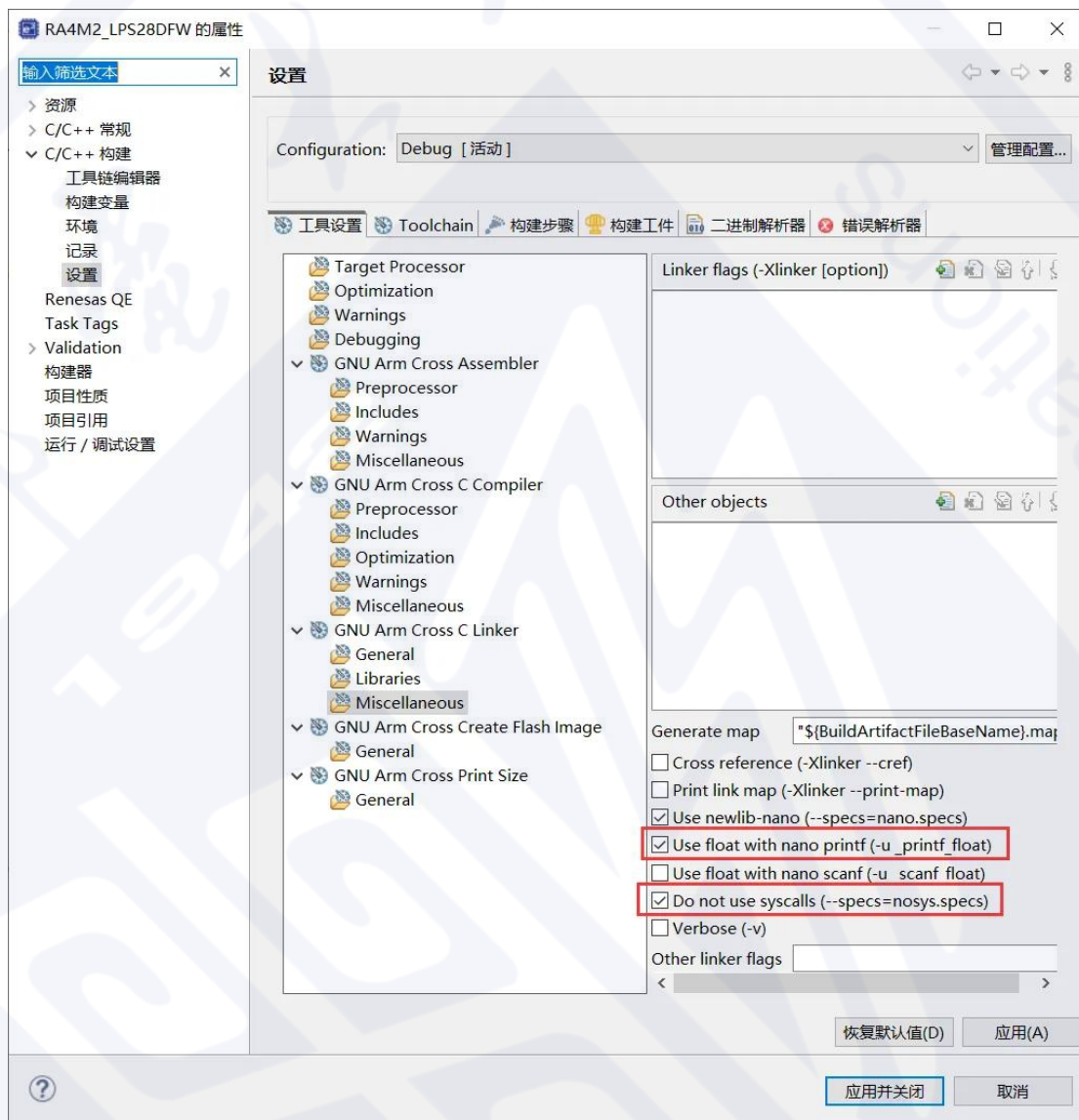
用途：这个选项链接了一个非常基本的系统库，这个库不提供任何系统服务的实现。

应用场景：适用于完全的裸机程序，其中程序不执行任何操作系统调用，比如不进行文件操作或者系统级输入输出。

特点：这是一个更“裸”的环境，没有任何操作系统支持。使用这个规格文件，程序不期望有操作系统层面的任何支持。

如果你的程序需要与宿主机进行交互（如在开发期间的调试），并且通过调试器进行基本的输入输出操作，则使用 `--specs=rdimon.specs`。

如果你的程序是完全独立的，不需要任何形式的操作系统服务，包括不进行任何系统级的输入输出，则使用 `--specs=nosys.specs`。



R_SCI_UART_Open() 函数原型

◆ R_SCI_UART_Open()

```
fsp_err_t R_SCI_UART_Open ( uart_ctrl_t *const p_api_ctrl,  
                           uart_cfg_t const *const p_cfg  
                           )
```

Configures the UART driver based on the input configurations. If reception is enabled at compile time, reception is enabled at the end of this function. Implements `uart_api_t::open`

Return values

FSP_SUCCESS	Channel opened successfully.
FSP_ERR_ASSERTION	Pointer to UART control block or configuration structure is NULL.
FSP_ERR_IP_CHANNEL_NOT_PRESENT	The requested channel does not exist on this MCU.
FSP_ERR_INVALID_ARGUMENT	Flow control is enabled but flow control pin is not defined or selected channel does not support "Hardware CTS and Hardware RTS" flow control. (or) restricted channel is selected.
FSP_ERR_ALREADY_OPEN	Control block has already been opened or channel is being used by another instance. Call <code>close()</code> then <code>open()</code> to reconfigure.
FSP_ERR_INVALID_CHANNEL	IrDA is requested for a channel that does not support IrDA.

Returns

See [Common Error Codes](#) or functions called by this function for other possible return codes. This function calls:

- `transfer_api_t::open`

故可以用 `R_SCI_UART_Open()`函数进行配置，开启和初始化 UART。

```
/* Open the transfer instance with initial configuration. */  
err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);  
assert(FSP_SUCCESS == err);
```

```
[RA4M2_MINI_Project11_1] FSP Configuration  *hal_entry.c x startup.c main.c
2
3 FSP_CPP_HEADER
4 void R_BSP_WarmStart(bsp_warm_start_event_t event);
5 FSP_CPP_FOOTER
6 /* unused parameter 'fd' [-Wunused-parameter] | no previous declaration for '_write' [-W
7
8
9
10
11
12
13
14
15 /* Open the transfer instance with initial configuration. */
16 err = R_SCI_UART_Open(&g_uart9_ctrl1, &g_uart9_cfg);
17 assert(FSP_SUCCESS == err);
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
```

回调函数 user_uart_callback ()

当数据发送的时候，可以查看 UART_EVENT_TX_COMPLETE 来判断是否发送完毕。

◆ write

fsp_err_t(* uart_api_t::write)(uart_ctrl_t *const p_ctrl, uint8_t const *const p_src, uint32_t const bytes)

Write to UART device. The write buffer is used until write is complete. Do not overwrite write buffer contents until the write is finished. When the write is complete (all bytes are fully transmitted on the wire), the callback called with event **UART_EVENT_TX_COMPLETE**. The maximum transfer size is reported by **infoGet[]**.

Implemented as

- R_SCI_UART_Write()
- R_SCI_B_UART_Write()

Parameters

[in] p_ctrl Pointer to the UART control block.

[in] p_src Source address to write data to.

[in] bytes Write data length.

Enumeration Type Documentation

◆ uart_event_t

enum `uart_event_t`

UART Event codes

Enumerator	
UART_EVENT_RX_COMPLETE	Receive complete event.
UART_EVENT_TX_COMPLETE	Transmit complete event.
UART_EVENT_RX_CHAR	Character received.
UART_EVENT_ERR_PARITY	Parity error event.
UART_EVENT_ERR_FRAMING	Mode fault error event.
UART_EVENT_ERR_OVERFLOW	FIFO Overflow error event.
UART_EVENT_BREAK_DETECT	Break detect error event.
UART_EVENT_TX_DATA_EMPTY	Last byte is transmitting, ready for more data.

可以检查检查 "p_args" 结构体中的 "event" 字段的值是否等于 "UART_EVENT_TX_COMPLETE"。如果条件为真，那么 if 语句后面的代码块将会执行。

```
fsp_err_t err = FSP_SUCCESS;
volatile bool uart_send_complete_flag = false;
void user_uart_callback (uart_callback_args_t * p_args)
{
    if(p_args->event == UART_EVENT_TX_COMPLETE)
    {
        uart_send_complete_flag = true;
    }
}
```

```
[RA4M2_MINI_Project11_1] FSP Configuration  *hal_entry.c x startup.c main.c
1 #include "hal_data.h"
2
3 FSP_CPP_HEADER
4 void R_BSP_WarmStart(bsp_warm_start_event_t event);
5 FSP_CPP_FOOTER
6
7 fsp_err_t err = FSP_SUCCESS;
8 volatile bool uart_send_complete_flag = false;
9 void user_uart_callback (uart_callback_args_t * p_args)
10 {
11     if(p_args->event == UART_EVENT_TX_COMPLETE)
12     {
13         uart_send_complete_flag = true;
14     }
15 }
16
17 unused parameter 'fd' [-Wunused-parameter] | no previous declaration for '_write' [-W
18
19 * main() is generated by the RA Configuration editor and is used to generate threads if
20 void hal_entry(void)
21 {
22     /* TODO: add your own code here */
23
24     /* Open the transfer instance with initial configuration. */
25     err = R_SCI_UART_Open(&g_uart9_ctrl1, &g_uart9_cfg);
26     assert(FSP_SUCCESS == err);
27
28     #if BSP_TZ_SECURE_BUILD
29     /* Enter non-secure code */
30     R_BSP_NonSecureEnter();
31     #endif
32
33     * This function is called at various points during the startup process. This implement
34     void R_BSP_WarmStart(bsp_warm_start_event_t event)
35     {
36         if (BSP_WARM_START_RESET == event)
37         {
38             #if BSP_FEATURE_FLASH_LP_VERSION != 0
```

printf 输出重定向到串口

打印最常用的方法是 `printf`，所以要解决的问题是将 `printf` 的输出重定向到串口，然后通过串口将数据发送出去。

注意一定要加上头文件 `#include <stdio.h>`

```
#ifdef __GNUC__ //串口重定向
#define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
#else
#endif

PUTCHAR_PROTOTYPE
{
    err = R_SCI_UART_Write(&g_uart9_ctrl1, (uint8_t *)&ch, 1);
    if(FSP_SUCCESS != err) __BKPT();
    while(uart_send_complete_flag == false){}
    uart_send_complete_flag = false;
    return ch;
}
```

```

int _write(int fd, char *pBuffer, int size)
{
    for(int i=0; i<size; i++)
    {
        __io_putchar(*pBuffer++);
    }

    return size;
}

```

```

1  #include "hal_data.h"
2  #include <stdio.h>
3  FSP_CPP_HEADER
4  void R_BSP_WarmStart(bsp_warm_start_event_t event);
5  FSP_CPP_FOOTER
6
7  fsp_err_t err = FSP_SUCCESS;
8  volatile bool uart_send_complete_flag = false;
9  void user_uart_callback (uart_callback_args_t * p_args)
10 {
11     if(p_args->event == UART_EVENT_TX_COMPLETE)
12     {
13         uart_send_complete_flag = true;
14     }
15 }
16
17
18 #ifdef __GNUC__ //串口重定向
19 #define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
20 #else
21 #define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
22 #endif
23
24
25 PUTCHAR_PROTOTYPE
26 {
27     err = R_SCI_UART_Write(&g_uart9_ctrl1, (uint8_t *)&ch, 1);
28     if(FSP_SUCCESS != err) __BKPT();
29     while(uart_send_complete_flag == false){}
30     uart_send_complete_flag = false;
31     return ch;
32 }
33
34 int _write(int fd, char *pBuffer, int size)
35 {
36     for(int i=0; i<size; i++)
37     {
38         __io_putchar(*pBuffer++);
39     }
40     return size;
41 }
42
43
44
45 * * main() is generated by the RA Configuration editor and is used to generate threads i
46 void hal_entry(void)
47 {
48     /* TODO: add your own code here */
49
50     /* Open the transfer instance with initial configuration. */
51     err = R_SCI_UART_Open(&g_uart9_ctrl1, &g_uart9_cfg);
52     assert(FSP_SUCCESS == err);
53
54
55
56

```

接口配置

RA4M2_MINI	LPS28DFW
3V3	3V3
GND	GND
P301	SCL
P302	SDA
P500	SA0

IIC 属性配置

查看手册,可以得知 LPS28DFW 的 IIC 地址为“1011100” 或者 “1011101”,即 0x5C 或 0x5D。

7.2.1 I²C operation

The transaction on the bus is started through a start (ST) signal. A start condition is defined as a high to low transition on the data line while the SCL line is held high. After the master has transmitted this, the bus is considered busy. The next data byte transmitted after the start condition contains the address of the slave in the first 7 bits and the eighth bit tells whether the master is receiving data from the slave or transmitting data to the slave. When an address is sent, each device in the system compares the first seven bits after a start condition with its address. If they match, the device considers itself addressed by the master.

The 7-bit slave address (SAD) associated to the LPS28DFW is 101110xb. The SA0 (pin #2) pin can be used to modify the less significant bit of the device address. If the SA0 pin is connected to the voltage supply, LSb is "1" (7-bit address 1011101b = 5Dh), otherwise if the SA0 pin is connected to GND, the LSb value is "0" (7-bit address 1011100b = 5Ch). This solution permits connecting and addressing two different LPS28DFW devices to the same I²C lines.

Data transfer with acknowledge is mandatory. The transmitter must release the SDA line during the acknowledge pulse. The receiver must then pull the data line low so that it remains stable low during the high period of the acknowledge clock pulse. A receiver which has been addressed is obliged to generate an acknowledge after each byte of data received.



The I²C embedded inside the ASIC behaves like a slave device and the following protocol must be adhered to. After the start condition (ST) a slave address is sent, once a slave acknowledge has been returned (SAK), an 8-bit subaddress is transmitted (SUB). The IF_ADD_INC bit in CTRL_REG2 (11h) enables subaddress auto increment (IF_ADD_INC is '1' by default), so if IF_ADD_INC = '1' the SUB (subaddress) is automatically increased to allow multiple data read/write.

The slave address is completed with a read/write bit. If the bit is '1' (read), a repeated start (SR) condition must be issued after the two subaddress bytes; if the bit is '0' (write), the master transmits to the slave with direction unchanged. The following table explains how the SAD + read/write bit pattern is composed, listing all the possible configurations.

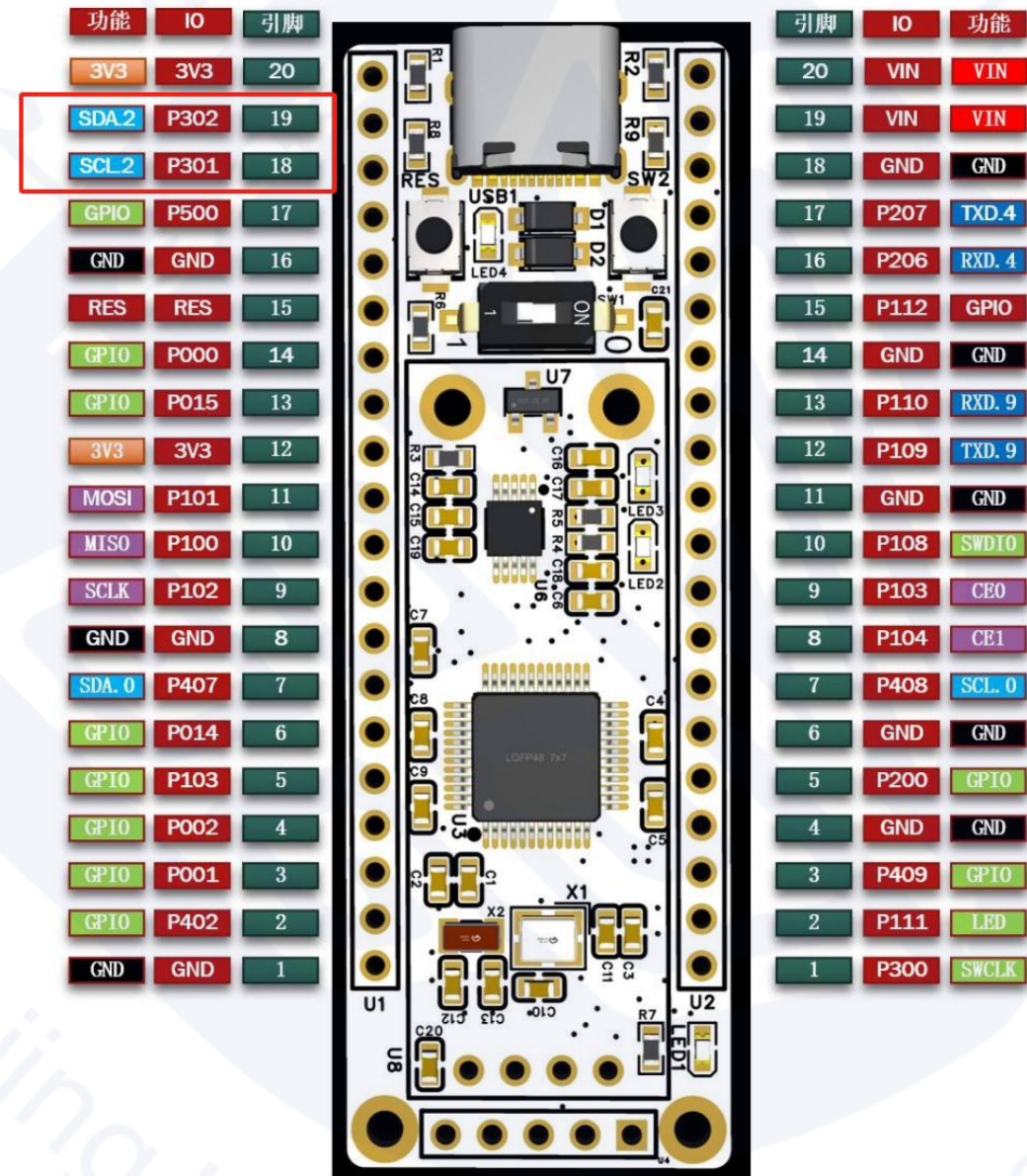
Table 12. SAD + read/write patterns

Command	SAD[6:1]	SAD[0] = SA0	R/W	SAD + R/W
Read	101110	0	1	10111001 (B9h)
Write	101110	0	0	10111000 (B8h)
Read	101110	1	1	10111011 (BBh)
Write	101110	1	0	10111010 (BAh)

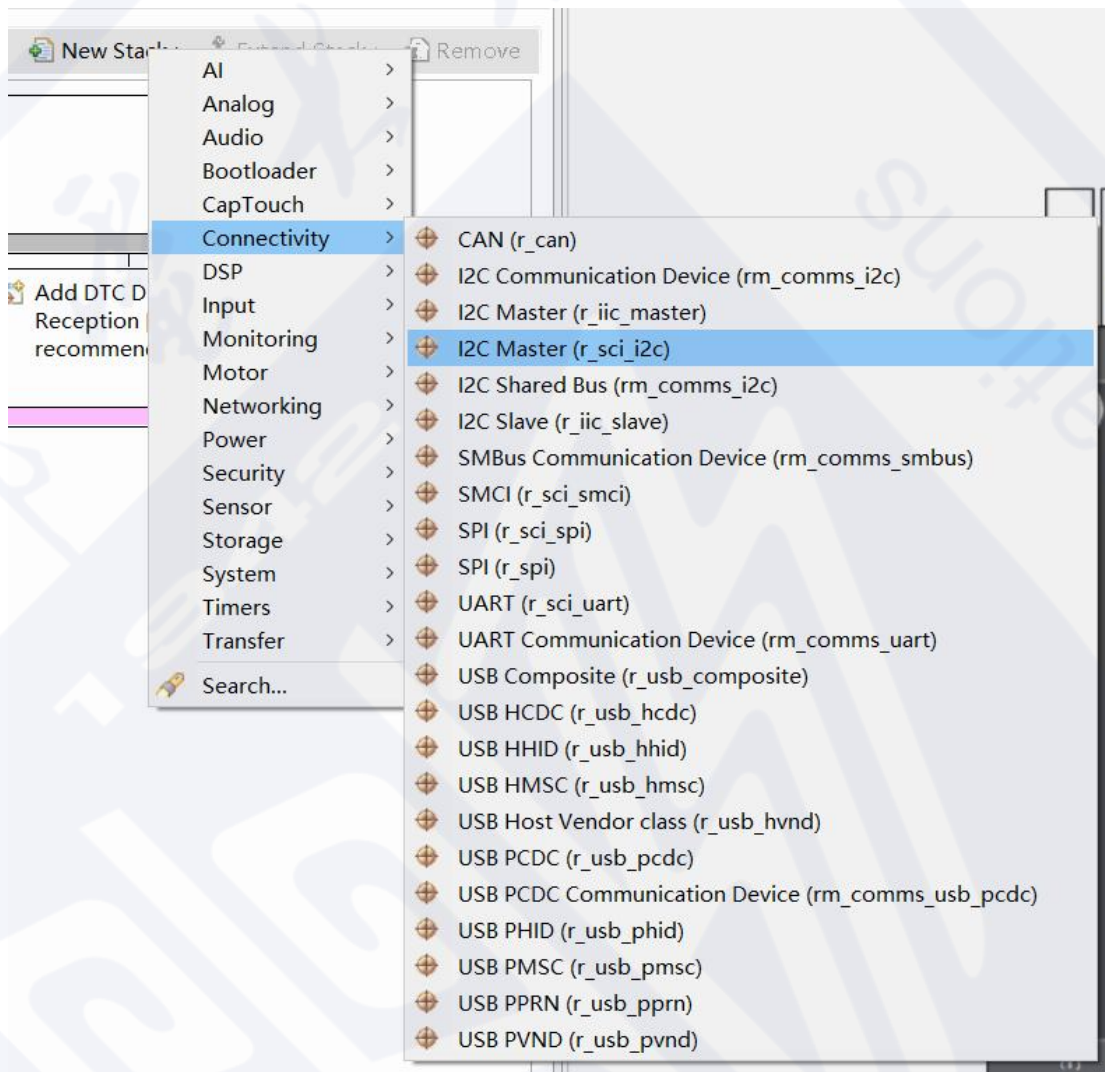
IIC 配置

配置 RA2M2 的 I2C 接口，使其作为 I2C master 进行通信。
查看开发板原理图，对应的 IIC 为 P301 和 P302。

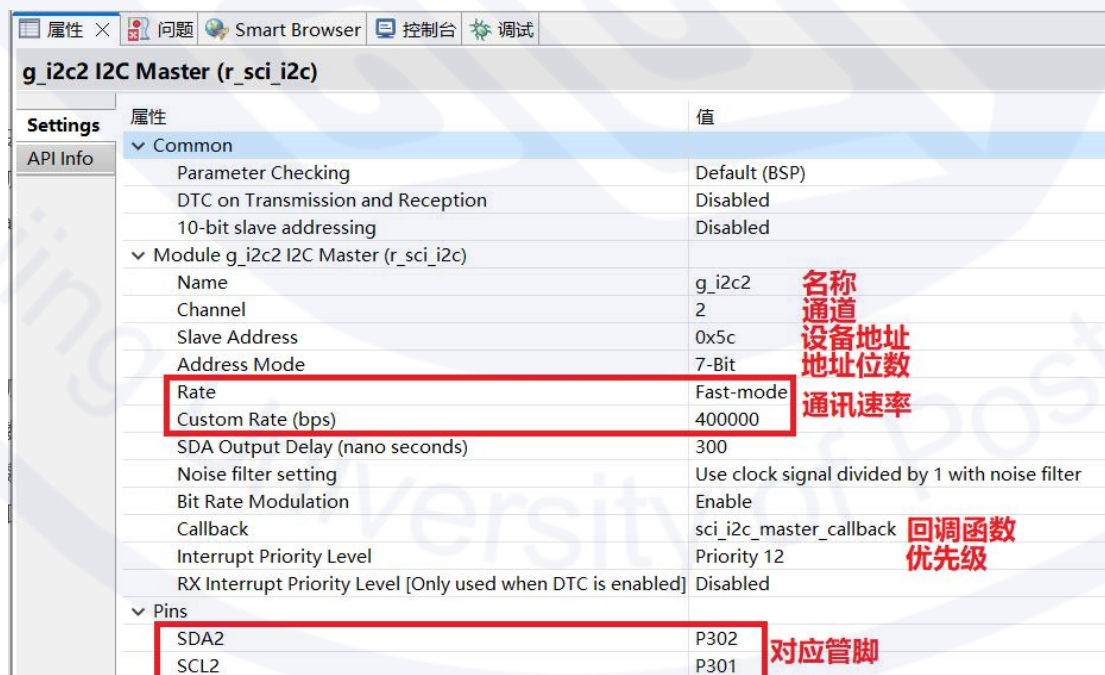
5. IO 预定义

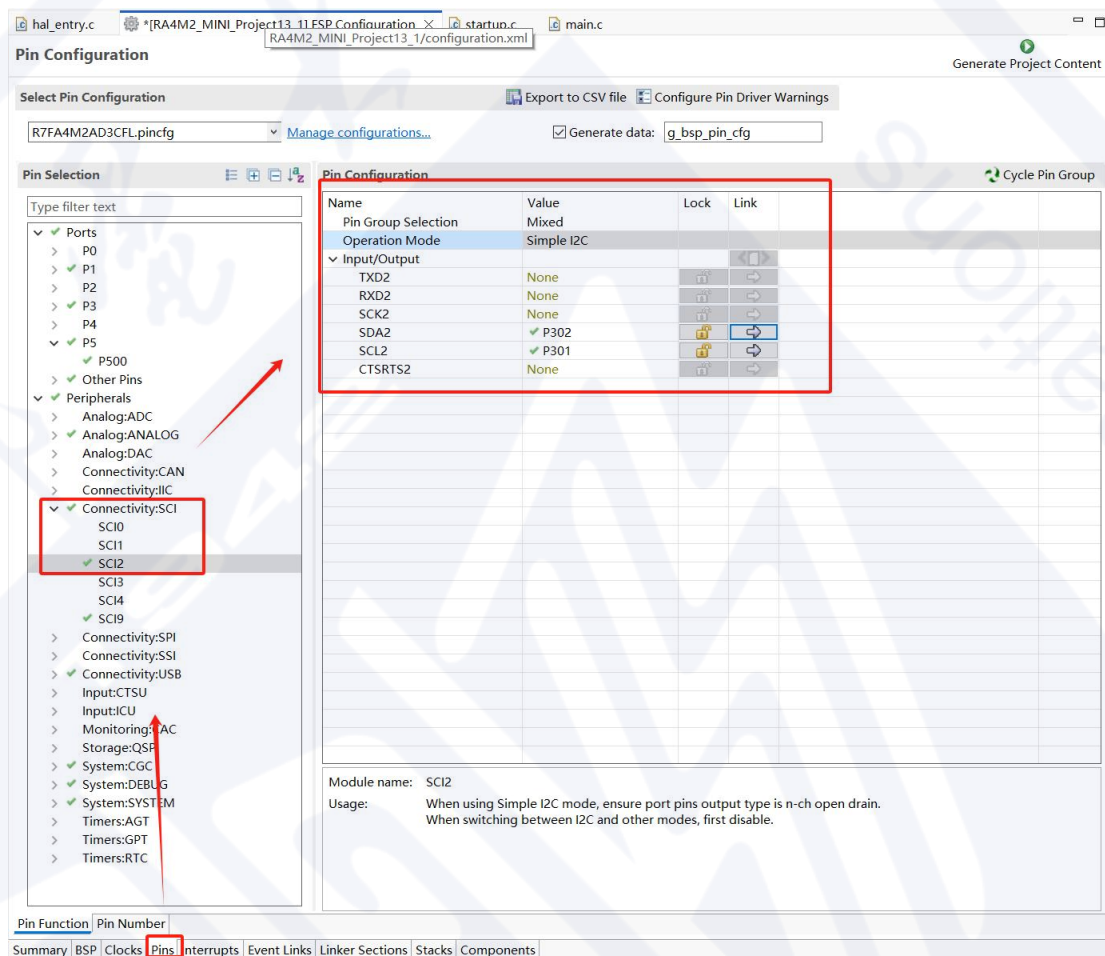


点击 Stacks->New Stack->Connectivity -> I2C Master(r_sci_i2c)。



设置 IIC 的配置，需要注意从机的地址。





R_SCI_I2C_Open() 函数原型

◆ R_SCI_I2C_Open()

```
fsp_err_t R_SCI_I2C_Open ( i2c_master_ctrl_t *const p_api_ctrl,
                           i2c_master_cfg_t const *const p_cfg
                           )
```

Opens the I2C device.

Return values

FSP_SUCCESS	Requested clock rate was set exactly.
FSP_ERR_ALREADY_OPEN	Module is already open.
FSP_ERR_IP_CHANNEL_NOT_PRESENT	Channel is not available on this MCU.
FSP_ERR_ASSERTION	Parameter check failure due to one or more reasons below: 1. p_api_ctrl or p_cfg is NULL. 2. extended parameter is NULL. 3. Callback parameter is NULL. 4. Clock rate requested is greater than 400KHz 5. Invalid IRQ number assigned

R_SCI_I2C_Open()函数为执行 IIC 初始化，开启配置如下所示。

```
/* Initialize the I2C module */
```

```
err = R_SCI_I2C_Open(&g_i2c2_ctrl, &g_i2c2_cfg);

/* Handle any errors. This function should be defined by the user. */

assert(FSP_SUCCESS == err);
```



```
41     }
42
43
44
45  * * main() is generated by the RA Configuration editor and is used to generate threads i
46
47  void hal_entry(void)
48  {
49      /* TODO: add your own code here */
50
51      /* Open the transfer instance with initial configuration. */
52      err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);
53      assert(FSP_SUCCESS == err);
54
55
56      /* Initialize the I2C module */
57      err = R_SCI_I2C_Open(&g_i2c2_ctrl, &g_i2c2_cfg);
58      /* Handle any errors. This function should be defined by the user. */
59      assert(FSP_SUCCESS == err);
60
61
62
63  #if BSP_TZ_SECURE_BUILD
64      /* Enter non-secure code */
65      R_BSP_NonSecureEnter();
66  #endif
67  }
68
69  * * This function is called at various points during the startup process. This implemen
70
71  void R_BSP_WarmStart(bsp_warm_start_event_t event)
72  {
73      if (BSP_WARM_START_RESET == event)
74      {
75          #if BSP_FEATURE_FLASH_LP_VERSION != 0
76
77              /* Enable reading from data flash. */
78              R_FACI_LP->DFLCTL = 1U;
79
80              /* Would normally have to wait tDSTOP(6us) for data flash recovery. Placing the
81               * C runtime initialization, should negate the need for a delay since the initi
82
83          #endif
84      }
85
86      if (BSP_WARM_START_POST_C == event)
87      {
88          /* C runtime environment and system clocks are setup. */
89
90          /* Configure pins. */
91          R_IOPORT_Open (&IOPORT_CFG_CTRL, &IOPORT_CFG_NAME);
92
93          #if BSP_CFG_SDRAM_ENABLED
94
95              /* Setup SDRAM and initialize it. Must configure pins first. */
96              R_BSP_SdramInit(true);
97          #endif
98      }
99  }
```

R_SCI_I2C_Write() 函数原型

◆ R_SCI_I2C_Write()

```
fsp_err_t R_SCI_I2C_Write ( i2c_master_ctrl_t *const p_api_ctrl,  
                           uint8_t *const      p_src,  
                           uint32_t const      bytes,  
                           bool const          restart  
                           )
```

Performs a write to the I2C device.

This function will fail if there is already an in-progress I2C transfer on the associated channel. Otherwise, the I2C write operation will begin. When no callback is provided by the user, this function performs a blocking write. Otherwise, the write operation is non-blocking and the caller will be notified when the operation has finished by an I2C_EVENT_TX_COMPLETE in the callback.

Return values

FSP_SUCCESS	Function executed without issue.
FSP_ERR_ASSERTION	p_ctrl, p_src is NULL.
FSP_ERR_INVALID_SIZE	Provided number of bytes more than uint16_t size (65535) while DTC is used for data transfer.
FSP_ERR_NOT_OPEN	Device was not even opened.

R_SCI_I2C_Write()函数是向 IIC 设备中写入数据，写入格式如下所示。

```
err = R_SCI_I2C_Write(&g_i2c2_ctrl, &reg, 1, true);  
assert(FSP_SUCCESS == err);
```

R_SCI_I2C_Read() 函数原型

◆ R_SCI_I2C_Read()

```
fsp_err_t R_SCI_I2C_Read ( i2c_master_ctrl_t *const p_api_ctrl,  
                           uint8_t *const      p_dest,  
                           uint32_t const      bytes,  
                           bool const          restart  
                           )
```

Performs a read from the I2C device. The caller will be notified when the operation has completed (successfully) by an I2C_MASTER_EVENT_RX_COMPLETE in the callback.

Return values

FSP_SUCCESS	Function executed without issue.
FSP_ERR_ASSERTION	The parameter p_ctrl, p_dest is NULL, bytes is 0.
FSP_ERR_INVALID_SIZE	Provided number of bytes more than uint16_t size (65535) while DTC is used for data transfer.
FSP_ERR_NOT_OPEN	Device was not even opened.

R_SCI_I2C_Read()函数是向 IIC 设备中读取数据，读取格式如下所示。

```
/* Read data from I2C slave */  
err = R_SCI_I2C_Read(&g_i2c2_ctrl, data, len+1, true);  
assert(FSP_SUCCESS == err);
```

sci_i2c_master_callback() 回调函数

对于数据是否发送完毕，可以查看是否获取到 I2C_MASTER_EVENT_TX_COMPLETE 字段。

◆ i2c_master_event_t

enum i2c_master_event_t

Callback events

Enumerator	
I2C_MASTER_EVENT_ABORTED	A transfer was aborted.
I2C_MASTER_EVENT_RX_COMPLETE	A receive operation was completed successfully.
I2C_MASTER_EVENT_TX_COMPLETE	A transmit operation was completed successfully.

```
/* Callback function */
i2c_master_event_t i2c_event = I2C_MASTER_EVENT_ABORTED;
uint32_t timeout_ms = 100000;
void sci_i2c_master_callback(i2c_master_callback_args_t *p_args)
{
    i2c_event = I2C_MASTER_EVENT_ABORTED;
    if (NULL != p_args)
    {
        /* capture callback event for validating the i2c transfer event*/
        i2c_event = p_args->event;
    }
}
```

```
*hal_entry.c x [RA4M2_MINI_Project13_1] FSP Configuration
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80

PUTCHAR_PROTOTYPE
{
    err = R_SCI_UART_Write(&g_uart9_ctrl, (uint8_t *)&ch, 1);
    if(FSP_SUCCESS != err) __BKPT();
    while(uart_send_complete_flag == false){}
    uart_send_complete_flag = false;
    return ch;
}

int _write(int fd, char *pBuffer, int size)
{
    for(int i=0; i<size; i++)
    {
        __io_putchar(*pBuffer++);
    }
    return size;
}

/* Callback function */
i2c_master_event_t i2c_event = I2C_MASTER_EVENT_ABORTED;
uint32_t timeout_ms = 100000;
void sci_i2c_master_callback(i2c_master_callback_args_t *p_args)
{
    i2c_event = I2C_MASTER_EVENT_ABORTED;
    if (NULL != p_args)
    {
        /* capture callback event for validating the i2c transfer event*/
        i2c_event = p_args->event;
    }
}

/* main() is generated by the RA Configuration editor and is used to generate threads i
void hal_entry(void)
{
    /* TODO: add your own code here */

    /* Open the transfer instance with initial configuration. */
    err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);
    assert(FSP_SUCCESS == err);

    /* Initialize the I2C module */
    err = R_SCI_I2C_Open(&g_i2c2_ctrl, &g_i2c2_cfg);
    /* Handle any errors. This function should be defined by the user. */
    assert(FSP_SUCCESS == err);

#ifdef BSP_TZ_SECURE_BUILD
    /* Enter non-secure code */
    R_BSP_NonSecureEnter();
#endif
}
```

SA0 地址设置

通过设置 SA0 管脚的高低电平可以改变模块的地址。

7.2.1

I²C operation

The transaction on the bus is started through a start (ST) signal. A start condition is defined as a high to low transition on the data line while the SCL line is held high. After the master has transmitted this, the bus is considered busy. The next data byte transmitted after the start condition contains the address of the slave in the first 7 bits and the eighth bit tells whether the master is receiving data from the slave or transmitting data to the slave. When an address is sent, each device in the system compares the first seven bits after a start condition with its address. If they match, the device considers itself addressed by the master.

The 7-bit slave address (SAD) associated to the LPS28DFW is 101110xb. The SA0 (pin #2) pin can be used to modify the less significant bit of the device address. If the SA0 pin is connected to the voltage supply, LSb is "1" (7-bit address 1011101b = 5Dh), otherwise if the SA0 pin is connected to GND, the LSb value is "0" (7-bit address 1011100b = 5Ch). This solution permits connecting and addressing two different LPS28DFW devices to the same I²C lines.

Data transfer with acknowledge is mandatory. The transmitter must release the SDA line during the acknowledge pulse. The receiver must then pull the data line low so that it remains stable low during the high period of the acknowledge clock pulse. A receiver which has been addressed is obliged to generate an acknowledge after each byte of data received.



The I²C embedded inside the ASIC behaves like a slave device and the following protocol must be adhered to. After the start condition (ST) a slave address is sent, once a slave acknowledge has been returned (SAK), an 8-bit subaddress is transmitted (SUB). The IF_ADD_INC bit in CTRL_REG2 (11h) enables subaddress auto increment (IF_ADD_INC is '1' by default), so if IF_ADD_INC = '1' the SUB (subaddress) is automatically increased to allow multiple data read/write.

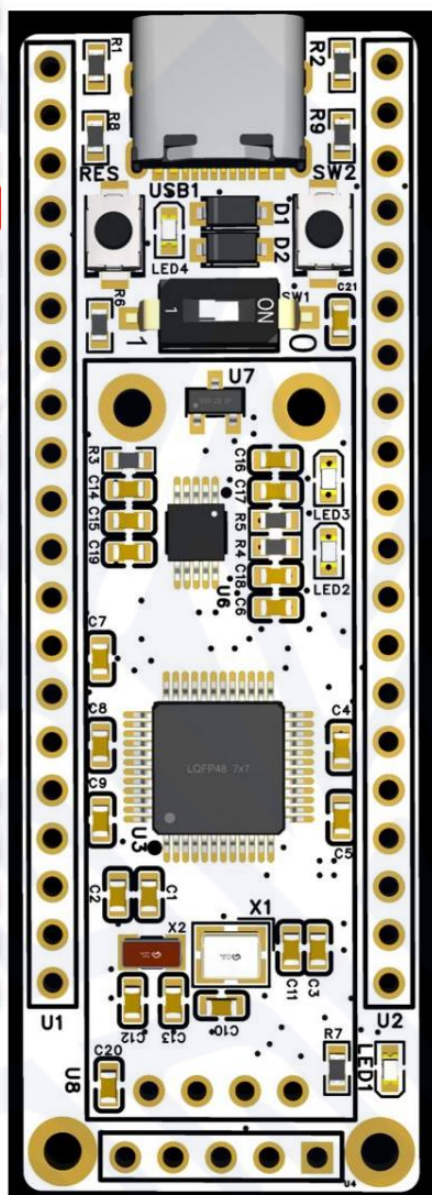
The slave address is completed with a read/write bit. If the bit is '1' (read), a repeated start (SR) condition must be issued after the two subaddress bytes; if the bit is '0' (write), the master transmits to the slave with direction unchanged. The following table explains how the SAD + read/write bit pattern is composed, listing all the possible configurations.

Table 12. SAD + read/write patterns

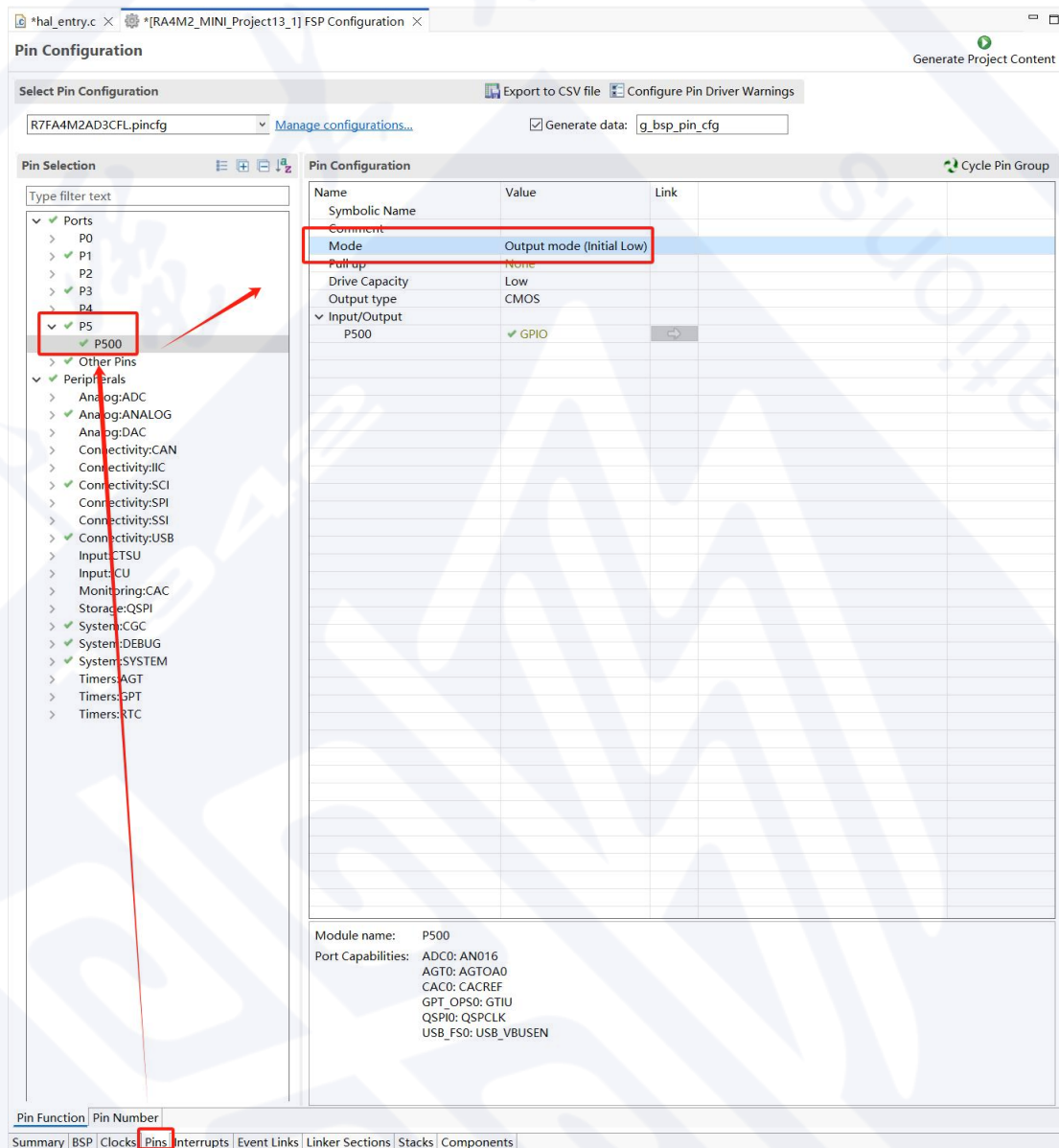
Command	SAD[6:1]	SAD[0] = SA0	R/W	SAD + R/W
Read	101110	0	1	10111001 (B9h)
Write	101110	0	0	10111000 (B8h)
Read	101110	1	1	10111011 (BBh)
Write	101110	1	0	10111010 (BAh)

这里设置 P500 驱动 SA0 管脚位输出管脚。

功能	IO	引脚
3V3	3V3	20
SDA_2	P302	19
SCL_2	P301	18
GPIO	P500	17
GND	GND	16
RES	RES	15
GPIO	P000	14
GPIO	P015	13
3V3	3V3	12
MOSI	P101	11
MISO	P100	10
SCLK	P102	9
GND	GND	8
SDA_0	P407	7
GPIO	P014	6
GPIO	P103	5
GPIO	P002	4
GPIO	P001	3
GPIO	P402	2
GND	GND	1

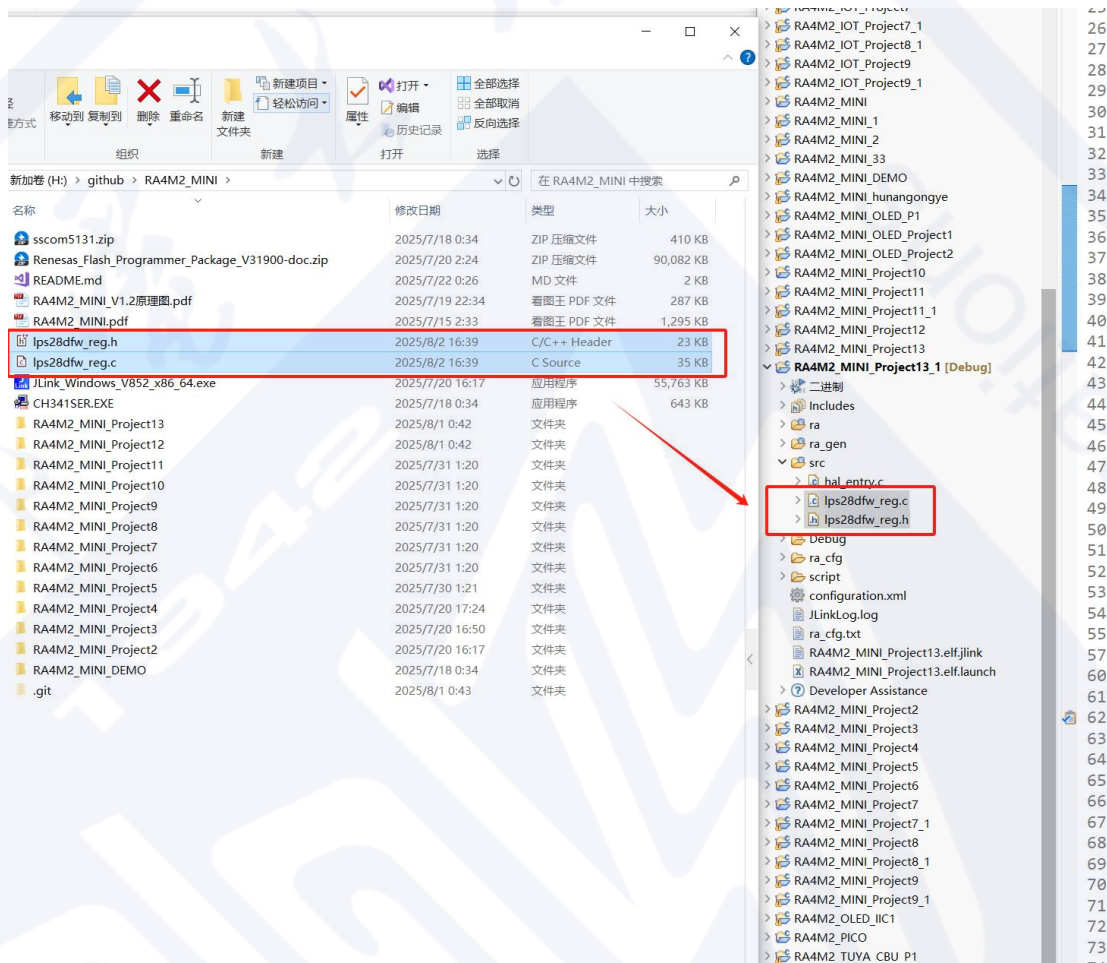


引脚	IO	功能
20	VIN	VIN
19	VIN	VIN
18	GND	GND
17	P207	TXD.4
16	P206	RXD.4
15	P112	GPIO
14	GND	GND
13	P110	RXD.9
12	P109	TXD.9
11	GND	GND
10	P108	SWDIO
9	P103	CE0
8	P104	CE1
7	P408	SCL.0
6	GND	GND
5	P200	GPIO
4	GND	GND
3	P409	GPIO
2	P111	LED
1	P300	SWCLK



参考程序

<https://github.com/STMicroelectronics/lps28dfw-pid>



添加头文件

```
#include "lps28dfw_reg.h"
```

```
*hal_entry.c x [RA4M2_MINI_Project13_1] FSP Configuration
1  #include "hal_data.h"
2  #include <stdio.h>
3  #include "lps28dfw_reg.h"
4  FSP_CPP_HEADER
5  void R_BSP_WarmStart(bsp_warm_start_event_t event);
6  FSP_CPP_FOOTER
7
8  fsp_err_t err = FSP_SUCCESS;
9  volatile bool uart_send_complete_flag = false;
10 void user_uart_callback (uart_callback_args_t * p_args)
11 {
12     if(p_args->event == UART_EVENT_TX_COMPLETE)
13     {
14         uart_send_complete_flag = true;
15     }
16 }
17
18
19 #ifdef __GNUC__ //串口重定向
20 #define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
21 #else
22 #define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
23 #endif
24
25
26 PUTCHAR_PROTOTYPE
27 {
28     err = R_SCI_UART_Write(&g_uart9_ctrl1, (uint8_t *)&ch, 1);
29     if(FSP_SUCCESS != err) __BKPT();
30     while(uart_send_complete_flag == false){}
31     uart_send_complete_flag = false;
32     return ch;
33 }
34
35 int _write(int fd, char *pBuffer, int size)
36 {
37     for(int i=0; i<size; i++)
38     {
39         __io_putchar(*pBuffer++);
40     }
41     return size;
42 }
43
44 /* Callback function */
45 i2c_master_event_t i2c_event = I2C_MASTER_EVENT_ABORTED;
46 uint32_t timeout_ms = 100000;
47 void sci_i2c_master_callback(i2c_master_callback_args_t *p_args)
48 {
49     i2c_event = I2C_MASTER_EVENT_ABORTED;
50     if (NULL != p_args)
51     {
52         /* capture callback event for validating the i2c transfer event*/
53     }
54 }
```

添加变量

```
/* Private macro -----*/
#define BOOT_TIME 10 //ms
#define SENSOR_BUS g_i2c2_ctrl
/* Private variables -----*/
static uint8_t tx_buffer[1000];
static lps28dfw_data_t data;

/* Extern variables -----*/

/* Private functions -----*/
```

```

/*
 * WARNING:
 * Functions declare in this section are defined at the end of this file
 * and are strictly related to the hardware platform used.
 *
 */

static int32_t platform_write(void *handle, uint8_t reg, const uint8_t *bufp,
                               uint16_t len);

static int32_t platform_read(void *handle, uint8_t reg, uint8_t *bufp,
                              uint16_t len);

static void tx_com( uint8_t *tx_buffer, uint16_t len );

static void platform_delay(uint32_t ms);

static void platform_init(void);

```

```

28     err = R_SCI_UART_Write(&g_uart9_ctrl, (uint8_t *)&ch, 1);
29     if(FSP_SUCCESS != err) __BKPT();
30     while(uart_send_complete_flag == false){}
31     uart_send_complete_flag = false;
32     return ch;
33 }
34
35 int _write(int fd, char *pBuffer, int size)
36 {
37     for(int i=0; i<size; i++)
38     {
39         __io_putchar(*pBuffer++);
40     }
41     return size;
42 }
43
44 /* Callback function */
45 i2c_master_event_t i2c_event = I2C_MASTER_EVENT_ABORTED;
46 uint32_t timeout_ms = 100000;
47 void sci_i2c_master_callback(i2c_master_callback_args_t *p_args)
48 {
49     i2c_event = I2C_MASTER_EVENT_ABORTED;
50     if (NULL != p_args)
51     {
52         /* capture callback event for validating the i2c transfer event*/
53         i2c_event = p_args->event;
54     }
55 }
56
57 /* Private macro -----*/
58 #define BOOT_TIME 10 //ms
59 #define SENSOR_BUS g_i2c2_ctrl
60 /* Private variables -----*/
61 static uint8_t tx_buffer[1000];
62 static lps28dfw_data_t data;
63
64 /* Extern variables -----*/
65
66 /* Private functions -----*/
67
68 /*
69  * WARNING:
70  * Functions declare in this section are defined at the end of this file
71  * and are strictly related to the hardware platform used.
72  *
73  */
74 static int32_t platform_write(void *handle, uint8_t reg, const uint8_t *bufp,
75                               uint16_t len);
76 static int32_t platform_read(void *handle, uint8_t reg, uint8_t *bufp,
77                              uint16_t len);
78 static void tx_com( uint8_t *tx_buffer, uint16_t len );
79 static void platform_delay(uint32_t ms);
80 static void platform_init(void);
81
82 /* * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used.
83 void hal_entry(void)
84 {
85     /* TODO: add your own code here */
86
87     /* Open the transfer instance with initial configuration. */
88     err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);
89     assert(FSP_SUCCESS == err);
90
91     /* Initialize the I2C module */
92     err = R_SCI_I2C_Open(&g_i2c2_ctrl, &g_i2c2_cfg);

```

添加 IIC 驱动代码

```
/*
 * @brief Write generic device register (platform dependent)
 *
 * @param handle    customizable argument. In this examples is used in
 *                  order to select the correct sensor bus handler.
 * @param reg       register to write
 * @param bufp      pointer to data to write in register reg
 * @param len       number of consecutive register to write
 */
static int32_t platform_write(void *handle, uint8_t reg, const uint8_t *bufp, uint16_t len)
{
    // 创建一个足够大的缓冲区来包含寄存器地址和数据
    uint8_t data[len + 1];
    data[0] = reg; // 将寄存器地址放在数据的开始
    memcpy(&data[1], bufp, len); // 复制数据到缓冲区

    err = R_SCI_I2C_Write(&g_i2c2_ctrl, data, len+1, true);
    assert(FSP_SUCCESS == err);
    /* Since there is nothing else to do, block until Callback triggers*/
    //while ((I2C_MASTER_EVENT_TX_COMPLETE != i2c_event) && timeout_ms)
    while ((I2C_MASTER_EVENT_TX_COMPLETE != i2c_event) && timeout_ms>0)
    {
        R_BSP_SoftwareDelay(1U, BSP_DELAY_UNITS_MICROSECONDS);
        timeout_ms--;
    }
    if (I2C_MASTER_EVENT_ABORTED == i2c_event)
    {
        __BKPT(0);
    }
    /* Read data back from the I2C slave */
    i2c_event = I2C_MASTER_EVENT_ABORTED;
    timeout_ms = 100000;
    return 0;
}

/*
 * @brief Read generic device register (platform dependent)
 *
 * @param handle    customizable argument. In this examples is used in
```

```

*          order to select the correct sensor bus handler.
* @param reg      register to read
* @param bufp     pointer to buffer that store the data read
* @param len      number of consecutive register to read
*
*/
static int32_t platform_read(void *handle, uint8_t reg, uint8_t *bufp, uint16_t len)
{
    err = R_SCI_I2C_Write(&g_i2c2_ctrl, &reg, 1, true);
    assert(FSP_SUCCESS == err);

    /* Since there is nothing else to do, block until Callback triggers*/
    //while ((I2C_MASTER_EVENT_TX_COMPLETE != i2c_event) && timeout_ms)
    while ((I2C_MASTER_EVENT_TX_COMPLETE != i2c_event) && timeout_ms>0)
    {
        R_BSP_SoftwareDelay(1U, BSP_DELAY_UNITS_MICROSECONDS);
        timeout_ms--;
    }

    if (I2C_MASTER_EVENT_ABORTED == i2c_event)
    {
        __BKPT(0);
    }

    /* Read data back from the I2C slave */
    i2c_event = I2C_MASTER_EVENT_ABORTED;
    timeout_ms = 100000;

    /* Read data from I2C slave */
    err = R_SCI_I2C_Read(&g_i2c2_ctrl, bufp, len, false);
    assert(FSP_SUCCESS == err);
    while ((I2C_MASTER_EVENT_RX_COMPLETE != i2c_event) && timeout_ms)
    {
        R_BSP_SoftwareDelay(1U, BSP_DELAY_UNITS_MILLISECONDS);
        timeout_ms--;
    }

    if (I2C_MASTER_EVENT_ABORTED == i2c_event)
    {
        __BKPT(0);
    }

    i2c_event = I2C_MASTER_EVENT_ABORTED;
    timeout_ms = 100000;
    return 0;
}

```

```

/*
 * @brief platform specific delay (platform dependent)
 *
 * @param ms      delay in ms
 *
 */

static void platform_delay(uint32_t ms)
{
    R_BSP_SoftwareDelay(ms, BSP_DELAY_UNITS_MILLISECONDS);
}

```

```

113
114 /* Wait sensor boot time */
115 platform_delay(BOOT_TIME);
116
117 #if BSP_TZ_SECURE_BUILD
118 /* Enter non-secure code */
119 R_BSP_NonSecureEnter();
120 #endif
121
122
123 /*
124 * @brief Write generic device register (platform dependent)
125 *
126 * @param handle      customizable argument. In this examples is used in
127 *                    order to select the correct sensor bus handler.
128 * @param reg         register to write
129 * @param bufp        pointer to data to write in register reg
130 * @param len         number of consecutive register to write
131 *
132 */
133
134 static int32_t platform_write(void *handle, uint8_t reg, const uint8_t *bufp, uint16_t len)
135 {
136     // 这里是一个足够大的缓冲区来存放寄存器数据
137     uint8_t data[len + 1];
138     data[0] = reg; // 寄存器地址放在数据的首个字节
139     memcpy(&data[1], bufp, len); // 把数据复制到缓冲区
140
141     err = R_SCI_I2C_Write(&i2c2_ctrl, data, len+1, true);
142     assert(FSP_SUCCESS == err);
143     /* Since there is nothing else to do, block until Callback triggers */
144     //while ((I2C_MASTER_EVENT_TX_COMPLETE != i2c_event) && timeout_ms)
145     while ((I2C_MASTER_EVENT_TX_COMPLETE != i2c_event) && timeout_ms > 0)
146     {
147         R_BSP_SoftwareDelay(1U, BSP_DELAY_UNITS_MICROSECONDS);
148         timeout_ms--;
149     }
150     if (I2C_MASTER_EVENT_ABORTED == i2c_event)
151     {
152         __BKPT(0);
153     }
154     /* Read data back from the I2C slave */
155     i2c_event = I2C_MASTER_EVENT_ABORTED;
156     timeout_ms = 1000000;
157     return 0;
158 }
159
160 /*
161 * @brief Read generic device register (platform dependent)
162 *
163 * @param handle      customizable argument. In this examples is used in
164 *                    order to select the correct sensor bus handler.
165 * @param reg         register to read
166 * @param bufp        pointer to buffer that store the data read
167 * @param len         number of consecutive register to read
168 *
169 */
170
171 static int32_t platform_read(void *handle, uint8_t reg, uint8_t *bufp, uint16_t len)
172 {
173     err = R_SCI_I2C_Write(&i2c2_ctrl, &reg, 1, true);
174     assert(FSP_SUCCESS == err);
175     /* Since there is nothing else to do, block until Callback triggers */
176     //while ((I2C_MASTER_EVENT_TX_COMPLETE != i2c_event) && timeout_ms)
177     while ((I2C_MASTER_EVENT_TX_COMPLETE != i2c_event) && timeout_ms > 0)
178     {
179         R_BSP_SoftwareDelay(1U, BSP_DELAY_UNITS_MICROSECONDS);
180         timeout_ms--;
181     }
182     if (I2C_MASTER_EVENT_ABORTED == i2c_event)
183     {
184         __BKPT(0);
185     }
186     /* Read data back from the I2C slave */
187     i2c_event = I2C_MASTER_EVENT_ABORTED;
188     timeout_ms = 1000000;
189     /* Read data from I2C slave */
190     err = R_SCI_I2C_Read(&i2c2_ctrl, bufp, len, false);
191     assert(FSP_SUCCESS == err);
192     while ((I2C_MASTER_EVENT_RX_COMPLETE != i2c_event) && timeout_ms)
193     {
194         R_BSP_SoftwareDelay(1U, BSP_DELAY_UNITS_MILLISECONDS);
195         timeout_ms--;
196     }
197     if (I2C_MASTER_EVENT_ABORTED == i2c_event)
198     {
199         __BKPT(0);
200     }
201     i2c_event = I2C_MASTER_EVENT_ABORTED;
202     timeout_ms = 1000000;
203     return 0;
204 }
205
206
207 /*
208 * @brief platform specific delay (platform dependent)

```

SA0 设置模块地址

使能 SA0 为低电平，配置模块地址。

Table 12. SAD + read/write patterns

Command	SAD[6:1]	SAD[0] = SA0	R/W	SAD + R/W
Read	101110	0	1	10111001 (B9h)
Write	101110	0	0	10111000 (B8h)
Read	101110	1	1	10111011 (BBh)
Write	101110	1	0	10111010 (BAh)

```
R_IOPORT_PinWrite(&g_ioport_ctrl, BSP_IO_PORT_05_PIN_00, BSP_IO_LEVEL_LOW);
```

```
lps28dfw_pin_int_route_t int_route;
```

```
lps28dfw_all_sources_t all_sources;
```

```
lps28dfw_bus_mode_t bus_mode;
```

```
lps28dfw_stat_t status;
```

```
stmdev_ctx_t dev_ctx;
```

```
lps28dfw_id_t id;
```

```
lps28dfw_md_t md;
```

```
/* Initialize mems driver interface */
```

```
dev_ctx.write_reg = platform_write;
```

```
dev_ctx.read_reg = platform_read;
```

```
dev_ctx.handle = &SENSOR_BUS;
```

```
/* Wait sensor boot time */
```

```
platform_delay(BOOT_TIME);
```

```

76         uint16_t len);
77 static void tx_com( uint8_t *tx_buffer, uint16_t len );
78 static void platform_delay(uint32_t ms);
79 static void platform_init(void);
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142

```

```

static void tx_com( uint8_t *tx_buffer, uint16_t len );
static void platform_delay(uint32_t ms);
static void platform_init(void);

/* main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used. */
void hal_entry(void)
{
    /* TODO: add your own code here */

    /* Open the transfer instance with initial configuration. */
    err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);
    assert(FSP_SUCCESS == err);

    /* Initialize the I2C module */
    err = R_SCI_I2C_Open(&g_i2c2_ctrl, &g_i2c2_cfg);
    /* Handle any errors. This function should be defined by the user. */
    assert(FSP_SUCCESS == err);

    R_IOPORT_PinWrite(&g_ioport_ctrl, BSP_IO_PORT_05_PIN_00, BSP_IO_LEVEL_LOW);

    lps28dfw_pin_int_route_t int_route;
    lps28dfw_all_sources_t all_sources;
    lps28dfw_bus_mode_t bus_mode;
    lps28dfw_stat_t status;
    stmdev_ctx_t dev_ctx;
    lps28dfw_id_t id;
    lps28dfw_md_t md;

    /* Initialize mems driver interface */
    dev_ctx.write_reg = platform_write;
    dev_ctx.read_reg = platform_read;
    dev_ctx.handle = &SENSOR_BUS;

    /* Wait sensor boot time */
    platform_delay(BOOT_TIME);

    #if BSP_TZ_SECURE_BUILD
    /* Enter non-secure code */
    R_BSP_NonSecureEnter();
    #endif
}

/*
 * @brief Write generic device register (platform dependent)
 *
 * @param handle customizable argument. In this examples is used in
 * order to select the correct sensor bus handler.
 * @param reg register to write
 * @param bufp pointer to data to write in register reg
 * @param len number of consecutive register to write
 */
static int32_t platform_write(void *handle, uint8_t reg, const uint8_t *bufp, uint16_t len)
{
    // 创建一个足够大的缓冲区来包含寄存器地址和数据
    uint8_t data[len + 1];
    data[0] = reg; // 将寄存器地址放在数据的开始
    memcpy(&data[1], bufp, len); // 复制数据到缓冲区

    err = R_SCI_I2C_Write(&g_i2c2_ctrl, data, len+1, true);
    assert(FSP_SUCCESS == err);
    /* Since there is nothing else to do, block until Callback triggers*/
}

```

获取 ID

可以向 WHO_AM_I (0Fh)获取固定值，判断是否为 0xB4

9.5 WHO_AM_I (0Fh)

Device Who am I

7	6	5	4	3	2	1	0
1	0	1	1	0	1	0	0

lps28dfw_id_get 为获取函数。

```

135  /**
136   * @brief Device "Who am I".[get]
137   *
138   * @param ctx    communication interface handler. (ptr)
139   * @param val    ID values. (ptr)
140   * @retval      interface status (MANDATORY: return 0 -> no Error)
141   *
142   */
143  int32_t lps28dfw_id_get(stmdev_ctx_t *ctx, lps28dfw_id_t *val)
144  {
145      uint8_t reg;
146      int32_t ret;
147
148      ret = lps28dfw_read_reg(ctx, LPS28DFW_WHO_AM_I, &reg, 1);
149      val->whoami = reg;
150
151      return ret;
152  }
153

```

对应的获取 ID 驱动程序,如下所示。

```

/* Check device ID */
lps28dfw_id_get(&dev_ctx, &id);

printf("LPS28DFW_ID=0x%x,id.whoami=0x%x\n", LPS28DFW_ID, id.whoami);

if (id.whoami != LPS28DFW_ID)
    while(1);

```

```

*hal_entry.c x [RA4M2_MINI_Project13_1] FSP Configuration
76 RA4M2_MINI_Project13_1/src/hal_entry.c uint16_t len);
77 static void tx_com(uint8_t *tx_buffer, uint16_t len);
78 static void platform_delay(uint32_t ms);
79 static void platform_init(void);
80
82 * main() is generated by the RA Configuration editor and is used to generate threads if an RTOS is used.
83 void hal_entry(void)
84 {
85     /* TODO: add your own code here */
86
87     /* Open the transfer instance with initial configuration. */
88     err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);
89     assert(FSP_SUCCESS == err);
90
91     /* Initialize the I2C module */
92     err = R_SCI_I2C_Open(&g_i2c2_ctrl, &g_i2c2_cfg);
93     /* Handle any errors. This function should be defined by the user. */
94     assert(FSP_SUCCESS == err);
95
96     R_IOPORT_PinWrite(&g_ioport_ctrl, BSP_IO_PORT_05_PIN_00, BSP_IO_LEVEL_LOW);
97
98     lps28dfw_pin_int_route_t int_route;
99     lps28dfw_all_sources_t all_sources;
100     lps28dfw_bus_mode_t bus_mode;
101     lps28dfw_stat_t status;
102     stmdev_ctx_t dev_ctx;
103     lps28dfw_id_t id;
104     lps28dfw_md_t md;
105
106     /* Initialize mems driver interface */
107     dev_ctx.write_reg = platform_write;
108     dev_ctx.read_reg = platform_read;
109     dev_ctx.handle = &SENSOR_BUS;
110
111     /* Wait sensor boot time */
112     platform_delay(BOOT_TIME);
113
114     /* Check device ID */
115     lps28dfw_id_get(&dev_ctx, &id);
116     printf("LPS28DFW_ID=0x%x, id.whoami=0x%x\n", LPS28DFW_ID, id.whoami);
117
118     if (id.whoami != LPS28DFW_ID)
119         while(1);
120
121 #if BSP_TZ_SECURE_BUILD
122     /* Enter non-secure code */
123     R_BSP_NonSecureEnter();
124 #endif
125
126 /*
127  * @brief Write generic device register (platform dependent)
128  *
129  * @param handle customizable argument. In this examples is used in
130  * order to select the correct sensor bus handler.
131  * @param reg register to write
132  * @param bufp pointer to data to write in register reg
133  * @param len number of consecutive register to write
134  */
135 static int32_t platform_write(void *handle, uint8_t reg, const uint8_t *bufp, uint16_t len)
136 {
137     // 创建一个足够大的缓冲区来包含寄存器地址和数据
138 }

```



轮询读取数据

对于压强和温度数据是否准备好，可以查看 STATUS (27h)的 Zyxda 位，判断是否有新数据到达。



9.20 STATUS (27h)

Status register (read only)

7	6	5	4	3	2	1	0
-	-	T_OR	P_OR	-	-	T_DA	P_DA

T_OR	Temperature data overrun. (0: no overrun has occurred; 1: new data for temperature has overwritten the previous data)
P_OR	Pressure data overrun. (0: no overrun has occurred; 1: new data for pressure has overwritten the previous data)
T_DA	Temperature data available. (0: new data for temperature is not yet available; 1: new temperature data is generated)
P_DA	Pressure data available. (0: new data for pressure is not yet available; 1: new pressure data is generated)

This register is updated every ODR cycle.

对于压强数据，主要在 PRESS_OUT_XL (28h)-PRESS_OUT_H (2Ah)。

9.21 PRESS_OUT_XL (28h)

Pressure output value LSB data (read only)

7	6	5	4	3	2	1	0
POUT7	POUT6	POUT5	POUT4	POUT3	POUT2	POUT1	POUT0

POUT[7:0]	This register contains the low part of the pressure output value.
-----------	---

The pressure output value is a 24-bit data that contains the measured pressure. It is composed of [PRESS_OUT_H \(2Ah\)](#), [PRESS_OUT_L \(29h\)](#) and [PRESS_OUT_XL \(28h\)](#). The value is expressed as 2's complement.

The output pressure register **PRESS_OUT** is provided as the difference between the measured pressure and the content of the register **RPDS** (18h, 19h).

This register contains the pressure value and the resolution is: 1 LSB = 1/4096 hPa (when **FS_MODE** = 0), otherwise 1 LSB = 1/2048 hPa (when **FS_MODE** = 1). Please refer to [Section 4.5 Interpreting pressure readings](#) for additional information.

9.22 PRESS_OUT_L (29h)

Pressure output value middle data (read only)

7	6	5	4	3	2	1	0
POUT15	POUT14	POUT13	POUT12	POUT11	POUT10	POUT9	POUT8

POUT[15:8]	This register contains the mid part of the pressure output value. Refer to PRESS_OUT_XL (28h)
------------	--



9.23 PRESS_OUT_H (2Ah)

Pressure output value MSB data (read only)

7	6	5	4	3	2	1	0
POUT23	POUT22	POUT21	POUT20	POUT19	POUT18	POUT17	POUT16

POUT[23:16]	This register contains the high part of the pressure output value. Refer to PRESS_OUT_XL (28h)
-------------	---

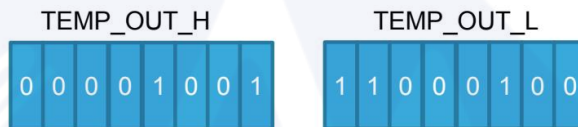
温度完整数据为 0x0806，换算为十进制为 2054，具体度为 2054/100°C=20.54°C

4.6 Interpreting temperature readings

The temperature data are stored in 2 registers: TEMP_OUT_H (2Ch) and TEMP_OUT_L (2Bh).

The value is expressed as 2's complement. To obtain the temperature in °C, take the two's complement of the complete 16-bit word and then divide by the sensitivity 100 LSB/°C.

Figure 7. Temperature readings



Temperature value (LSB) = TEMP_OUT_H (2Ch) & TEMP_OUT_L (2Bh)
= 09C4 = 2500 LSB (decimal signed)

$$\text{Temperature (}^{\circ}\text{C)} = \frac{\text{Temperature value (LSB)}}{\text{Sensitivity}} = \frac{2500 \text{ LSB}}{100 \text{ LSB}/^{\circ}\text{C}} = 25.00^{\circ}\text{C}$$

```
/* Restore default configuration */
lps28dfw_init_set(&dev_ctx, LPS28DFW_RESET);

do {
    lps28dfw_status_get(&dev_ctx, &status);
} while (status.sw_reset);

/* Set bdu and if_inc recommended for driver usage */
lps28dfw_init_set(&dev_ctx, LPS28DFW_DRV_RDY);

/* Select bus interface */
bus_mode.filter = LPS28DFW_AUTO;
bus_mode.interface = LPS28DFW_SEL_BY_HW;
lps28dfw_bus_mode_set(&dev_ctx, &bus_mode);

/* Set Output Data Rate */
md.odr = LPS28DFW_4Hz;
md.avg = LPS28DFW_16_AVG;
md.lpf = LPS28DFW_LPF_ODR_DIV_4;
md.fs = LPS28DFW_1260hPa;
lps28dfw_mode_set(&dev_ctx, &md);

/* Configure interrupt pins */
lps28dfw_pin_int_route_get(&dev_ctx, &int_route);
int_route.drdy_pres = PROPERTY_DISABLE;
lps28dfw_pin_int_route_set(&dev_ctx, &int_route);
```

```
/* Read samples in polling mode (no int) */
while(1)
{
    /* Read output only if new values are available */
    lps28dfw_all_sources_get(&dev_ctx, &all_sources);
    if ( all_sources.drdy_pres | all_sources.drdy_temp ) {
        lps28dfw_data_get(&dev_ctx, &md, &data);
        printf( "pressure [hPa]:%.2f temperature [degC]:%.2f\r\n",
                data.pressure.hpa, data.heat.deg_c);
    }
}
```

```
100 lps28dfw_pin_int_route_t int_route;
101 lps28dfw_all_sources_t all_sources;
102 lps28dfw_bus_mode_t bus_mode;
103 lps28dfw_stat_t status;
104 stmdev_ctx_t dev_ctx;
105 lps28dfw_id_t id;
106 lps28dfw_md_t md;
107
108 /* Initialize mems driver interface */
109 00000328 dev_ctx.write_reg = platform_write;
110 dev_ctx.read_reg = platform_read;
111 00000336 dev_ctx.handle = &SENSOR_BUS;
112
113
114 /* Wait sensor boot time */
115 platform_delay(BOOT_TIME);
116
117 /* Check device ID */
118 0000033c lps28dfw_id_get(&dev_ctx, &id);
119 00000344 printf("LPS28DFW_ID=0x%x,id.whoami=0x%x\n",LPS28DFW_ID,id.whoami);
120
121 00000350 if (id.whoami != LPS28DFW_ID)
122 00000358 while(1);
123
124 /* Restore default configuration */
125 0000035a lps28dfw_init_set(&dev_ctx, LPS28DFW_RESET);
126 do {
127 00000362 lps28dfw_status_get(&dev_ctx, &status);
128 0000036a } while (status.sw_reset);
129
130 /* Set bdu and if_inc recommended for driver usage */
131 00000374 lps28dfw_init_set(&dev_ctx, LPS28DFW_DRV_RDY);
132
133 /* Select bus interface */
134 bus_mode.filter = LPS28DFW_AUTO;
135 0000037c bus_mode.interface = LPS28DFW_SEL_BY_HW;
136 00000384 lps28dfw_bus_mode_set(&dev_ctx, &bus_mode);
137
138 /* Set Output Data Rate */
139 md.odr = LPS28DFW_4Hz;
140 md.avg = LPS28DFW_16_AVG;
141 md.lpf = LPS28DFW_LPF_ODR_DIV_4;
142 00000388 md.fs = LPS28DFW_1260hPa;
143 00000390 lps28dfw_mode_set(&dev_ctx, &md);
144
145 /* Configure interrupt pins */
146 00000394 lps28dfw_pin_int_route_get(&dev_ctx, &int_route);
147 0000039c int_route.drdy_pres = PROPERTY_DISABLE;
148 000003ac lps28dfw_pin_int_route_set(&dev_ctx, &int_route);
149
150 /* Read samples in polling mode (no int) */
151 while(1)
152 {
153
154 /* Read output only if new values are available */
155 000003b4 lps28dfw_all_sources_get(&dev_ctx, &all_sources);
156 000003bc if ( all_sources.drdy_pres | all_sources.drdy_temp ) {
157 000003c8 lps28dfw_data_get(&dev_ctx, &md, &data);
158 000003d2 printf( "pressure [hPa]:%6.2f temperature [degC]:%6.2f\r\n",
159 data.pressure.hpa, data.heat.deg_c);
160 }
161 }
162
163 #if BSP_TZ_SECURE_BUILD
164 /* Enter non-secure code */
165 R_BSP_NonSecureEnter();
166 #endif
167 }
168
169
170 /*
171 * @brief Write generic device register (platform dependent)
172 *
173 * @param handle customizable argument. In this examples is used in
174 * order to select the correct sensor bus handler.
175 * @param reg register to write
176 * @param bufc pointer to data to write in register reg
177 */
```

串口打印数据如下。

[15:09:36.632] 收 ← ◆ pressure [hPa]:1013.86 temperature [degC]: 20.54

演示

```
[14:57:57.137]收←◆pressure [hPa]:994.03 temperature [degC]: 21.44
[14:57:57.392]收←◆pressure [hPa]:994.03 temperature [degC]: 21.45
[14:57:57.647]收←◆pressure [hPa]:994.03 temperature [degC]: 21.46
[14:57:57.902]收←◆pressure [hPa]:994.04 temperature [degC]: 21.47
[14:57:58.157]收←◆pressure [hPa]:994.03 temperature [degC]: 21.48
[14:57:58.412]收←◆pressure [hPa]:994.03 temperature [degC]: 21.49
[14:57:58.667]收←◆pressure [hPa]:994.02 temperature [degC]: 21.49
[14:57:58.923]收←◆pressure [hPa]:994.02 temperature [degC]: 21.51
[14:57:59.178]收←◆pressure [hPa]:994.03 temperature [degC]: 21.52
[14:57:59.433]收←◆pressure [hPa]:994.02 temperature [degC]: 21.53
[14:57:59.688]收←◆pressure [hPa]:994.04 temperature [degC]: 21.54
```

正常气压为 50hPa 到 1050hPa 之间。

一般气压多少算正常

1013.25hPa

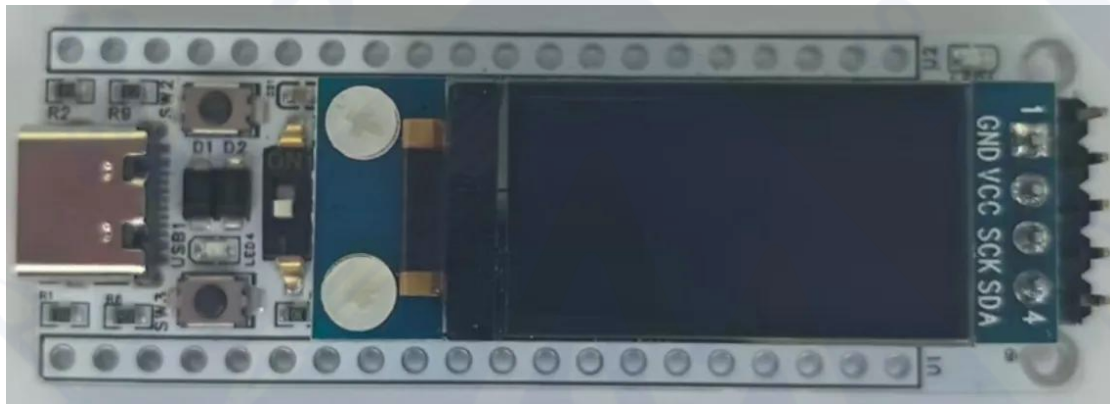
“一般情况下,标准大气压被定义为海平面上的气压,其数值为1013.25hPa。这个数值是国际标准大气模型中的一个参考值,也是气象学和其他科学领域中常用的标准值。具体分析如下:在海平面上,标准大气压的平均值为1013.25hPa,气压的数值会因地理位置、天气、季节等因素而有所变化。在不同的地方和不同的时间,气压的数值可能会高于或低于标准大气压。一般来说,当气压在950hPa到1050hPa之间时,被认为是正常的气压范围... [更多](#)”

project15: 配置 RTC 时钟及显示时间

概述

本文将详细讲解如何借助 e2studio 来对瑞萨微控制器进行实时时钟 (RTC) 的设置和配置,以便实现日历功能和一秒钟产生的中断,从而通过串口输出实时数据。实时时钟 (RTC) 模块是一种时间管理外设,主要用于记录和控制日期和时间。与常见的微控制器 (MCU) 中的定时器不同,RTC 时钟提供了两种计时方式:日期模式和计时模式。RTC 时钟的常用功能包括设置时间、设定闹钟、配置周期性中断以及启动或停止操作。

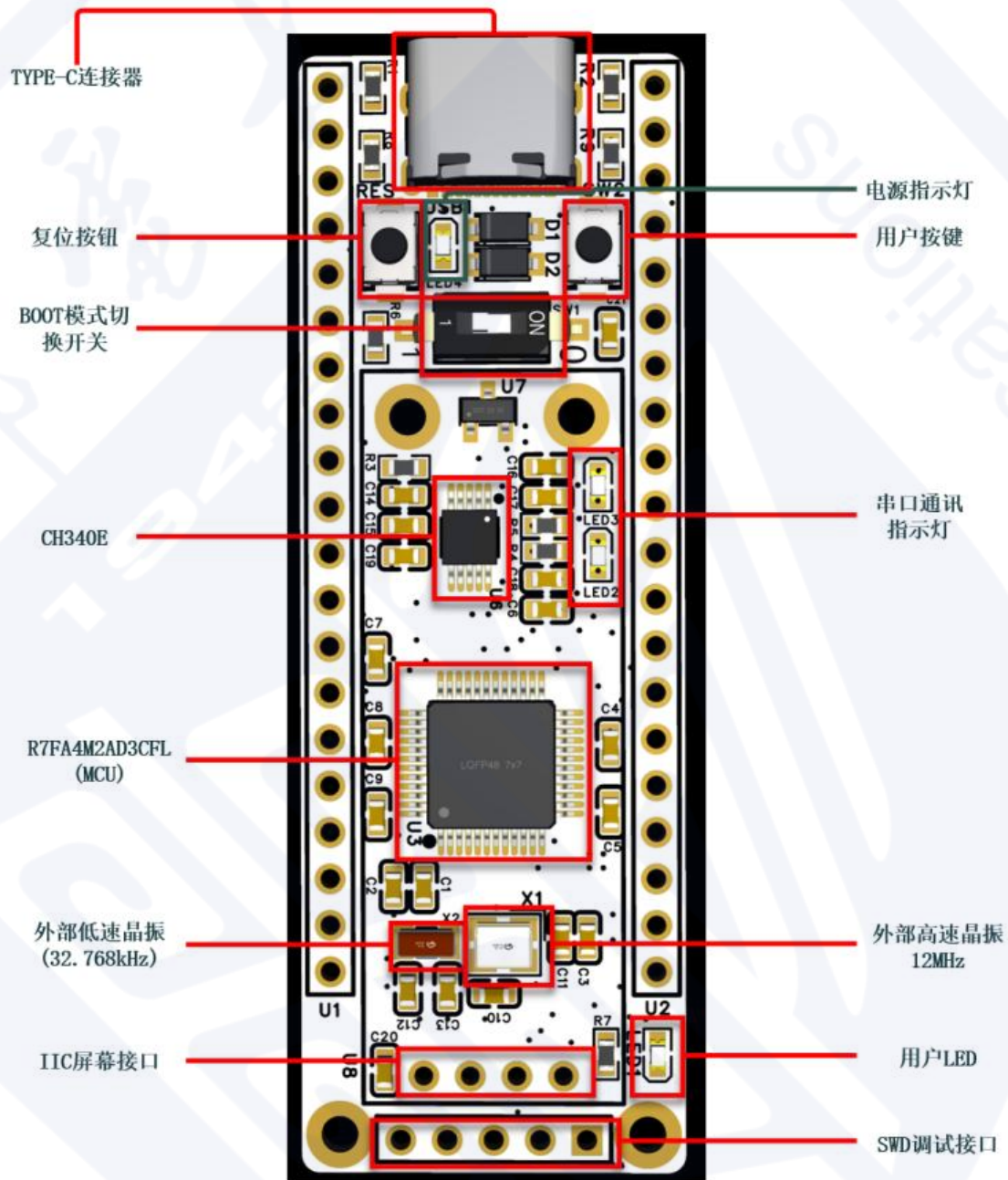
通过使用 e2studio 工具，我们可以轻松地对瑞萨微控制器进行 RTC 配置，从而实现高精度的时间和日期管理。在本文中，我们将重点讨论如何设置 RTC 时钟日历和产生一秒钟的中断，使得串口能够实时打印数据。



硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL

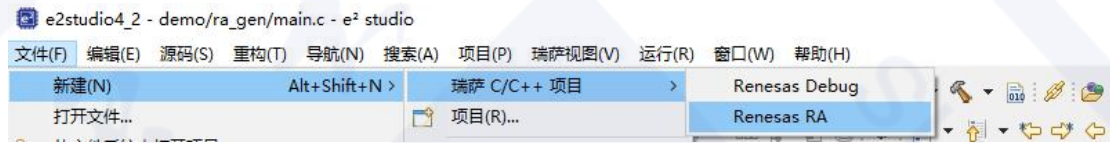


参考程序

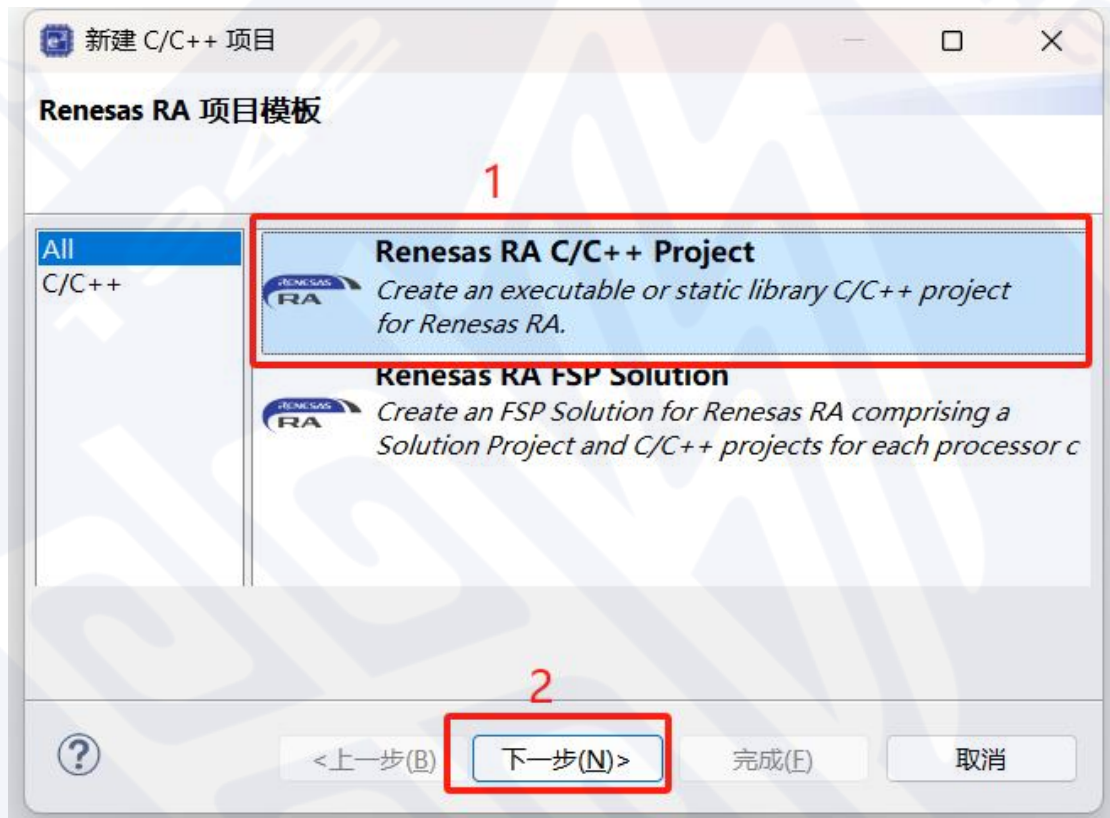
https://github.com/CoreMaker-lab/RA4M2_MINI

https://gitee.com/CoreMaker/RA4M2_MINI

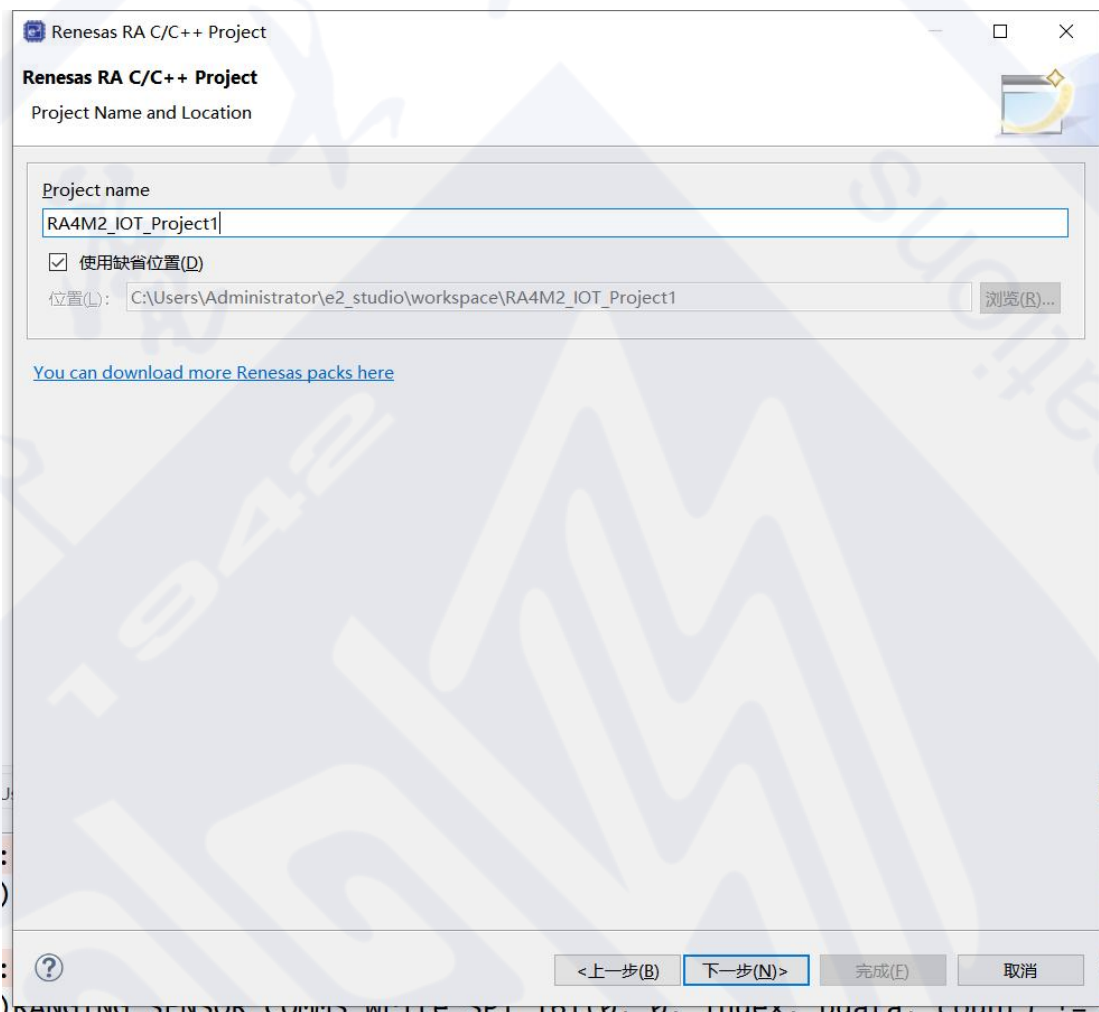
新建工程



工程模板



保存工程路径



芯片配置

本文中使用 R7FA4M2AD3CFL 来进行演示。

Renesas RA C/C++ Project

Renesas RA C/C++ Project

Device and Tools Selection

Device Selection

FSP Version: 5.9.0 **版本**

Board: Custom User Board (Any Device) ...

Device: R7FA4M2AD3CFL **芯片型号**

Core: CM33

Language: ☒ C ☐ C++

Board Description

Device Details

TrustZone	Yes
Pins	48
Processor	Cortex-M33

IDE Project Type

e2 studio managed build

Toolchains

LLVM Embedded Toolchain for Arm

GNU ARM Embedded **工具链**

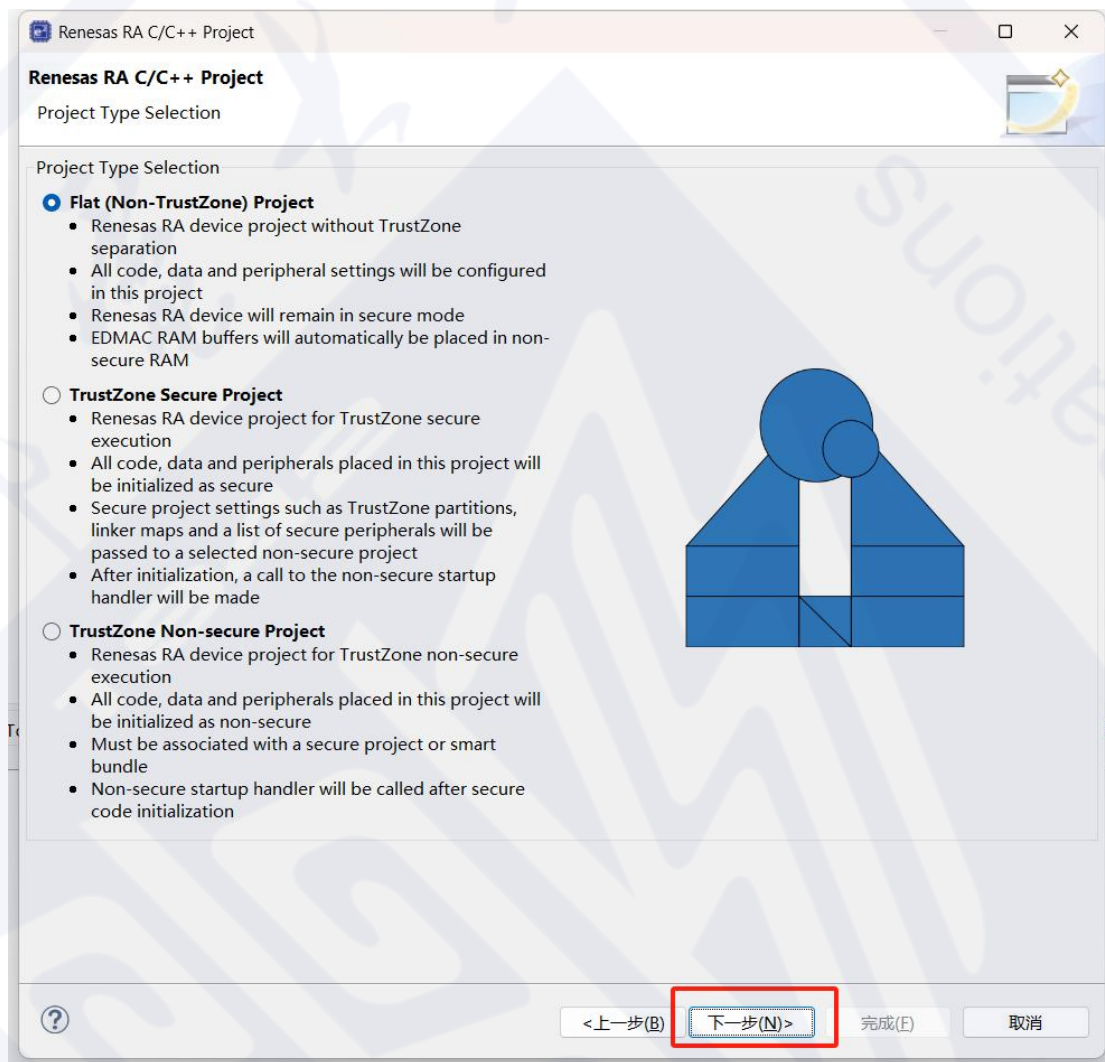
13.2.1.arm-13-7 [Manage Toolchains...](#)

Debugger

J-Link ARM **使用JLink进行调试**

[?](#) <上一步(B) 下一步(N)> 完成(F) 取消

工程模板选择



Renesas RA C/C++ Project

Renesas RA C/C++ Project

Preceding Project or Smart Bundle Selection

☒ None

Choose this option when creating a project for the primary processor core (no preceding project or Smart Bundle).

☐ Preceding Project:

Choose this option if you have access to the preceding bootloader project or TrustZone secure project source code. A debug configuration for Secure Software Debug (SSD) state will be generated where applicable and both secure and non-secure images will be downloaded to the target device. Should you wish to test Non-Secure Software Debug (NSECSO) state, use the Renesas Device Partition Manager to change the Device Lifecycle state as needed.

☐ Smart Bundle:

Resolved location:

Workspace...

File System...

Variables...

Choose this option if you only have access to a pre-programmed device containing TrustZone secure code. The Smart Bundle file (*.sbd) should be obtained from the TrustZone Secure Project developer. A debug configuration for Non-Secure Software Debug (NSECSO) state will be generated and only the non-secure code image will be downloaded to the target device.

Preceding Project/Smart Bundle Details

FSP version

Toolchain

Toolchain version

Board

Device

Core

Zones

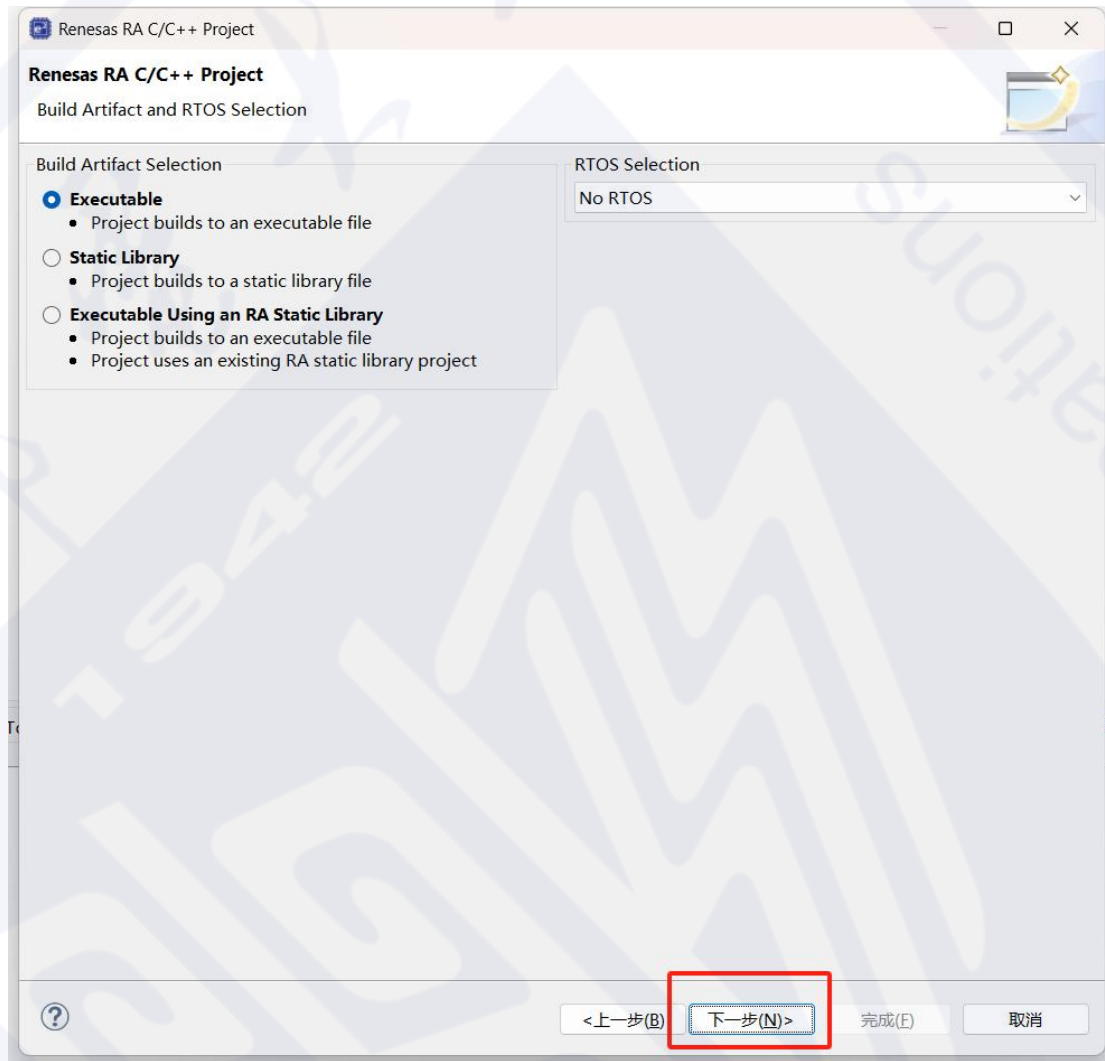
?

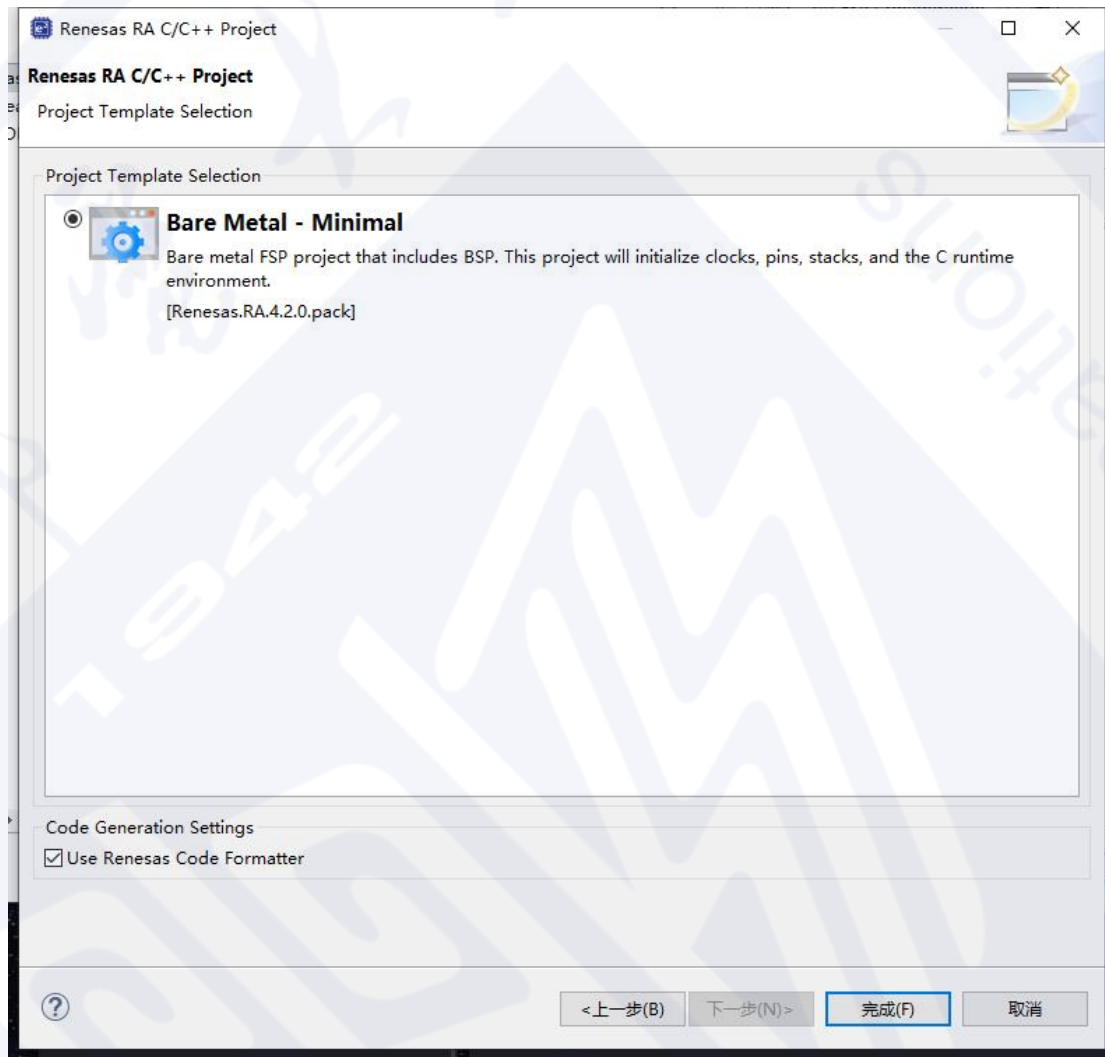
< 上一步(B)

下一步(N) >

完成(E)

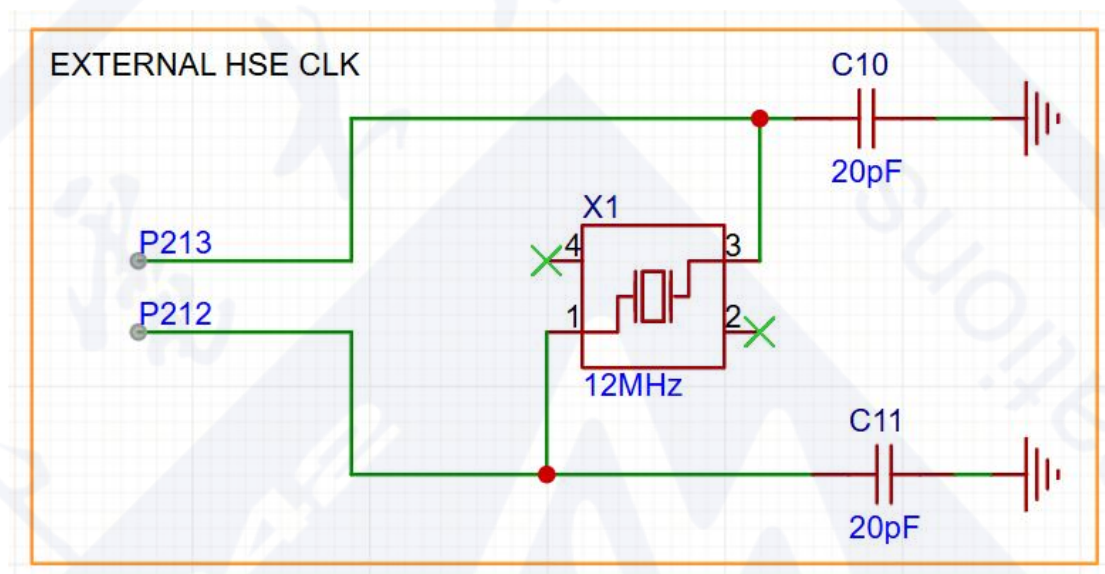
取消



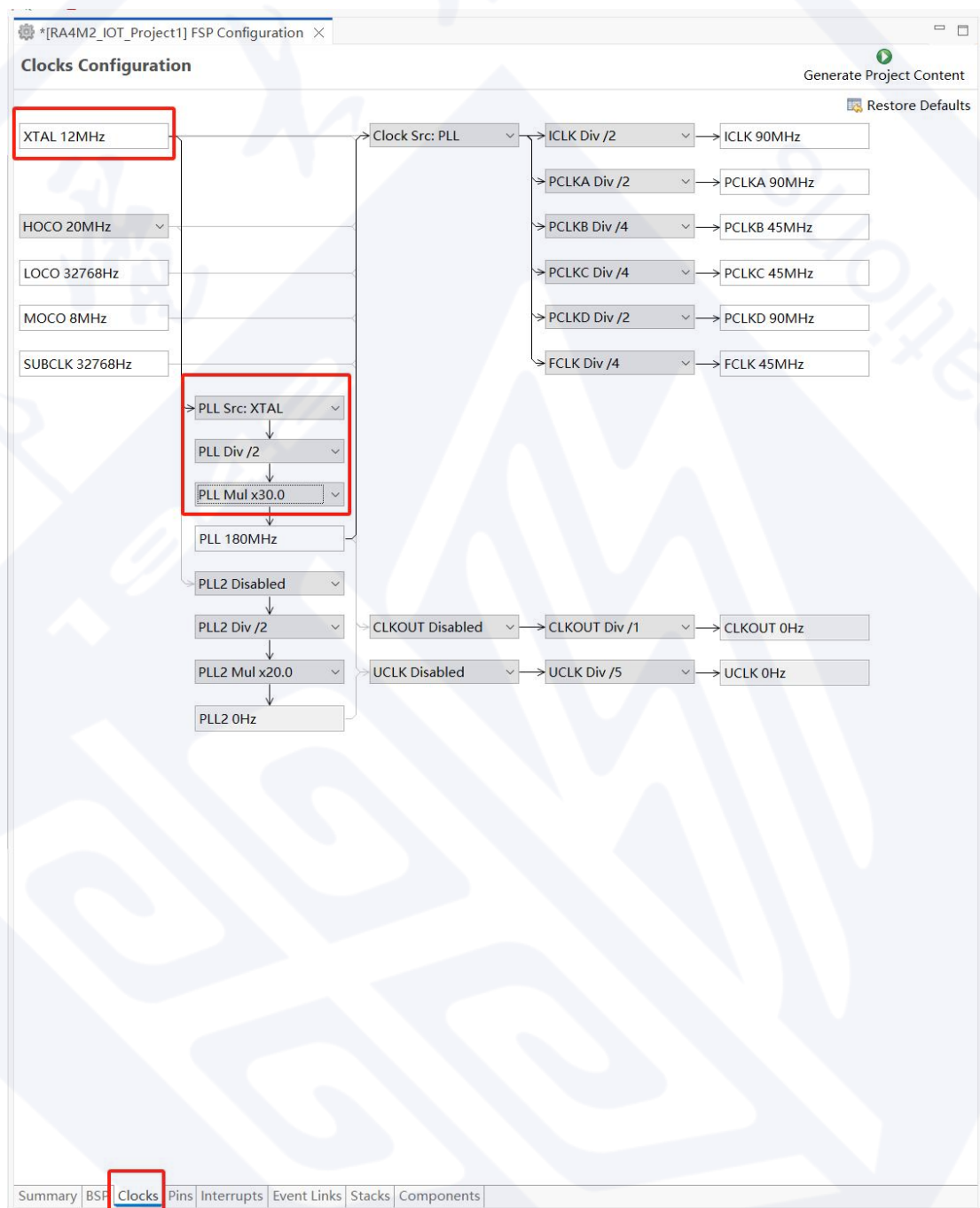


时钟设置

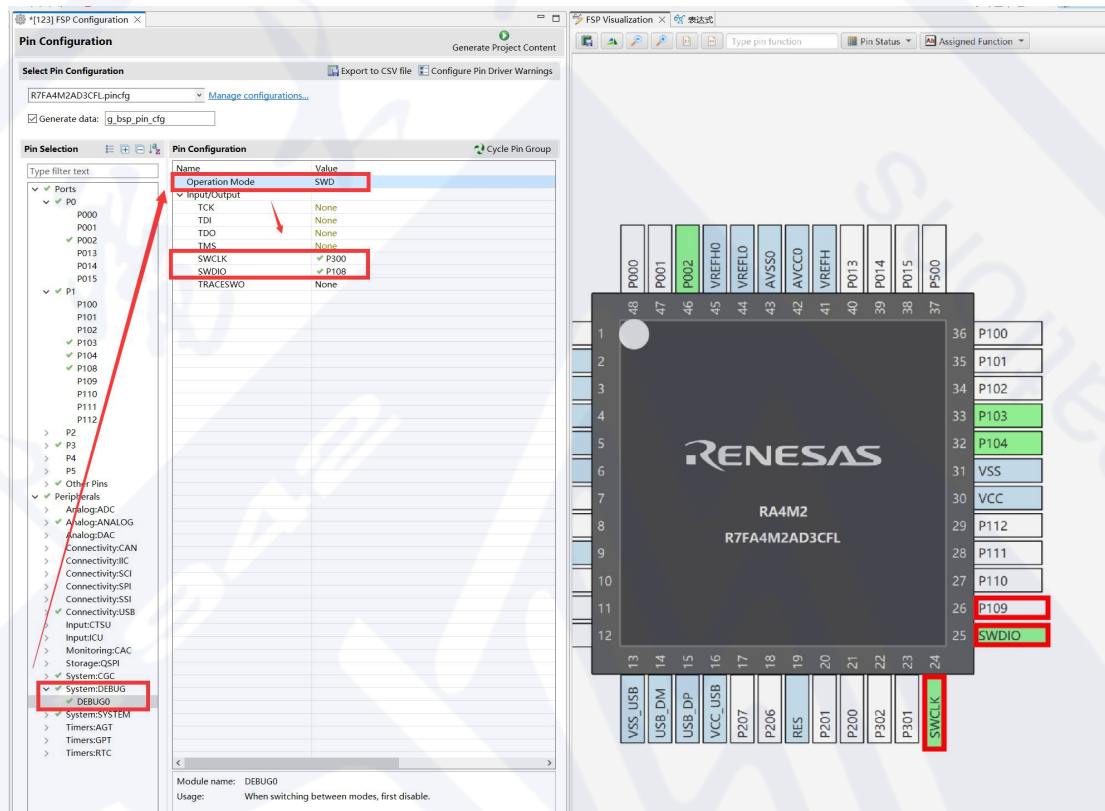
开发板上的外部高速晶振为 12M.



需要修改 XTAL 为 12M。

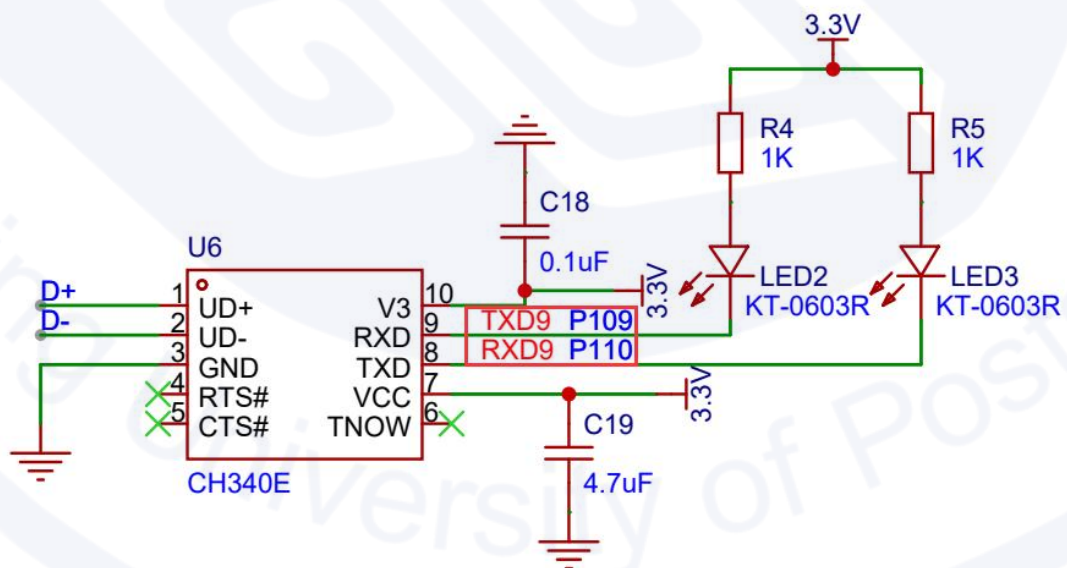


SWD 调试口设置

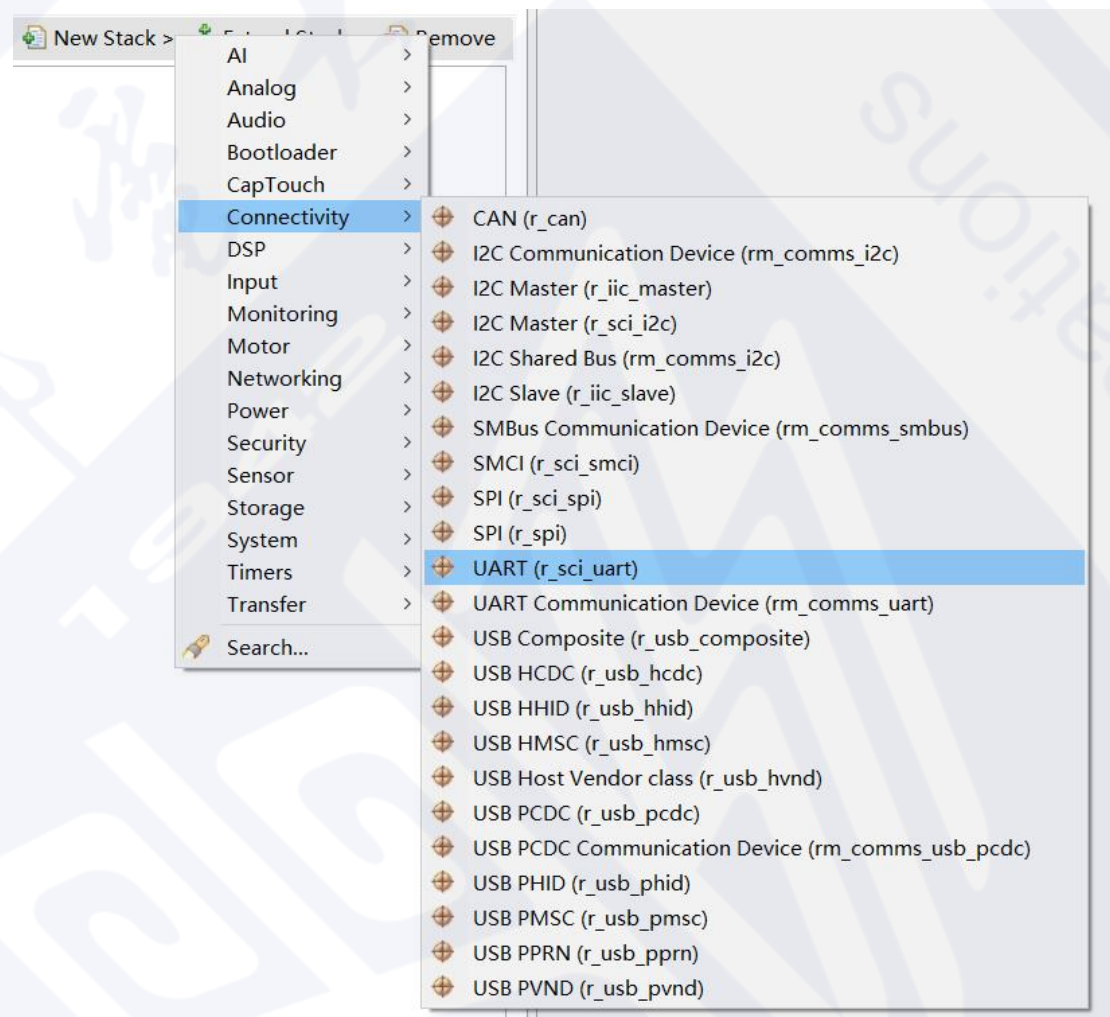


UART 配置

串口CH340E



点击 Stacks->New Stack->Connectivity -> UART(r_sci_uart)。



UART 属性配置

属性 ×	问题	Smart Browser	控制台	调试
g_uart9 UART (r_sci_uart)				
Settings	属性		值	
API Info	▼ Common			
	Parameter Checking		Default (BSP)	
	FIFO Support		Disable	
	DTC Support		Disable	
	Flow Control Support		Disable	
	RS-485 Support		Disable	
	IrDA Support		Disabled	
	▼ Module g_uart9 UART (r_sci_uart)			
	▼ General			
	Name		g_uart9	
	Channel		9	
	Data Bits		8bits	
	Parity		None	
	Stop Bits		1bit	
	▼ Baud			
	Baud Rate		115200	
	Baud Rate Modulation		Enabled	
	Max Error (%)		5	
	► Flow Control			
	► Extra			
	▼ Interrupts			
	Callback		user_uart_callback	
	Receive Interrupt Priority		Priority 2	
	Transmit Data Empty Interrupt Priority		Priority 2	
	Transmit End Interrupt Priority		Priority 2	
	Error Interrupt Priority		Priority 2	
	▼ Pins			
	TXD9		P109	
	RXD9		P110	
	CTS9		<unavailable>	

名称
通道
数据位
无校验位
停止位

波特率
启用波特率调制

回调函数名称

串口发送端口
串口接收端口

设置 e2studio 堆栈

printf 函数通常需要设置堆栈大小。这是因为 printf 函数在运行时需要使用栈空间来存储临时变量和函数调用信息。如果堆栈大小不足，可能会导致程序崩溃或不可预期的行为。

printf 函数使用了可变参数列表，它会在调用时使用栈来存储参数，在函数调用结束时再清除参数，这需要足够的栈空间。另外 printf 也会使用一些临时变量，如果栈空间不足，会导致程序崩溃。

因此，为了避免这类问题，应该根据程序的需求来合理设置堆栈大小。

项目资源管理器 | [RA2M4_print] FSP Configuration

Board Support Package Configuration

Generate Project Content | Restore Defaults

Device Selection

FSP version: 4.2.0

Board: Custom User Board (Any Device)

Device: R7FA4M2AD3CFP

Core: CM33

RTOS: No RTOS

Board Details

Summary **BSP** Clocks Pins Interrupts Event Links Stacks Components

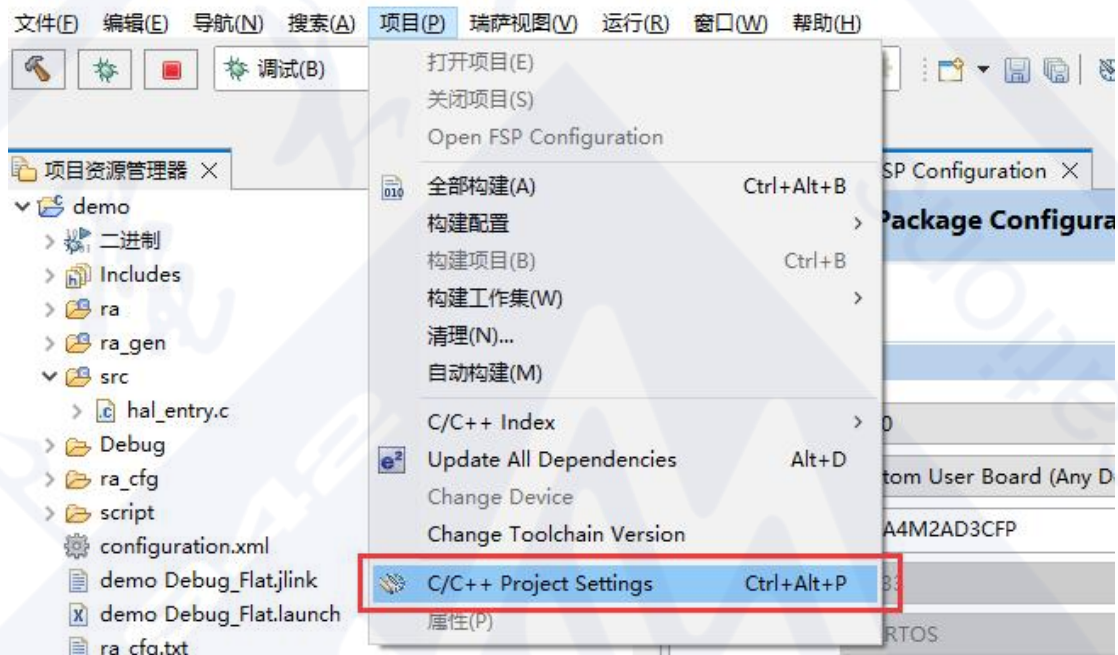
Custom User Board (Any Device)

Settings | 属性 | 问题 | Smart Browser | 控制台

属性	值
√ R7FA4M2AD3CFP	
part_number	R7FA4M2AD3CFP
rom_size_bytes	524288
ram_size_bytes	131072
data_flash_size_bytes	8192
package_style	LQFP
package_pins	100
√ RA4M2	
series	4
√ RA4M2 Family	
> Security	
> OFS0 register settings	
> OFS1 register settings	
> Block Protection Settings (BPS)	
> Permanent Block Protection Settings (PBPS)	
> Clocks	
Startup C-Cache Line Size	32 Bytes
Main Oscillator Wait Time	8163 cycles
√ RA Common	
Main stack size (bytes)	0x400
Heap size (bytes)	0x2000
MCU Vcc (mV)	3300
Parameter checking	Disabled
Assert Failures	Return FSP_ERR_ASSERTION
Error Log	No Error Log
Clock Registers not Reset Values during Startup	Disabled
Main Oscillator Populated	Populated
PFS Protect	Enabled
C Runtime Initialization	Enabled
Early BSP Initialization	Disabled
Main Oscillator Clock Source	Crystal or Resonator
Subclock Populated	Populated
Subclock Drive (Drive capacitance availability varies by MCU)	Standard/Normal mode
Subclock Stabilization Time (ms)	1000

3.设置堆栈大小

e2studio 的重定向 printf 设置



在嵌入式系统的开发中，尤其是在使用 GNU 编译器集合（GCC）时，`--specs` 参数用于指定链接时使用的系统规格（specs）文件。这些规格文件控制了编译器和链接器的行为，尤其是关于系统库和启动代码的链接。`--specs=rdimon.specs` 和 `--specs=nosys.specs` 是两种常见的规格文件，它们用于不同的场景。

`--specs=rdimon.specs`

用途：这个选项用于链接“Redlib”库，这是为裸机（bare-metal）和半主机（semihosting）环境设计的 C 库的一个变体。半主机环境是一种特殊的运行模式，允许嵌入式程序通过宿主机（如开发 PC）的调试器进行输入输出操作。

应用场景：当你在没有完整操作系统的环境中运行程序，但同时需要使用调试器来处理输入输出（例如打印到宿主机的终端），这个选项非常有用。

特点：它提供了一些基本的系统调用，通过调试接口与宿主机通信。

`--specs=nosys.specs`

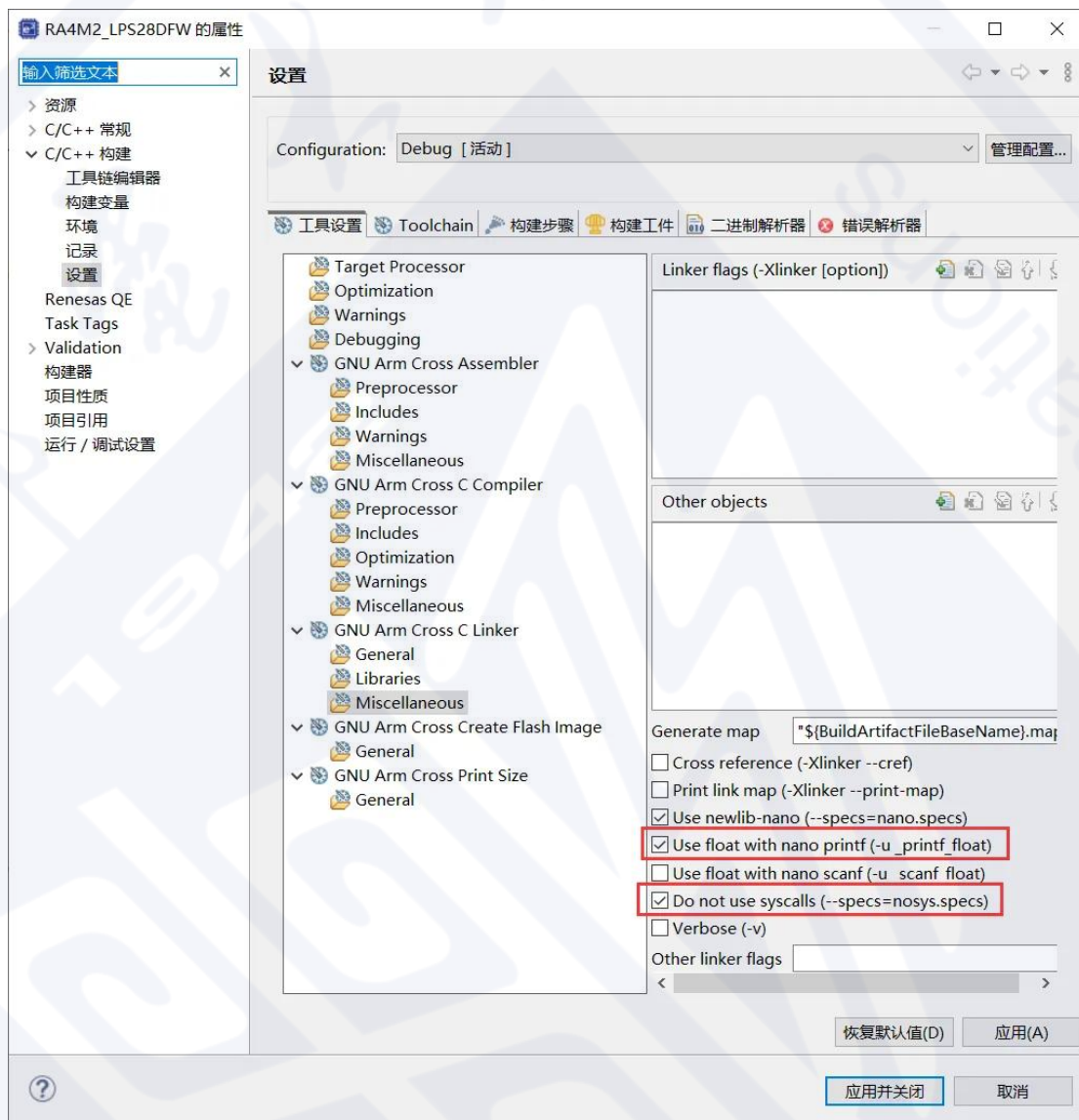
用途：这个选项链接了一个非常基本的系统库，这个库不提供任何系统服务的实现。

应用场景：适用于完全的裸机程序，其中程序不执行任何操作系统调用，比如不进行文件操作或者系统级输入输出。

特点：这是一个更“裸”的环境，没有任何操作系统支持。使用这个规格文件，程序不期望有操作系统层面的任何支持。

如果你的程序需要与宿主机进行交互（如在开发期间的调试），并且通过调试器进行基本的输入输出操作，则使用 `--specs=rdimon.specs`。

如果你的程序是完全独立的，不需要任何形式的操作系统服务，包括不进行任何系统级的输入输出，则使用 `--specs=nosys.specs`。



R_SCI_UART_Open() 函数原型

◆ R_SCI_UART_Open()

```
fsp_err_t R_SCI_UART_Open ( uart_ctrl_t *const p_api_ctrl,  
                           uart_cfg_t const *const p_cfg  
                           )
```

Configures the UART driver based on the input configurations. If reception is enabled at compile time, reception is enabled at the end of this function. Implements `uart_api_t::open`

Return values

FSP_SUCCESS	Channel opened successfully.
FSP_ERR_ASSERTION	Pointer to UART control block or configuration structure is NULL.
FSP_ERR_IP_CHANNEL_NOT_PRESENT	The requested channel does not exist on this MCU.
FSP_ERR_INVALID_ARGUMENT	Flow control is enabled but flow control pin is not defined or selected channel does not support "Hardware CTS and Hardware RTS" flow control. (or) restricted channel is selected.
FSP_ERR_ALREADY_OPEN	Control block has already been opened or channel is being used by another instance. Call <code>close()</code> then <code>open()</code> to reconfigure.
FSP_ERR_INVALID_CHANNEL	IrDA is requested for a channel that does not support IrDA.

Returns

See [Common Error Codes](#) or functions called by this function for other possible return codes. This function calls:

- `transfer_api_t::open`

故可以用 `R_SCI_UART_Open()`函数进行配置，开启和初始化 UART。

```
/* Open the transfer instance with initial configuration. */  
err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);  
assert(FSP_SUCCESS == err);
```

```
[RA4M2_MINI_Project11_1] FSP Configuration  *hal_entry.c x startup.c main.c
2
3 FSP_CPP_HEADER
4 void R_BSP_WarmStart(bsp_warm_start_event_t event);
5 FSP_CPP_FOOTER
6 unused parameter 'fd' [-Wunused-parameter] | no previous declaration for '_write' [-W
7
8
9
10
11
12
13
14
15 /* Open the transfer instance with initial configuration. */
16 err = R_SCI_UART_Open(&g_uart9_ctrl1, &g_uart9_cfg);
17 assert(FSP_SUCCESS == err);
18
19
20
21 #if BSP_TZ_SECURE_BUILD
22 /* Enter non-secure code */
23 R_BSP_NonSecureEnter();
24 #endif
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
```

回调函数 user_uart_callback ()

当数据发送的时候，可以查看 UART_EVENT_TX_COMPLETE 来判断是否发送完毕。

◆ write

```
fsp_err_t (*uart_api_t::write)(uart_ctrl_t *const p_ctrl, uint8_t const *const p_src, uint32_t const bytes)
```

Write to UART device. The write buffer is used until write is complete. Do not overwrite write buffer contents until the write is finished. When the write is complete (all bytes are fully transmitted on the wire), the callback called with event **UART_EVENT_TX_COMPLETE**. The maximum transfer size is reported by [infoGet\(\)](#).

Implemented as

- R_SCI_UART_Write()
- R_SCI_B_UART_Write()

Parameters

[in] p_ctrl Pointer to the UART control block.

[in] p_src Source address to write data to.

[in] bytes Write data length.

Enumeration Type Documentation

◆ uart_event_t

enum `uart_event_t`

UART Event codes

Enumerator	
UART_EVENT_RX_COMPLETE	Receive complete event.
UART_EVENT_TX_COMPLETE	Transmit complete event.
UART_EVENT_RX_CHAR	Character received.
UART_EVENT_ERR_PARITY	Parity error event.
UART_EVENT_ERR_FRAMING	Mode fault error event.
UART_EVENT_ERR_OVERFLOW	FIFO Overflow error event.
UART_EVENT_BREAK_DETECT	Break detect error event.
UART_EVENT_TX_DATA_EMPTY	Last byte is transmitting, ready for more data.

可以检查检查 "p_args" 结构体中的 "event" 字段的值是否等于 "UART_EVENT_TX_COMPLETE"。如果条件为真，那么 if 语句后面的代码块将会执行。

```
fsp_err_t err = FSP_SUCCESS;
volatile bool uart_send_complete_flag = false;
void user_uart_callback (uart_callback_args_t * p_args)
{
    if(p_args->event == UART_EVENT_TX_COMPLETE)
    {
        uart_send_complete_flag = true;
    }
}
```

```
[RA4M2_MINI_Project11_1] FSP Configuration  *hal_entry.c x startup.c main.c
1 #include "hal_data.h"
2
3 FSP_CPP_HEADER
4 void R_BSP_WarmStart(bsp_warm_start_event_t event);
5 FSP_CPP_FOOTER
6
7 fsp_err_t err = FSP_SUCCESS;
8 volatile bool uart_send_complete_flag = false;
9 void user_uart_callback (uart_callback_args_t * p_args)
10 {
11     if(p_args->event == UART_EVENT_TX_COMPLETE)
12     {
13         uart_send_complete_flag = true;
14     }
15 }
16
17 unused parameter 'fd' [-Wunused-parameter] | no previous declaration for '_write' [-W
18
19 * main() is generated by the RA Configuration editor and is used to generate threads if
20 void hal_entry(void)
21 {
22     /* TODO: add your own code here */
23
24     /* Open the transfer instance with initial configuration. */
25     err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);
26     assert(FSP_SUCCESS == err);
27
28     #if BSP_TZ_SECURE_BUILD
29     /* Enter non-secure code */
30     R_BSP_NonSecureEnter();
31     #endif
32 }
33
34 * This function is called at various points during the startup process. This implement
35 void R_BSP_WarmStart(bsp_warm_start_event_t event)
36 {
37     if (BSP_WARM_START_RESET == event)
38     {
39         #if BSP_FEATURE_FLASH_LP_VERSION != 0
40
```

printf 输出重定向到串口

打印最常用的方法是 printf，所以要解决的问题是将 printf 的输出重定向到串口，然后通过串口将数据发送出去。

注意一定要加上头文件#include <stdio.h>

```
#ifdef __GNUC__ //串口重定向
#define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
#else
#endif

PUTCHAR_PROTOTYPE
{
    err = R_SCI_UART_Write(&g_uart9_ctrl, (uint8_t *)&ch, 1);
    if(FSP_SUCCESS != err) __BKPT();
    while(uart_send_complete_flag == false){}
    uart_send_complete_flag = false;
    return ch;
}
```

```

}

int _write(int fd, char *pBuffer, int size)
{
    for(int i=0; i<size; i++)
    {
        __io_putchar(*pBuffer++);
    }

    return size;
}

```

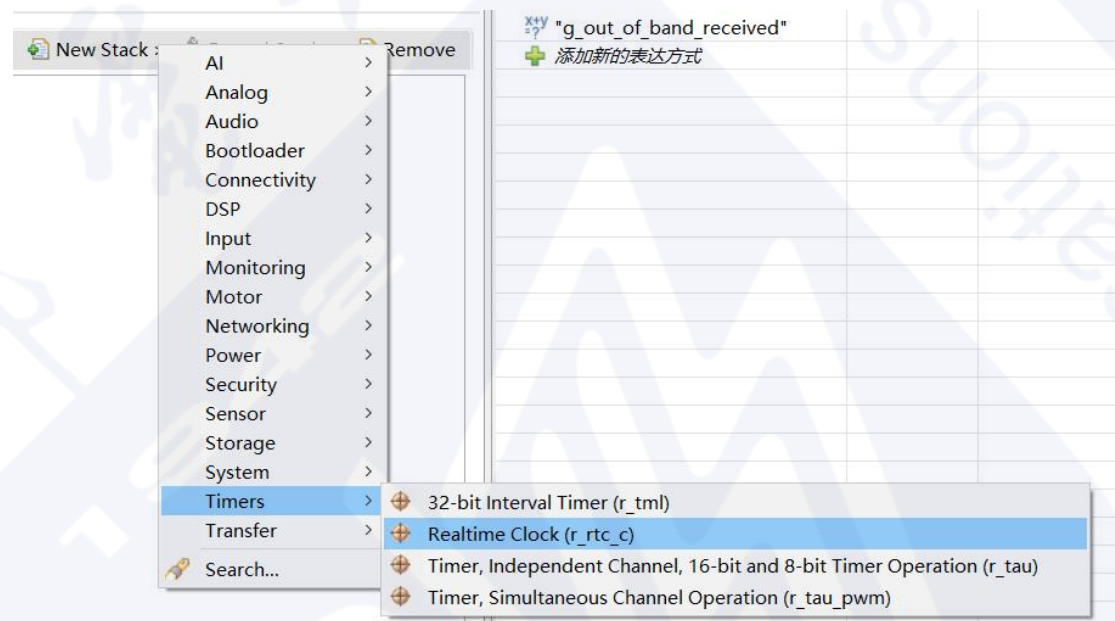
```

1  #include "hal_data.h"
2  #include <stdio.h>
3  FSP_CPP_HEADER
4  void R_BSP_WarmStart(bsp_warm_start_event_t event);
5  FSP_CPP_FOOTER
6
7  fsp_err_t err = FSP_SUCCESS;
8  volatile bool uart_send_complete_flag = false;
9  void user_uart_callback (uart_callback_args_t * p_args)
10 {
11     if(p_args->event == UART_EVENT_TX_COMPLETE)
12     {
13         uart_send_complete_flag = true;
14     }
15 }
16
17
18 #ifdef __GNUC__ //串口重定向
19 #define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
20 #else
21 #define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
22 #endif
23
24
25 PUTCHAR_PROTOTYPE
26 {
27     err = R_SCI_UART_Write(&g_uart9_ctrl1, (uint8_t *)&ch, 1);
28     if(FSP_SUCCESS != err) __BKPT();
29     while(uart_send_complete_flag == false){}
30     uart_send_complete_flag = false;
31     return ch;
32 }
33
34 int _write(int fd, char *pBuffer, int size)
35 {
36     for(int i=0; i<size; i++)
37     {
38         __io_putchar(*pBuffer++);
39     }
40     return size;
41 }
42
43
44
45 * * main() is generated by the RA Configuration editor and is used to generate threads i
46
47 void hal_entry(void)
48 {
49     /* TODO: add your own code here */
50
51     /* Open the transfer instance with initial configuration. */
52     err = R_SCI_UART_Open(&g_uart9_ctrl1, &g_uart9_cfg);
53     assert(FSP_SUCCESS == err);
54
55
56

```

RTC 配置

点击 Stacks->New Stack->Timers -> Realtime Clock(r_rtc_c)。



RTC 属性配置

属性

问题

Smart Browser

控制台

g_rtc0 Realtime Clock (r_rtc)

Settings

API Info

属性

值

▼ Common

Parameter Checking

Set Source Clock in Open

▼ Module g_rtc0 Realtime Clock (r_rtc)

Name

Clock Source

Frequency Comparision Value (LOCO)

Automatic Adjustment Mode

Automatic Adjustment Period

Adjustment Type (Plus-Minus)

Error Adjustment Value

Callback

Alarm Interrupt Priority

Period Interrupt Priority

Carry Interrupt Priority

▼ Pins

RTCOUNT

Default (BSP)

Enabled

g_rtc0

Sub-Clock

255

Enabled

10 Seconds

NONE

0

rtc_callback

Priority 2

Priority 2

Priority 2

<unavailable>

名称

时钟源, SUBCLK时钟精度高

误差调整模式

自动调整的误差调整周期

调整值

回调函数

闹钟报警中断优先级

周期中断优先级

其中 LOCO 为内部低速时钟，需要准确定时还是需要外部低速晶振 Sub-clock。

Leading-performance 100 MHz Arm Cortex-M33 core, up to 512 KB code flash memory with background operation, 8 KB Data flash memory, and 128 KB SRAM with Parity/ECC. High-integration with USB 2.0 Full-Speed, SDHI, Quad SPI, and advanced analog. Integrated Secure Crypto Engine with cryptography accelerators, key management support, tamper detection and power analysis resistance in concert with Arm TrustZone for integrated secure element functionality.

Features

- Arm® Cortex®-M33 Core
 - Armv8-M architecture with the main extension
 - Maximum operating frequency: 100 MHz
 - Arm Memory Protection Unit (Arm MPU)
 - Protected Memory System Architecture (PMSAv8)
 - Secure MPU (MPU_S): 8 regions
 - Non-secure MPU (MPU_NS): 8 regions
 - SysTick timer
 - Embeds two SysTick timers: Secure and Non-secure instance
 - Driven by LOCO or system clock
 - CoreSight™ ETM-M33
- Memory
 - Up to 512-KB code flash memory
 - 8-KB data flash memory (100,000 program/erase (P/E) cycles)
 - 128-KB SRAM
- Connectivity
 - Serial Communications Interface (SCI) × 6
 - Asynchronous interfaces
 - 8-bit clock synchronous interface
 - Smart card interface
 - Simple IIC
 - Simple SPI
 - Manchester coding (SCI3, SCI4)
 - I²C bus interface (IIC) × 2
 - Serial Peripheral Interface (SPI)
 - Quad Serial Peripheral Interface (QSPI)
 - USB 2.0 Full-Speed Module (USBFS)
 - Control Area Network module (CAN)
 - SD/MMC Host Interface (SDHI)
 - Serial Sound Interface Enhanced (SSIE)
- Independent Watchdog Timer (IWDT)
- Human Machine Interface (HMI)
 - Capacitive Touch Sensing Unit (CTSU)
- Multiple Clock Sources
 - Main clock oscillator (MOSC) (8 to 24 MHz)
 - Sub-clock oscillator (SOSC) (32.768 kHz)
 - High-speed on-chip oscillator (HOCO) (16/18/20 MHz)
 - Middle-speed on-chip oscillator (MOCO) (8 MHz)
 - Low-speed on-chip oscillator (LOCO) (32.768 kHz)
 - IWDT-dedicated on-chip oscillator (15 kHz)
 - Clock trim function for HOCO/MOCO/LOCO
 - PLL/PLL2
 - Clock out support
- General-Purpose I/O Ports
 - 5-V tolerance, open drain, input pull-up, switchable driving ability
- Operating Voltage
 - VCC: 2.7 to 3.6 V
- Operating Temperature and Packages
 - Ta = -40°C to +105°C
 - 100-pin LQFP (14 mm × 14 mm, 0.5 mm pitch)
 - 64-pin LQFP (10 mm × 10 mm, 0.5 mm pitch)
 - 48-pin LQFP (7 mm × 7 mm, 0.5 mm pitch)
 - 48-pin QFN (7 mm × 7 mm, 0.5 mm pitch)

Table 2.17 Clock timing except for sub-clock oscillator (2 of 2)

Parameter	Symbol	Min	Typ	Max	Unit	Test conditions
PLL/PLL2 period jitter	$f_{PLL}, f_{PLL2} \geq 120\text{MHz}$	—	—	±100	—	ps
	$f_{PLL}, f_{PLL2} < 120\text{MHz}$	—	—	±120	—	ps
PLL/PLL2 long term jitter	—	—	—	±300	—	ps Term: 1μs, 10μs

- Note 1. When setting up the main clock oscillator, ask the oscillator manufacturer for an oscillation evaluation, and use the results as the recommended oscillation stabilization time. Set the MOSCWTCR register to a value equal to or greater than the recommended value.
After changing the setting in the MOSCCR.MOSTP bit to start main clock operation, read the OSCSF.MOSCSF flag to confirm that it is 1, and then start using the main clock oscillator.
- Note 2. This is the time from release from reset state until the HOCO oscillation frequency (f_{HOCO}) reaches the range for guaranteed operation.

Table 2.18 Clock timing for the sub-clock oscillator

Parameter	Symbol	Min	Typ	Max	Unit	Test conditions
Sub-clock frequency	f_{SUB}	—	32.768	—	kHz	—
Sub-clock oscillation stabilization wait time	$t_{SUBOSCWT}$	—	—	—*1	s	Figure 2.11

- Note 1. When setting up the sub-clock oscillator, ask the oscillator manufacturer for an oscillation evaluation and use the results as the recommended oscillation stabilization time.
After changing the setting in the SOSCCR.SOSTP bit to start sub-clock operation, only start using the sub-clock oscillator after the sub-clock oscillation stabilization time elapses with an adequate margin. A value that is two times the value shown is recommended.

内部低速晶振的偏差过大，不如外置低速晶振。

2.3.2 Clock Timing

Table 2.17 Clock timing except for sub-clock oscillator (1 of 2)

Parameter	Symbol	Min	Typ	Max	Unit	Test conditions
EXTAL external clock input cycle time	t_{EXcyc}	41.66	—	—	ns	Figure 2.7
EXTAL external clock input high pulse width	t_{EXH}	15.83	—	—	ns	
EXTAL external clock input low pulse width	t_{EXL}	15.83	—	—	ns	
EXTAL external clock rise time	t_{EXr}	—	—	5.0	ns	
EXTAL external clock fall time	t_{EXf}	—	—	5.0	ns	
Main clock oscillator frequency	f_{MAIN}	8	—	24	MHz	—
Main clock oscillation stabilization wait time (crystal)*1	$t_{MAINOSCWT}$	—	—	—*1	ms	Figure 2.8
LOCO clock oscillation frequency	f_{LOCO}	29.4912	32.768	36.0448	kHz	—
LOCO clock oscillation stabilization wait time	t_{LOCOWT}	—	—	60.4	μs	Figure 2.9
ILOCO clock oscillation frequency	f_{ILOCO}	13.5	15	16.5	kHz	—
MOCO clock oscillation frequency	f_{MOCO}	6.8	8	9.2	MHz	—
MOCO clock oscillation stabilization wait time	t_{MOCOWT}	—	—	15.0	μs	—
HOCO clock oscillator oscillation frequency	Without FLL	f_{HOCO16}	15.78	16	16.22	MHz –20 ≤ Ta ≤ 105°C
		f_{HOCO18}	17.75	18	18.25	
		f_{HOCO20}	19.72	20	20.28	
		f_{HOCO16}	15.71	16	16.29	–40 ≤ Ta ≤ –20°C
		f_{HOCO18}	17.68	18	18.32	
		f_{HOCO20}	19.64	20	20.36	
	With FLL	f_{HOCO16}	15.960	16	16.040	–40 ≤ Ta ≤ 105°C Sub-clock frequency accuracy is ±50 ppm.
		f_{HOCO18}	17.955	18	18.045	
		f_{HOCO20}	19.950	20	20.050	
HOCO clock oscillation stabilization wait time*2	t_{HOCOWT}	—	—	64.7	μs	—
HOCO period jitter	—	—	±85	—	ps	—
FLL stabilization wait time	t_{FLLWT}	—	—	1.8	ms	—
PLL clock frequency	f_{PLL}	100	—	200	MHz	—
PLL2 clock frequency	f_{PLL2}	120	—	240	MHz	—
PLL/PLL2 clock oscillation stabilization wait time	t_{PLLWT}	—	—	174.9	μs	Figure 2.10

设定时间

在启动 RTC 后，需要为其设定当前时间。您可以使用 `R_RTC_CalendarTimeSet(&g_rtc0_ctrl, &set_time);` 函数来实现这一目标。具体的时间参数可以通过修改 `set_time` 变量来调整。

◆ R_RTC_CalendarTimeSet()

```
fsp_err_t R_RTC_CalendarTimeSet ( rtc_ctrl_t *const p_ctrl,  
                                rtc_time_t *const p_time  
                                )
```

Set the calendar time.

Implements `rtc_api_t::calendarTimeSet`.

Return values

FSP_SUCCESS	Calendar time set operation was successful.
FSP_ERR_ASSERTION	Invalid input argument.
FSP_ERR_NOT_OPEN	Driver not open already for operation.
FSP_ERR_INVALID_ARGUMENT	Invalid time parameter field.

```
//RTC 变量  
/* rtc_time_t is an alias for the C Standard time.h struct 'tm' */  
rtc_time_t set_time =  
{  
    .tm_sec = 50,      /* 秒, 范围从 0 到 59 */  
    .tm_min = 59,      /* 分, 范围从 0 到 59 */  
    .tm_hour = 23,     /* 小时, 范围从 0 到 23*/  
    .tm_mday = 28,     /* 一月中的第几天, 范围从 1 到 31*/  
    .tm_mon = 1,       /* 月份, 范围从 0 到 11*/  
    .tm_year = 125,    /* 自 1900 起的年数, 2025 为 125*/  
    .tm_wday = 5,      /* 一周中的第几天, 范围从 0 到 6*/  
    .tm_yday=0,        /* 一年中的第几天, 范围从 0 到 365*/  
    .tm_isdst=0;       /* 夏令时*/  
};
```

设定周期性中断

如果您想要使用 RTC 实现固定延迟中断，可以通过 `R_RTC_PeriodicIrqRateSet` 函数来实现。例如，要设置 1 秒的周期性中断，您可以使用如下代码：

```
R_RTC_PeriodicIrqRateSet(&g_rtc0_ctrl, RTC_PERIODIC_IRQ_SELECT_1_SECOND);
```

每次周期性中断产生时，系统将触发回调函数的事件 `RTC_EVENT_PERIODIC_IRQ`。

◆ R_RTC_PeriodicIrqRateSet()

```
fsp_err_t R_RTC_PeriodicIrqRateSet ( rtc_ctrl_t *const p_ctrl,  
                                     rtc_periodic_irq_select_t const rate  
                                   )
```

Set the periodic interrupt rate and enable periodic interrupt.

Implements `rtc_api_t::periodicIrqRateSet`

Note

To start the RTC `R_RTC_CalendarTimeSet` must be called at least once.

Example:

```
/* Set the periodic interrupt rate to 1 second */  
R_RTC_PeriodicIrqRateSet(&g_rtc0_ctrl, RTC_PERIODIC_IRQ_SELECT_1_SECOND);
```

Return values

<code>FSP_SUCCESS</code>	The periodic interrupt rate was successfully set.
<code>FSP_ERR_ASSERTION</code>	Invalid input argument.
<code>FSP_ERR_NOT_OPEN</code>	Driver not open already for operation.
<code>FSP_ERR_IRQ_BSP_DISABLED</code>	User IRQ parameter not valid

设定日历闹钟时间

在启动 RTC 后，您可以设置日历闹钟时间。通过使用 `R_RTC_CalendarAlarmSet(&g_rtc0_ctrl, &set_alarm_time);` 函数，可以设定闹钟时间。具体的时间参数可以通过修改 `set_alarm_time` 变量来调整。具体设置方法如下。在这个示例中，我们仅设置了 0 点 0 分进行闹钟触发，而且只在周三触发。

```
//RTC 闹钟变量  
rtc_alarm_time_t set_alarm_time=  
{  
    .time.tm_sec = 10,      /* 秒，范围从 0 到 59 */  
    .time.tm_min = 30,      /* 分，范围从 0 到 59 */  
    .time.tm_hour = 12,     /* 小时，范围从 0 到 23*/  
    .time.tm_mday = 1,      /* 一月中的第几天，范围从 1 到 31*/  
    .time.tm_mon = 2,       /* 月份，范围从 0 到 11*/  
    .time.tm_year = 125,    /* 自 1900 起的年数，2025 为 125*/  
    .time.tm_wday = 6,      /* 一周中的第几天，范围从 0 到 6*/  
  
    .sec_match = 1,  
    .min_match = 0,  
    .hour_match = 0,  
    .mday_match = 0,  
    .mon_match = 0,  
    .year_match = 0,  
    .dayofweek_match = 0,  
};
```

◆ R_RTC_C_CalendarAlarmSet()

```
fsp_err_t R_RTC_C_CalendarAlarmSet ( rtc_ctrl_t *const    p_ctrl,  
                                     rtc_alarm_time_t *const p_alarm  
                                     )
```

Set the calendar alarm time. For the p_alarm, only minutes, hours and weekdays are valid. Minutes 0 to 59. Hours 0 to 23. Weekdays 0 to 127.

Implements `rtc_api_t::calendarAlarmSet`.

Precondition

The calendar counter must be running before the alarm can be set.

Return values

FSP_SUCCESS	Calendar alarm time set operation was successful.
FSP_ERR_INVALID_ARGUMENT	Invalid time parameter field.
FSP_ERR_ASSERTION	Invalid input argument.
FSP_ERR_NOT_OPEN	Driver not open already for operation.
FSP_ERR_IRQ_BSP_DISABLED	Interrupt must be enabled to use the alarm.

回调函数

可以触发进入回调函数的事件如下所示，RTC_EVENT_PERIODIC_IRQ 为设置的实时性事件，例如 1s 一次，RTC_EVENT_ALARM_IRQ 为闹钟事件。

enum rtc_event_t

Events that can trigger a callback function

Enumerator

RTC_EVENT_ALARM_IRQ	Real Time Clock ALARM IRQ.
RTC_EVENT_PERIODIC_IRQ	Real Time Clock PERIODIC IRQ.

```
//RTC 回调函数  
volatile bool rtc_flag = 0; //RTC 延时 1s 标志位  
volatile bool rtc_alarm_flag = 0; //RTC 闹钟  
/* Callback function */  
void rtc_callback(rtc_callback_args_t *p_args)  
{  
    /* TODO: add your own code here */  
    if(p_args->event == RTC_EVENT_PERIODIC_IRQ)  
        rtc_flag=1;  
    else if(p_args->event == RTC_EVENT_ALARM_IRQ)  
        rtc_alarm_flag=1;  
}
```

R_RTC_Open 函数

R_RTC_Open 函数可以开启 RTC。

◆ R_RTC_Open()

```
fsp_err_t R_RTC_Open ( rtc_ctrl_t *const p_ctrl,  
                      rtc_cfg_t const *const p_cfg  
                      )
```

Opens and configures the RTC driver module. Implements `rtc_api_t::open`. Configuration includes clock source, and interrupt callback function.

Example:

```
/* Open the RTC module */  
err = R_RTC_Open(&g_rtc0_ctrl, &g_rtc0_cfg);
```

Return values

FSP_SUCCESS	Initialization was successful and RTC has started.
FSP_ERR_ASSERTION	Invalid p_ctrl or p_cfg pointer.
FSP_ERR_ALREADY_OPEN	Module is already open.
FSP_ERR_INVALID_ARGUMENT	Invalid time parameter field.

同时主程序中开启 RTC 已经设置时间和闹钟。

```
/* *****RTC 开启***** */  
/* Initialize the RTC module */  
err = R_RTC_Open(&g_rtc0_ctrl, &g_rtc0_cfg);  
  
/* Handle any errors. This function should be defined by the user. */  
assert(FSP_SUCCESS == err);  
  
/* Set the RTC clock source. Can be skipped if "Set Source Clock in Open" property is enabled.  
*/  
R_RTC_ClockSourceSet(&g_rtc0_ctrl);  
/* R_RTC_CalendarTimeSet must be called at least once to start the RTC */  
R_RTC_CalendarTimeSet(&g_rtc0_ctrl, &set_time);  
/* Set the periodic interrupt rate to 1 second */  
R_RTC_PeriodicIrqRateSet(&g_rtc0_ctrl, RTC_PERIODIC_IRQ_SELECT_1_SECOND);  
  
R_RTC_CalendarAlarmSet(&g_rtc0_ctrl, &set_alarm_time);  
uint8_t rtc_second= 0;    //秒  
uint8_t rtc_minute =0;    //分  
uint8_t rtc_hour =0;      //时  
uint8_t rtc_day =0;       //日  
uint8_t rtc_month =0;     //月  
uint16_t rtc_year =0;      //年  
uint8_t rtc_week =0;      //周
```

```
rtc_time_t get_time;
```

同时主函数的 **while** 循环中添加打印和中断处理，以及当前时间显示。

```
while(1)
{
    if(rtc_flag)
    {
        R_RTC_CalendarTimeGet(&g_rtc0_ctrl, &get_time); //获取 RTC 计数时间
        rtc_flag=0;
        rtc_second=get_time.tm_sec; //秒
        rtc_minute=get_time.tm_min; //分
        rtc_hour=get_time.tm_hour; //时
        rtc_day=get_time.tm_mday; //日
        rtc_month=get_time.tm_mon; //月
        rtc_year=get_time.tm_year; //年
        rtc_week=get_time.tm_wday; //周
        printf(" %d y %d m %d d %d h %d m %d s %d
w\n", rtc_year+1900, rtc_month+1, rtc_day, rtc_hour, rtc_minute, rtc_second, rtc_week);

    }

    if(rtc_alarm_flag)
    {
        rtc_alarm_flag=0;
        printf("/*****Alarm Clock*****/\n");
    }

    R_BSP_SoftwareDelay(10U, BSP_DELAY_UNITS_MILLISECONDS);
}
```

演示效果

设置 2 月 28 日，当过了凌晨 0 点后自动切换为 3 月 1 号显示。

[11:09:16.641]收←◆hello!
[11:09:17.651]收←◆ 2025 y 2 m 28 d 23 h 59 m 51 s 3 w
[11:09:18.651]收←◆ 2025 y 2 m 28 d 23 h 59 m 52 s 3 w
[11:09:19.651]收←◆ 2025 y 2 m 28 d 23 h 59 m 53 s 3 w
[11:09:20.651]收←◆ 2025 y 2 m 28 d 23 h 59 m 54 s 3 w
[11:09:21.650]收←◆ 2025 y 2 m 28 d 23 h 59 m 55 s 3 w
[11:09:22.651]收←◆ 2025 y 2 m 28 d 23 h 59 m 56 s 3 w
[11:09:23.650]收←◆ 2025 y 2 m 28 d 23 h 59 m 57 s 3 w
[11:09:24.651]收←◆ 2025 y 2 m 28 d 23 h 59 m 58 s 3 w
[11:09:25.650]收←◆ 2025 y 2 m 28 d 23 h 59 m 59 s 3 w
[11:09:26.651]收←◆ 2025 y 3 m 1 d 0 h 0 m 0 s 4 w
[11:09:27.650]收←◆ 2025 y 3 m 1 d 0 h 0 m 1 s 4 w
[11:09:28.650]收←◆ 2025 y 3 m 1 d 0 h 0 m 2 s 4 w
[11:09:29.648]收←◆ 2025 y 3 m 1 d 0 h 0 m 3 s 4 w

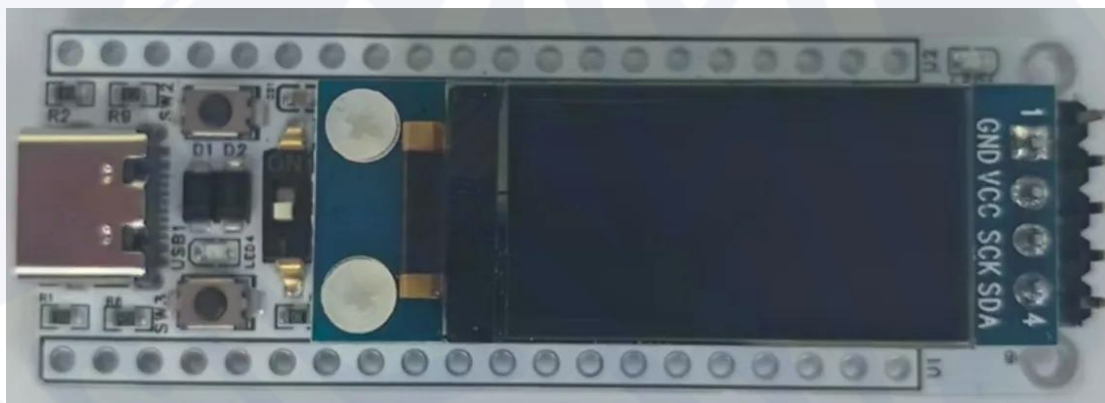
设置每过 1s 打印一次当前时间，设置过 1 分钟，在 10s 时候闹铃。

[11:09:28.650]收←◆ 2025 y 3 m 1 d 0 h 0 m 2 s 4 w
[11:09:29.648]收←◆ 2025 y 3 m 1 d 0 h 0 m 3 s 4 w
[11:09:30.648]收←◆ 2025 y 3 m 1 d 0 h 0 m 4 s 4 w
[11:09:31.648]收←◆ 2025 y 3 m 1 d 0 h 0 m 5 s 4 w
[11:09:32.647]收←◆ 2025 y 3 m 1 d 0 h 0 m 6 s 4 w
[11:09:33.646]收←◆ 2025 y 3 m 1 d 0 h 0 m 7 s 4 w
[11:09:34.647]收←◆ 2025 y 3 m 1 d 0 h 0 m 8 s 4 w
[11:09:35.646]收←◆ 2025 y 3 m 1 d 0 h 0 m 9 s 4 w
[11:09:36.646]收←◆ 2025 y 3 m 1 d 0 h 0 m 10 s 4 w
/*****Alarm
Clock*****/
[11:09:37.653]收←◆ 2025 y 3 m 1 d 0 h 0 m 11 s 4 w
[11:09:38.653]收←◆ 2025 y 3 m 1 d 0 h 0 m 12 s 4 w
[11:09:39.652]收←◆ 2025 y 3 m 1 d 0 h 0 m 13 s 4 w
[11:09:40.652]收←◆ 2025 y 3 m 1 d 0 h 0 m 14 s 4 w

project16: ADC 单通道采集电压

概述

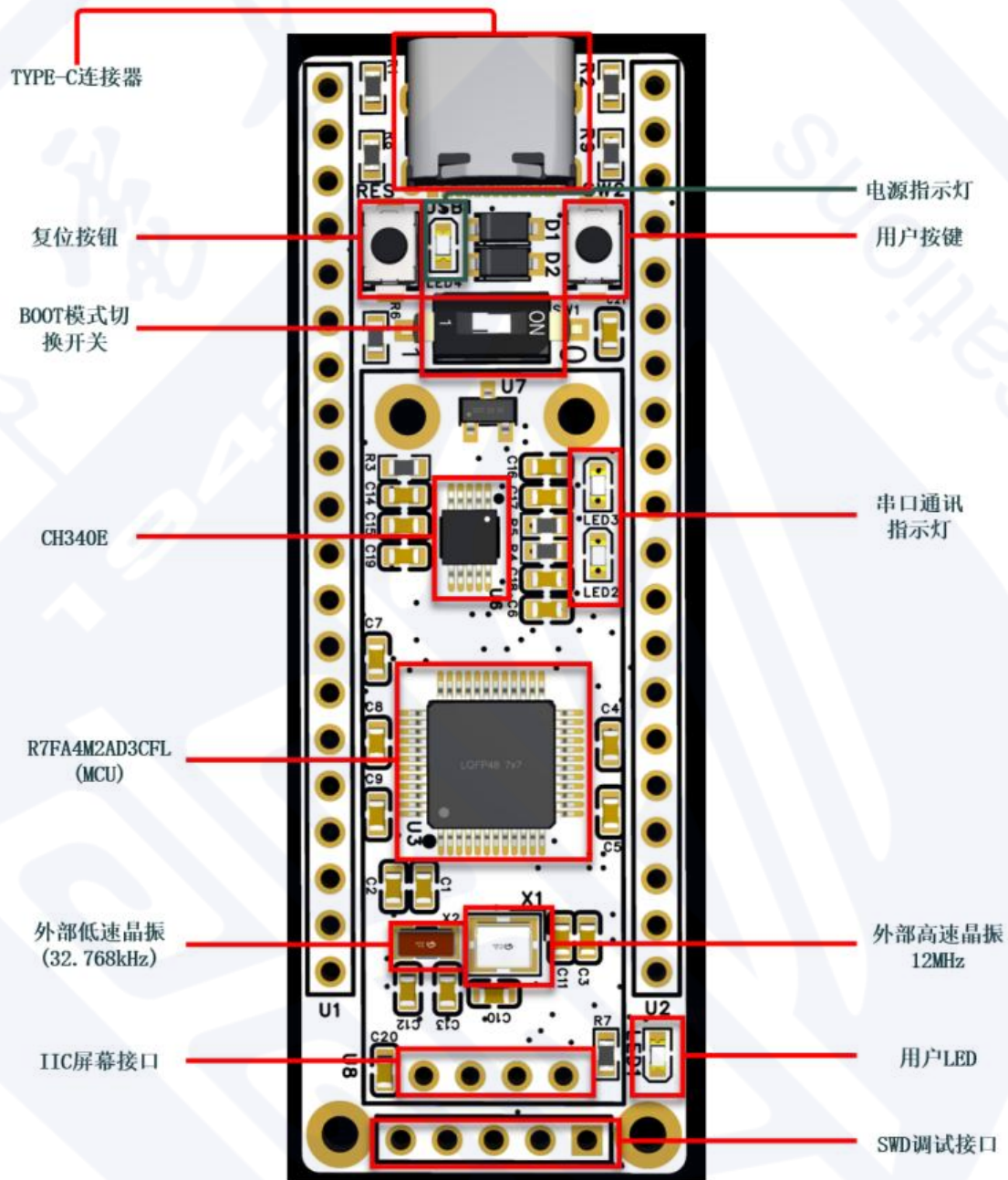
本篇文章主要介绍如何使用 e2studio 对瑞萨单片机进行 ADC 单通道采集电压。



硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL

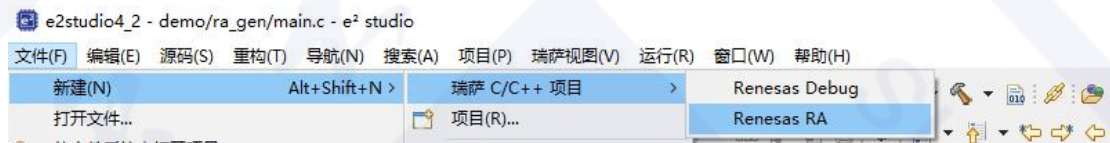


参考程序

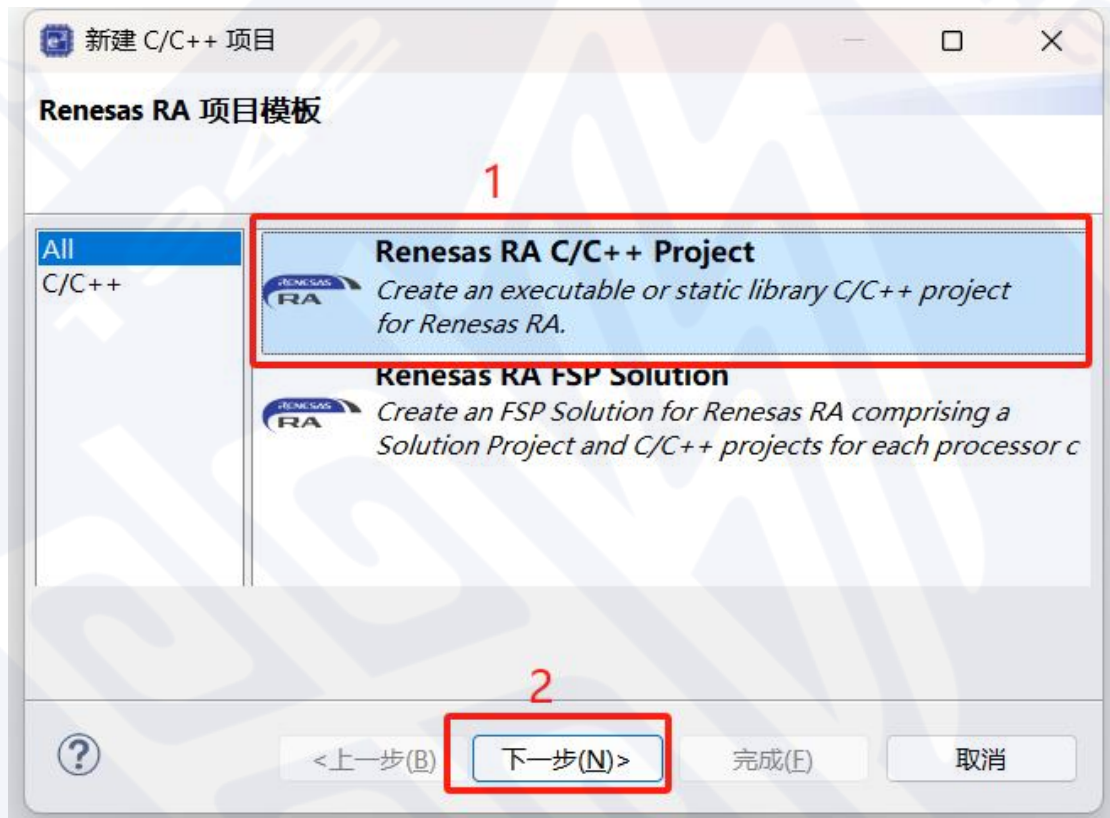
https://github.com/CoreMaker-lab/RA4M2_MINI

https://gitee.com/CoreMaker/RA4M2_MINI

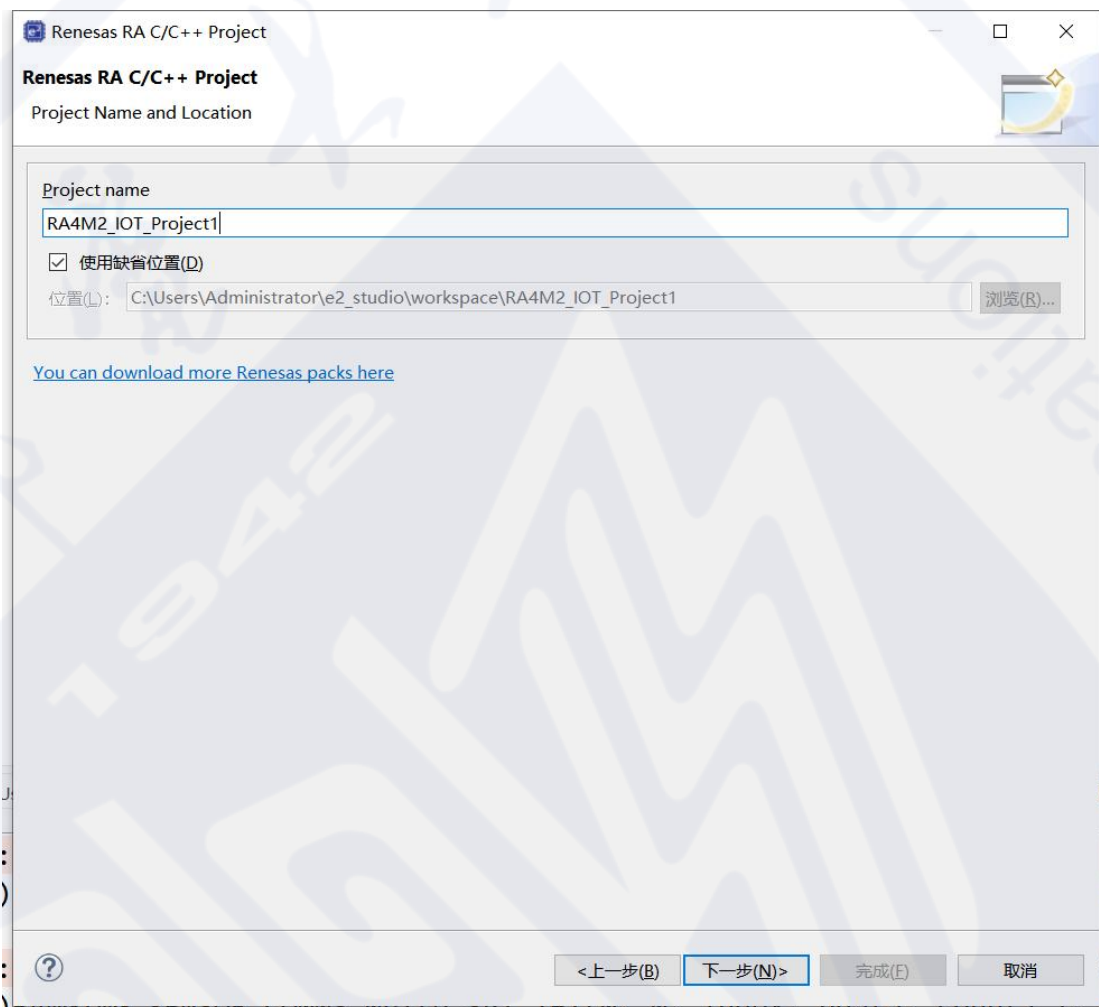
新建工程



工程模板



保存工程路径



芯片配置

本文中使用 R7FA4M2AD3CFL 来进行演示。

Renesas RA C/C++ Project

Renesas RA C/C++ Project

Device and Tools Selection

Device Selection

FSP Version: 5.9.0 **版本**

Board: Custom User Board (Any Device) ...

Device: R7FA4M2AD3CFL **芯片型号**

Core: CM33

Language: ☒ C ☐ C++

Board Description

Device Details

TrustZone	Yes
Pins	48
Processor	Cortex-M33

IDE Project Type

e2 studio managed build

Toolchains

LLVM Embedded Toolchain for Arm

GNU ARM Embedded **工具链**

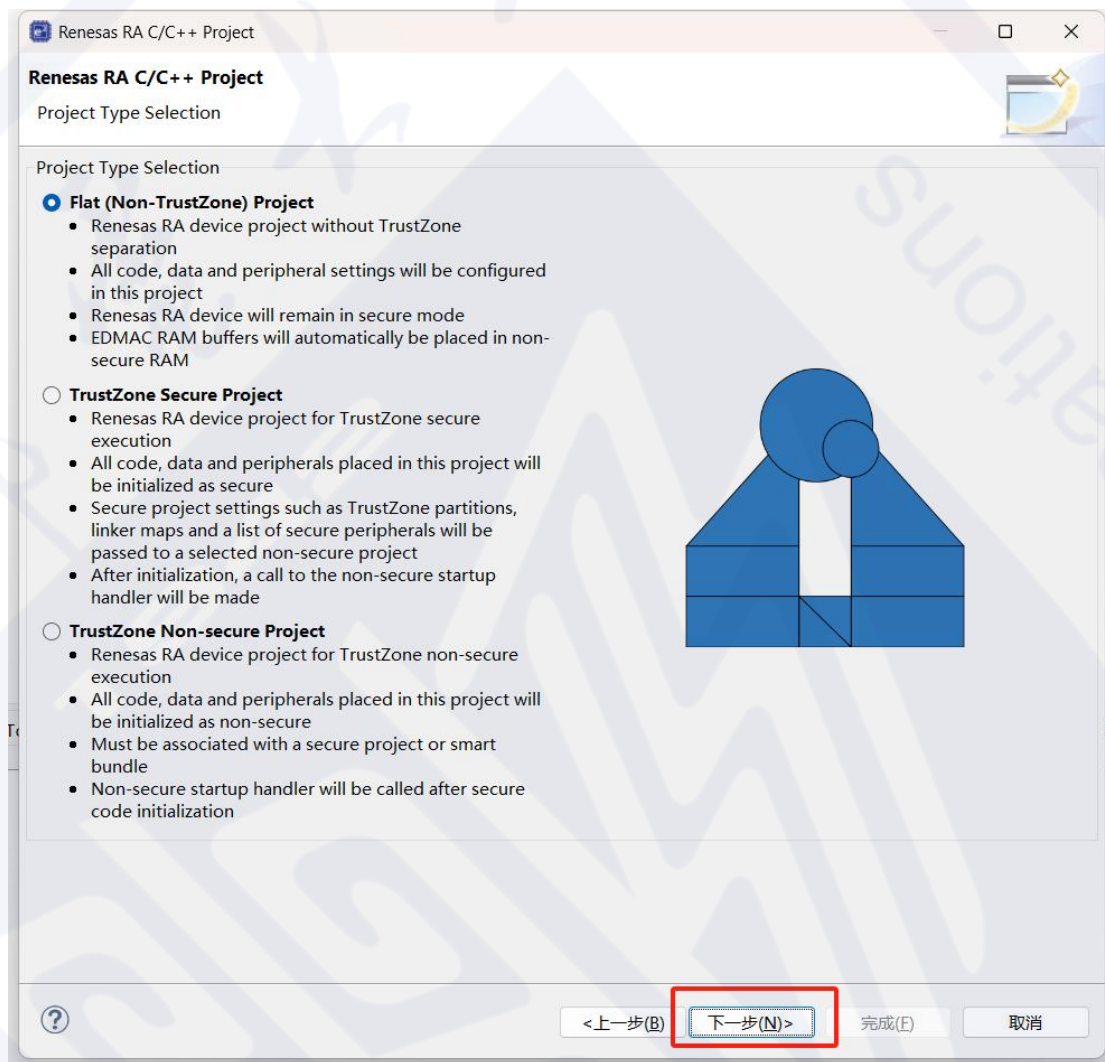
13.2.1.arm-13-7 [Manage Toolchains...](#)

Debugger

J-Link ARM **使用JLink进行调试**

[?](#) <上一步(B) 下一步(N)> 完成(F) 取消

工程模板选择



Renesas RA C/C++ Project

Renesas RA C/C++ Project

Preceding Project or Smart Bundle Selection

☒ None

Choose this option when creating a project for the primary processor core (no preceding project or Smart Bundle).

☐ Preceding Project:

Choose this option if you have access to the preceding bootloader project or TrustZone secure project source code. A debug configuration for Secure Software Debug (SSD) state will be generated where applicable and both secure and non-secure images will be downloaded to the target device. Should you wish to test Non-Secure Software Debug (NSECSO) state, use the Renesas Device Partition Manager to change the Device Lifecycle state as needed.

☐ Smart Bundle:

Resolved location:

Workspace...

File System...

Variables...

Choose this option if you only have access to a pre-programmed device containing TrustZone secure code. The Smart Bundle file (*.sbd) should be obtained from the TrustZone Secure Project developer. A debug configuration for Non-Secure Software Debug (NSECSO) state will be generated and only the non-secure code image will be downloaded to the target device.

Preceding Project/Smart Bundle Details

FSP version

Toolchain

Toolchain version

Board

Device

Core

Zones

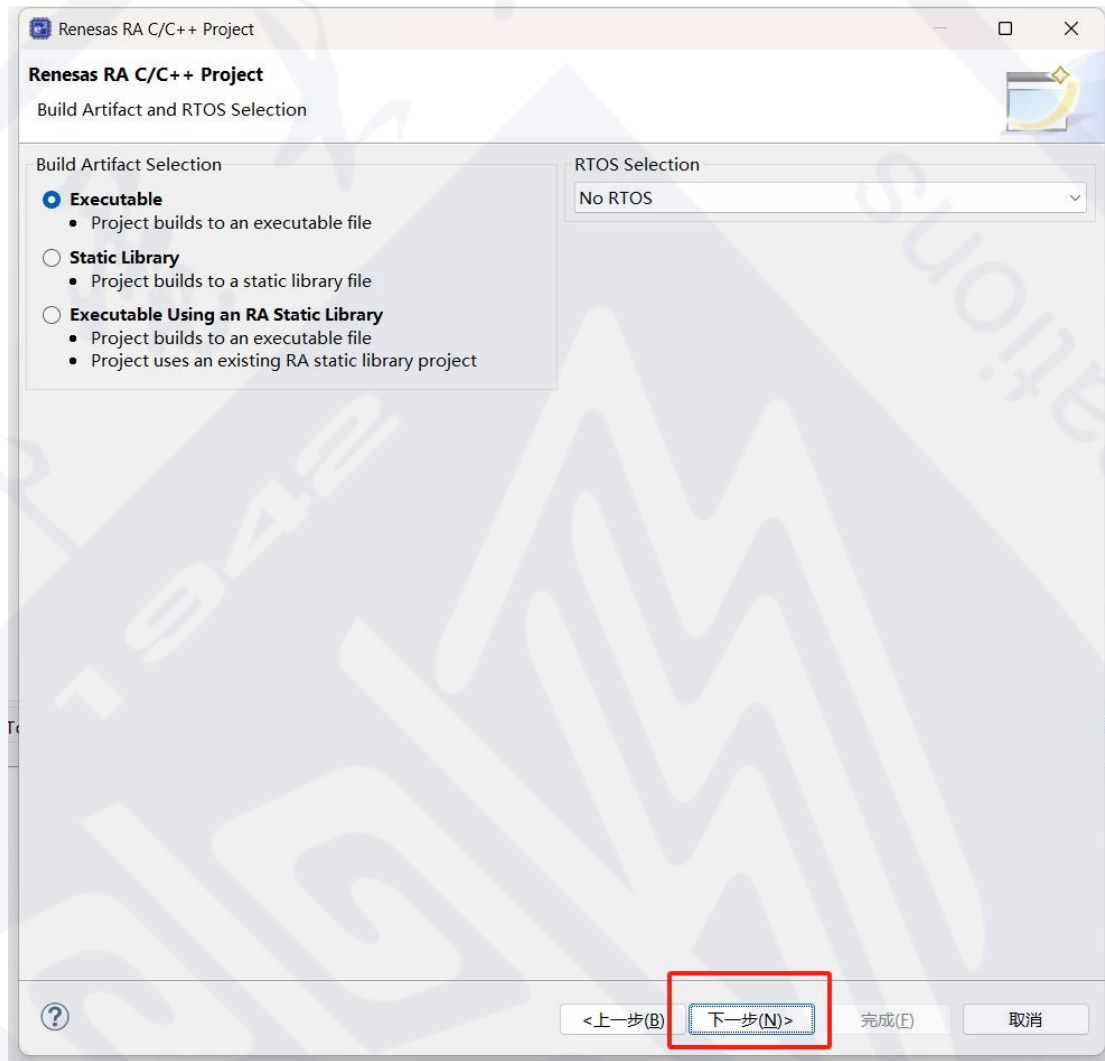
?

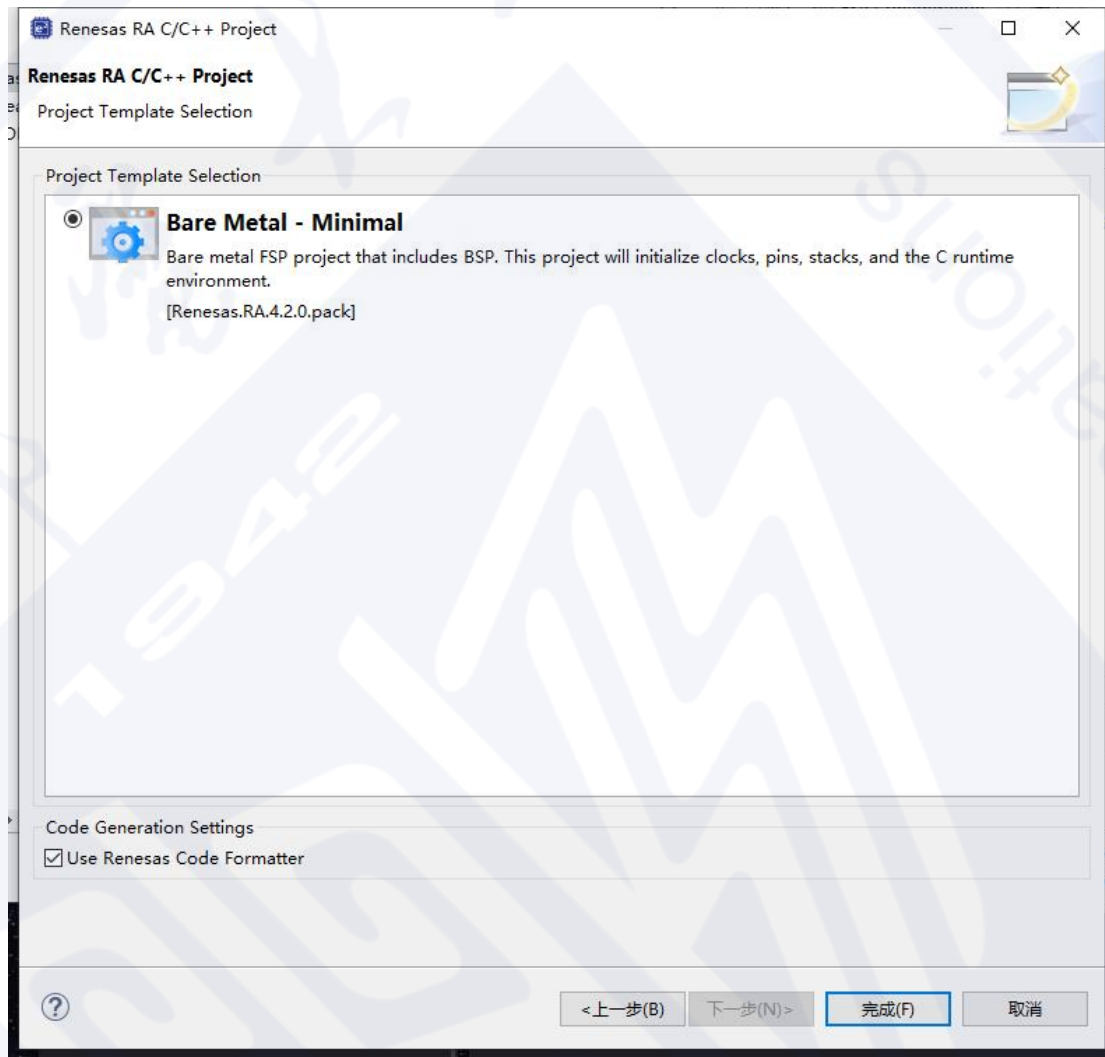
< 上一步(B)

下一步(N) >

完成(E)

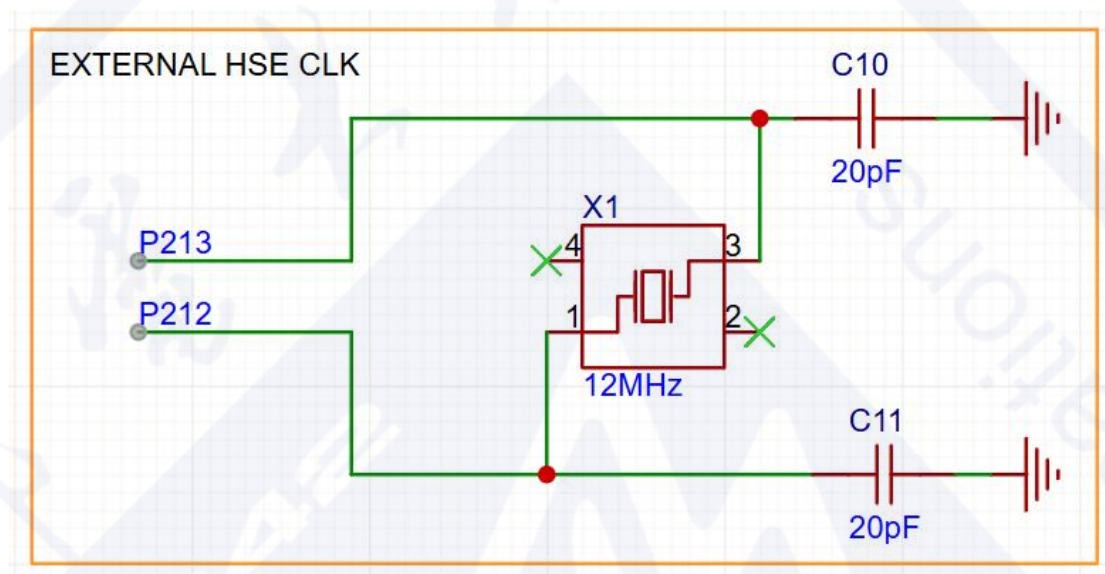
取消



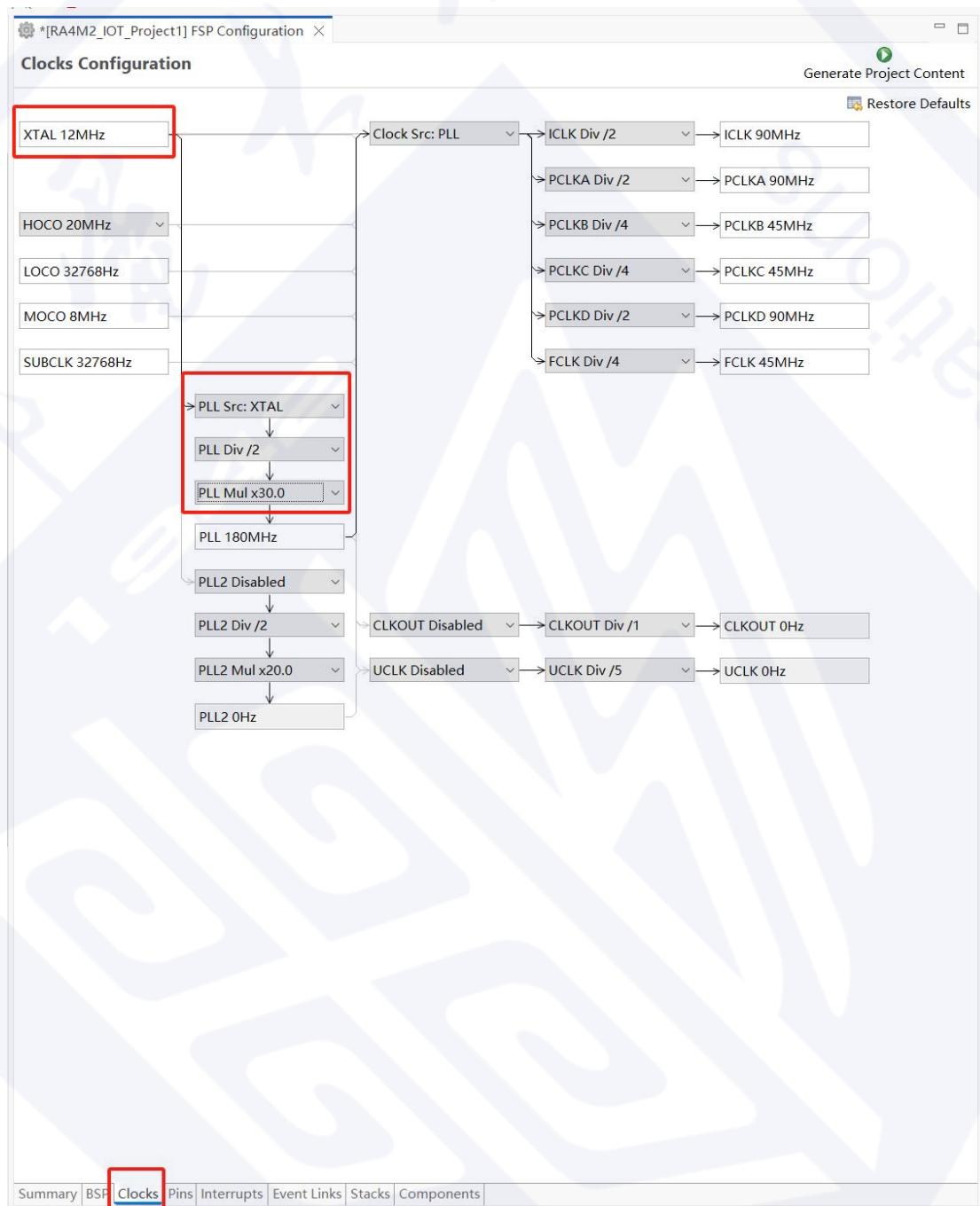


时钟设置

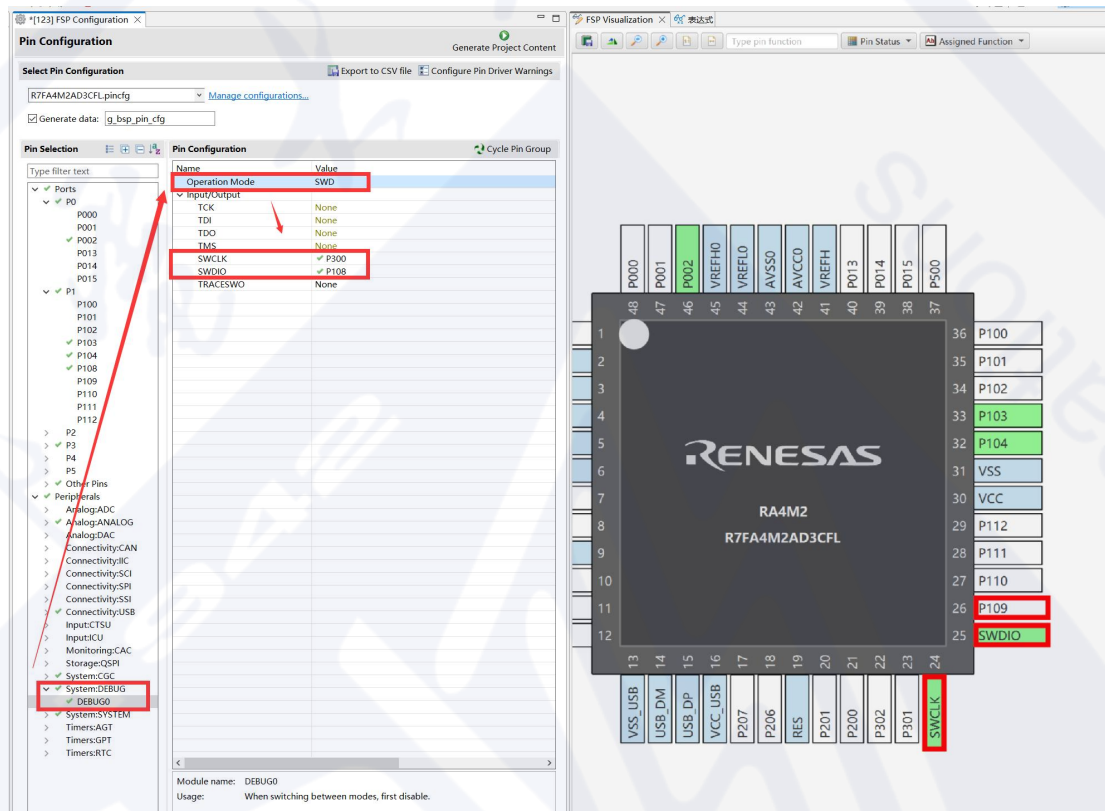
开发板上的外部高速晶振为 12M.



需要修改 XTAL 为 12M。

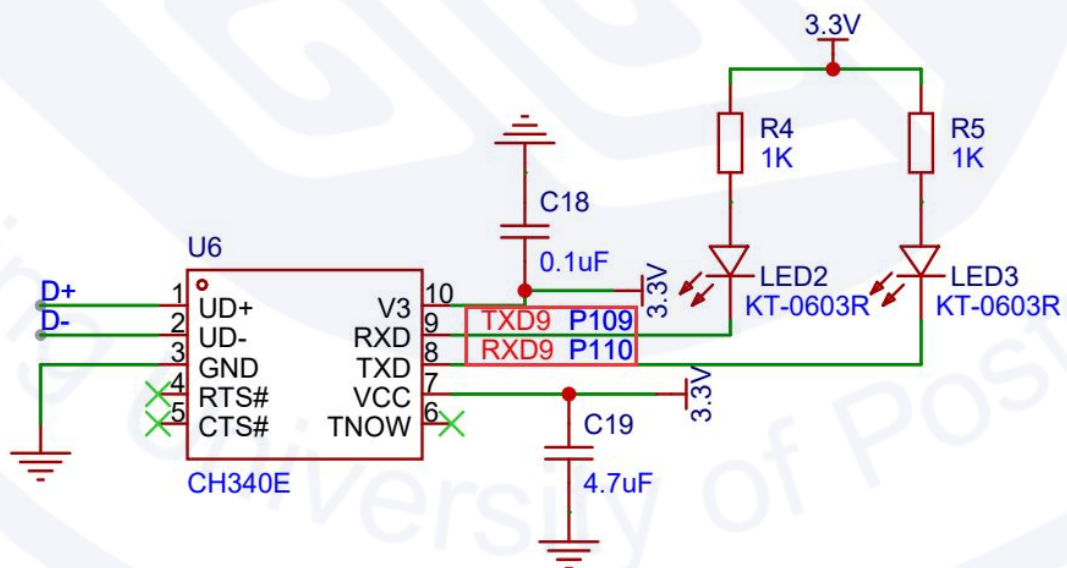


SWD 调试口设置

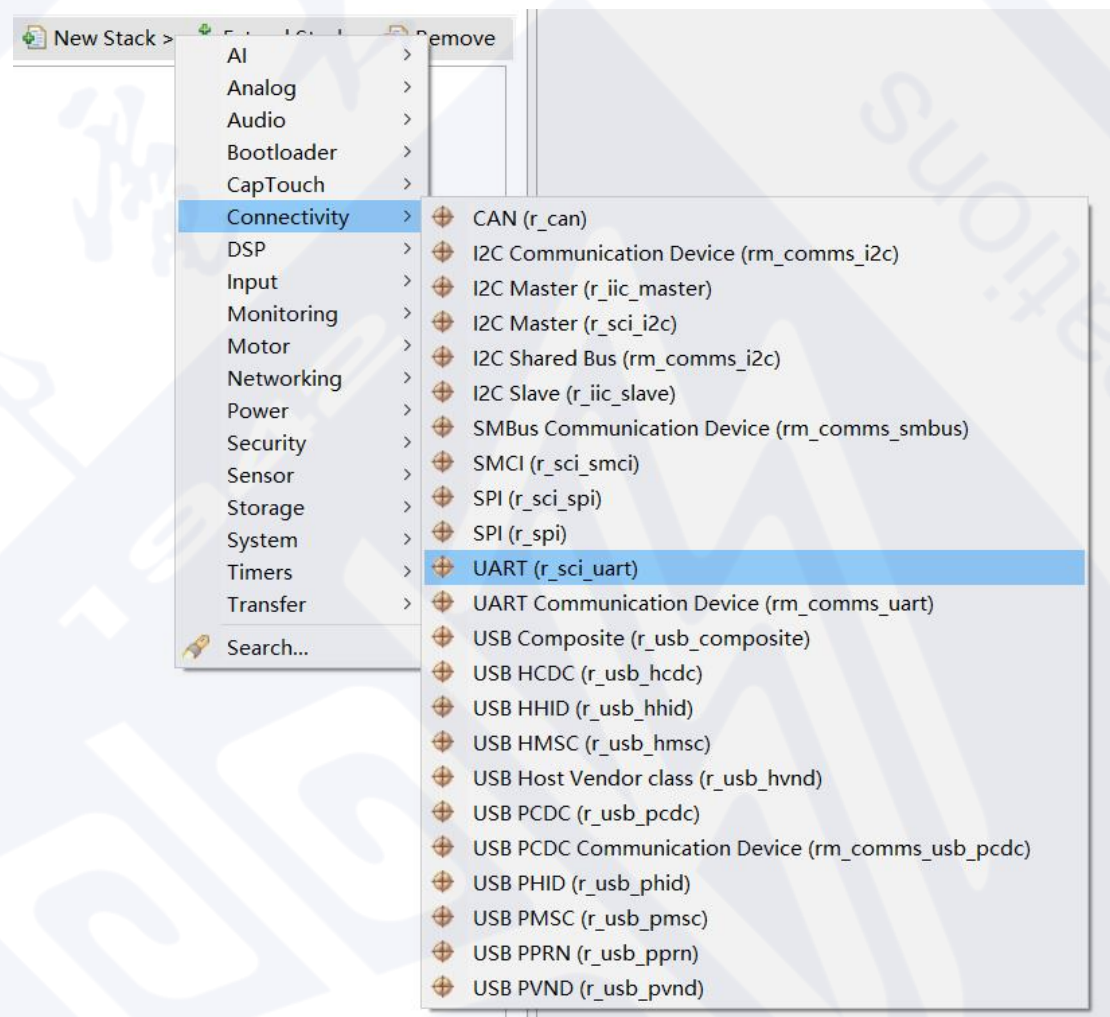


UART 配置

串口CH340E



点击 Stacks->New Stack->Connectivity -> UART(r_sci_uart)。



UART 属性配置

属性 ×	问题	Smart Browser	控制台	调试
g_uart9 UART (r_sci_uart)				
Settings	属性		值	
API Info	▼ Common			
	Parameter Checking		Default (BSP)	
	FIFO Support		Disable	
	DTC Support		Disable	
	Flow Control Support		Disable	
	RS-485 Support		Disable	
	IrDA Support		Disabled	
	▼ Module g_uart9 UART (r_sci_uart)			
	▼ General			
	Name		g_uart9	
	Channel		9	
	Data Bits		8bits	
	Parity		None	
	Stop Bits		1bit	
	▼ Baud			
	Baud Rate		115200	
	Baud Rate Modulation		Enabled	
	Max Error (%)		5	
	► Flow Control			
	► Extra			
	▼ Interrupts			
	Callback		user_uart_callback	
	Receive Interrupt Priority		Priority 2	
	Transmit Data Empty Interrupt Priority		Priority 2	
	Transmit End Interrupt Priority		Priority 2	
	Error Interrupt Priority		Priority 2	
	▼ Pins			
	TXD9		P109	
	RXD9		P110	
	CTS9		<unavailable>	

名称
通道
数据位
无校验位
停止位

波特率
启用波特率调制

回调函数名称

串口发送端口
串口接收端口

设置 e2studio 堆栈

printf 函数通常需要设置堆栈大小。这是因为 printf 函数在运行时需要使用栈空间来存储临时变量和函数调用信息。如果堆栈大小不足，可能会导致程序崩溃或不可预期的行为。

printf 函数使用了可变参数列表，它会在调用时使用栈来存储参数，在函数调用结束时再清除参数，这需要足够的栈空间。另外 printf 也会使用一些临时变量，如果栈空间不足，会导致程序崩溃。

因此，为了避免这类问题，应该根据程序的需求来合理设置堆栈大小。

项目资源管理器 | [RA2M4_print] FSP Configuration

Board Support Package Configuration

Generate Project Content | Restore Defaults

Device Selection

FSP version: 4.2.0 | Board: Custom User Board (Any Device) | Device: R7FA4M2AD3CFP | Core: CM33 | RTOS: No RTOS

Board Details

Summary **BSP** | Clocks | Pins | Interrupts | Event Links | Stacks | Components

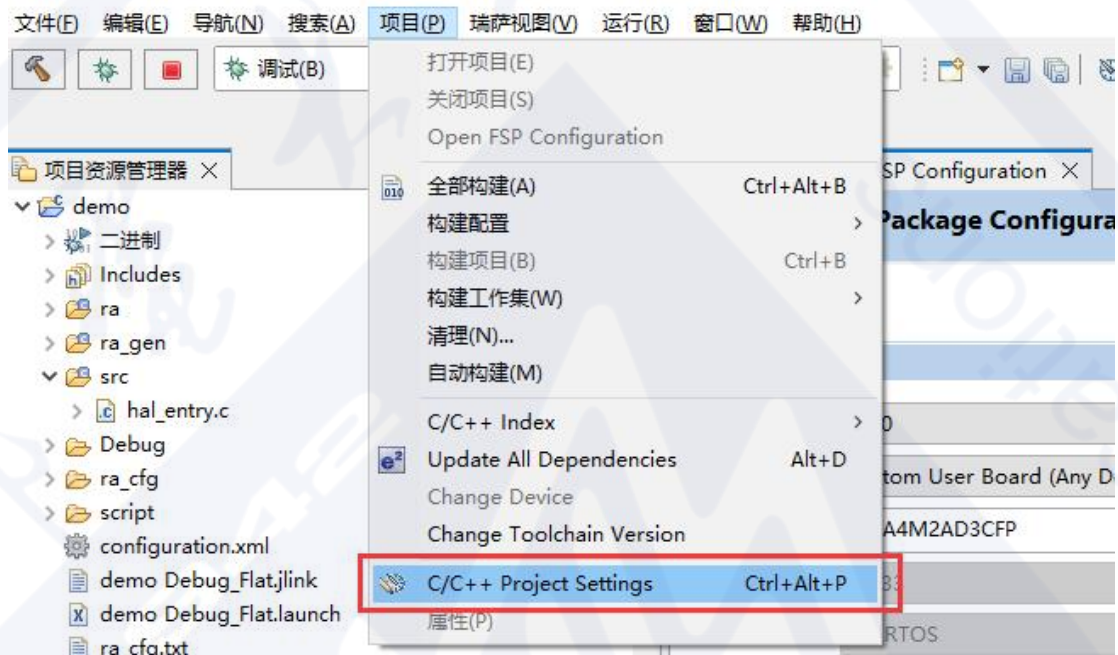
Custom User Board (Any Device)

Settings | 属性 | 问题 | Smart Browser | 控制台

属性	值
√ R7FA4M2AD3CFP	
part_number	R7FA4M2AD3CFP
rom_size_bytes	524288
ram_size_bytes	131072
data_flash_size_bytes	8192
package_style	LQFP
package_pins	100
√ RA4M2	
series	4
√ RA4M2 Family	
> Security	
> OFS0 register settings	
> OFS1 register settings	
> Block Protection Settings (BPS)	
> Permanent Block Protection Settings (PBPS)	
> Clocks	
Startup C-Cache Line Size	32 Bytes
Main Oscillator Wait Time	8163 cycles
√ RA Common	
Main stack size (bytes)	0x400
Heap size (bytes)	0x2000
MCU Vcc (mV)	3300
Parameter checking	Disabled
Assert Failures	Return FSP_ERR_ASSERTION
Error Log	No Error Log
Clock Registers not Reset Values during Startup	Disabled
Main Oscillator Populated	Populated
PFS Protect	Enabled
C Runtime Initialization	Enabled
Early BSP Initialization	Disabled
Main Oscillator Clock Source	Crystal or Resonator
Subclock Populated	Populated
Subclock Drive (Drive capacitance availability varies by MCU)	Standard/Normal mode
Subclock Stabilization Time (ms)	1000

3.设置堆栈大小

e2studio 的重定向 printf 设置



在嵌入式系统的开发中，尤其是在使用 GNU 编译器集合（GCC）时，`--specs` 参数用于指定链接时使用的系统规格（specs）文件。这些规格文件控制了编译器和链接器的行为，尤其是关于系统库和启动代码的链接。`--specs=rdimon.specs` 和 `--specs=nosys.specs` 是两种常见的规格文件，它们用于不同的场景。

`--specs=rdimon.specs`

用途：这个选项用于链接“Redlib”库，这是为裸机（bare-metal）和半主机（semihosting）环境设计的 C 库的一个变体。半主机环境是一种特殊的运行模式，允许嵌入式程序通过宿主机（如开发 PC）的调试器进行输入输出操作。

应用场景：当你需要在没有完整操作系统的环境中运行程序，但同时需要使用调试器来处理输入输出（例如打印到宿主机的终端），这个选项非常有用。

特点：它提供了一些基本的系统调用，通过调试接口与宿主机通信。

`--specs=nosys.specs`

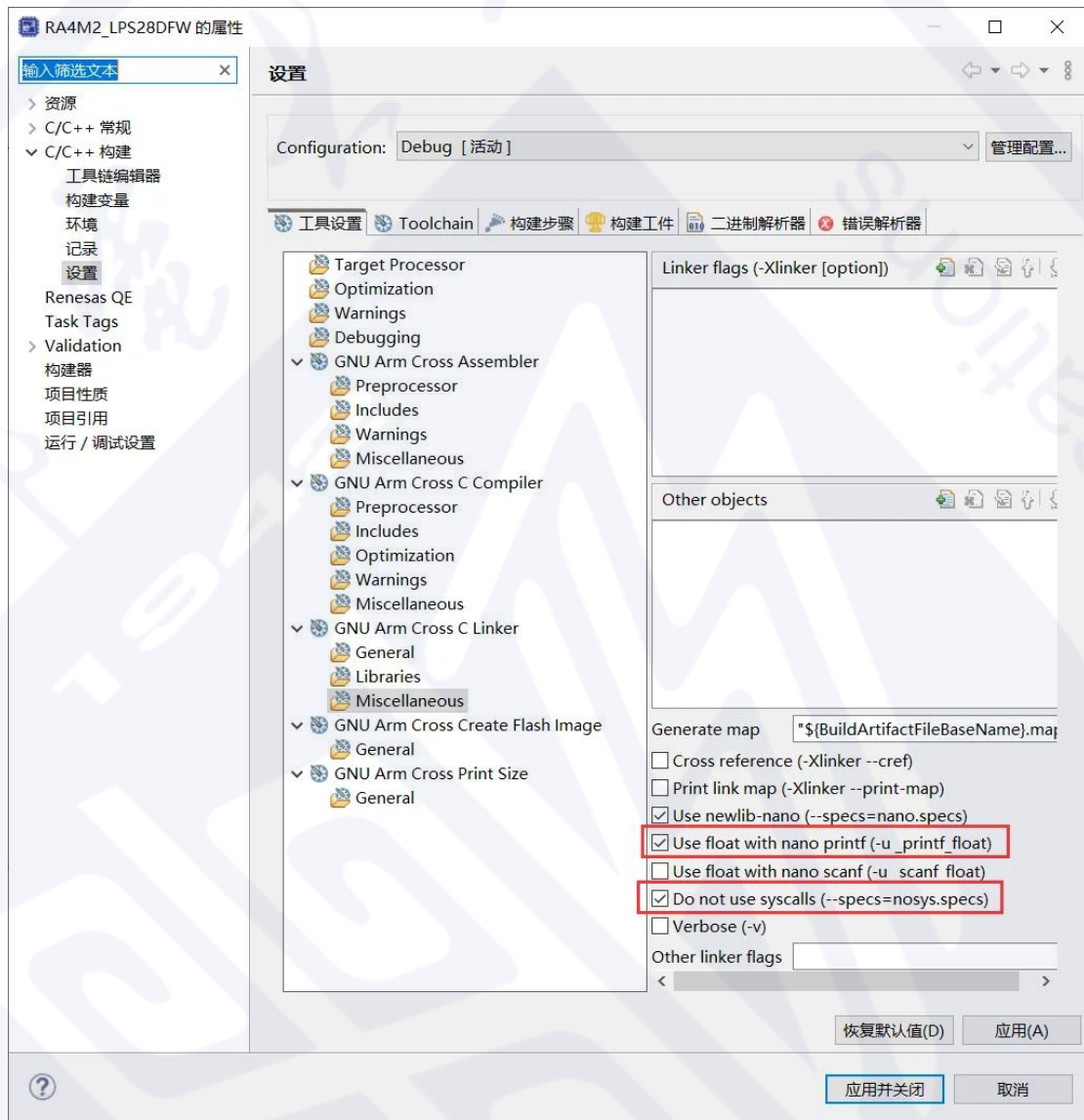
用途：这个选项链接了一个非常基本的系统库，这个库不提供任何系统服务的实现。

应用场景：适用于完全的裸机程序，其中程序不执行任何操作系统调用，比如不进行文件操作或者系统级输入输出。

特点：这是一个更“裸”的环境，没有任何操作系统支持。使用这个规格文件，程序不期望有操作系统层面的任何支持。

如果你的程序需要与宿主机进行交互（如在开发期间的调试），并且通过调试器进行基本的输入输出操作，则使用 `--specs=rdimon.specs`。

如果你的程序是完全独立的，不需要任何形式的操作系统服务，包括不进行任何系统级的输入输出，则使用 `--specs=nosys.specs`。



R_SCI_UART_Open() 函数原型

◆ R_SCI_UART_Open()

```
fsp_err_t R_SCI_UART_Open ( uart_ctrl_t *const p_api_ctrl,  
                           uart_cfg_t const *const p_cfg  
                           )
```

Configures the UART driver based on the input configurations. If reception is enabled at compile time, reception is enabled at the end of this function. Implements `uart_api_t::open`

Return values

FSP_SUCCESS	Channel opened successfully.
FSP_ERR_ASSERTION	Pointer to UART control block or configuration structure is NULL.
FSP_ERR_IP_CHANNEL_NOT_PRESENT	The requested channel does not exist on this MCU.
FSP_ERR_INVALID_ARGUMENT	Flow control is enabled but flow control pin is not defined or selected channel does not support "Hardware CTS and Hardware RTS" flow control. (or) restricted channel is selected.
FSP_ERR_ALREADY_OPEN	Control block has already been opened or channel is being used by another instance. Call <code>close()</code> then <code>open()</code> to reconfigure.
FSP_ERR_INVALID_CHANNEL	IrDA is requested for a channel that does not support IrDA.

Returns

See [Common Error Codes](#) or functions called by this function for other possible return codes. This function calls:

- `transfer_api_t::open`

故可以用 `R_SCI_UART_Open()`函数进行配置，开启和初始化 UART。

```
/* Open the transfer instance with initial configuration. */  
err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);  
assert(FSP_SUCCESS == err);
```

```
[RA4M2_MINI_Project11_1] FSP Configuration  *hal_entry.c x startup.c main.c
2
3 FSP_CPP_HEADER
4 void R_BSP_WarmStart(bsp_warm_start_event_t event);
5 FSP_CPP_FOOTER
6 /* unused parameter 'fd' [-Wunused-parameter] | /* no previous declaration for '_write' [-W
7
8
9
10
11
12
13
14
15 /* Open the transfer instance with initial configuration. */
16 err = R_SCI_UART_Open(&g_uart9_ctrl1, &g_uart9_cfg);
17 assert(FSP_SUCCESS == err);
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
```

回调函数 user_uart_callback ()

当数据发送的时候，可以查看 UART_EVENT_TX_COMPLETE 来判断是否发送完毕。

◆ write

```
fsp_err_t (*uart_api_t::write)(uart_ctrl_t *const p_ctrl, uint8_t const *const p_src, uint32_t const bytes)
```

Write to UART device. The write buffer is used until write is complete. Do not overwrite write buffer contents until the write is finished. When the write is complete (all bytes are fully transmitted on the wire), the callback called with event **UART_EVENT_TX_COMPLETE**. The maximum transfer size is reported by `infoGet()`.

Implemented as

- R_SCI_UART_Write[]
- R_SCI_B_UART_Write[]

Parameters

[in] p_ctrl Pointer to the UART control block.

[in] p_src Source address to write data to.

[in] bytes Write data length.

Enumeration Type Documentation

◆ uart_event_t

enum `uart_event_t`

UART Event codes

Enumerator	
UART_EVENT_RX_COMPLETE	Receive complete event.
UART_EVENT_TX_COMPLETE	Transmit complete event.
UART_EVENT_RX_CHAR	Character received.
UART_EVENT_ERR_PARITY	Parity error event.
UART_EVENT_ERR_FRAMING	Mode fault error event.
UART_EVENT_ERR_OVERFLOW	FIFO Overflow error event.
UART_EVENT_BREAK_DETECT	Break detect error event.
UART_EVENT_TX_DATA_EMPTY	Last byte is transmitting, ready for more data.

可以检查检查 "p_args" 结构体中的 "event" 字段的值是否等于 "UART_EVENT_TX_COMPLETE"。如果条件为真，那么 if 语句后面的代码块将会执行。

```
fsp_err_t err = FSP_SUCCESS;
volatile bool uart_send_complete_flag = false;
void user_uart_callback (uart_callback_args_t * p_args)
{
    if(p_args->event == UART_EVENT_TX_COMPLETE)
    {
        uart_send_complete_flag = true;
    }
}
```

```
[RA4M2_MINI_Project11_1] FSP Configuration  *hal_entry.c  startup.c  main.c
1  #include "hal_data.h"
2
3  FSP_CPP_HEADER
4  void R_BSP_WarmStart(bsp_warm_start_event_t event);
5  FSP_CPP_FOOTER
6
7  fsp_err_t err = FSP_SUCCESS;
8  volatile bool uart_send_complete_flag = false;
9  void user_uart_callback (uart_callback_args_t * p_args)
10 {
11     if(p_args->event == UART_EVENT_TX_COMPLETE)
12     {
13         uart_send_complete_flag = true;
14     }
15 }
16
17 unused parameter 'fd' [-Wunused-parameter] | no previous declaration for '_write' [-W
18
19 * main() is generated by the RA Configuration editor and is used to generate threads if
20 void hal_entry(void)
21 {
22     /* TODO: add your own code here */
23
24     /* Open the transfer instance with initial configuration. */
25     err = R_SCI_UART_Open(&g_uart9_ctrl1, &g_uart9_cfg);
26     assert(FSP_SUCCESS == err);
27
28     #if BSP_TZ_SECURE_BUILD
29     /* Enter non-secure code */
30     R_BSP_NonSecureEnter();
31     #endif
32
33     * This function is called at various points during the startup process. This implement
34     void R_BSP_WarmStart(bsp_warm_start_event_t event)
35     {
36         if (BSP_WARM_START_RESET == event)
37         {
38             #if BSP_FEATURE_FLASH_LP_VERSION != 0
39
```

printf 输出重定向到串口

打印最常用的方法是 `printf`，所以要解决的问题是将 `printf` 的输出重定向到串口，然后通过串口将数据发送出去。

注意一定要加上头文件 `#include <stdio.h>`

```
#ifdef __GNUC__
//串口重定向
#define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
#else
#endif

PUTCHAR_PROTOTYPE
{
    err = R_SCI_UART_Write(&g_uart9_ctrl1, (uint8_t *)&ch, 1);
    if(FSP_SUCCESS != err) __BKPT();
    while(uart_send_complete_flag == false){}
    uart_send_complete_flag = false;
}
```

```

        return ch;
    }

    int _write(int fd, char *pBuffer, int size)
    {
        for(int i=0; i<size; i++)
        {
            __io_putchar(*pBuffer++);
        }

        return size;
    }
}

```

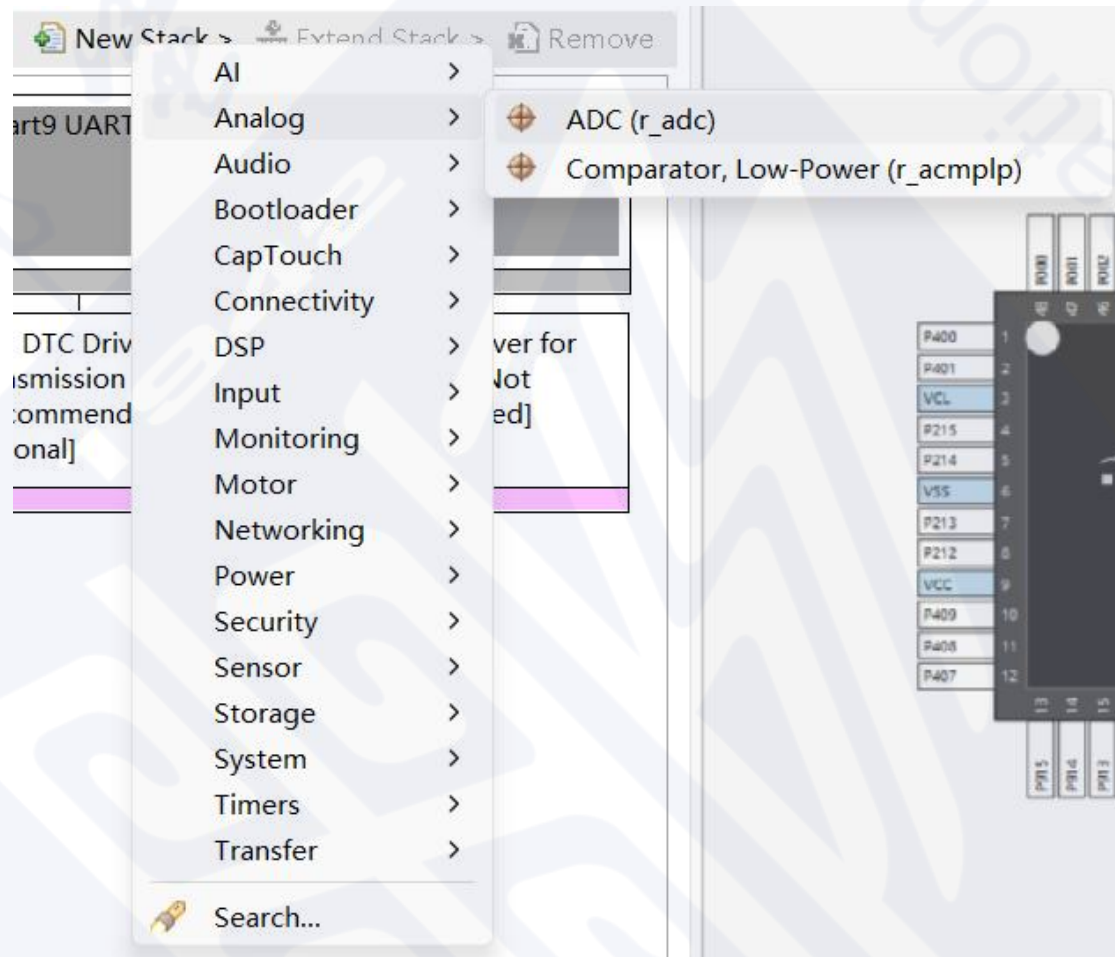
```

1  #include "hal_data.h"
2  #include <stdio.h>
3  FSP_CPP_HEADER
4  void R_BSP_WarmStart(bsp_warm_start_event_t event);
5  FSP_CPP_FOOTER
6
7  fsp_err_t err = FSP_SUCCESS;
8  volatile bool uart_send_complete_flag = false;
9  void user_uart_callback (uart_callback_args_t * p_args)
10 {
11     if(p_args->event == UART_EVENT_TX_COMPLETE)
12     {
13         uart_send_complete_flag = true;
14     }
15 }
16
17
18 #ifdef __GNUC__ //串口重定向
19 #define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
20 #else
21 #define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
22 #endif
23
24
25 PUTCHAR_PROTOTYPE
26 {
27     err = R_SCI_UART_Write(&g_uart9_ctrl, (uint8_t *)&ch, 1);
28     if(FSP_SUCCESS != err) __BKPT();
29     while(uart_send_complete_flag == false){}
30     uart_send_complete_flag = false;
31     return ch;
32 }
33
34 int _write(int fd, char *pBuffer, int size)
35 {
36     for(int i=0; i<size; i++)
37     {
38         __io_putchar(*pBuffer++);
39     }
40     return size;
41 }
42
43
44
45 * * main() is generated by the RA Configuration editor and is used to generate threads i
46 void hal_entry(void)
47 {
48     /* TODO: add your own code here */
49
50     /* Open the transfer instance with initial configuration. */
51     err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);
52     assert(FSP_SUCCESS == err);
53
54
55

```

ADC 配置

点击 Stacks->New Stack->Driver->Analog -> ADC (r_adc)。



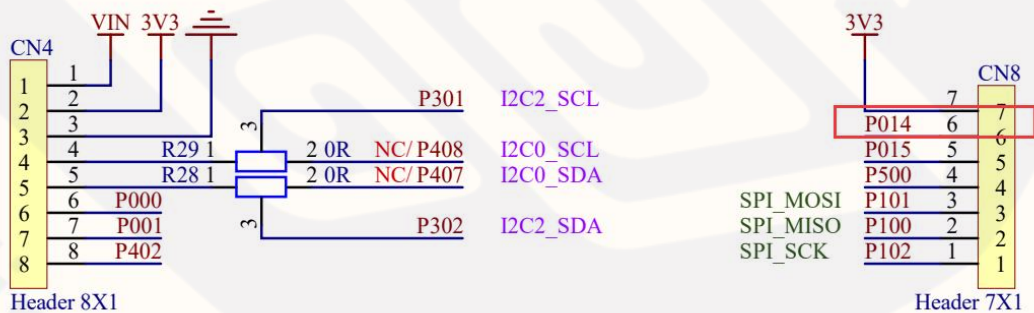
ADC 属性配置

- **Name:** `g_adc0` 该 ADC 实例的名称，在代码中用于引用此 ADC 设备。
- **Unit:** `0` 表示使用的是 ADC0 设备（MCU 可能有多个 ADC 单元，如 ADC0、ADC1 等）。
- **Resolution:** `12-Bit` 选择 ADC 分辨率为 12 位，即转换后的数值范围为 `0~4095` ($2^{12} = 4096$)。
- **Alignment:** `Right` 选择数据右对齐，即 ADC 结果存储时，低 12 位有效，高位填充 `0`。
- **Clear after read:** `On` 每次读取 ADC 结果后，寄存器会自动清除，防止旧数据干扰新采样。
- **Mode:** `Single Scan` 单次扫描模式，即每次触发 ADC 转换后，仅采样一次选中的通道。

属性 × 问题 Smart Browser 控制台 调试	
g_adc0 ADC (r_adc)	
属性	值
Settings	
API Info	
▼ Common	
Parameter Checking	Default (BSP)
▼ Module g_adc0 ADC (r_adc)	
▼ General	
Name	g_adc0
Unit	0
Resolution	12-Bit
Alignment	Right
Clear after read	On
Mode	Single Scan
Double-trigger	Disabled
▼ Input	
▼ Channel Scan Mask (channel availability varies by MCU)	
Channel 0	☐
Channel 1	☐
Channel 2	☐
Channel 3	☐
Channel 4	☐
Channel 5	☐
Channel 6	☐
Channel 7	☐
Channel 8	☐
Channel 9	☐
Channel 10	☐
Channel 11	☐
Channel 12	☒
Channel 13	☐
Channel 14	☐
Channel 15	☐
Channel 16	☐
Channel 17	☐
Channel 18	☐
Channel 19	☐
Channel 20	☐
Channel 21	☐
Channel 22	☐
Channel 23	☐
Channel 24	☐
Channel 25	☐
Channel 26	☐
Channel 27	☐
Channel 28	☐
Temperature Sensor	☐
Internal Reference Voltage	☐

名称
ADC0 设备
ADC 分辨率为 12 位
右对齐
寄存器自动清除
单次扫描模式

对 ADC 通道 12 进行采样



- Normal/Group A Trigger：SoftwareADC 触发方式为软件触发，即需要手动调用 API `R_ADC_ScanStart()` 来启动转换。
- Callback（回调函数）：指定回调函数 `adc_callback`，用于 ADC 转换完成后的事件处理。
- Scan End Interrupt Priority（扫描结束中断优先级）：Priority 2ADC 扫描结束中断的优先级设置为 2（数值越小，优先级越高）。

- AN012 → P014 : ADC 通道 12 (AN012) 映射到 P014 引脚, 即 ADC 采样的模拟信号输入在 P014 引脚。

▼ Interrupts		
Normal/Group A Trigger	Software	ADC 触发方式为软件触发
Group B Trigger	Disabled	
Group Priority (Valid only in Group Scan Mode)	Group A cannot interrupt Group B	
Callback	adc_callback	回调函数
Scan End Interrupt Priority	Priority 2	优先级
Scan End Group B Interrupt Priority	Disabled	
Window Compare A Interrupt Priority	Disabled	
Window Compare B Interrupt Priority	Disabled	
> Extra		
▼ Pins		
ADTRG0	None	
AN000	None	
AN001	None	
AN002	None	
AN011	None	
AN012	P014	P014管脚
AN013	None	
AN016	None	

R_ADC_Open() 函数原型

◆ R_ADC_Open()

```
fsp_err_t R_ADC_Open ( adc_ctrl_t* p_ctrl,
                      adc_cfg_t const *const p_cfg
                      )
```

Sets the operational mode, trigger sources, interrupt priority, and configurations for the peripheral as a whole. If interrupt is enabled, the function registers a callback function pointer for notifying the user whenever a scan has completed.

Return values

FSP_SUCCESS	Module is ready for use.
FSP_ERR_ASSERTION	An input argument is invalid.
FSP_ERR_ALREADY_OPEN	The instance control structure has already been opened.
FSP_ERR_IRQ_BSP_DISABLED	A callback is provided, but the interrupt is not enabled.
FSP_ERR_IP_CHANNEL_NOT_PRESENT	The requested unit does not exist on this MCU.
FSP_ERR_INVALID_HW_CONDITION	The ADC clock must be at least 1 MHz

故可以用 R_ADC_Open() 函数进行配置, 开启和初始化 ADC 模式。

```
/* Initializes the module. */
err = R_ADC_Open(&g_adc0_ctrl, &g_adc0_cfg);

/* Handle any errors. This function should be defined by the user. */
assert(FSP_SUCCESS == err);
```

R_ADC_ScanCfg() 函数原型

◆ R_ADC_ScanCfg()

```
fsp_err_t R_ADC_ScanCfg ( adc_ctrl_t * p_ctrl,  
                        void const *const p_channel_cfg  
                        )
```

Configures the ADC scan parameters. Channel specific settings are set in this function. Pass a pointer to `adc_channel_cfg_t` to `p_channel_cfg`.

Note

This starts group B scans if `adc_channel_cfg_t::priority_group_a` is set to `ADC_GROUP_A_GROUP_B_CONTINUOUS_SCAN`.

Return values

FSP_SUCCESS Channel specific settings applied.
FSP_ERR_ASSERTION An input argument is invalid.
FSP_ERR_NOT_OPEN Unit is not open.

配置 ADC 扫描参数，赋能 ADC 通道。在此函数中设置通道特定设置。

```
/* Enable channels. */  
  
err = R_ADC_ScanCfg(&g_adc0_ctrl, &g_adc0_channel_cfg);  
  
assert(FSP_SUCCESS == err);
```

R_ADC_ScanStart() 函数原型

◆ R_ADC_ScanStart()

```
fsp_err_t R_ADC_ScanStart ( adc_ctrl_t * p_ctrl )
```

Starts a software scan or enables the hardware trigger for a scan depending on how the triggers were configured in the `R_ADC_Open` call. If the unit was configured for ELC or external hardware triggering, then this function allows the trigger signal to get to the ADC unit. The function is not able to control the generation of the trigger itself. If the unit was configured for software triggering, then this function starts the software triggered scan.

Precondition

Call `R_ADC_ScanCfg` after `R_ADC_Open` before starting a scan.
On MCUs that support calibration, call `R_ADC_Calibrate` and wait for calibration to complete before starting a scan.

Return values

FSP_SUCCESS Scan started (software trigger) or hardware triggers enabled.
FSP_ERR_ASSERTION An input argument is invalid.
FSP_ERR_NOT_OPEN Unit is not open.
FSP_ERR_NOT_INITIALIZED Unit is not initialized.
FSP_ERR_IN_USE Another scan is still in progress (software trigger).

启动扫描，若设置为单次扫描，每次扫描完一次都需要重新开启。

```
/* Enable scan triggering from ELC events. */  
  
(void) R_ADC_ScanStart(&g_adc0_ctrl);
```

回调函数 `adc_callback ()`

每次扫描完之后都会进入回调函数中。

```
volatile bool scan_complete_flag = false;  
  
void adc_callback (adc_callback_args_t * p_args)  
{
```

```

//宏将告知编译器回调函数不使用参数 p_args，从而避免编译器发出警告，
FSP_PARAMETER_NOT_USED(p_args);

scan_complete_flag = true;

}

```

故需要在扫描时候在主程序中等待其扫描完毕。

```

scan_complete_flag = false;
while (!scan_complete_flag)
{
    /* Wait for callback to set flag. */
}

```

R_ADC_Read() 函数原型

◆ R_ADC_Read()

```

fsp_err_t R_ADC_Read ( adc_ctrl_t*      p_ctrl,
                        adc_channel_t const reg_id,
                        uint16_t*const    p_data
                      )

```

Reads conversion results from a single channel or sensor.

Return values

FSP_SUCCESS	Data read into provided p_data.
FSP_ERR_ASSERTION	An input argument is invalid.
FSP_ERR_NOT_OPEN	Unit is not open.
FSP_ERR_NOT_INITIALIZED	Unit is not initialized.

读取单个通道的 ADC 值。

```

err = R_ADC_Read(&g_adc0_ctrl, ADC_CHANNEL_12, &adc_data12);
assert(FSP_SUCCESS == err);
a12=(double) (adc_data12/4095.0)*3.3;
printf("P014 (AN12)=%d,voltage=%f\n",adc_data12,a12);

```

主循环

```

while(1)
{
    uint16_t adc_data12=0;
    double a12;

```

```
/* Enable scan triggering from ELC events. */  
  
(void) R_ADC_ScanStart(&g_adc0_ctrl);  
scan_complete_flag = false;  
while (!scan_complete_flag)  
{  
    /* Wait for callback to set flag. */  
}  
  
err = R_ADC_Read(&g_adc0_ctrl, ADC_CHANNEL_12, &adc_data12);  
assert(FSP_SUCCESS == err);  
a12=(double) (adc_data12/4095.0)*3.3;  
printf("P014 (AN12)=%d,voltage=%f\n",adc_data12,a12);  
  
R_BSP_SoftwareDelay (1000, BSP_DELAY_UNITS_MILLISECONDS);  
}
```

演示效果

接入 gnd 时，显示如下所示。

[01:39:09.237] 收 ← ◆ P014 (AN12)=0, voltage=0.000000
[01:39:10.245] 收 ← ◆ P014 (AN12)=0, voltage=0.000000
[01:39:11.254] 收 ← ◆ P014 (AN12)=0, voltage=0.000000
[01:39:12.261] 收 ← ◆ P014 (AN12)=0, voltage=0.000000
[01:39:13.269] 收 ← ◆ P014 (AN12)=0, voltage=0.000000
[01:39:14.278] 收 ← ◆ P014 (AN12)=0, voltage=0.000000
[01:39:15.286] 收 ← ◆ P014 (AN12)=0, voltage=0.000000
[01:39:16.294] 收 ← ◆ P014 (AN12)=1, voltage=0.000806
[01:39:17.302] 收 ← ◆ P014 (AN12)=0, voltage=0.000000
[01:39:18.310] 收 ← ◆ P014 (AN12)=0, voltage=0.000000
[01:39:19.319] 收 ← ◆ P014 (AN12)=0, voltage=0.000000
[01:39:20.327] 收 ← ◆ P014 (AN12)=0, voltage=0.000000
[01:39:21.335] 收 ← ◆ P014 (AN12)=0, voltage=0.000000
[01:39:22.342] 收 ← ◆ P014 (AN12)=0, voltage=0.000000
[01:39:23.351] 收 ← ◆ P014 (AN12)=0, voltage=0.000000

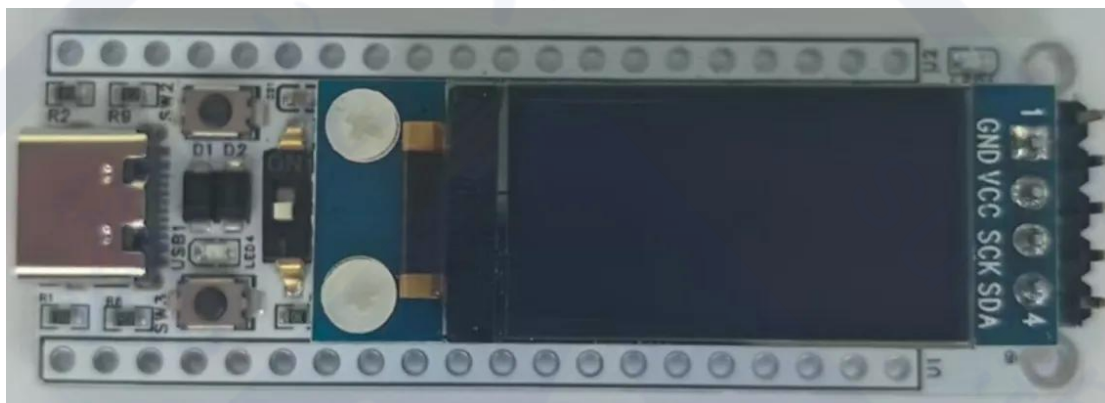
接入 3.3V 时，显示如下所示。

```
[01:40:40.984] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
[01:40:41.993] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
[01:40:43.002] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
[01:40:44.011] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
[01:40:45.020] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
[01:40:46.029] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
[01:40:47.038] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
[01:40:48.047] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
[01:40:49.056] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
[01:40:50.065] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
[01:40:51.073] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
[01:40:52.082] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
[01:40:53.092] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
[01:40:54.100] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
[01:40:55.109] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
```

project17: ADC 扫描多通道采样

概述

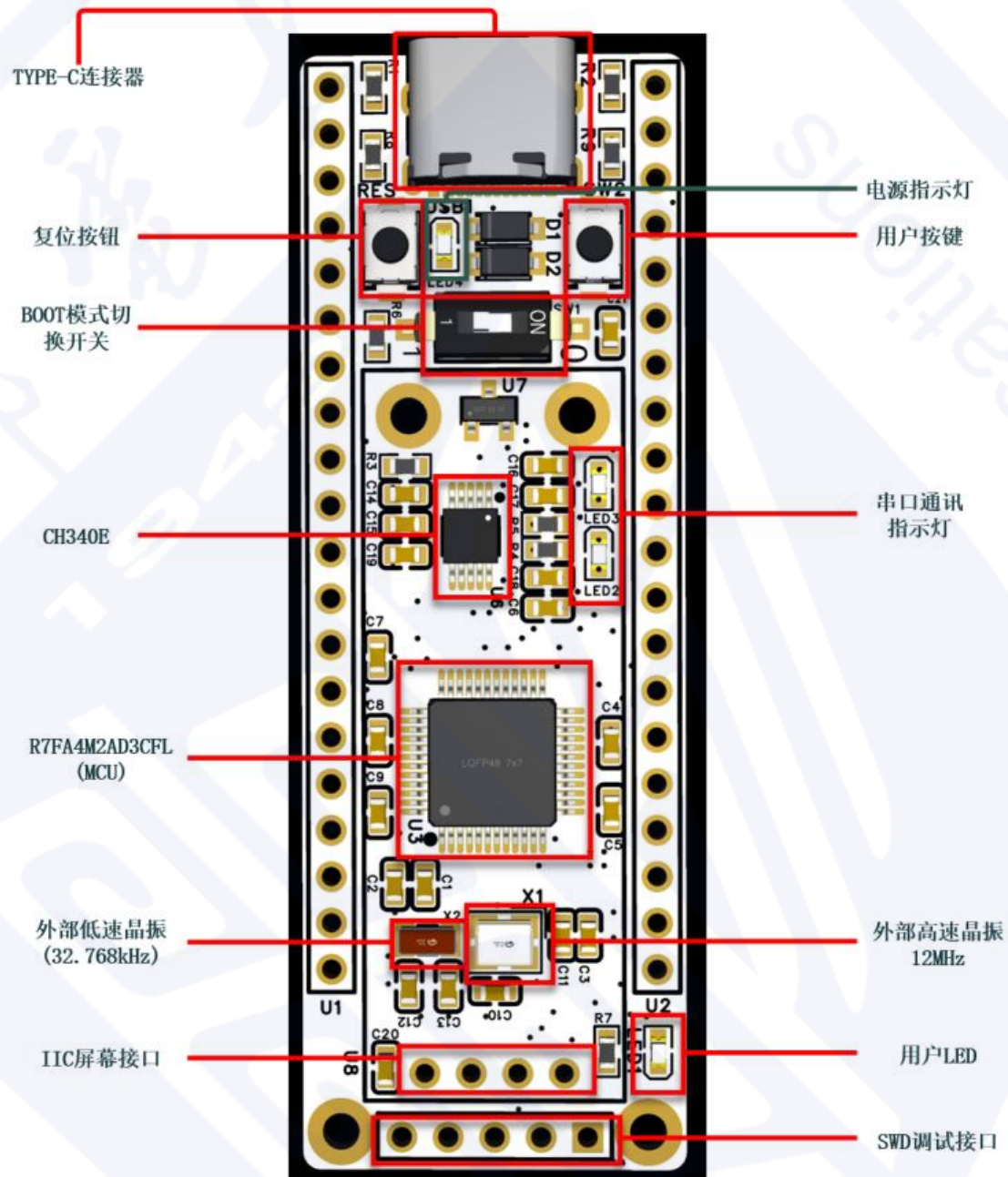
在嵌入式系统中，ADC（模数转换器）是一个非常重要的组件，它将模拟信号转换为数字信号。为了提高采样效率和扩展应用，瑞萨 MCU 支持多通道扫描模式，允许同时采集多个模拟信号。在本文中，将在上篇文章基础上探讨如何配置 ADC 模块进行多通道扫描，并采集来自不同的输入模拟信号。



硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL



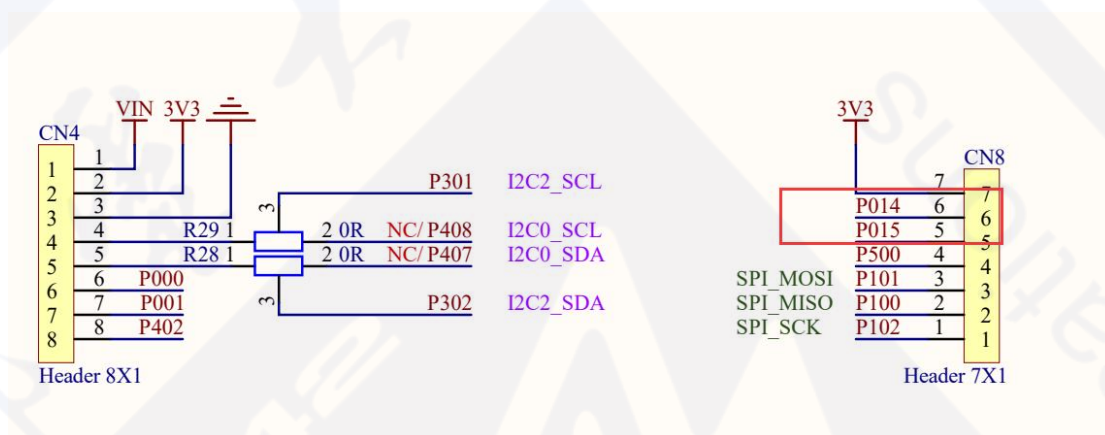
参考程序

https://github.com/CoreMaker-lab/RA4M2_MINI

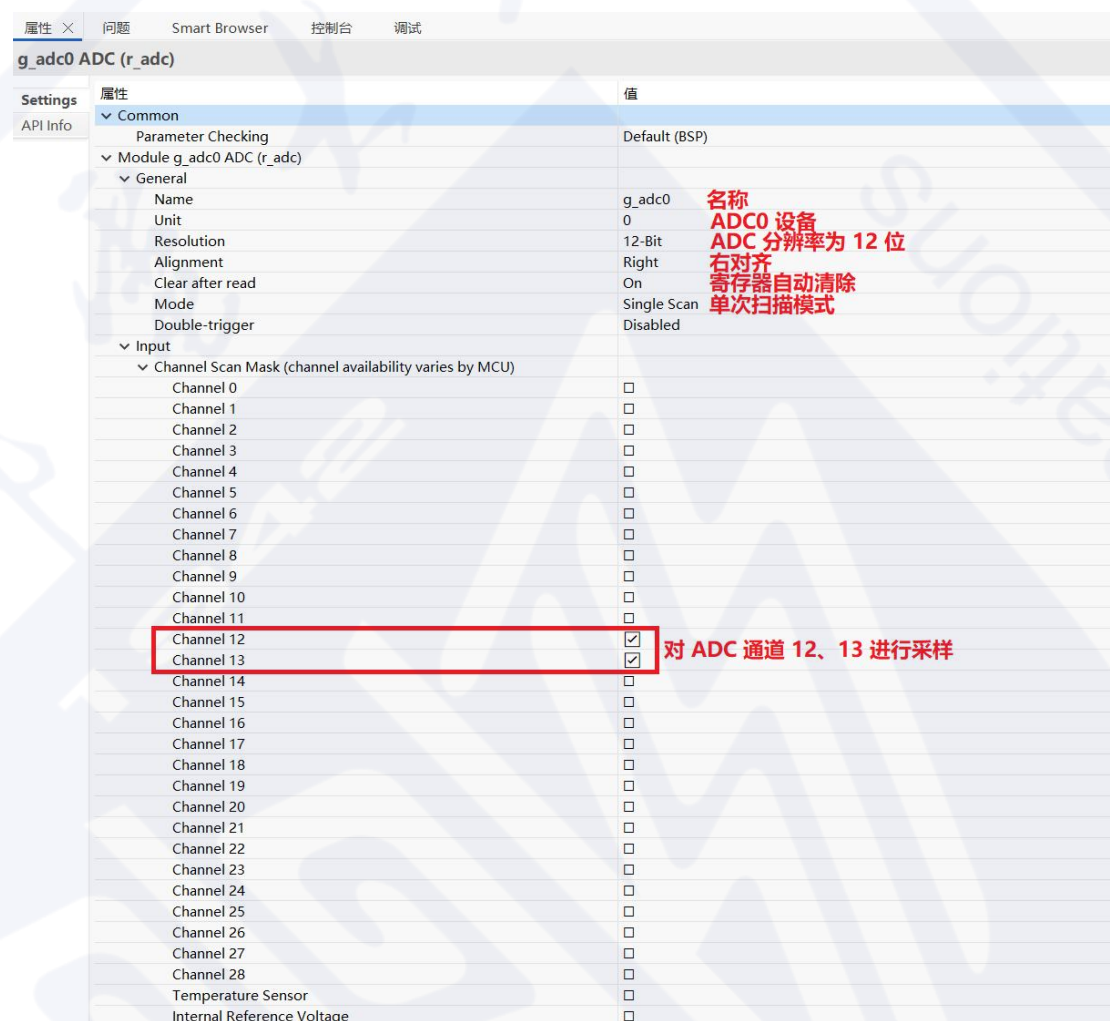
https://gitee.com/CoreMaker/RA4M2_MINI

ADC 属性配置

添加 P015 通道。



- **Name:** `g_adc0` 该 ADC 实例的名称，在代码中用于引用此 ADC 设备。
- **Unit:** `0` 表示使用的是 ADC0 设备（MCU 可能有多个 ADC 单元，如 ADC0、ADC1 等）。
- **Resolution:** 12-Bit 选择 ADC 分辨率为 12 位，即转换后的数值范围为 $0 \sim 4095$ ($2^{12} = 4096$)。
- **Alignment:** Right 选择数据右对齐，即 ADC 结果存储时，低 12 位有效，高位填充 0。
- **Clear after read:** On 每次读取 ADC 结果后，寄存器会自动清除，防止旧数据干扰新采样。
- **Mode:** Single Scan 单次扫描模式，即每次触发 ADC 转换后，仅采样一次选中的通道。



- Normal/Group A Trigger：SoftwareADC 触发方式为软件触发，即需要手动调用 API `R_ADC_ScanStart()` 来启动转换。
- Callback（回调函数）：指定回调函数 `adc_callback`，用于 ADC 转换完成后的事件处理。
- Scan End Interrupt Priority（扫描结束中断优先级）：Priority 2ADC 扫描结束中断的优先级设置为 2（数值越小，优先级越高）。
- AN012 → P014：ADC 通道 12（AN012）映射到 P014 引脚，即 ADC 采样的模拟信号输入在 P014 引脚。
- AN013 → P015：ADC 通道 13（AN013）映射到 P015 引脚，即 ADC 采样的模拟信号输入在 P015 引脚。

▼ Interrupts	
Normal/Group A Trigger	Software ADC 触发方式为软件触发
Group B Trigger	Disabled
Group Priority (Valid only in Group Scan Mode)	Group A cannot interrupt Group B
Callback	adc_callback 回调函数
Scan End Interrupt Priority	Priority 2 优先级
Scan End Group B Interrupt Priority	Disabled
Window Compare A Interrupt Priority	Disabled
Window Compare B Interrupt Priority	Disabled
> Extra	
▼ Pins	
ADTRG0	None
AN000	None
AN001	None
AN002	None
AN011	None
AN012	P014
AN013	P015 P014、P015管脚
AN016	None

回调函数

实现了一个 ADC 回调函数，并在扫描完成时设置 `scan_complete_flag` 标志位为 `true`，以便通知主程序扫描已完成。

```
volatile bool scan_complete_flag = false;
void adc_callback (adc_callback_args_t * p_args)
{
    //宏将告知编译器回调函数不使用参数 p_args，从而避免编译器发出警告，
    FSP_PARAMETER_NOT_USED(p_args);
    scan_complete_flag = true;
}
```

主程序

通过 ADC 对多个通道进行采样，并计算采样数据的平均值与相应的电压值，最后打印出每个通道的数字输出和电压值。

```
while(1)
{
    uint16_t adc_data12=0;
    double a12;
    uint16_t adc_data13=0;
    double a13;
    /* Enable scan triggering from ELC events. */
    (void) R_ADC_ScanStart(&g_adc0_ctrl);
    scan_complete_flag = false;
    while (!scan_complete_flag)
    {
        /* Wait for callback to set flag. */
```

```
}

err = R_ADC_Read(&g_adc0_ctrl, ADC_CHANNEL_12, &adc_data12);
assert(FSP_SUCCESS == err);
a12=(double) (adc_data12/4095.0)*3.3;
printf("P014 (AN12)=%d,voltage=%f\n",adc_data12,a12);

err = R_ADC_Read(&g_adc0_ctrl, ADC_CHANNEL_13, &adc_data13);
assert(FSP_SUCCESS == err);
a13=(double) (adc_data13/4095.0)*3.3;
printf("P015 (AN13)=%d,voltage=%f\n",adc_data13,a13);

R_BSP_SoftwareDelay (1000, BSP_DELAY_UNITS_MILLISECONDS);
}
```

演示结果

给 P014 接入 3.3V，给 P015 接入 GND，结果如下所示。

[01:43:21.588] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
P015 (AN13)=1, voltage=0.000806

[01:43:22.602] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
P015 (AN13)=1, voltage=0.000806

[01:43:23.615] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
P015 (AN13)=0, voltage=0.000000

[01:43:24.629] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
P015 (AN13)=1, voltage=0.000806

[01:43:25.642] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
P015 (AN13)=0, voltage=0.000000

[01:43:26.656] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
P015 (AN13)=1, voltage=0.000806

[01:43:27.669] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
P015 (AN13)=0, voltage=0.000000

[01:43:28.682] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
P015 (AN13)=0, voltage=0.000000

[01:43:29.695] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
P015 (AN13)=1, voltage=0.000806

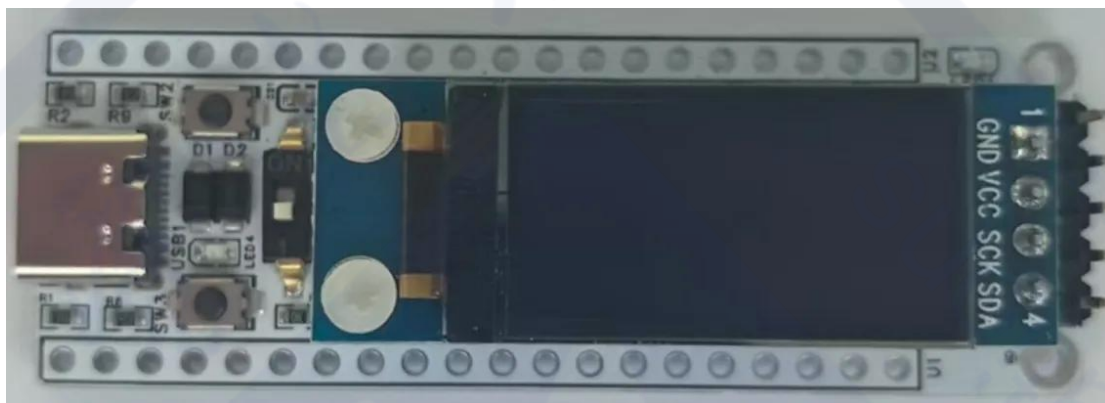
[01:43:30.709] 收 ← ◆ P014 (AN12)=4095, voltage=3.300000
P015 (AN13)=0, voltage=0.000000

project18: 看门狗 WDT

概述

本文主要阐述了如何借助 e2studio 工具对瑞萨独立看门狗 WDT 进行配置设置，同时设置 RTC 时钟产生每秒的周期性中断，并通过串口输出观察独立看门狗 IWDG 的计数情况。主要介绍 RA 系列 MCU 的看门狗定时器 (WDT)，包括其功能、配置方法、API 说明及示例代码。看门狗定时器主要用于检测系统异常，防止程序陷入死循环或卡死，从而提高系统稳定性。

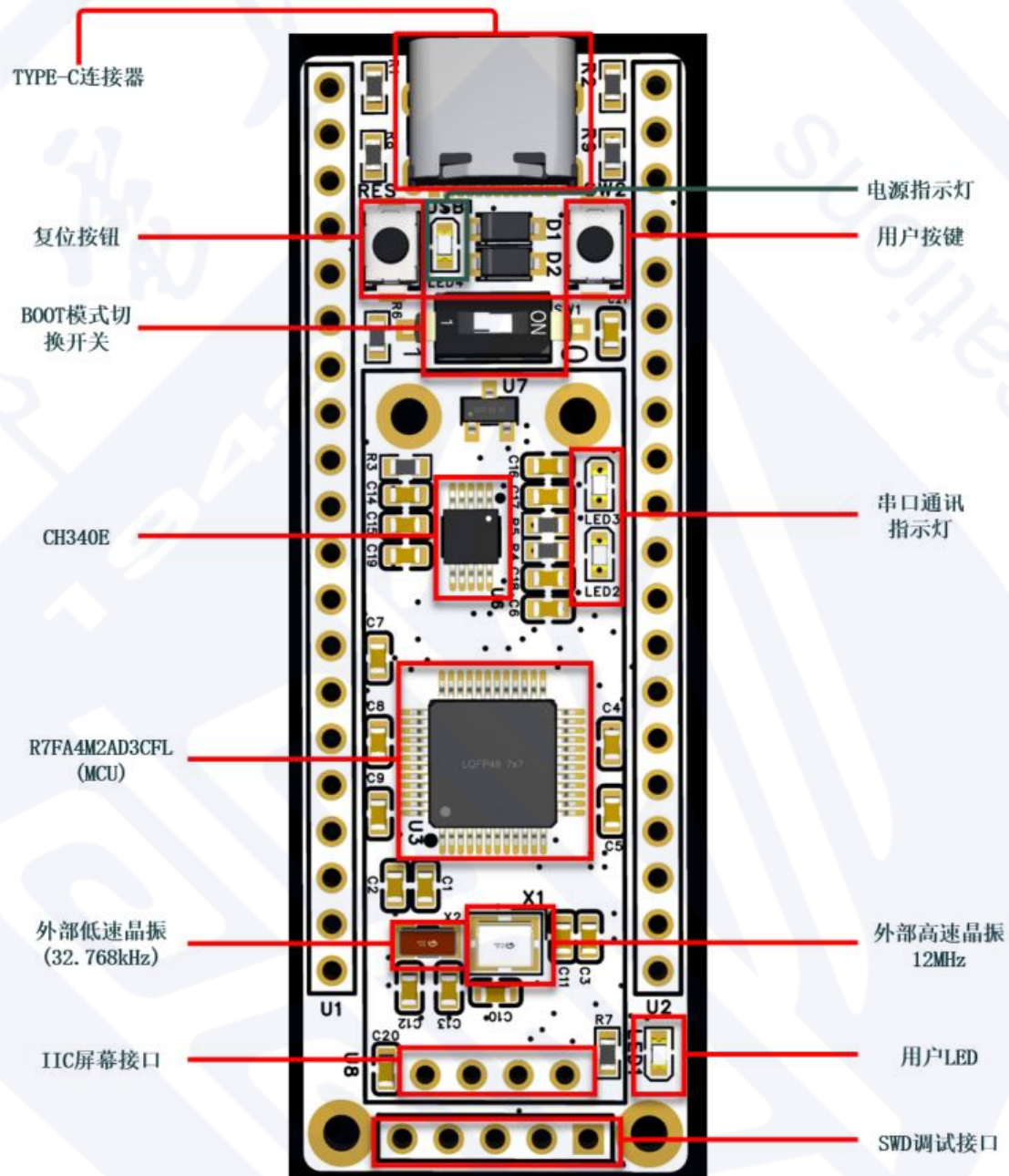
看门狗定时器 (WDT) 是一种超时保护机制，当 MCU 在一定时间内没有刷新 (喂狗) 时，WDT 会触发系统复位或中断，确保系统能够恢复运行。



硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL

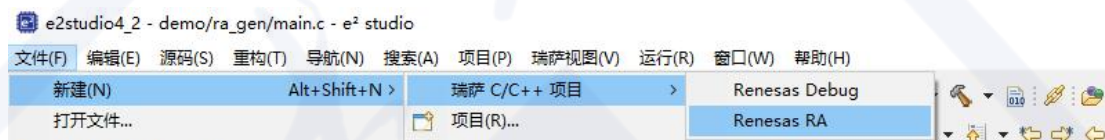


参考程序

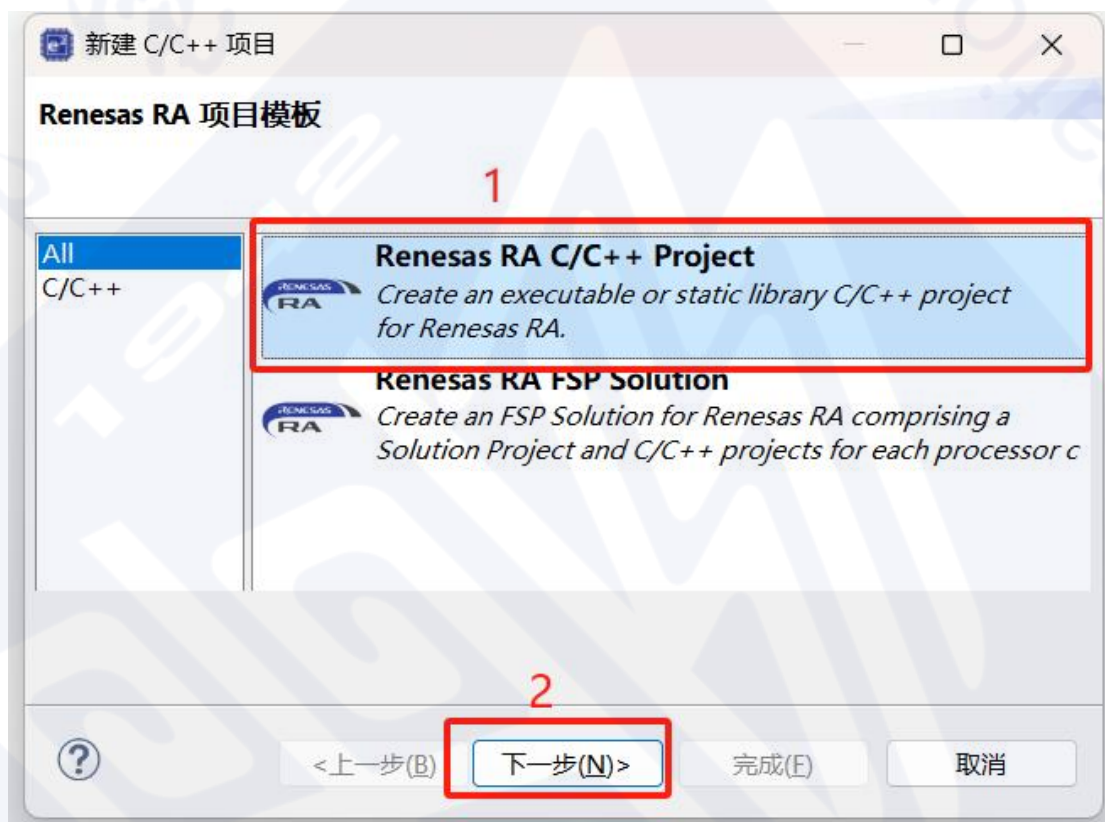
https://github.com/CoreMaker-lab/RA4M2_MINI

https://gitee.com/CoreMaker/RA4M2_MINI

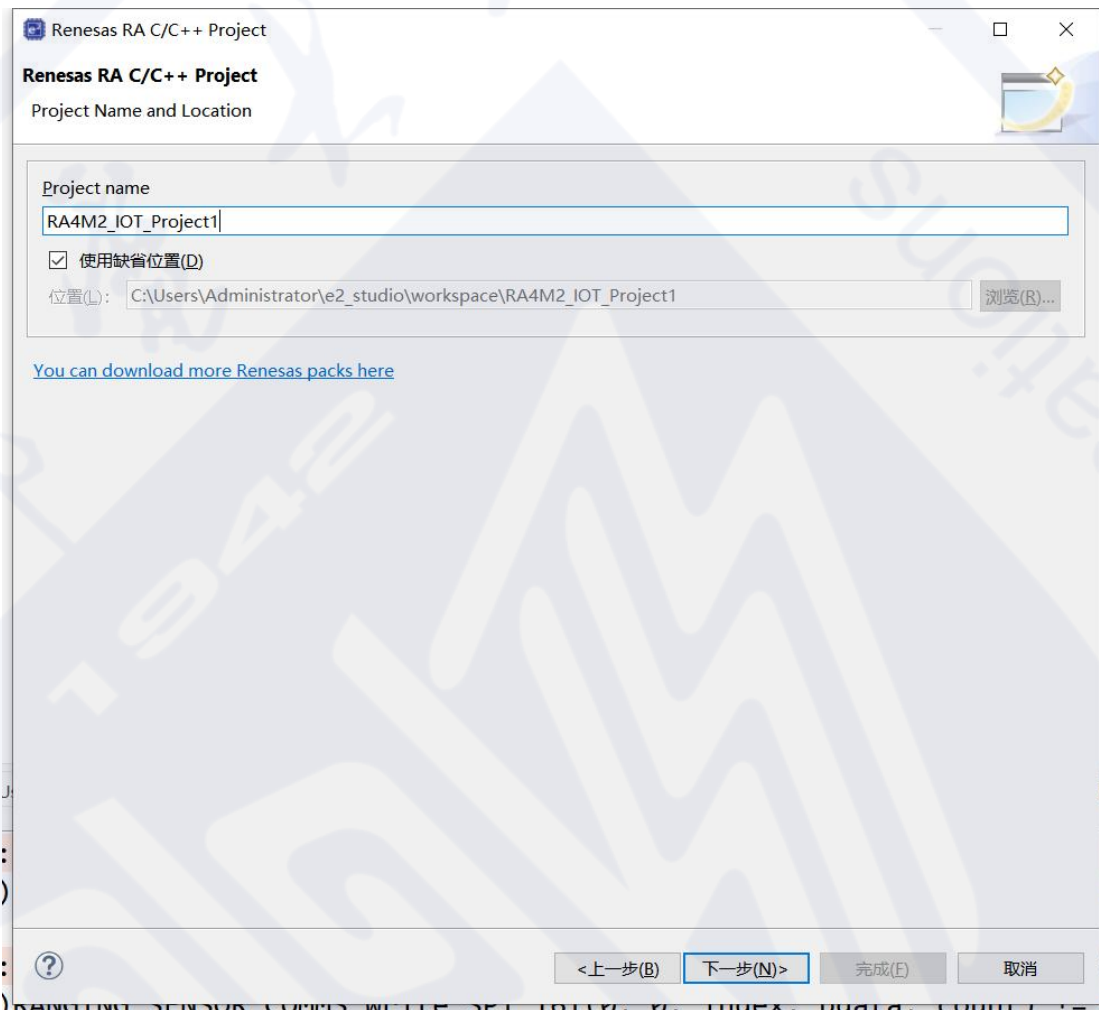
新建工程



工程模板



保存工程路径



芯片配置

本文中使用 R7FA4M2AD3CFL 来进行演示。

Renesas RA C/C++ Project

Renesas RA C/C++ Project

Device and Tools Selection

Device Selection

FSP Version: 5.9.0 **版本**

Board: Custom User Board (Any Device) ...

Device: R7FA4M2AD3CFL **芯片型号**

Core: CM33

Language: ☒ C ☐ C++

Board Description

Device Details

TrustZone	Yes
Pins	48
Processor	Cortex-M33

IDE Project Type

e2 studio managed build

Toolchains

LLVM Embedded Toolchain for Arm

GNU ARM Embedded **工具链**

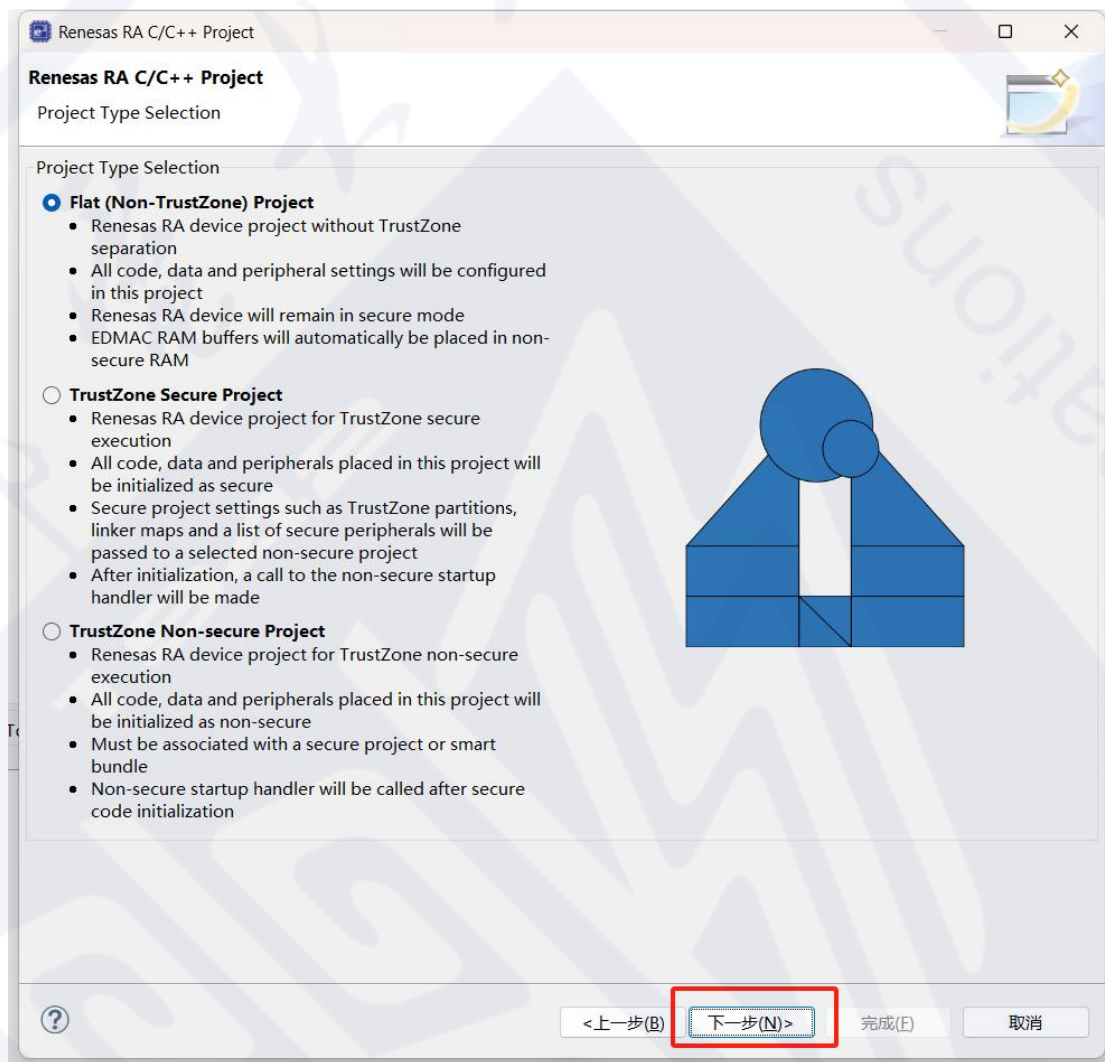
13.2.1.arm-13-7 [Manage Toolchains...](#)

Debugger

J-Link ARM **使用JLink进行调试**

[?](#) <上一步(B) 下一步(N)> 完成(F) 取消

工程模板选择



Renesas RA C/C++ Project

Renesas RA C/C++ Project

Preceding Project or Smart Bundle Selection

☒ None

Choose this option when creating a project for the primary processor core (no preceding project or Smart Bundle).

☐ Preceding Project:

Choose this option if you have access to the preceding bootloader project or TrustZone secure project source code. A debug configuration for Secure Software Debug (SSD) state will be generated where applicable and both secure and non-secure images will be downloaded to the target device. Should you wish to test Non-Secure Software Debug (NSECSO) state, use the Renesas Device Partition Manager to change the Device Lifecycle state as needed.

☐ Smart Bundle:

Resolved location:

Workspace...

File System...

Variables...

Choose this option if you only have access to a pre-programmed device containing TrustZone secure code. The Smart Bundle file (*.sbd) should be obtained from the TrustZone Secure Project developer. A debug configuration for Non-Secure Software Debug (NSECSO) state will be generated and only the non-secure code image will be downloaded to the target device.

Preceding Project/Smart Bundle Details

FSP version

Toolchain

Toolchain version

Board

Device

Core

Zones

?

< 上一步(B)

下一步(N) >

完成(E)

取消

Renesas RA C/C++ Project

Renesas RA C/C++ Project
Build Artifact and RTOS Selection

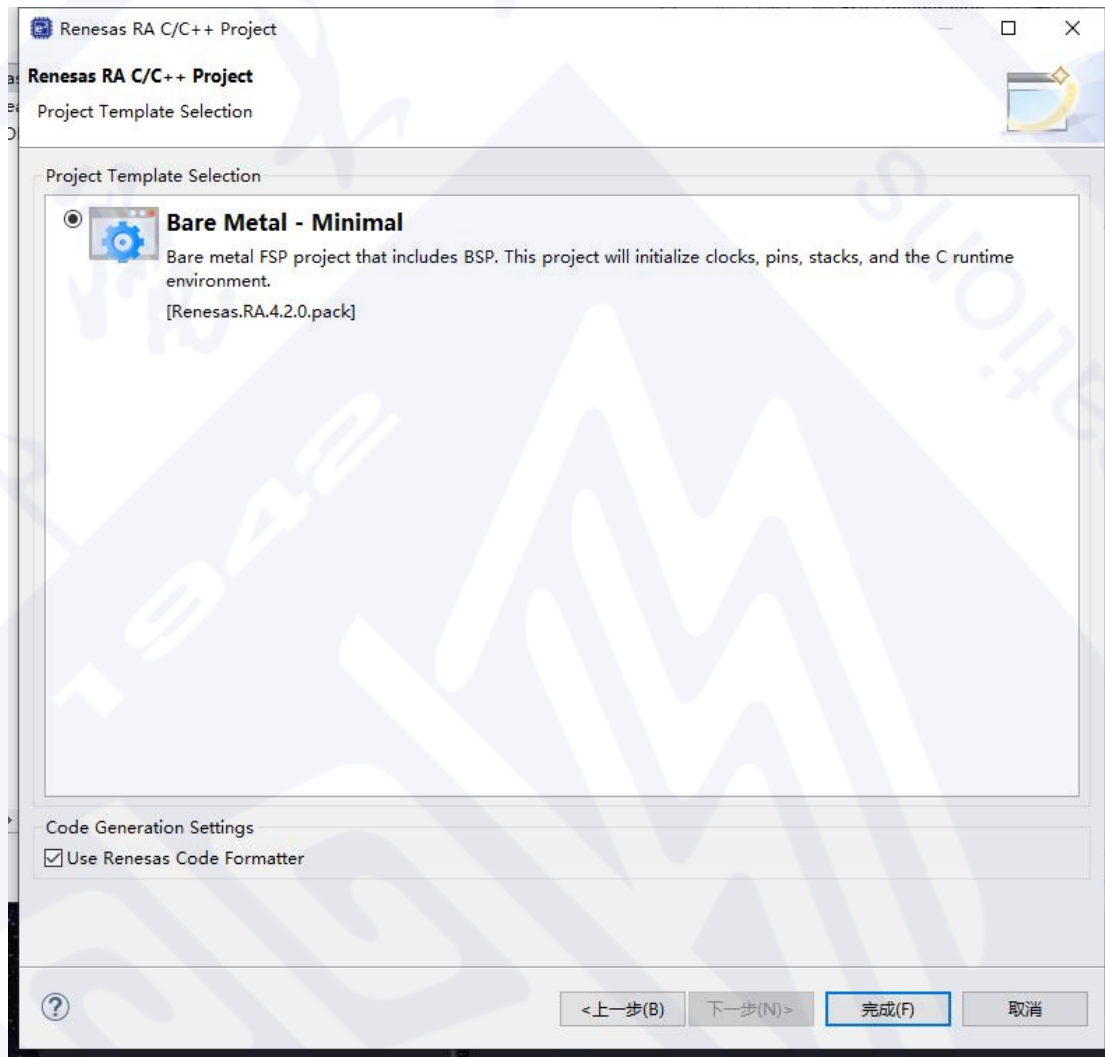
Build Artifact Selection

- ☒ **Executable**
 - Project builds to an executable file
- ☐ **Static Library**
 - Project builds to a static library file
- ☐ **Executable Using an RA Static Library**
 - Project builds to an executable file
 - Project uses an existing RA static library project

RTOS Selection

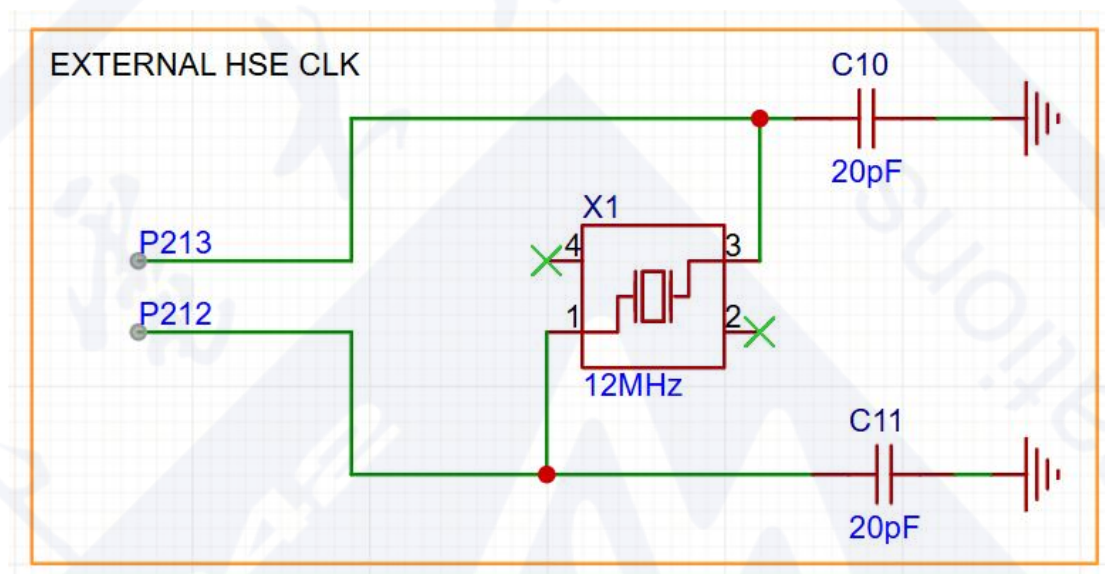
No RTOS

? <上一步(B) **下一步(N)>** 完成(F) 取消

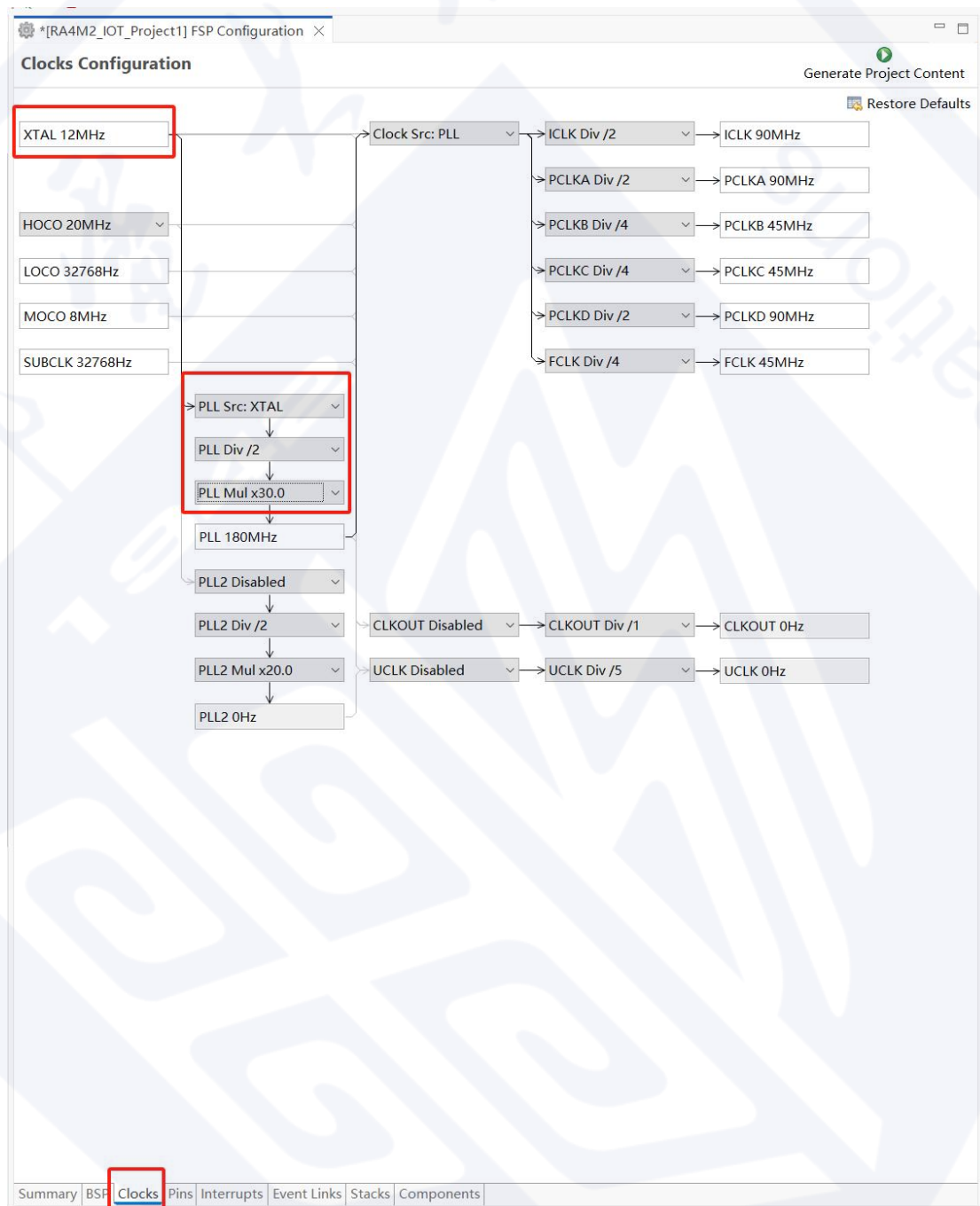


时钟设置

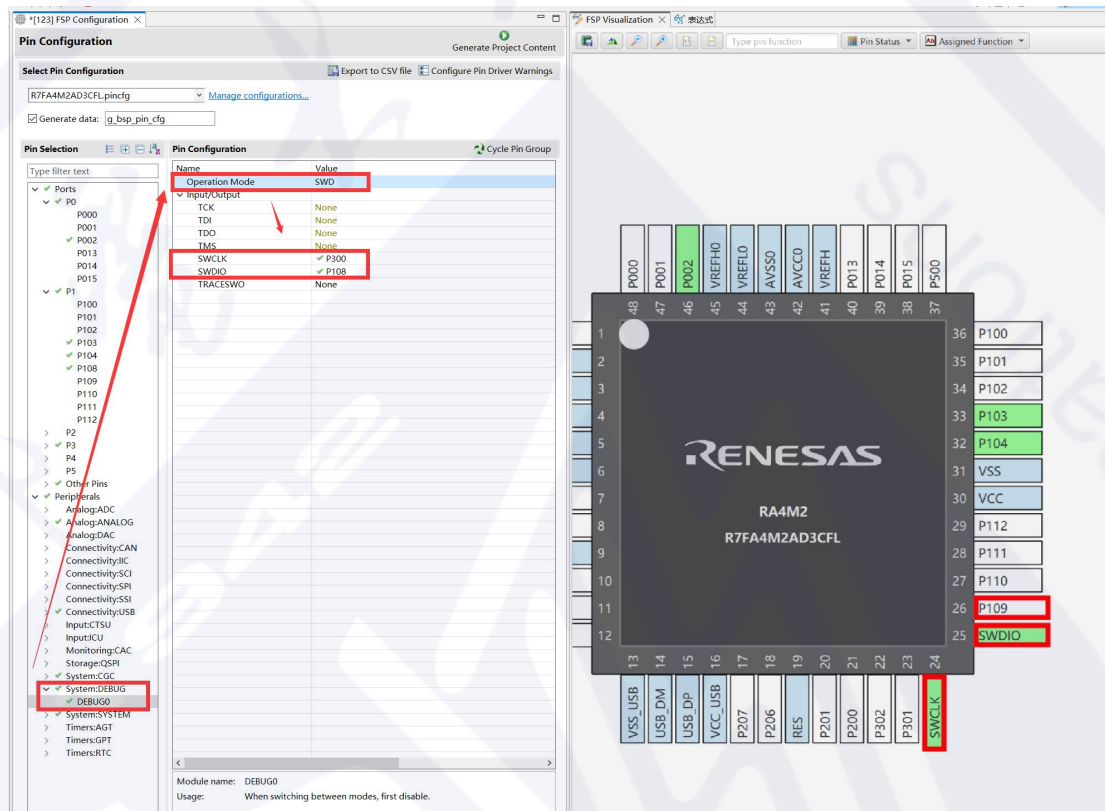
开发板上的外部高速晶振为 12M.



需要修改 XTAL 为 12M。

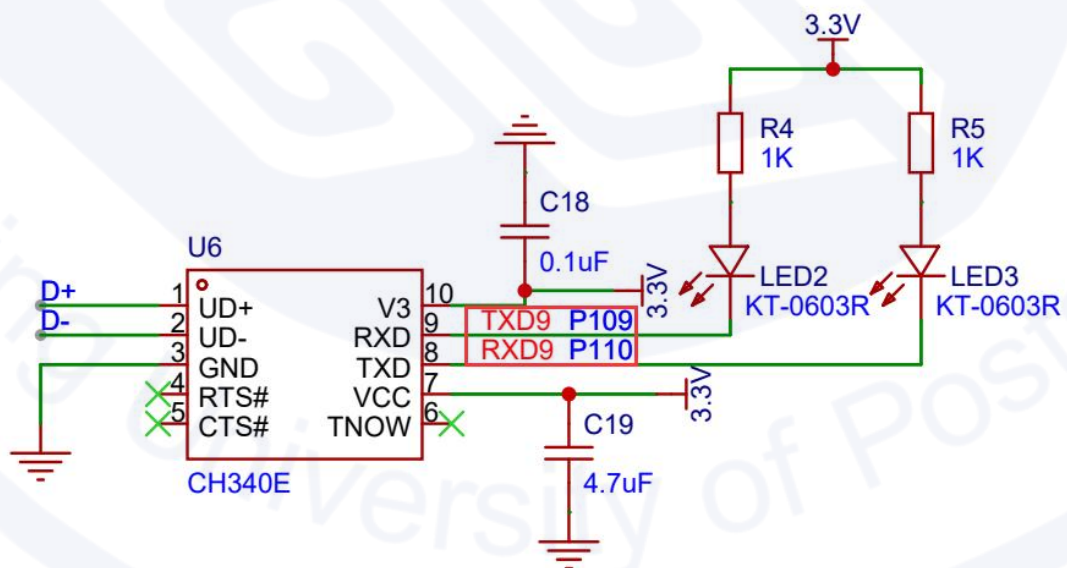


SWD 调试口设置

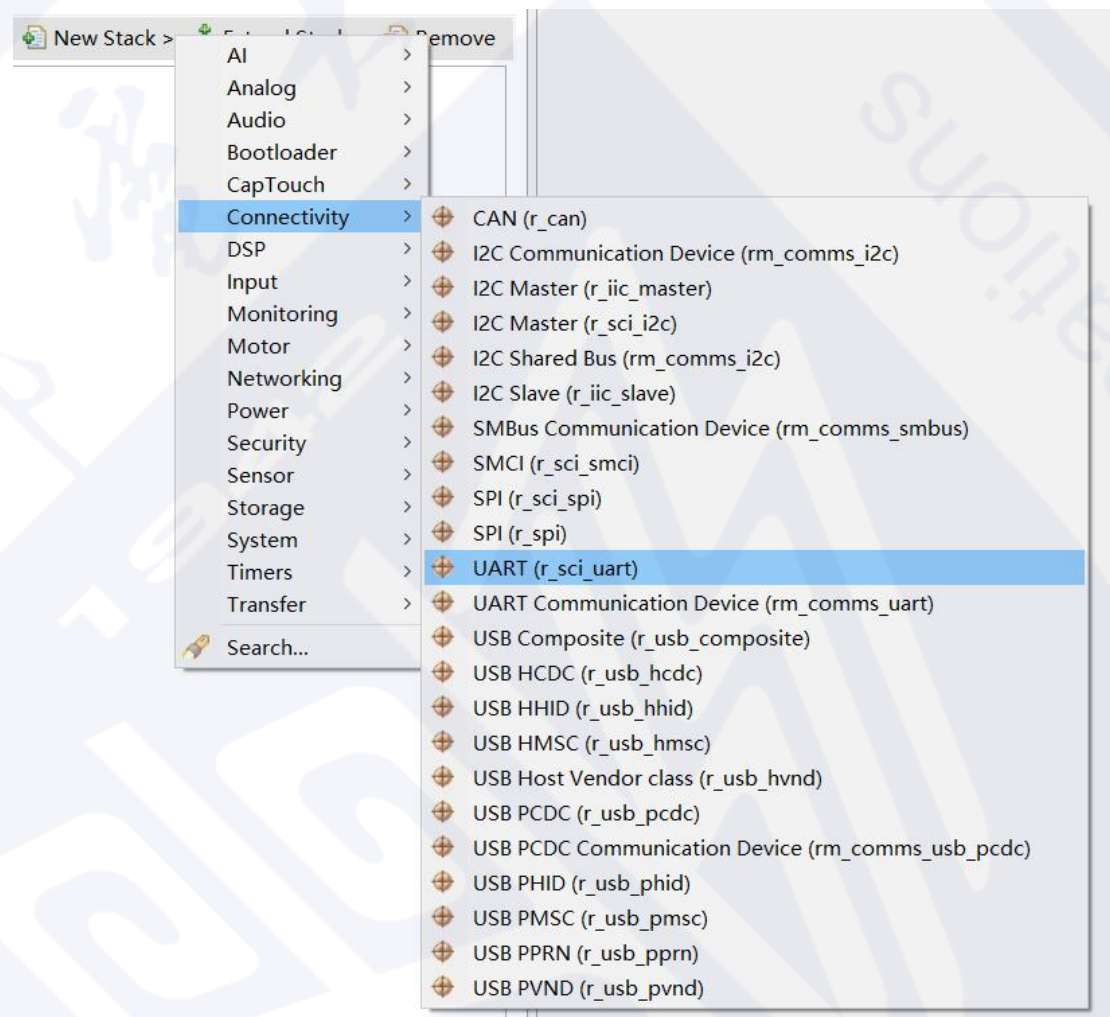


UART 配置

串口CH340E



点击 Stacks->New Stack->Connectivity -> UART(r_sci_uart)。



UART 属性配置

属性 ×	问题	Smart Browser	控制台	调试
g_uart9 UART (r_sci_uart)				
Settings	属性		值	
API Info	▼ Common			
	Parameter Checking		Default (BSP)	
	FIFO Support		Disable	
	DTC Support		Disable	
	Flow Control Support		Disable	
	RS-485 Support		Disable	
	IrDA Support		Disabled	
	▼ Module g_uart9 UART (r_sci_uart)			
	▼ General			
	Name		g_uart9	
	Channel		9	
	Data Bits		8bits	
	Parity		None	
	Stop Bits		1bit	
	▼ Baud			
	Baud Rate		115200	
	Baud Rate Modulation		Enabled	
	Max Error (%)		5	
	► Flow Control			
	► Extra			
	▼ Interrupts			
	Callback		user_uart_callback	
	Receive Interrupt Priority		Priority 2	
	Transmit Data Empty Interrupt Priority		Priority 2	
	Transmit End Interrupt Priority		Priority 2	
	Error Interrupt Priority		Priority 2	
	▼ Pins			
	TXD9		P109	
	RXD9		P110	
	CTS9		<unavailable>	

名称
通道
数据位
无校验位
停止位

波特率
启用波特率调制

回调函数名称

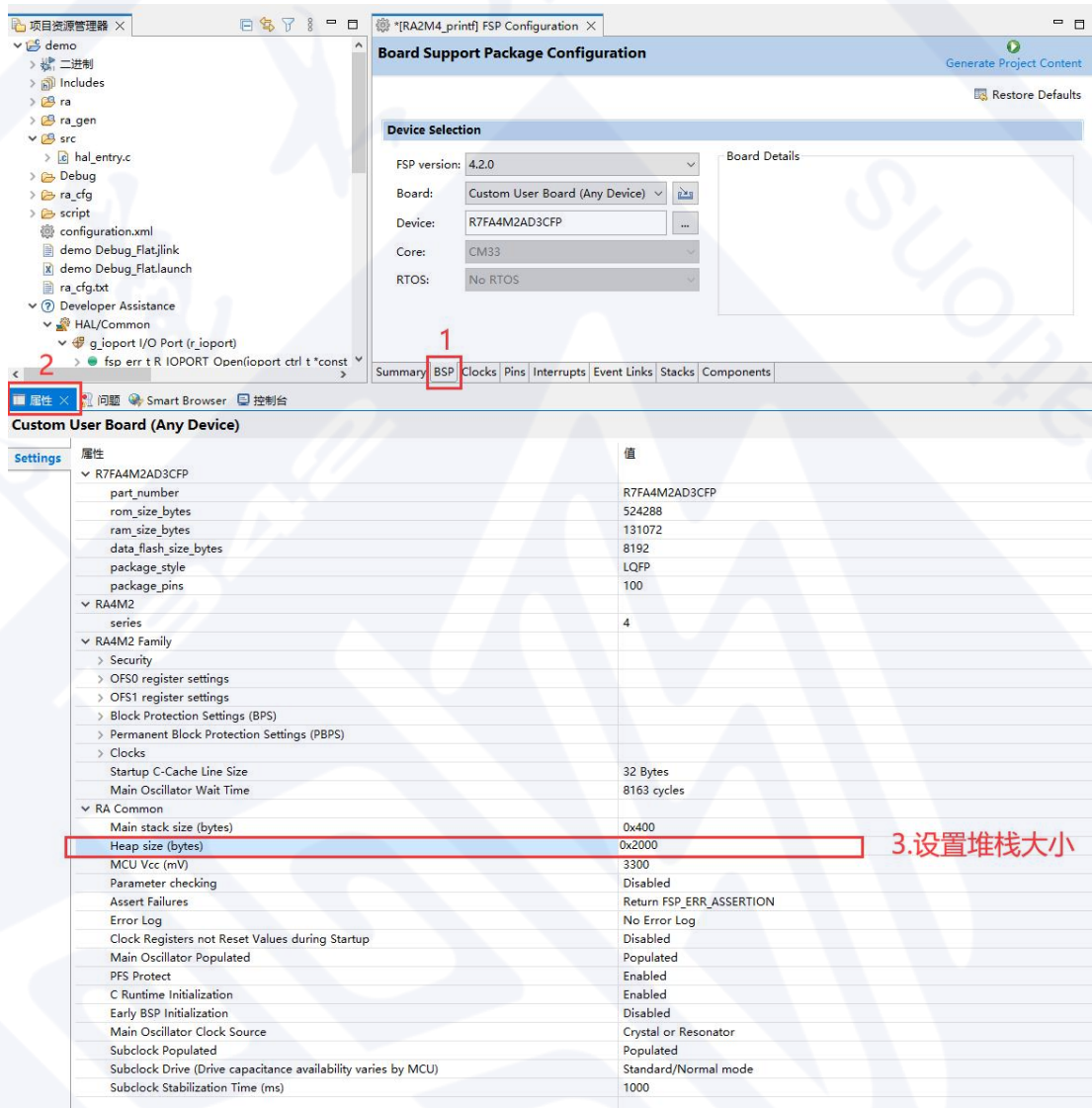
串口发送端口
串口接收端口

设置 e2studio 堆栈

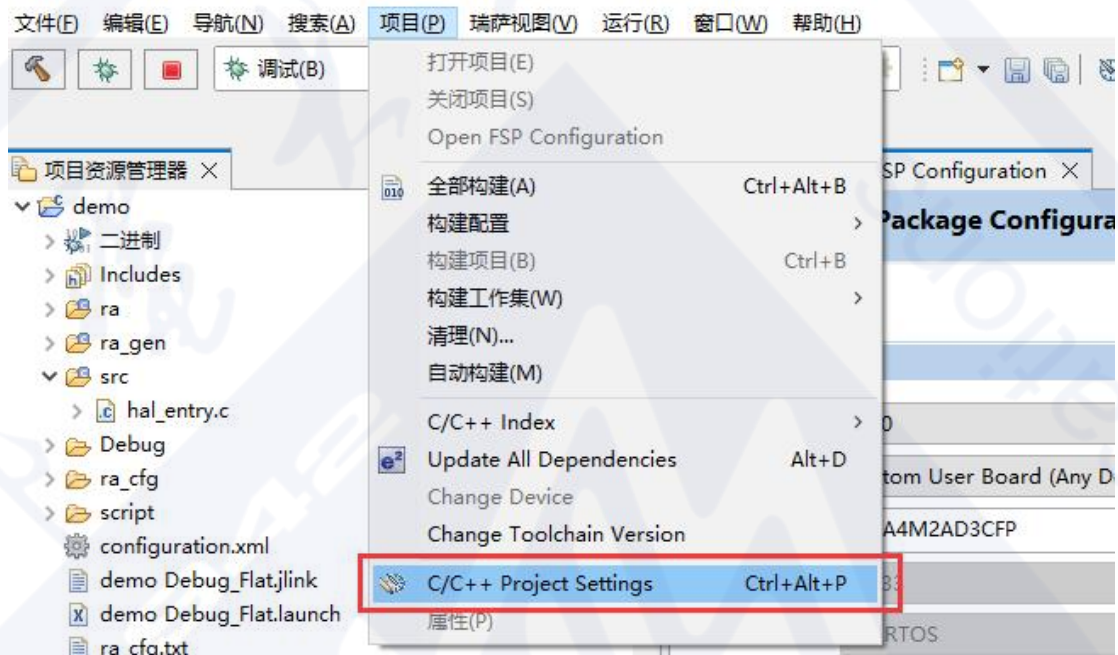
printf 函数通常需要设置堆栈大小。这是因为 printf 函数在运行时需要使用栈空间来存储临时变量和函数调用信息。如果堆栈大小不足，可能会导致程序崩溃或不可预期的行为。

printf 函数使用了可变参数列表，它会在调用时使用栈来存储参数，在函数调用结束时再清除参数，这需要足够的栈空间。另外 printf 也会使用一些临时变量，如果栈空间不足，会导致程序崩溃。

因此，为了避免这类问题，应该根据程序的需求来合理设置堆栈大小。



e2studio 的重定向 printf 设置



在嵌入式系统的开发中，尤其是在使用 GNU 编译器集合（GCC）时，`--specs` 参数用于指定链接时使用的系统规格（specs）文件。这些规格文件控制了编译器和链接器的行为，尤其是关于系统库和启动代码的链接。`--specs=rdimon.specs` 和 `--specs=nosys.specs` 是两种常见的规格文件，它们用于不同的场景。

`--specs=rdimon.specs`

用途：这个选项用于链接“Redlib”库，这是为裸机（bare-metal）和半主机（semihosting）环境设计的 C 库的一个变体。半主机环境是一种特殊的运行模式，允许嵌入式程序通过宿主机（如开发 PC）的调试器进行输入输出操作。

应用场景：当你需要在没有完整操作系统的环境中运行程序，但同时需要使用调试器来处理输入输出（例如打印到宿主机的终端），这个选项非常有用。

特点：它提供了一些基本的系统调用，通过调试接口与宿主机通信。

`--specs=nosys.specs`

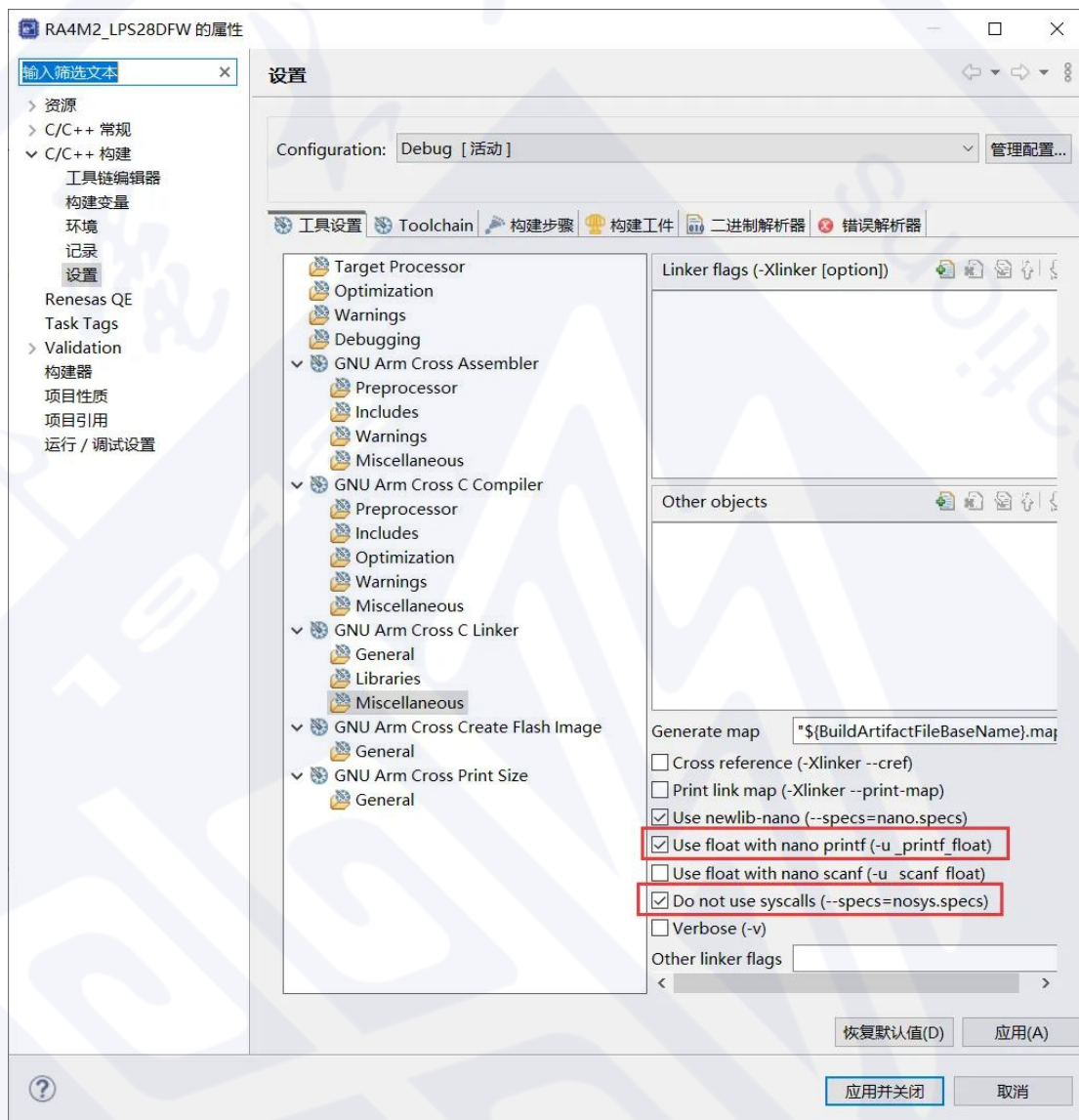
用途：这个选项链接了一个非常基本的系统库，这个库不提供任何系统服务的实现。

应用场景：适用于完全的裸机程序，其中程序不执行任何操作系统调用，比如不进行文件操作或者系统级输入输出。

特点：这是一个更“裸”的环境，没有任何操作系统支持。使用这个规格文件，程序不期望有操作系统层面的任何支持。

如果你的程序需要与宿主机进行交互（如在开发期间的调试），并且通过调试器进行基本的输入输出操作，则使用 `--specs=rdimon.specs`。

如果你的程序是完全独立的，不需要任何形式的操作系统服务，包括不进行任何系统级的输入输出，则使用 `--specs=nosys.specs`。



R_SCI_UART_Open() 函数原型

◆ R_SCI_UART_Open()

```
fsp_err_t R_SCI_UART_Open ( uart_ctrl_t *const p_api_ctrl,  
                           uart_cfg_t const *const p_cfg  
                           )
```

Configures the UART driver based on the input configurations. If reception is enabled at compile time, reception is enabled at the end of this function. Implements `uart_api_t::open`

Return values

FSP_SUCCESS	Channel opened successfully.
FSP_ERR_ASSERTION	Pointer to UART control block or configuration structure is NULL.
FSP_ERR_IP_CHANNEL_NOT_PRESENT	The requested channel does not exist on this MCU.
FSP_ERR_INVALID_ARGUMENT	Flow control is enabled but flow control pin is not defined or selected channel does not support "Hardware CTS and Hardware RTS" flow control. (or) restricted channel is selected.
FSP_ERR_ALREADY_OPEN	Control block has already been opened or channel is being used by another instance. Call <code>close()</code> then <code>open()</code> to reconfigure.
FSP_ERR_INVALID_CHANNEL	IrDA is requested for a channel that does not support IrDA.

Returns

See [Common Error Codes](#) or functions called by this function for other possible return codes. This function calls:

- `transfer_api_t::open`

故可以用 `R_SCI_UART_Open()`函数进行配置，开启和初始化 UART。

```
/* Open the transfer instance with initial configuration. */  
err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);  
assert(FSP_SUCCESS == err);
```

```
[RA4M2_MINI_Project11_1] FSP Configuration  *hal_entry.c x startup.c main.c
2
3 FSP_CPP_HEADER
4 void R_BSP_WarmStart(bsp_warm_start_event_t event);
5 FSP_CPP_FOOTER
6 /* unused parameter 'fd' [-Wunused-parameter] | no previous declaration for '_write' [-W
7
8
9
10
11
12
13
14
15 /* Open the transfer instance with initial configuration. */
16 err = R_SCI_UART_Open(&g_uart9_ctrl1, &g_uart9_cfg);
17 assert(FSP_SUCCESS == err);
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
```

回调函数 user_uart_callback ()

当数据发送的时候，可以查看 UART_EVENT_TX_COMPLETE 来判断是否发送完毕。

◆ write

```
fsp_err_t(*uart_api_t::write)(uart_ctrl_t *const p_ctrl, uint8_t const *const p_src, uint32_t const bytes)
```

Write to UART device. The write buffer is used until write is complete. Do not overwrite write buffer contents until the write is finished. When the write is complete (all bytes are fully transmitted on the wire), the callback called with event **UART_EVENT_TX_COMPLETE**. The maximum transfer size is reported by **infoGet[]**.

Implemented as

- R_SCI_UART_Write[]
- R_SCI_B_UART_Write[]

Parameters

[in] p_ctrl Pointer to the UART control block.

[in] p_src Source address to write data to.

[in] bytes Write data length.

Enumeration Type Documentation

◆ uart_event_t

enum `uart_event_t`

UART Event codes

Enumerator	
UART_EVENT_RX_COMPLETE	Receive complete event.
UART_EVENT_TX_COMPLETE	Transmit complete event.
UART_EVENT_RX_CHAR	Character received.
UART_EVENT_ERR_PARITY	Parity error event.
UART_EVENT_ERR_FRAMING	Mode fault error event.
UART_EVENT_ERR_OVERFLOW	FIFO Overflow error event.
UART_EVENT_BREAK_DETECT	Break detect error event.
UART_EVENT_TX_DATA_EMPTY	Last byte is transmitting, ready for more data.

可以检查检查 "p_args" 结构体中的 "event" 字段的值是否等于 "UART_EVENT_TX_COMPLETE"。如果条件为真，那么 if 语句后面的代码块将会执行。

```
fsp_err_t err = FSP_SUCCESS;
volatile bool uart_send_complete_flag = false;
void user_uart_callback (uart_callback_args_t * p_args)
{
    if(p_args->event == UART_EVENT_TX_COMPLETE)
    {
        uart_send_complete_flag = true;
    }
}
```

```
[RA4M2_MINI_Project11_1] FSP Configuration  *hal_entry.c x startup.c main.c
1 #include "hal_data.h"
2
3 FSP_CPP_HEADER
4 void R_BSP_WarmStart(bsp_warm_start_event_t event);
5 FSP_CPP_FOOTER
6
7 fsp_err_t err = FSP_SUCCESS;
8 volatile bool uart_send_complete_flag = false;
9 void user_uart_callback (uart_callback_args_t * p_args)
10 {
11     if(p_args->event == UART_EVENT_TX_COMPLETE)
12     {
13         uart_send_complete_flag = true;
14     }
15 }
16
17
18
19 * main() is generated by the RA Configuration editor and is used to generate threads if
20 void hal_entry(void)
21 {
22     /* TODO: add your own code here */
23
24     /* Open the transfer instance with initial configuration. */
25     err = R_SCI_UART_Open(&g_uart9_ctrl1, &g_uart9_cfg);
26     assert(FSP_SUCCESS == err);
27
28     #if BSP_TZ_SECURE_BUILD
29     /* Enter non-secure code */
30     R_BSP_NonSecureEnter();
31     #endif
32
33     * This function is called at various points during the startup process. This implement
34     void R_BSP_WarmStart(bsp_warm_start_event_t event)
35     {
36         if (BSP_WARM_START_RESET == event)
37         {
38             #if BSP_FEATURE_FLASH_LP_VERSION != 0
```

printf 输出重定向到串口

打印最常用的方法是 `printf`，所以要解决的问题是将 `printf` 的输出重定向到串口，然后通过串口将数据发送出去。

注意一定要加上头文件 `#include <stdio.h>`

```
#ifdef __GNUC__ //串口重定向
#define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
#else
#endif

PUTCHAR_PROTOTYPE
{
    err = R_SCI_UART_Write(&g_uart9_ctrl1, (uint8_t *)&ch, 1);
    if(FSP_SUCCESS != err) __BKPT();
    while(uart_send_complete_flag == false){}
    uart_send_complete_flag = false;
    return ch;
}
```

```

int _write(int fd, char *pBuffer, int size)
{
    for(int i=0; i<size; i++)
    {
        __io_putchar(*pBuffer++);
    }

    return size;
}

```

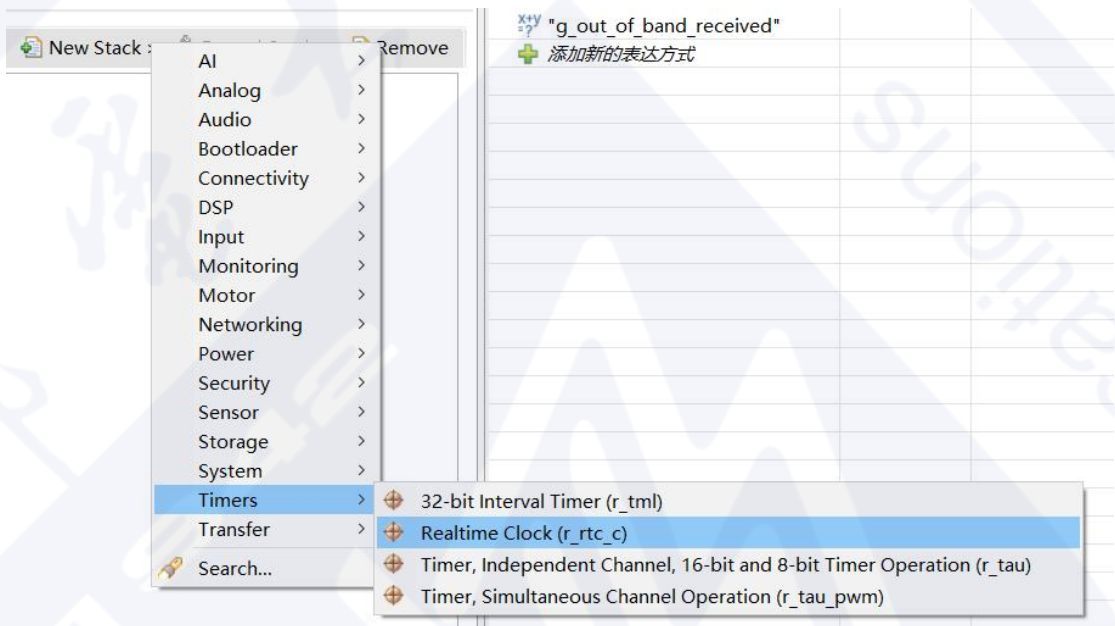
```

1  #include "hal_data.h"
2  #include <stdio.h>
3  FSP_CPP_HEADER
4  void R_BSP_WarmStart(bsp_warm_start_event_t event);
5  FSP_CPP_FOOTER
6
7  fsp_err_t err = FSP_SUCCESS;
8  volatile bool uart_send_complete_flag = false;
9  void user_uart_callback (uart_callback_args_t * p_args)
10 {
11     if(p_args->event == UART_EVENT_TX_COMPLETE)
12     {
13         uart_send_complete_flag = true;
14     }
15 }
16
17
18 #ifdef __GNUC__ //串口重定向
19 #define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
20 #else
21 #define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
22 #endif
23
24
25 PUTCHAR_PROTOTYPE
26 {
27     err = R_SCI_UART_Write(&g_uart9_ctrl1, (uint8_t *)&ch, 1);
28     if(FSP_SUCCESS != err) __BKPT();
29     while(uart_send_complete_flag == false){}
30     uart_send_complete_flag = false;
31     return ch;
32 }
33
34 int _write(int fd, char *pBuffer, int size)
35 {
36     for(int i=0; i<size; i++)
37     {
38         __io_putchar(*pBuffer++);
39     }
40     return size;
41 }
42
43
44
45 * * main() is generated by the RA Configuration editor and is used to generate threads i
46 void hal_entry(void)
47 {
48     /* TODO: add your own code here */
49
50     /* Open the transfer instance with initial configuration. */
51     err = R_SCI_UART_Open(&g_uart9_ctrl1, &g_uart9_cfg);
52     assert(FSP_SUCCESS == err);
53
54
55
56

```

RTC 配置

点击 Stacks->New Stack->Timers -> Realtime Clock(r_rtc_c)。



RTC 属性配置

属性	问题	Smart Browser	控制台
g_rtc0 Realtime Clock (r_rtc)			
Settings	属性		值
API Info	▼ Common		
	Parameter Checking		Default (BSP)
	Set Source Clock in Open		Enabled
	▼ Module g_rtc0 Realtime Clock (r_rtc)		
	Name		g_rtc0
	Clock Source		Sub-Clock
	Frequency Comparision Value (LOCO)		255
	Automatic Adjustment Mode		Enabled
	Automatic Adjustment Period		10 Seconds
	Adjustment Type (Plus-Minus)		NONE
	Error Adjustment Value		0
	Callback		rtc_callback
	Alarm Interrupt Priority		Priority 2
	Period Interrupt Priority		Priority 2
Carry Interrupt Priority		Priority 2	
▼ Pins			
RTCOUNT		<unavailable>	

名称
时钟源，SUBCLK时钟精度高

误差调整模式
自动调整的误差调整周期

调整值
回调函数
闹钟报警中断优先级
周期中断优先级

设定时间

在启动 RTC 后，需要为其设定当前时间。您可以使用 `R_RTC_CalendarTimeSet(&g_rtc0_ctrl, &set_time);` 函数来实现这一目标。具体的时间参数可以通过修改 `set_time` 变量来调整。

◆ `R_RTC_CalendarTimeSet()`

```
fsp_err_t R_RTC_CalendarTimeSet ( rtc_ctrl_t *const p_ctrl,
                                  rtc_time_t *const p_time
                                )
```

Set the calendar time.

Implements `rtc_api_t::calendarTimeSet`.

Return values

FSP_SUCCESS	Calendar time set operation was successful.
FSP_ERR_ASSERTION	Invalid input argument.
FSP_ERR_NOT_OPEN	Driver not open already for operation.
FSP_ERR_INVALID_ARGUMENT	Invalid time parameter field.

```
//RTC 变量
/* rtc_time_t is an alias for the C Standard time.h struct 'tm' */
rtc_time_t set_time =
{
    .tm_sec = 50,      /* 秒, 范围从 0 到 59 */
    .tm_min = 59,      /* 分, 范围从 0 到 59 */
    .tm_hour = 23,      /* 小时, 范围从 0 到 23 */
    .tm_mday = 28,      /* 一月中的第几天, 范围从 1 到 31 */
    .tm_mon = 1,        /* 月份, 范围从 0 到 11 */
    .tm_year = 125,      /* 自 1900 起的年数, 2025 为 125 */
    .tm_wday = 5,        /* 一周中的第几天, 范围从 0 到 6 */
    // .tm_yday=0,      /* 一年中的第几天, 范围从 0 到 365 */
    // .tm_isdst=0;      /* 夏令时 */
};
```

回调函数

可以触发进入回调函数的事件如下所示，`RTC_EVENT_PERIODIC_IRQ` 为设置的实时性事件，例如 1s 一次，`RTC_EVENT_ALARM_IRQ` 为闹钟事件。

enum rtc_event_t

Events that can trigger a callback function

Enumerator

RTC_EVENT_ALARM_IRQ	Real Time Clock ALARM IRQ.
RTC_EVENT_PERIODIC_IRQ	Real Time Clock PERIODIC IRQ.

```
//RTC 回调函数
volatile bool rtc_flag = 0;//RTC 延时 1s 标志位
volatile bool rtc_alarm_flag = 0;//RTC 闹钟
/* Callback function */
void rtc_callback(rtc_callback_args_t *p_args)
{
    /* TODO: add your own code here */
    if(p_args->event == RTC_EVENT_PERIODIC_IRQ)
        rtc_flag=1;
    else if(p_args->event == RTC_EVENT_ALARM_IRQ)
        rtc_alarm_flag=1;
}
```

R_RTC_Open 函数

R_RTC_Open 函数可以开启 RTC。

◆ R_RTC_Open()

```
fsp_err_t R_RTC_Open ( rtc_ctrl_t *const p_ctrl,  
                      rtc_cfg_t const *const p_cfg  
                      )
```

Opens and configures the RTC driver module. Implements `rtc_api_t::open`. Configuration includes clock source, and interrupt callback function.

Example:

```
/* Open the RTC module */  
err = R_RTC_Open(&g_rtc0_ctrl, &g_rtc0_cfg);
```

Return values

FSP_SUCCESS	Initialization was successful and RTC has started.
FSP_ERR_ASSERTION	Invalid p_ctrl or p_cfg pointer.
FSP_ERR_ALREADY_OPEN	Module is already open.
FSP_ERR_INVALID_ARGUMENT	Invalid time parameter field.

同时主程序中开启 RTC 已经设置时间和闹钟。

```
/* *****RTC 开启***** */  
/* Initialize the RTC module */  
err = R_RTC_Open(&g_rtc0_ctrl, &g_rtc0_cfg);  
  
/* Handle any errors. This function should be defined by the user. */  
assert(FSP_SUCCESS == err);  
  
/* Set the RTC clock source. Can be skipped if "Set Source Clock in Open" property is enabled. */  
R_RTC_ClockSourceSet(&g_rtc0_ctrl);  
  
/* R_RTC_CalendarTimeSet must be called at least once to start the RTC */  
R_RTC_CalendarTimeSet(&g_rtc0_ctrl, &set_time);  
  
/* Set the periodic interrupt rate to 1 second */  
R_RTC_PeriodicIrqRateSet(&g_rtc0_ctrl, RTC_PERIODICC
```

WDT 配置

RA4M2 内部自带看门狗定时器（WDT）。

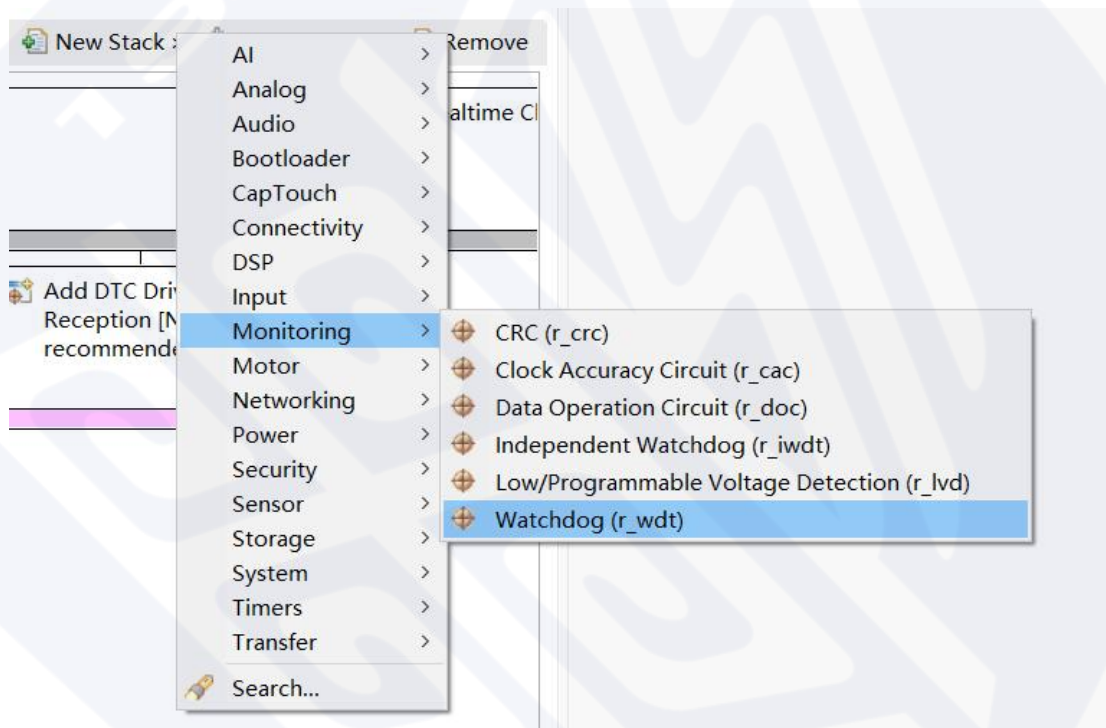
■ System and Power Management

- Low power modes
- Battery backup function (VBATT)
- Realtime Clock (RTC) with calendar and VBATT support
- Event Link Controller (ELC)
- Data Transfer Controller (DTC)
- DMA Controller (DMAC) × 8
- Power-on reset
- Low Voltage Detection (LVD) with voltage settings
- Watchdog Timer (WDT)

R01DS0367EJ0130 Rev.1.30
Jul 28, 2023

RENESAS

点击 Stacks->New Stack->Monitoring -> Watchdog(r_wdt)。

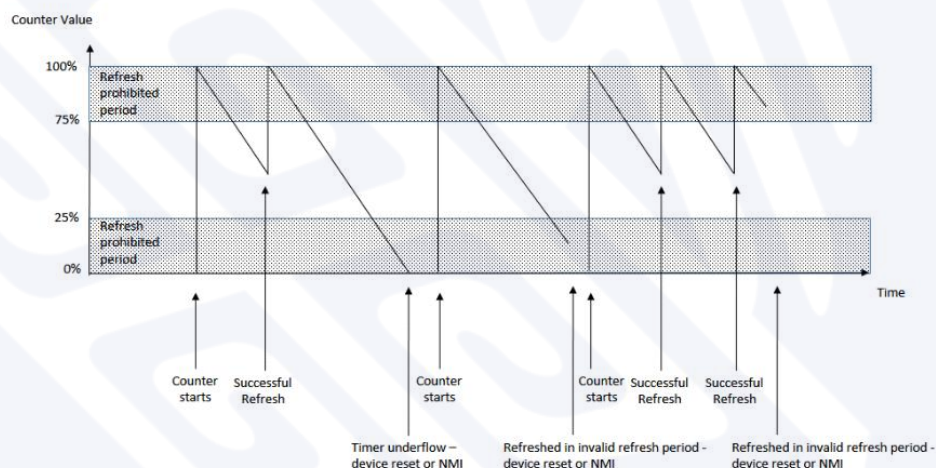


WDT 属性配置

r_wdt0 Watchdog (r_wdt)		
Settings	属性	值
API Info	▼ Common	
	Parameter Checking	Default (BSP)
	Register Start NMI Support	Disabled
	▼ Module g_wdt0 Watchdog (r_wdt)	
	Name	g_wdt0 名称
	Timeout	16,384 Cycles 周期
	Clock Division Ratio	PCLK/8192 分频
	Window Start Position	100 (Window Position Not Specified)
	Window End Position	0 (Window Position Not Specified)
	Reset Control	Reset Output
	Stop Control	WDT Count Disabled in Low Power Mode
	NMI Callback	NULL

看门狗周期起点
看门狗周期结束
看门狗触发模式
低功耗状态
回调函数

WDT 计数器开始递减，初始计数值为 100%。
在有效的 允许刷新时间窗口 内刷新（喂狗），计数器重新回到最大值 100%。
如果在允许的时间窗口内没有刷新，计数器递减到 0%，触发 MCU 复位(Reset)
或 非屏蔽中断（NMI）。



Watchdog Timer Operation Example

R_WDT_Open() 函数原型

◆ R_WDT_Open()

```
fsp_err_t R_WDT_Open ( wdt_ctrl_t *const p_ctrl,  
                      wdt_cfg_t const *const p_cfg  
                      )
```

Configure the WDT in register start mode. In auto-start_mode the NMI callback can be registered. Implements `wdt_api_t::open`.

This function should only be called once as WDT configuration registers can only be written to once so subsequent calls will have no effect.

Example:

```
/* Initializes the module. */  
err = R_WDT_Open(&g_wdt0_ctrl, &g_wdt0_cfg);
```

Return values

FSP_SUCCESS	WDT successfully configured.
FSP_ERR_ASSERTION	Null pointer, or one or more configuration options is invalid.
FSP_ERR_ALREADY_OPEN	Module is already open. This module can only be opened once.
FSP_ERR_INVALID_STATE	The security state of the NMI and the module do not match.

Note

In auto start mode the only valid configuration option is for registering the callback for the NMI ISR if NMI output has been selected.

可以用 `R_WDT_Open()` 函数进行初始化和开启 WDT。

```
/* Open the module. */  
  
err = R_WDT_Open(&g_wdt0_ctrl, &g_wdt0_cfg);  
  
/* Handle any errors. This function should be defined by the user. */  
assert(FSP_SUCCESS == err);
```

R_WDT_Refresh() 函数原型

◆ R_WDT_Refresh()

```
fsp_err_t R_WDT_Refresh ( wdt_ctrl_t *const p_ctrl )
```

Refresh the watchdog timer. Implements `wdt_api_t::refresh`.

In addition to refreshing the watchdog counter this function can be used to start the counter in register start mode.

Example:

```
/* Refresh before the counter underflows to prevent reset or NMI. */  
err = R_WDT_Refresh(&g_wdt0_ctrl);  
assert(FSP_SUCCESS == err);
```

Return values

FSP_SUCCESS	WDT successfully refreshed.
FSP_ERR_ASSERTION	<code>p_ctrl</code> is NULL.
FSP_ERR_NOT_OPEN	Instance control block is not initialized.

Note

This function only returns `FSP_SUCCESS`. If the refresh fails due to being performed outside of the permitted refresh period the device will either reset or trigger an NMI ISR to run.

可以用 `R_WDT_Refresh()` 函数进行喂狗操作。

```
/* Refresh before the counter underflows to prevent reset or NMI. */  
  
err = R_WDT_Refresh(&g_wdt0_ctrl);  
  
assert(FSP_SUCCESS == err);  
  
printf("starting up !\n");  
  
uint32_t wdt_counter = 0U;
```

R_WDT_CounterGet() 函数原型

◆ R_WDT_CounterGet()

```
fsp_err_t R_WDT_CounterGet( wdt_ctrl_t *const p_ctrl,  
                             uint32_t *const p_count  
)
```

Read the current count value of the WDT. Implements `wdt_api_t::counterGet`.

Example:

```
/* Read the current WDT counter value. */  
err = R_WDT_CounterGet(&g_wdt0_ctrl, &wdt_counter);  
assert(FSP_SUCCESS == err);
```

Return values

FSP_SUCCESS WDT current count successfully read.
FSP_ERR_ASSERTION Null pointer passed as a parameter.
FSP_ERR_NOT_OPEN Instance control block is not initialized.

可以用 R_WDT_CounterGet()函数获取当前的计数值。

```
/* Read the current WDT counter value. */  
  
err = R_WDT_CounterGet(&g_wdt0_ctrl, &wdt_counter);  
  
assert(FSP_SUCCESS == err);
```

WDT 周期设定

通过查阅数据手册，可以得知 WDT 使用的时钟为 PCLKB。

24. Watchdog Timer (WDT)

24.1 Overview

The Watchdog Timer (WDT) is a 14-bit down counter that can be used to reset the MCU when the counter underflows because the system has run out of control and is unable to refresh the WDT. In addition, the WDT can be used to generate a non-maskable interrupt or an underflow interrupt.

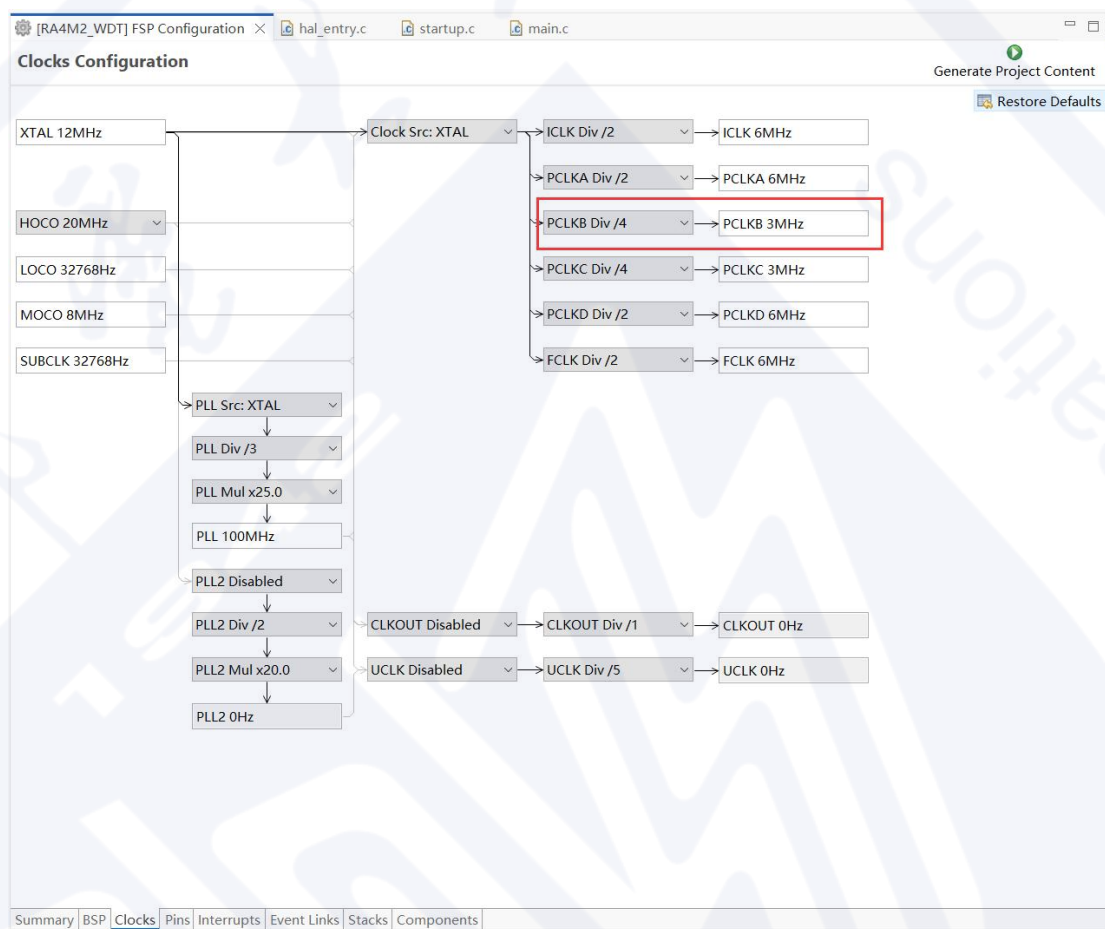
Table 24.1 lists the WDT specifications and Figure 24.1 shows a block diagram.

Table 24.1 WDT specifications

Parameter	Specifications
Count source ^{*1}	Peripheral clock (PCLKB)
Clock division ratio	Division by 4, 64, 128, 512, 2048, or 8192
Counter operation	Counting down using a 14-bit down-counter
Condition for starting the counter	- Auto start mode: Counting automatically starts after a reset or after an underflow or refresh error occurs - Register start mode: Counting is started with a refresh by writing to the WDTRR register - Only secure developer can select Auto-start mode or Register-start mode
Conditions for stopping the counter	- Reset (the down-counter and other registers return to their initial values) - A counter underflows or a refresh error is generated
Window function	Window start and end positions can be specified (refresh-permitted and refresh-prohibited periods)
Watchdog timer reset sources	- Down-counter underflows - Refreshing outside the refresh-permitted period (refresh error)
Non-maskable interrupt/interrupt sources	- Down-counter underflows - Refreshing outside the refresh-permitted period (refresh error)
Reading of the counter value	The down-counter value can be read by the WDTSR register
Event link function (output)	- Down-counter underflow event output - Refresh error event output
Output signal (internal signal)	- Reset output - Interrupt request output - Sleep-mode count stop control output
TrustZone Filter	Security attribution can be set

Note 1. Satisfy the frequency of the peripheral module clock (PCLKB) $\geq 4 \times$ (the frequency of the count clock source after division).

在本案例中，使用的 PCLKB 时钟为 3MHz。



WDT 从 PCLKB 运行，依据上文的设定，PCLKB 周期如下所示。

参数	值
PLCKB	3MHz
分频	PLCK/8192
超时周期	16384 cycles
WDT 时钟频率	$3\text{MHz} / 8192 = 366.2109375\text{ Hz}$
单周期时间	$1 / 366.2109375\text{ Hz} = 2.73067\text{ms}$
溢出时间	$2.73067\text{ms} * 16384\text{ cycles} = 44.739\text{ seconds}$

上述可以看到在该设置下的溢出时间为 44.739s，那么 1s 的计数为 $1\text{s}/2.73067\text{ms}=366.21$ 。

注意事项

如果使用 J-Link 调试器进行调试的话需要加上这一句话。

```
R_DEBUG->DBGSTOPCR_b.DBGSTOP_WDT = 0;
```

主程序

```
while(1)
{
    if(rtc_flag)
    {
        /* Read the current WDT counter value. */
        err = R_WDT_CounterGet(&g_wdt0_ctrl, &wdt_counter);
        assert(FSP_SUCCESS == err);
        printf("wdt_counter=%d\n", wdt_counter);
        rtc_flag=0;
        /* Refresh before the counter underflows to prevent reset or NMI. */
        err = R_WDT_Refresh(&g_wdt0_ctrl);
        assert(FSP_SUCCESS == err);
    }
}
```

演示效果

设置每过 1s 打印一次当前时间，分别设置喂狗和不喂狗，结果如下。
延迟 1s 的计数为 $1s/2.73067ms=366.21$ ，打印为 16018，由于是向下计数， $16384-366=16018$ ，符合计算值。

```
[17:44:04.398]收←◆wdt_counter=16018
[17:44:05.398]收←◆wdt_counter=16019
[17:44:06.398]收←◆wdt_counter=16019
[17:44:07.398]收←◆wdt_counter=16019
[17:44:08.398]收←◆wdt_counter=16018
[17:44:09.397]收←◆wdt_counter=16019
[17:44:10.397]收←◆wdt_counter=16019
[17:44:11.397]收←◆wdt_counter=16019
```

当不执行喂狗时候，计数值到 0 时会进行复位，2 个复位之间为 44s 左右，符合计算的 44.739。

[17:45:00.246]收←◆\0
[17:45:03.827]收←◆hello world!
starting up !

[17:45:04.831]收←◆wdt_counter=16017
[17:45:05.831]收←◆wdt_counter=15651
[17:45:06.830]收←◆wdt_counter=15285
[17:45:07.831]收←◆wdt_counter=14919
[17:45:08.830]收←◆wdt_counter=14553
[17:45:09.830]收←◆wdt_counter=14187
[17:45:10.830]收←◆wdt_counter=13820
[17:45:11.830]收←◆wdt_counter=13454
[17:45:12.829]收←◆wdt_counter=13088
[17:45:13.830]收←◆wdt_counter=12722
[17:45:14.830]收←◆wdt_counter=12356
[17:45:15.830]收←◆wdt_counter=11989
[17:45:16.829]收←◆wdt_counter=11623
[17:45:17.829]收←◆wdt_counter=11257
[17:45:18.829]收←◆wdt_counter=10891
[17:45:19.829]收←◆wdt_counter=10525
[17:45:20.828]收←◆wdt_counter=10159
[17:45:21.829]收←◆wdt_counter=9792
[17:45:22.828]收←◆wdt_counter=9426
[17:45:23.828]收←◆wdt_counter=9060
[17:45:24.828]收←◆wdt_counter=8694
[17:45:25.827]收←◆wdt_counter=8328
[17:45:26.828]收←◆wdt_counter=7961
[17:45:27.827]收←◆wdt_counter=7595
[17:45:28.828]收←◆wdt_counter=7229
[17:45:29.827]收←◆wdt_counter=6863
[17:45:30.827]收←◆wdt_counter=6497
[17:45:31.827]收←◆wdt_counter=6131
[17:45:32.827]收←◆wdt_counter=5764
[17:45:33.827]收←◆wdt_counter=5398
[17:45:34.826]收←◆wdt_counter=5032
[17:45:35.827]收←◆wdt_counter=4666
[17:45:36.827]收←◆wdt_counter=4300
[17:45:37.826]收←◆wdt_counter=3933
[17:45:38.826]收←◆wdt_counter=3567
[17:45:39.826]收←◆wdt_counter=3201
[17:45:40.826]收←◆wdt_counter=2835
[17:45:41.826]收←◆wdt_counter=2469
[17:45:42.826]收←◆wdt_counter=2103
[17:45:43.826]收←◆wdt_counter=1736
[17:45:44.825]收←◆wdt_counter=1370
[17:45:45.825]收←◆wdt_counter=1004
[17:45:46.825]收←◆wdt_counter=638

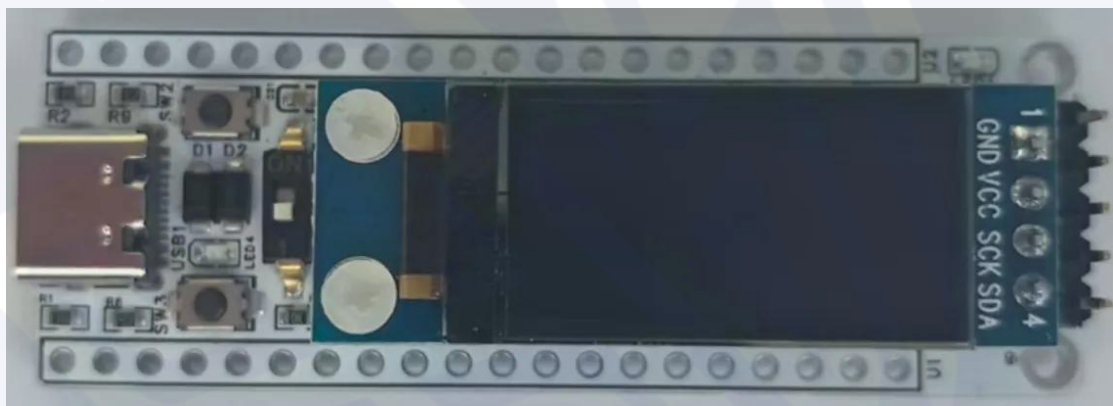
主程序

project19: 独立看门狗 IWDT

概述

本文主要阐述了如何借助 e2studio 工具对瑞萨独立看门狗 IWDT 进行配置设置，同时设置 RTC 时钟产生每秒的周期性中断，并通过串口输出观察独立看门狗 IWDT 的计数情况。

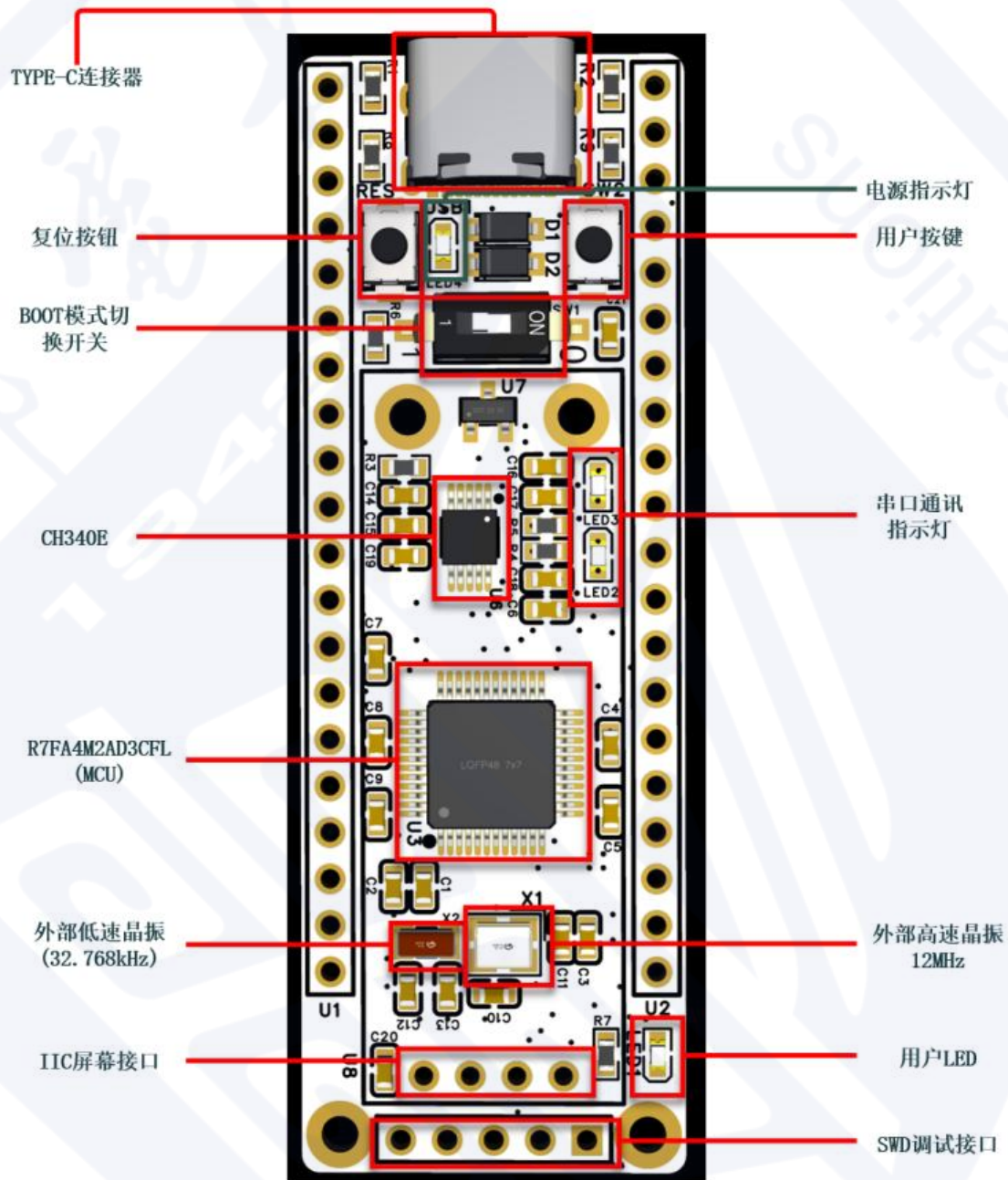
配置看门狗定时器以提高系统稳定性，防止系统长时间处于死循环。



硬件准备

首先需要准备一个开发板，这里我准备的是自己绘制的开发板，需要的可以进行申请。

主控为 R7FA4M2AD3CFL

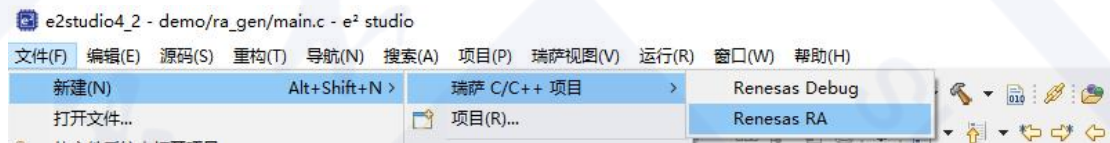


参考程序

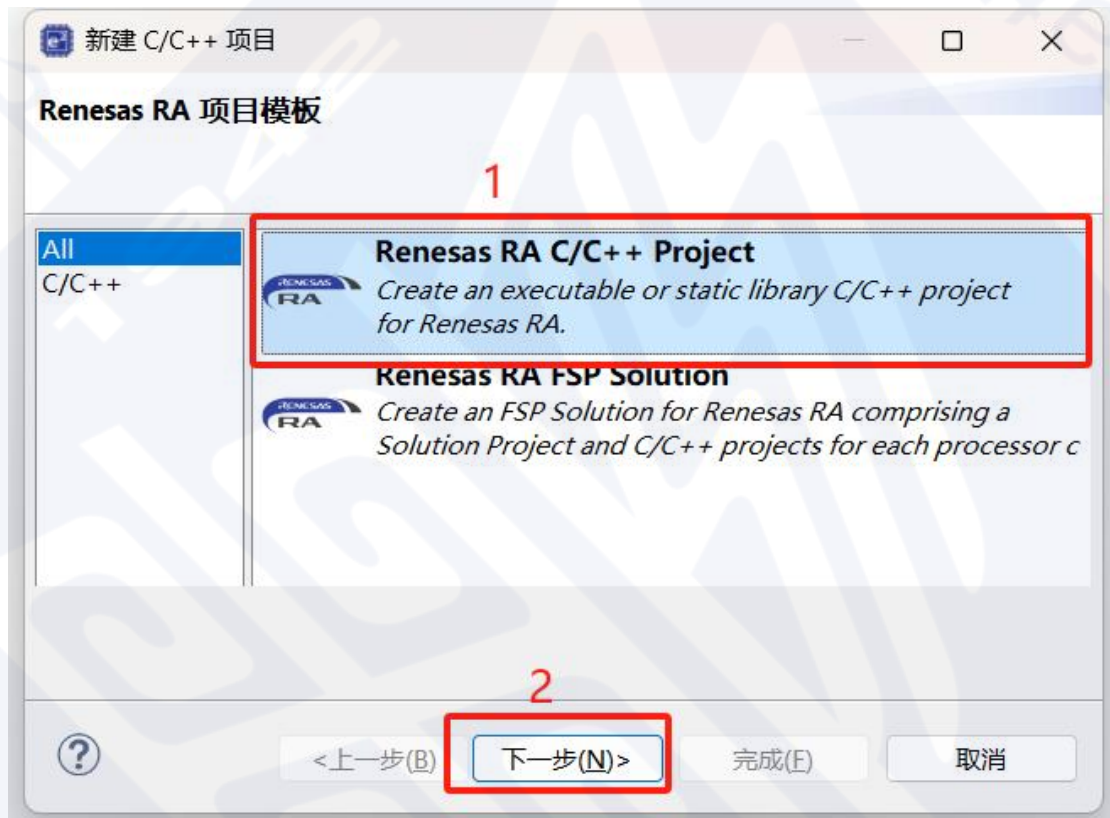
https://github.com/CoreMaker-lab/RA4M2_MINI

https://gitee.com/CoreMaker/RA4M2_MINI

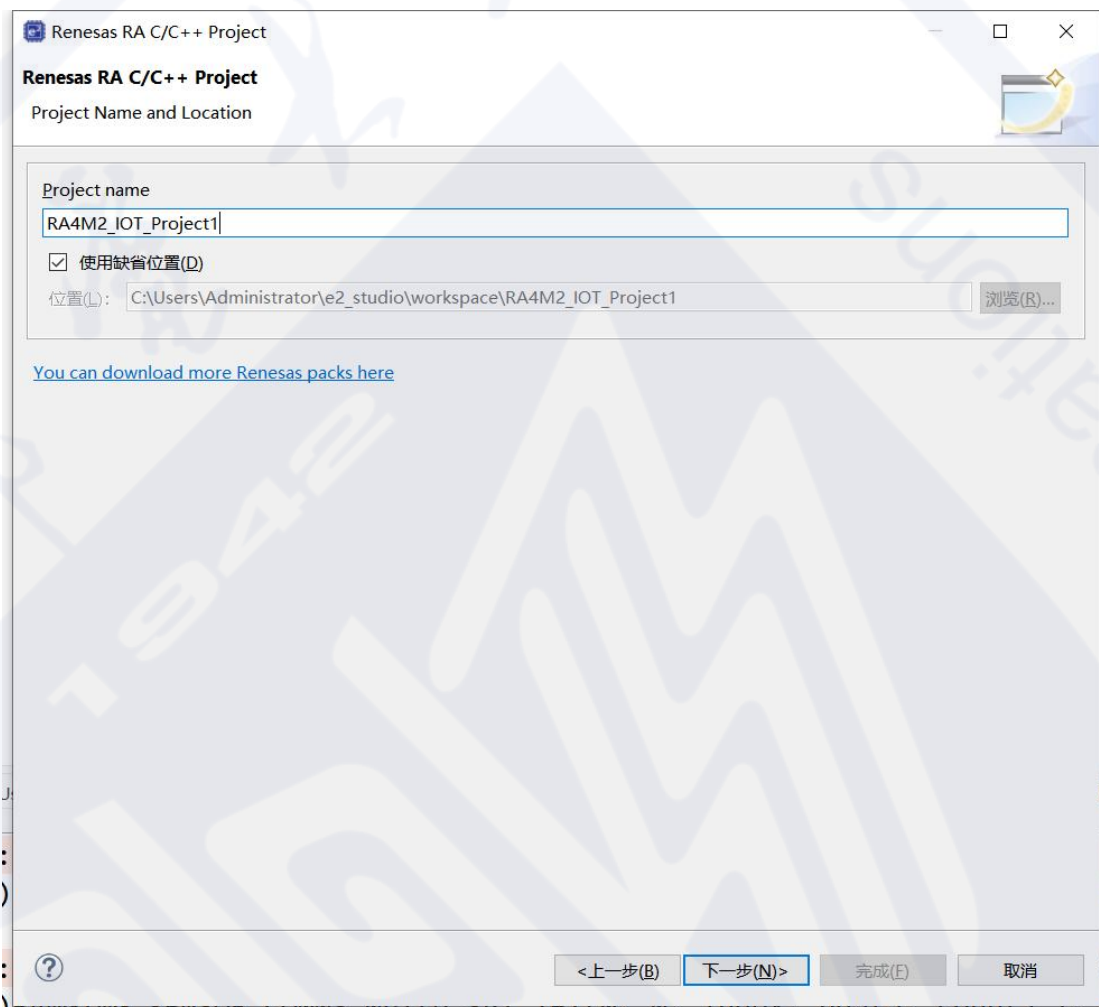
新建工程



工程模板



保存工程路径



芯片配置

本文中使用 R7FA4M2AD3CFL 来进行演示。

Renesas RA C/C++ Project

Renesas RA C/C++ Project

Device and Tools Selection

Device Selection

FSP Version: 5.9.0 **版本**

Board: Custom User Board (Any Device) ...

Device: R7FA4M2AD3CFL **芯片型号**

Core: CM33

Language: ☒ C ☐ C++

Board Description

Device Details

TrustZone	Yes
Pins	48
Processor	Cortex-M33

IDE Project Type

e2 studio managed build

Toolchains

LLVM Embedded Toolchain for Arm

GNU ARM Embedded **工具链**

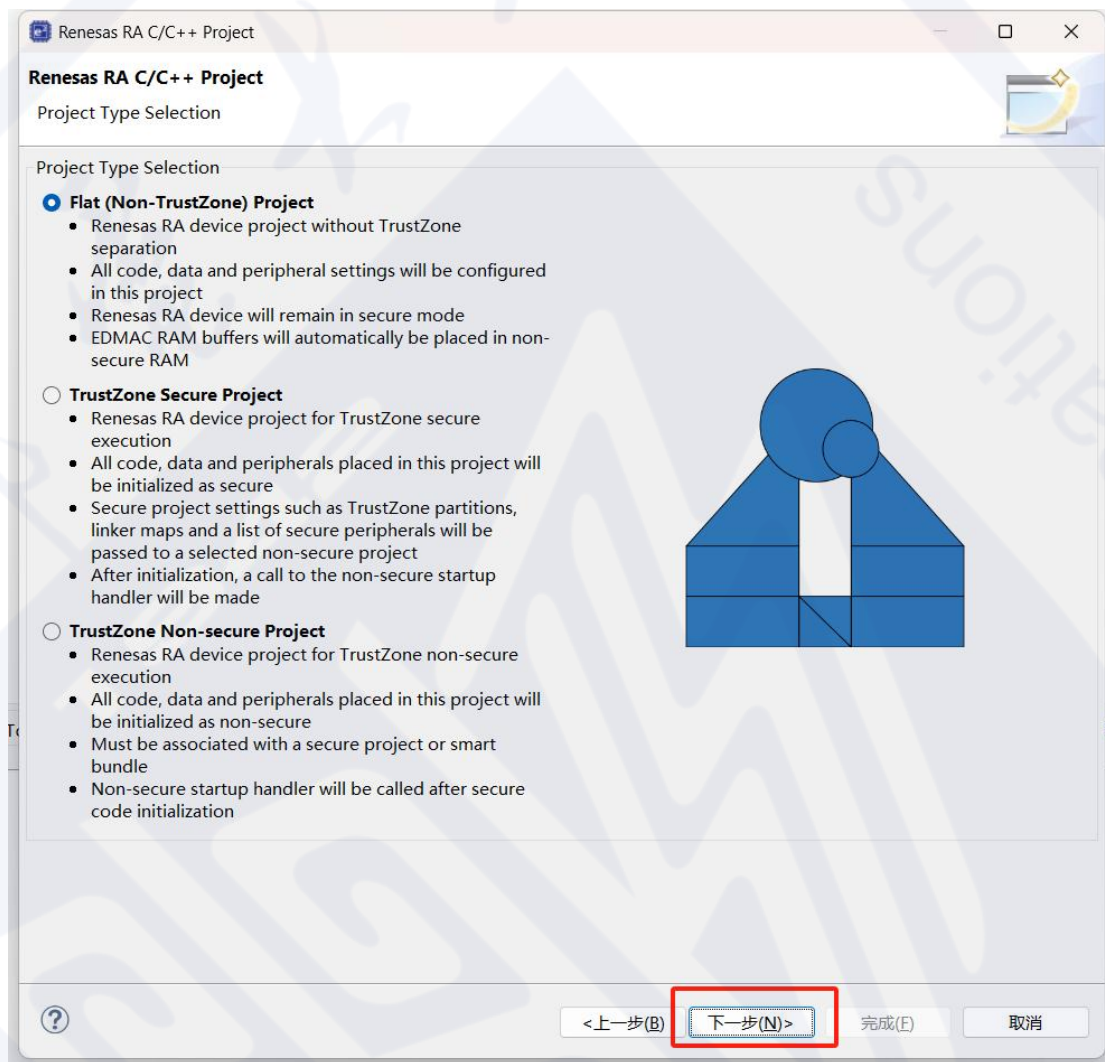
13.2.1.arm-13-7 [Manage Toolchains...](#)

Debugger

J-Link ARM **使用JLink进行调试**

[?](#) <上一步(B) 下一步(N)> 完成(F) 取消

工程模板选择



Renesas RA C/C++ Project

Renesas RA C/C++ Project

Preceding Project or Smart Bundle Selection

☒ None

Choose this option when creating a project for the primary processor core (no preceding project or Smart Bundle).

☐ Preceding Project:

Choose this option if you have access to the preceding bootloader project or TrustZone secure project source code. A debug configuration for Secure Software Debug (SSD) state will be generated where applicable and both secure and non-secure images will be downloaded to the target device. Should you wish to test Non-Secure Software Debug (NSECSO) state, use the Renesas Device Partition Manager to change the Device Lifecycle state as needed.

☐ Smart Bundle:

Resolved location:

Workspace...

File System...

Variables...

Choose this option if you only have access to a pre-programmed device containing TrustZone secure code. The Smart Bundle file (*.sbd) should be obtained from the TrustZone Secure Project developer. A debug configuration for Non-Secure Software Debug (NSECSO) state will be generated and only the non-secure code image will be downloaded to the target device.

Preceding Project/Smart Bundle Details

FSP version

Toolchain

Toolchain version

Board

Device

Core

Zones

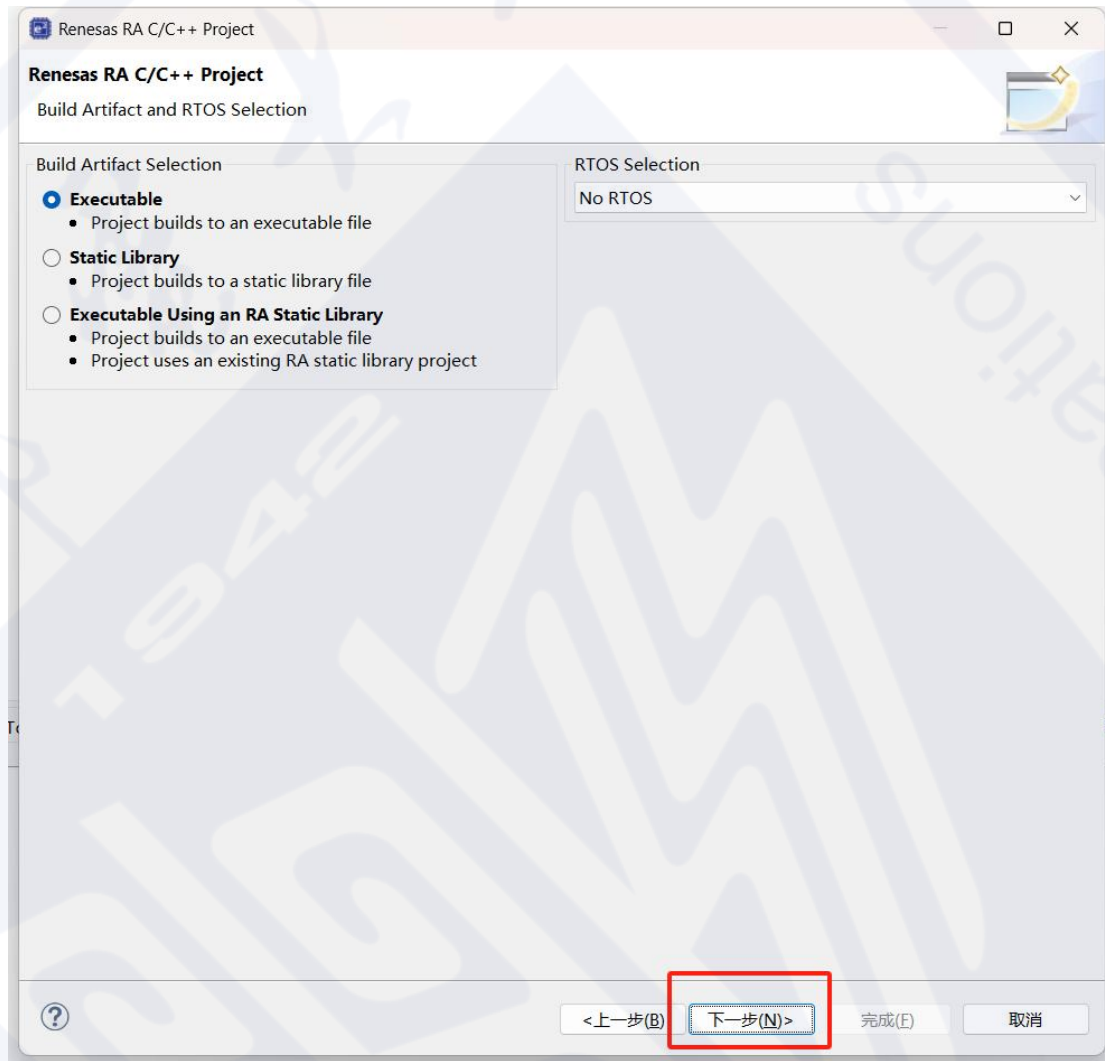
?

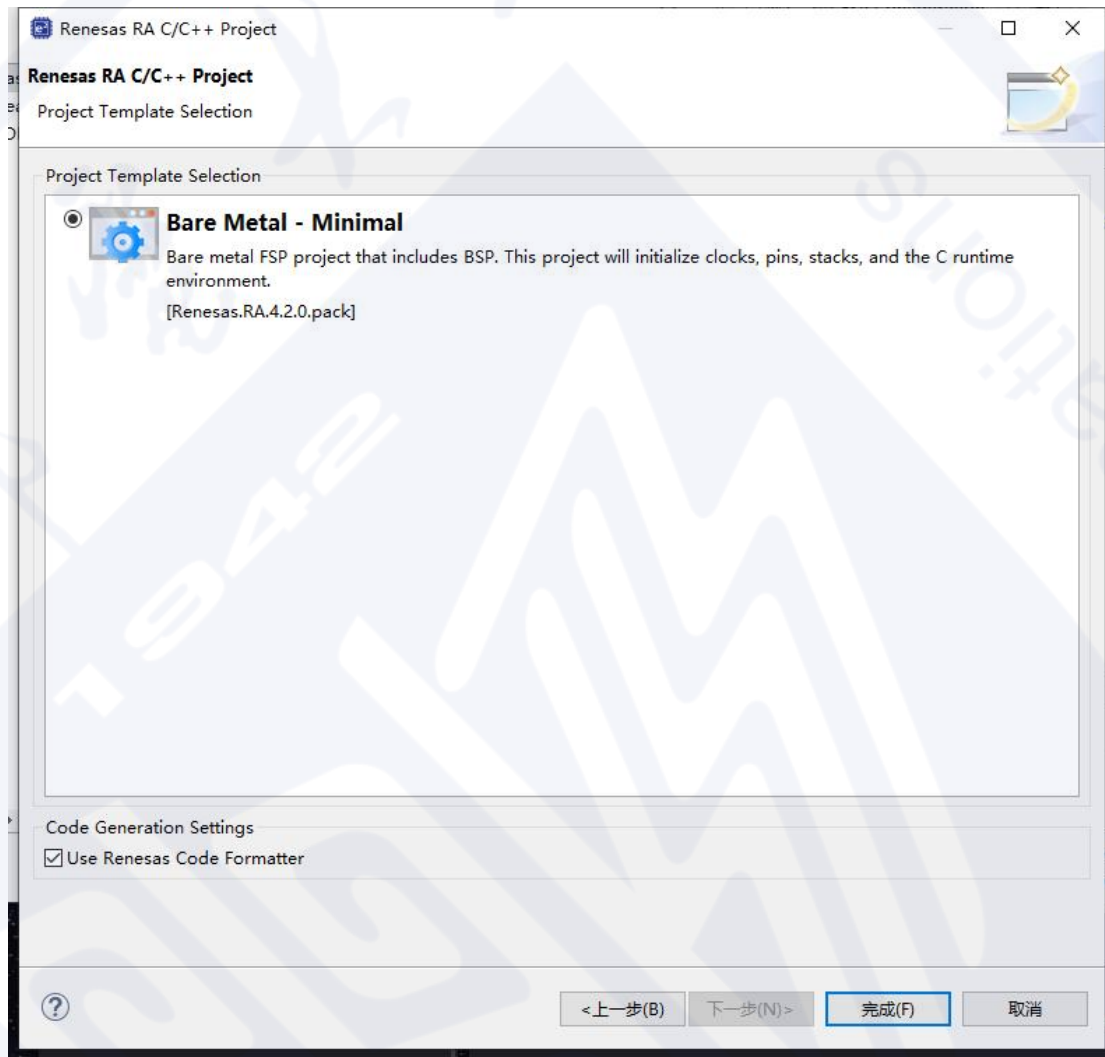
< 上一步(B)

下一步(N) >

完成(E)

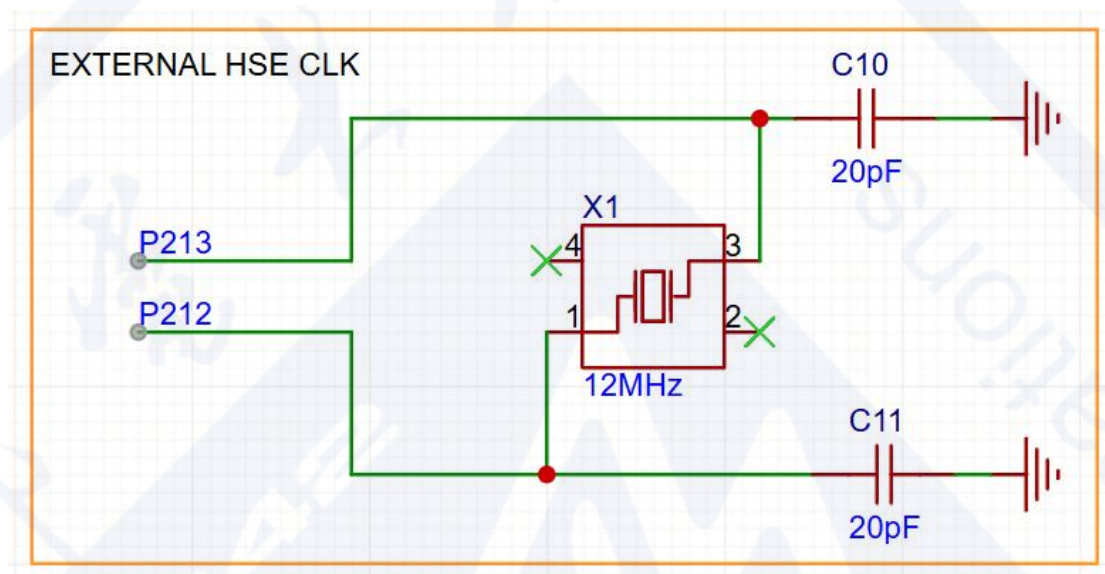
取消



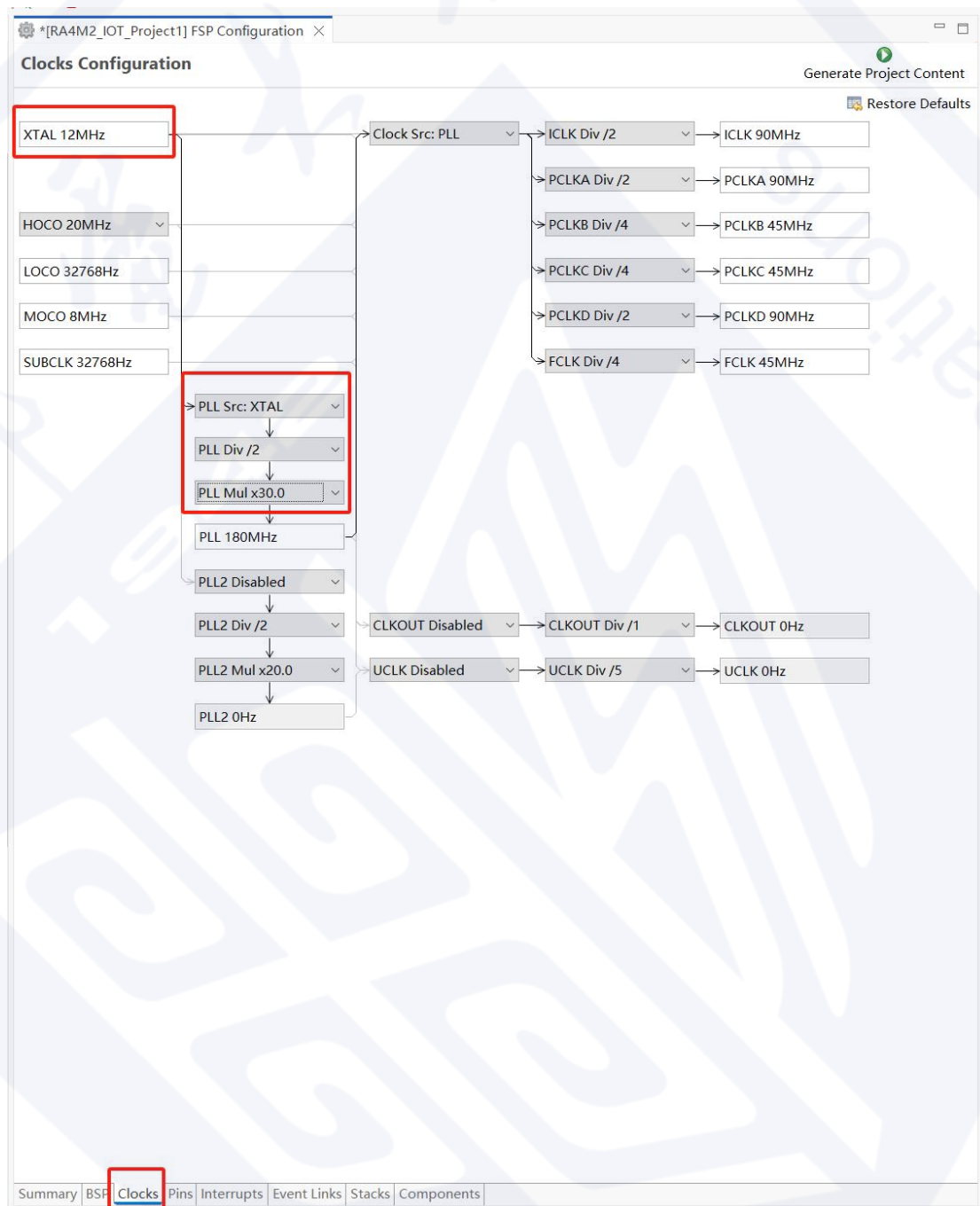


时钟设置

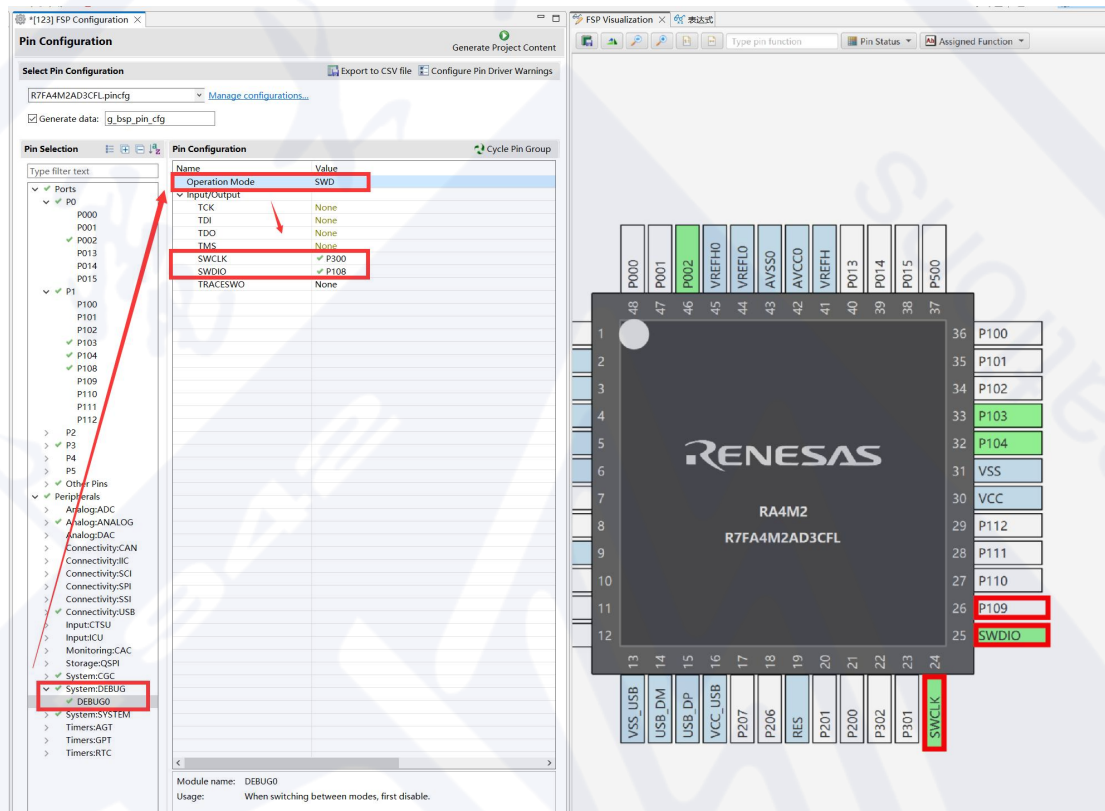
开发板上的外部高速晶振为 12M.



需要修改 XTAL 为 12M。

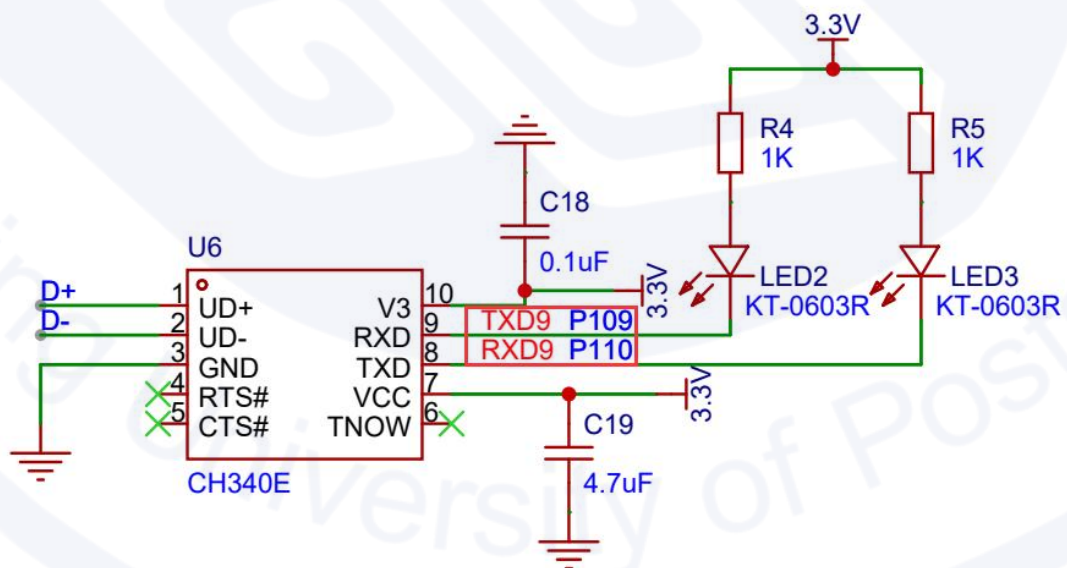


SWD 调试口设置

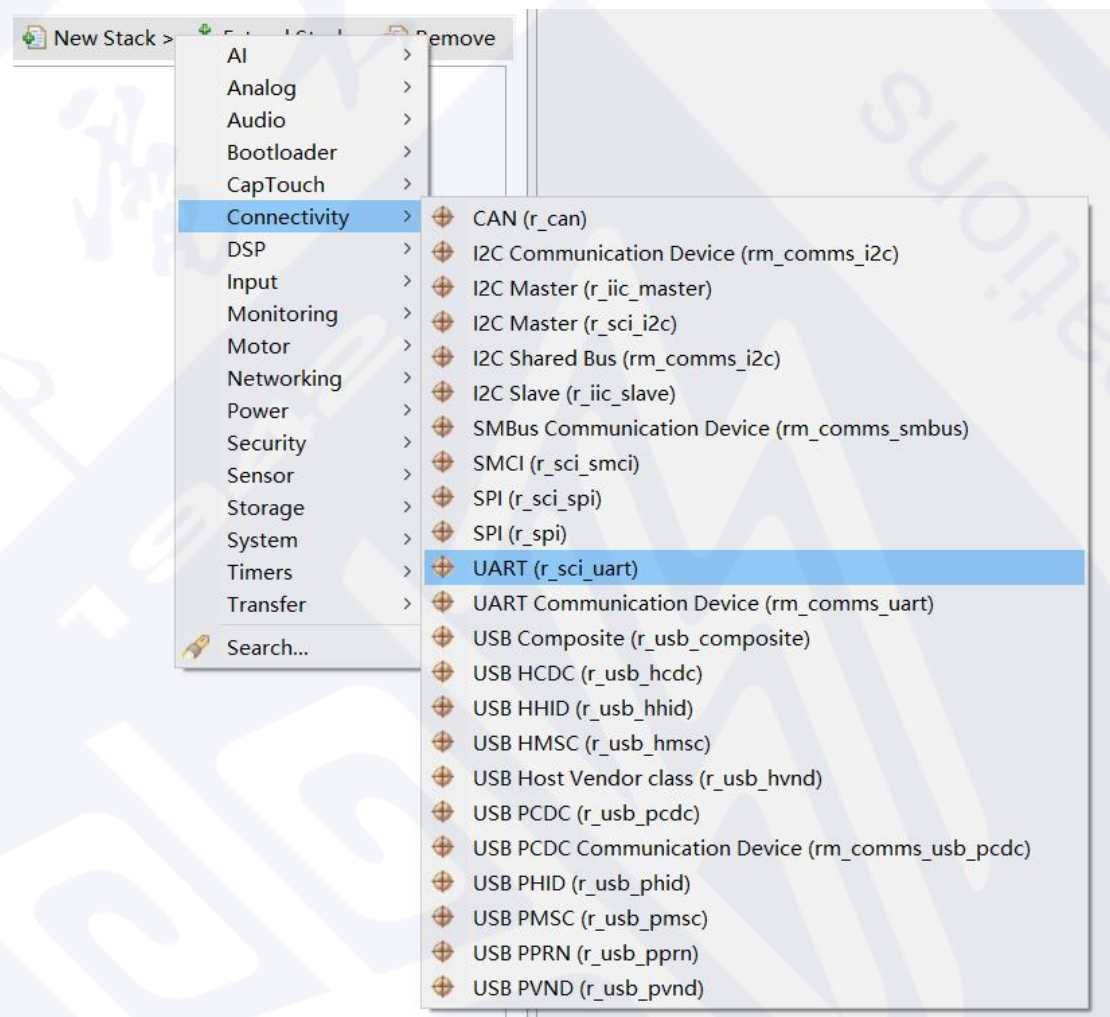


UART 配置

串口CH340E

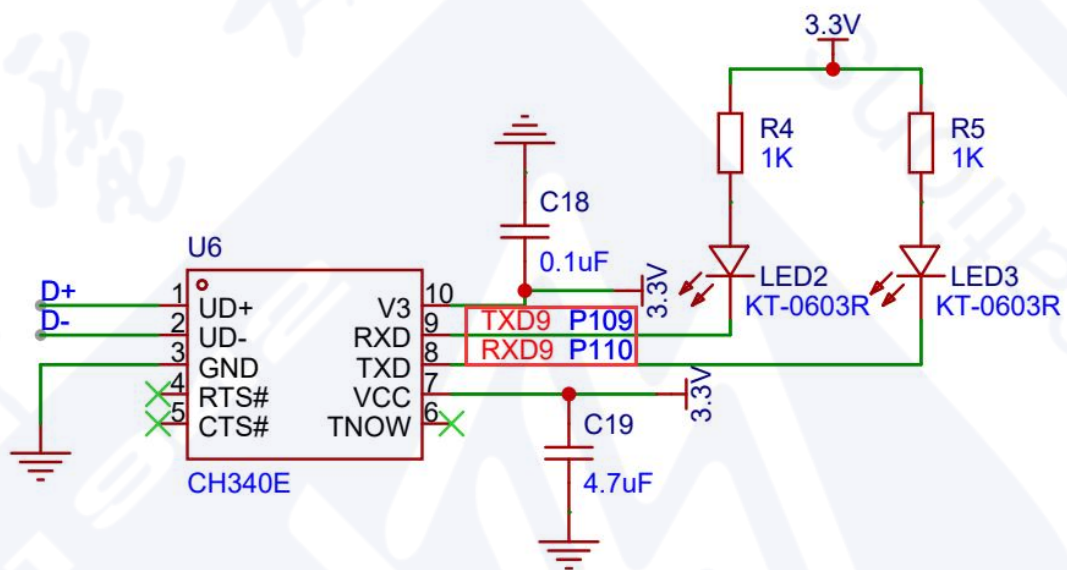


点击 Stacks->New Stack->Connectivity -> UART(r_sci_uart)。

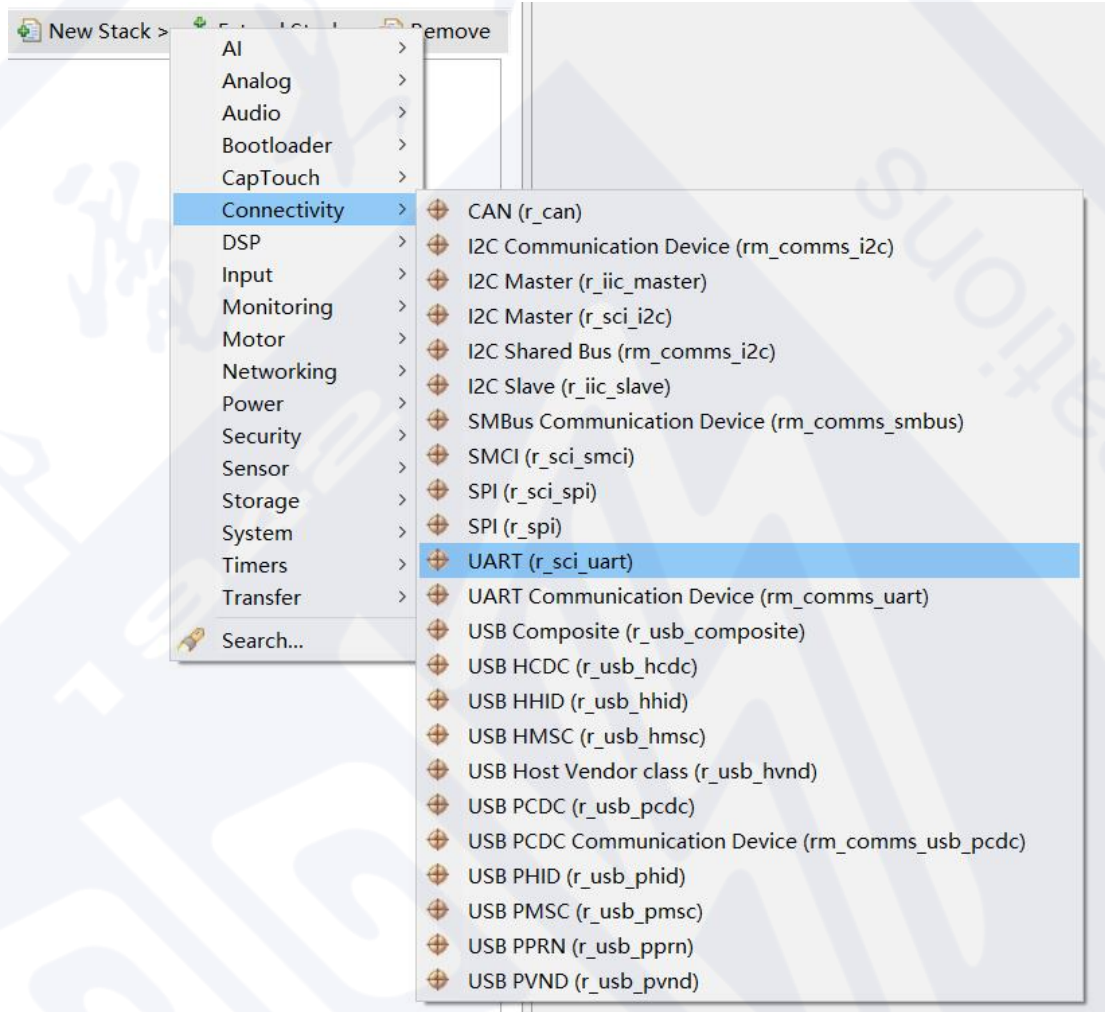


UART 配置

串口CH340E



点击 Stacks->New Stack->Connectivity -> UART(r_sci_uart)。



UART 属性配置

属性 × 问题 Smart Browser 控制台 调试		
g_uart9 UART (r_sci_uart)		
Settings	属性	值
API Info	▼ Common	
	Parameter Checking	Default (BSP)
	FIFO Support	Disable
	DTC Support	Disable
	Flow Control Support	Disable
	RS-485 Support	Disable
	IrDA Support	Disabled
	▼ Module g_uart9 UART (r_sci_uart)	
	▼ General	
	Name	g_uart9 名称
	Channel	9 通道
	Data Bits	8bits 数据位
	Parity	None 无校验位
	Stop Bits	1bit 停止位
	▼ Baud	
	Baud Rate	115200 波特率
	Baud Rate Modulation	Enabled 启用波特率调制
	Max Error (%)	5
	> Flow Control	
	> Extra	
	▼ Interrupts	
	Callback	user_uart_callback 回调函数名称
	Receive Interrupt Priority	Priority 2
	Transmit Data Empty Interrupt Priority	Priority 2
	Transmit End Interrupt Priority	Priority 2
	Error Interrupt Priority	Priority 2
	▼ Pins	
	TXD9	P109 串口发送端口
	RXD9	P110 串口接收端口
	CTS9	<unavailable>

设置 e2studio 堆栈

printf 函数通常需要设置堆栈大小。这是因为 printf 函数在运行时需要使用栈空间来存储临时变量和函数调用信息。如果堆栈大小不足，可能会导致程序崩溃或不可预期的行为。

printf 函数使用了可变参数列表，它会在调用时使用栈来存储参数，在函数调用结束时再清除参数，这需要足够的栈空间。另外 printf 也会使用一些临时变量，如果栈空间不足，会导致程序崩溃。

因此，为了避免这类问题，应该根据程序的需求来合理设置堆栈大小。

项目资源管理器

demo

- 二进制
- Includes
- ra
- ra_gen
- src
 - hal_entry.c
 - Debug
 - ra_cfg
 - script
 - configuration.xml
 - demo Debug_FlatJlink
 - demo Debug_FlatLaunch
 - ra_cfg.txt
- Developer Assistance
 - HAL/Common
 - g_ioport I/O Port (r_ioport)
 - fs_err t R IOPORT Open(ioport ctrl t *const

属性

Board Support Package Configuration

Device Selection

FSP version: 4.2.0

Board: Custom User Board (Any Device)

Device: R7FA4M2AD3CFP

Core: CM33

RTOS: No RTOS

Board Details

Summary **BSP** Clocks Pins Interrupts Event Links Stacks Components

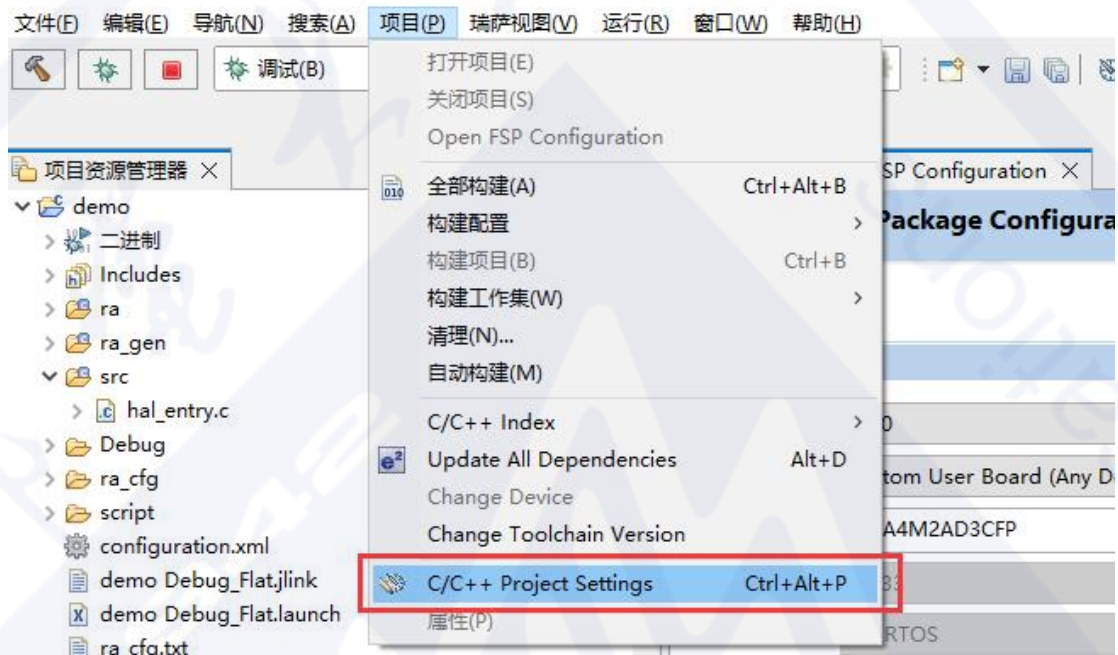
Custom User Board (Any Device)

Settings

属性	值
√ R7FA4M2AD3CFP	
part_number	R7FA4M2AD3CFP
rom_size_bytes	524288
ram_size_bytes	131072
data_flash_size_bytes	8192
package_style	LQFP
package_pins	100
√ RA4M2	
series	4
√ RA4M2 Family	
> Security	
> OFS0 register settings	
> OFS1 register settings	
> Block Protection Settings (BPS)	
> Permanent Block Protection Settings (PBPS)	
> Clocks	
Startup C-Cache Line Size	32 Bytes
Main Oscillator Wait Time	8163 cycles
√ RA Common	
Main stack size (bytes)	0x400
Heap size (bytes)	0x2000
MCU Vcc (mV)	3300
Parameter checking	Disabled
Assert Failures	Return FSP_ERR_ASSERTION
Error Log	No Error Log
Clock Registers not Reset Values during Startup	Disabled
Main Oscillator Populated	Populated
PFS Protect	Enabled
C Runtime Initialization	Enabled
Early BSP Initialization	Disabled
Main Oscillator Clock Source	Crystal or Resonator
Subclock Populated	Populated
Subclock Drive (Drive capacitance availability varies by MCU)	Standard/Normal mode
Subclock Stabilization Time (ms)	1000

3.设置堆栈大小

e2studio 的重定向 printf 设置



在嵌入式系统的开发中，尤其是在使用 GNU 编译器集合（GCC）时，`--specs` 参数用于指定链接时使用的系统规格（specs）文件。这些规格文件控制了编译器和链接器的行为，尤其是关于系统库和启动代码的链接。`--specs=rdimon.specs` 和 `--specs=nosys.specs` 是两种常见的规格文件，它们用于不同的场景。

`--specs=rdimon.specs`

用途：这个选项用于链接“Redlib”库，这是为裸机（bare-metal）和半主机（semihosting）环境设计的 C 库的一个变体。半主机环境是一种特殊的运行模式，允许嵌入式程序通过宿主机（如开发 PC）的调试器进行输入输出操作。

应用场景：当你需要在没有完整操作系统的环境中运行程序，但同时需要使用调试器来处理输入输出（例如打印到宿主机的终端），这个选项非常有用。

特点：它提供了一些基本的系统调用，通过调试接口与宿主机通信。

`--specs=nosys.specs`

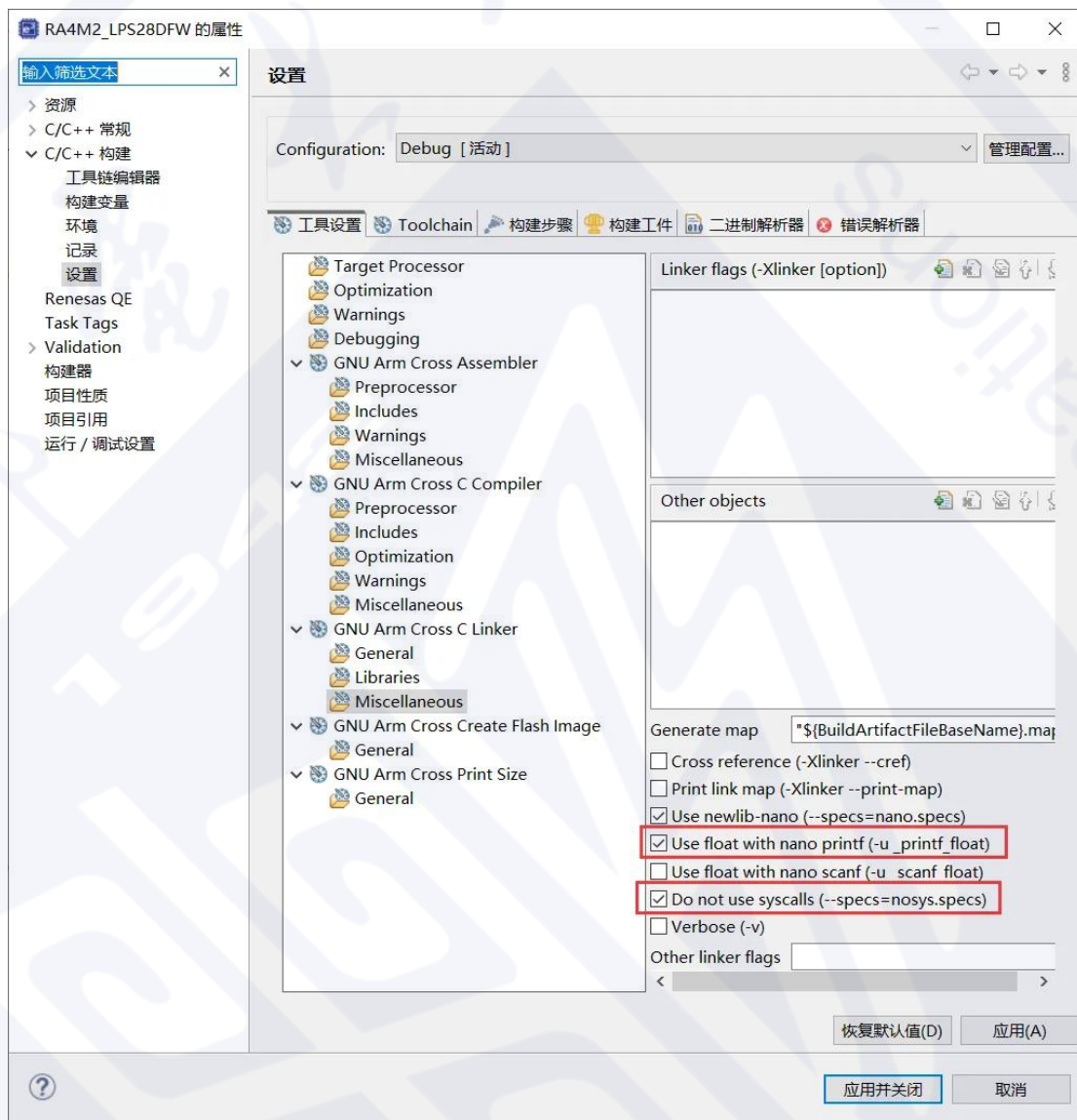
用途：这个选项链接了一个非常基本的系统库，这个库不提供任何系统服务的实现。

应用场景：适用于完全的裸机程序，其中程序不执行任何操作系统调用，比如不进行文件操作或者系统级输入输出。

特点：这是一个更“裸”的环境，没有任何操作系统支持。使用这个规格文件，程序不期望有操作系统层面的任何支持。

如果你的程序需要与宿主机进行交互（如在开发期间的调试），并且通过调试器进行基本的输入输出操作，则使用 `--specs=rdimon.specs`。

如果你的程序是完全独立的，不需要任何形式的操作系统服务，包括不进行任何系统级的输入输出，则使用 `--specs=nosys.specs`。



R_SCI_UART_Open() 函数原型

◆ R_SCI_UART_Open()

```
fsp_err_t R_SCI_UART_Open ( uart_ctrl_t *const    p_api_ctrl,  
                           uart_cfg_t const *const p_cfg  
                           )
```

Configures the UART driver based on the input configurations. If reception is enabled at compile time, reception is enabled at the end of this function. Implements `uart_api_t::open`

Return values

FSP_SUCCESS	Channel opened successfully.
FSP_ERR_ASSERTION	Pointer to UART control block or configuration structure is NULL.
FSP_ERR_IP_CHANNEL_NOT_PRESENT	The requested channel does not exist on this MCU.
FSP_ERR_INVALID_ARGUMENT	Flow control is enabled but flow control pin is not defined or selected channel does not support "Hardware CTS and Hardware RTS" flow control. (or) restricted channel is selected.
FSP_ERR_ALREADY_OPEN	Control block has already been opened or channel is being used by another instance. Call <code>close()</code> then <code>open()</code> to reconfigure.
FSP_ERR_INVALID_CHANNEL	IrDA is requested for a channel that does not support IrDA.

Returns

See [Common Error Codes](#) or functions called by this function for other possible return codes. This function calls:

- `transfer_api_t::open`

故可以用 `R_SCI_UART_Open()`函数进行配置，开启和初始化 UART。

```
/* Open the transfer instance with initial configuration. */  
err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);  
assert(FSP_SUCCESS == err);
```

```

2
3 FSP_CPP_HEADER
4 void R_BSP_WarmStart(bsp_warm_start_event_t event);
5 FSP_CPP_FOOTER
6 /* unused parameter 'fd' [-Wunused-parameter] | no previous declaration for '_write' [-W
7
8
9
10
11
12
13
14
15 /* Open the transfer instance with initial configuration. */
16 err = R_SCI_UART_Open(&g_uart9_ctrl1, &g_uart9_cfg);
17 assert(FSP_SUCCESS == err);
18
19
20
21 #if BSP_TZ_SECURE_BUILD
22 /* Enter non-secure code */
23 R_BSP_NonSecureEnter();
24 #endif
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48

```

回调函数 user_uart_callback ()

当数据发送的时候，可以查看 UART_EVENT_TX_COMPLETE 来判断是否发送完毕。

◆ write

```

fsp_err_t (*uart_api_t::write)(uart_ctrl_t *const p_ctrl, uint8_t const *const p_src, uint32_t const bytes)

```

Write to UART device. The write buffer is used until write is complete. Do not overwrite write buffer contents until the write is finished. When the write is complete (all bytes are fully transmitted on the wire), the callback called with event **UART_EVENT_TX_COMPLETE**. The maximum transfer size is reported by [infoGet\(\)](#).

Implemented as

- R_SCI_UART_Write()
- R_SCI_B_UART_Write()

Parameters

[in] p_ctrl Pointer to the UART control block.

[in] p_src Source address to write data to.

[in] bytes Write data length.

Enumeration Type Documentation

◆ uart_event_t

enum `uart_event_t`

UART Event codes

Enumerator	
UART_EVENT_RX_COMPLETE	Receive complete event.
UART_EVENT_TX_COMPLETE	Transmit complete event.
UART_EVENT_RX_CHAR	Character received.
UART_EVENT_ERR_PARITY	Parity error event.
UART_EVENT_ERR_FRAMING	Mode fault error event.
UART_EVENT_ERR_OVERFLOW	FIFO Overflow error event.
UART_EVENT_BREAK_DETECT	Break detect error event.
UART_EVENT_TX_DATA_EMPTY	Last byte is transmitting, ready for more data.

可以检查检查 "p_args" 结构体中的 "event" 字段的值是否等于 "UART_EVENT_TX_COMPLETE"。如果条件为真，那么 if 语句后面的代码块将会执行。

```
fsp_err_t err = FSP_SUCCESS;
volatile bool uart_send_complete_flag = false;
void user_uart_callback (uart_callback_args_t * p_args)
{
    if(p_args->event == UART_EVENT_TX_COMPLETE)
    {
        uart_send_complete_flag = true;
    }
}
```

```
[RA4M2_MINI_Project11_1] FSP Configuration  *hal_entry.c  startup.c  main.c
1  #include "hal_data.h"
2
3  FSP_CPP_HEADER
4  void R_BSP_WarmStart(bsp_warm_start_event_t event);
5  FSP_CPP_FOOTER
6
7  fsp_err_t err = FSP_SUCCESS;
8  volatile bool uart_send_complete_flag = false;
9  void user_uart_callback (uart_callback_args_t * p_args)
10 {
11     if(p_args->event == UART_EVENT_TX_COMPLETE)
12     {
13         uart_send_complete_flag = true;
14     }
15 }
16
17 unused parameter 'fd' [-Wunused-parameter] | no previous declaration for '_write' [-W
18
19 * main() is generated by the RA Configuration editor and is used to generate threads if
20 void hal_entry(void)
21 {
22     /* TODO: add your own code here */
23
24     /* Open the transfer instance with initial configuration. */
25     err = R_SCI_UART_Open(&g_uart9_ctrl1, &g_uart9_cfg);
26     assert(FSP_SUCCESS == err);
27
28     #if BSP_TZ_SECURE_BUILD
29     /* Enter non-secure code */
30     R_BSP_NonSecureEnter();
31     #endif
32
33     * This function is called at various points during the startup process. This implement
34     void R_BSP_WarmStart(bsp_warm_start_event_t event)
35     {
36         if (BSP_WARM_START_RESET == event)
37         {
38             #if BSP_FEATURE_FLASH_LP_VERSION != 0
39
```

printf 输出重定向到串口

打印最常用的方法是 `printf`，所以要解决的问题是将 `printf` 的输出重定向到串口，然后通过串口将数据发送出去。

注意一定要加上头文件 `#include <stdio.h>`

```
#ifdef __GNUC__ //串口重定向
#define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
#else
#endif

PUTCHAR_PROTOTYPE
{
    err = R_SCI_UART_Write(&g_uart9_ctrl1, (uint8_t *)&ch, 1);
    if(FSP_SUCCESS != err) __BKPT();
    while(uart_send_complete_flag == false){}
    uart_send_complete_flag = false;
}
```

```

        return ch;
    }

    int _write(int fd, char *pBuffer, int size)
    {
        for(int i=0; i<size; i++)
        {
            __io_putchar(*pBuffer++);
        }

        return size;
    }
}

```

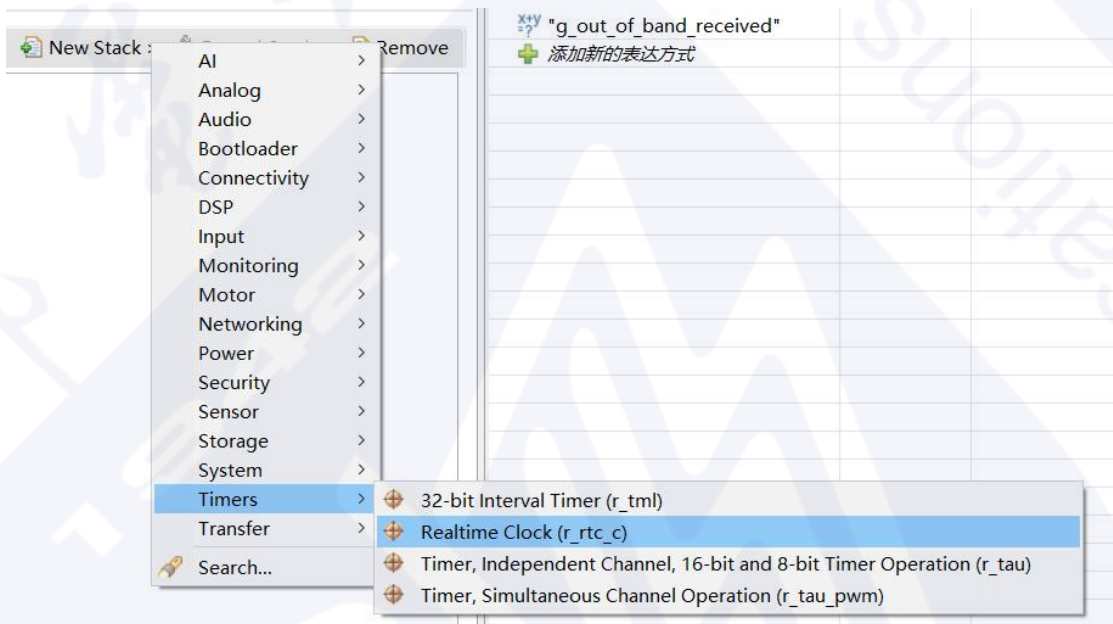
```

1  #include "hal_data.h"
2  #include <stdio.h>
3  FSP_CPP_HEADER
4  void R_BSP_WarmStart(bsp_warm_start_event_t event);
5  FSP_CPP_FOOTER
6
7  fsp_err_t err = FSP_SUCCESS;
8  volatile bool uart_send_complete_flag = false;
9  void user_uart_callback (uart_callback_args_t * p_args)
10 {
11     if(p_args->event == UART_EVENT_TX_COMPLETE)
12     {
13         uart_send_complete_flag = true;
14     }
15 }
16
17
18 #ifdef __GNUC__ //串口重定向
19 #define PUTCHAR_PROTOTYPE int __io_putchar(int ch)
20 #else
21 #define PUTCHAR_PROTOTYPE int fputc(int ch, FILE *f)
22 #endif
23
24
25 PUTCHAR_PROTOTYPE
26 {
27     err = R_SCI_UART_Write(&g_uart9_ctrl, (uint8_t *)&ch, 1);
28     if(FSP_SUCCESS != err) __BKPT();
29     while(uart_send_complete_flag == false){}
30     uart_send_complete_flag = false;
31     return ch;
32 }
33
34 int _write(int fd, char *pBuffer, int size)
35 {
36     for(int i=0; i<size; i++)
37     {
38         __io_putchar(*pBuffer++);
39     }
40     return size;
41 }
42
43
44
45 * * main() is generated by the RA Configuration editor and is used to generate threads i
46 void hal_entry(void)
47 {
48     /* TODO: add your own code here */
49
50     /* Open the transfer instance with initial configuration. */
51     err = R_SCI_UART_Open(&g_uart9_ctrl, &g_uart9_cfg);
52     assert(FSP_SUCCESS == err);
53
54
55

```

RTC 配置

点击 Stacks->New Stack->Timers -> Realtime Clock(r_rtc_c)。



RTC 属性配置

属性 × 问题 Smart Browser 控制台		
g_rtc0 Realtime Clock (r_rtc)		
Settings	属性	值
API Info	▼ Common	
	Parameter Checking	Default (BSP)
	Set Source Clock in Open	Enabled
	▼ Module g_rtc0 Realtime Clock (r_rtc)	
	Name	g_rtc0
	Clock Source	Sub-Clock
	Frequency Comparison Value (LOCO)	255
	Automatic Adjustment Mode	Enabled
	Automatic Adjustment Period	10 Seconds
	Adjustment Type (Plus-Minus)	NONE
	Error Adjustment Value	0
	Callback	rtc_callback
	Alarm Interrupt Priority	Priority 2
	Period Interrupt Priority	Priority 2
	Carry Interrupt Priority	Priority 2
	▼ Pins	
	RTCOUNT	<unavailable>

名称
时钟源，SUBCLK时钟精度高

误差调整模式
自动调整的误差调整周期

调整值
回调函数

闹钟报警中断优先级
周期中断优先级

设定时间

在启动 RTC 后，需要为其设定当前时间。您可以使用 `R_RTC_CalendarTimeSet(&g_rtc0_ctrl, &set_time);` 函数来实现这一目标。具体的时间参数可以通过修改 `set_time` 变量来调整。

◆ `R_RTC_CalendarTimeSet()`

```
fsp_err_t R_RTC_CalendarTimeSet ( rtc_ctrl_t *const p_ctrl,
                                rtc_time_t *const p_time
                                )
```

Set the calendar time.
Implements `rtc_api_t::calendarTimeSet`.

Return values

FSP_SUCCESS	Calendar time set operation was successful.
FSP_ERR_ASSERTION	Invalid input argument.
FSP_ERR_NOT_OPEN	Driver not open already for operation.
FSP_ERR_INVALID_ARGUMENT	Invalid time parameter field.

```
//RTC 变量
/* rtc_time_t is an alias for the C Standard time.h struct 'tm' */
rtc_time_t set_time =
{
    .tm_sec = 50,      /* 秒, 范围从 0 到 59 */
    .tm_min = 59,      /* 分, 范围从 0 到 59 */
    .tm_hour = 23,     /* 小时, 范围从 0 到 23 */
    .tm_mday = 28,     /* 一月中的第几天, 范围从 1 到 31 */
    .tm_mon = 1,       /* 月份, 范围从 0 到 11 */
    .tm_year = 125,    /* 自 1900 起的年数, 2025 为 125 */
    .tm_wday = 5,      /* 一周中的第几天, 范围从 0 到 6 */
    // .tm_yday=0,      /* 一年中的第几天, 范围从 0 到 365 */
    // .tm_isdst=0;     /* 夏令时 */
};
```

回调函数

可以触发进入回调函数的事件如下所示，`RTC_EVENT_PERIODIC_IRQ` 为设置的实时性事件，例如 1s 一次，`RTC_EVENT_ALARM_IRQ` 为闹钟事件。

enum rtc_event_t

Events that can trigger a callback function

Enumerator

RTC_EVENT_ALARM_IRQ	Real Time Clock ALARM IRQ.
RTC_EVENT_PERIODIC_IRQ	Real Time Clock PERIODIC IRQ.

```
//RTC 回调函数
volatile bool rtc_flag = 0;//RTC 延时 1s 标志位
volatile bool rtc_alarm_flag = 0;//RTC 闹钟
/* Callback function */
void rtc_callback(rtc_callback_args_t *p_args)
{
    /* TODO: add your own code here */
    if(p_args->event == RTC_EVENT_PERIODIC_IRQ)
        rtc_flag=1;
    else if(p_args->event == RTC_EVENT_ALARM_IRQ)
        rtc_alarm_flag=1;
}
```

R_RTC_Open 函数

R_RTC_Open 函数可以开启 RTC。

◆ R_RTC_Open()

```
fsp_err_t R_RTC_Open ( rtc_ctrl_t *const p_ctrl,  
                      rtc_cfg_t const *const p_cfg  
                      )
```

Opens and configures the RTC driver module. Implements `rtc_api_t::open`. Configuration includes clock source, and interrupt callback function.

Example:

```
/* Open the RTC module */  
err = R_RTC_Open(&g_rtc0_ctrl, &g_rtc0_cfg);
```

Return values

FSP_SUCCESS	Initialization was successful and RTC has started.
FSP_ERR_ASSERTION	Invalid p_ctrl or p_cfg pointer.
FSP_ERR_ALREADY_OPEN	Module is already open.
FSP_ERR_INVALID_ARGUMENT	Invalid time parameter field.

同时主程序中开启 RTC 已经设置时间和闹钟。

```
/* *****RTC 开启***** */  
/* Initialize the RTC module */  
err = R_RTC_Open(&g_rtc0_ctrl, &g_rtc0_cfg);  
  
/* Handle any errors. This function should be defined by the user. */  
assert(FSP_SUCCESS == err);  
  
/* Set the RTC clock source. Can be skipped if "Set Source Clock in Open" property is enabled.  
*/  
R_RTC_ClockSourceSet(&g_rtc0_ctrl);  
  
/* R_RTC_CalendarTimeSet must be called at least once to start the RTC */  
R_RTC_CalendarTimeSet(&g_rtc0_ctrl, &set_time);  
  
/* Set the periodic interrupt rate to 1 second */  
R_RTC_PeriodicIrqRateSet(&g_rtc0_ctrl, RTC_PERIODIC_IRQ_SELECT_1_SECOND);
```

IWDT 周期设定

独立看门狗定时器（IWDT）的时钟源为(IWDTCLK)。这种设计确保看门狗使用的是独立于主时钟的时钟源，即使主时钟停止工作，看门狗仍然能正常运行，从而提供更高的可靠性。

25. Independent Watchdog Timer (IWDT)

25.1 Overview

The Independent Watchdog Timer (IWDT) consists of a 14-bit down counter that must be serviced periodically to prevent counter underflow. The IWDT provides functionality to reset the MCU or to generate a non-maskable interrupt or an underflow interrupt. Because the timer operates with an independent, dedicated clock source, it is particularly useful in returning the MCU to a known state as a fail-safe mechanism when the system runs out of control. The IWDT can be triggered automatically by a reset, underflow, refresh error, or a refresh of the count value in the registers.

The IWDT functions differ from those of the WDT in the following respects:

- The divided IWDT-dedicated clock (IWDTCCLK) is used as the count source (not affected by PCLKB)
- IWDT does not support register start mode

Table 25.1 lists the IWDT specifications and Figure 25.1 shows a block diagram.

Table 25.1 IWDT specifications

Parameter	Description
Count source*1	IWDT-dedicated clock (IWDTCCLK)
Clock division ratio	Division by 1, 16, 32, 64, 128, or 256
Counter operation	Counting down using a 14-bit down-counter
Condition for starting the counter	• Counting automatically starts after a reset • Only secure developer can start the IWDT
Conditions for stopping the counter	• Reset (the down-counter and other registers return to their initial values) • A counter underflows or a refresh error is generated (counting restarts automatically).
Window function	Window start and end positions can be specified (refresh-permitted and refresh-prohibited periods)
Reset output sources	• Down-counter underflows • Refreshing outside the refresh-permitted period (refresh error).
Non-maskable interrupt/interrupt sources	• Down-counter underflows • Refreshing outside the refresh-permitted period (refresh error).
Reading the counter value	The down-counter value can be read by the IWDTSR register
Event link function	• Down-counter underflow event output • Refresh error event output.
Output signal (internal signal)	• Reset output • Interrupt request output • Sleep-mode count stop control output.
Auto start mode	Configurable to the following triggers: • Clock frequency division ratio after a reset (OFS0.IWDTCCKS[3:0] bits) • Timeout period of the Independent Watchdog Timer (OFS0.IWDTTOS[1:0] bits) • Window start position in the Independent Watchdog Timer (OFS0.IWDTWPSS[1:0] bits) • Window end position in the Independent Watchdog Timer (OFS0.IWDTWPES[1:0] bits) • Reset output or interrupt request output (OFS0.IWDTSTIRQS bit) • Down-count stop function at transition to Sleep, Snooze, or Software Standby mode (OFS0.IWDTSTPCTL bit).
TrustZone Filter	Security attribution can be set

Note 1. Satisfy the frequency of the peripheral module clock (PCLKB) $\geq 4 \times$ (the frequency of the count clock source after division).

The bus interface and registers operate with PCLKB, and the 14-bit counter and control circuits operate with IWDTCCLK.

IWDTCCLK 频率为 15k。

Table 8.2 Clock generation circuit specifications for the internal clocks

Item	Clock source	Clock supply	Specification
System clock (ICLK)	MOSC/SOSC/HOCO/MOCO/LOCO/PLL	CPU, DTC, DMAC, Flash, RAM	Up to 100 MHz Division ratios: 1/2/4/8/16/32/64
Peripheral module clock A (PCLKA)	MOSC/SOSC/HOCO/MOCO/LOCO/PLL	Peripheral modules (QSPI, SCI, SPI, CRC, DOC, ADC12, DAC12, SCE9, GPT bus clock)	Up to 100 MHz Division ratio: 1/2/4/8/16/32/64
Peripheral module clock B (PCLKB)	MOSC/SOSC/HOCO/MOCO/LOCO/PLL	Peripheral modules (CAC, ELC, I/O ports, POEG, RTC, WDT, IWDI, AGT, IIC, CAN, USBFS, SSIE, SDHI, TSN, CTSU, Standby SRAM)	Up to 50 MHz Division ratio: 1/2/4/8/16/32/64
Peripheral module clock C (PCLKC)	MOSC/SOSC/HOCO/MOCO/LOCO/PLL	Peripheral module (ADC12 conversion clock)	Up to 50 MHz Division ratio: 1/2/4/8/16/32/64
Peripheral module clock D (PCLKD)	MOSC/SOSC/HOCO/MOCO/LOCO/PLL	Peripheral module(GPT count clock)	Up to 100 MHz Division ratio: 1/2/4/8/16/32/64
FlashIF clock (FCLK)	MOSC/SOSC/HOCO/MOCO/LOCO/PLL	FlashIF	4 MHz to 50 MHz(P/E) Up to 50 MHz(read) Division ratio: 1/2/4/8/16/32/64
USB clock (USBCLK)	PLL/PLL2	USBFS	48 MHz Division ratio: 3/4/5
CAN clock (CANMCLK)	MOSC	CAN	8 MHz to 24 MHz
AGT clock (AGTSCLK)	SOSC	AGT	32.768 kHz
AGT clock (AGTLCLK)	LOCO	AGT	32.768 kHz
CAC Main clock (CACMCLK)	MOSC	CAC	Up to 24 MHz
CAC Sub clock (CACSCLK)	SOSC	CAC	32.768 kHz
CAC LOCO clock (CACLCLK)	LOCO	CAC	32.768 kHz
CAC MOCO clock (CACMOCLK)	MOCO	CAC	8 MHz
CAC HOCO clock (CACHCLK)	HOCO	CAC	16/18/20 MHz
CAC IWDTLCO clock (CACILCLK)	IWDTLCO	CAC	15 kHz
RTC clock (RTCCLK)	SOSC/LOCO	RTC	32.768 kHz
IWDT clock (IWDTCCLK)	IWDTLCO	IWDT	15 kHz
SysTick timer clock (SYSTICCLK)	LOCO	SysTick timer	32.768 kHz
JTAG clock (JTAGTCK)	TCK	JTAG	Up to 25 MHz
Serial wire clock (SWCLK)	SWCLK	OCD	Up to 25 MHz
Trace clock (TRCLK)	MOSC/SOSC/HOCO/MOCO/LOCO/PLL	CPU-OCD	Up to 50 MHz, Division ratio: 1/2/4
TCLK pin output (TCLK)	1/2 TRCLK	TCLK pin	Up to 25 MHz
Clock/buzzer output (CLKOUT)	MOSC/SOSC/LOCO/MOCO/HOCO	CLKOUT pin	Up to 60 MHz Division ratios: 1/2/4/8/16/32/64/128

Note: Restrictions on setting clock frequency: $ICLK \geq PCLKA \geq PCLKB$, $PCLKD \geq PCLKA \geq PCLKB$

$ICLK \geq FCLK$

Restrictions on clock frequency ratio: (N: integer, and up to 64)

$ICLK:FCLK = N:1$, $ICLK:PCLKA = N:1$, $ICLK:PCLKB = N:1$, $ICLK:PCLKC = N:1$ or $1:N$, $ICLK:PCLKD = N:1$ or $1:N$, $ICLK:TRCLK = N:1$ or $1:N$

If the A/D converter is enabled, the clock frequency ratio is constrained as follows:

$PCLKA:PCLKC = 1:1$ or $2:1$ or $4:1$ or $8:1$ or $1:2$ or $1:4$

Note: Restrictions on the minimum FCLK frequency 4MHz when P/E.

Note: The multiplication of PLL and PLL2 should be set to be within the output frequency range of PLL and PLL2, taking the frequency of HOCO into consideration when not using the FLL function.

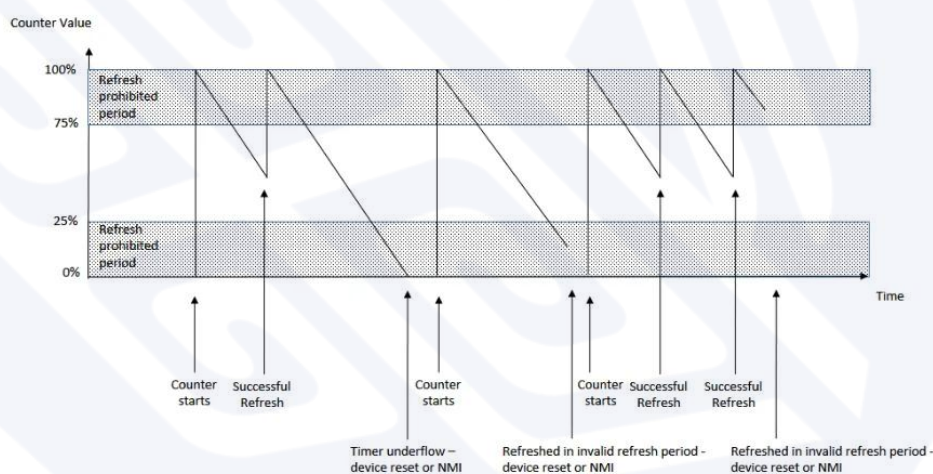
Note: Clocks have a permissible frequency range (See Table 8.2).

依据上文的设定，IWDT 周期如下所示。

参数	值
IWDTCLK	15k
分频(设置为 32)	0.46875k
超时周期	2048 cycles
单周期时间	$1/0.46875k=2.1333ms$
溢出时间	$2.1333ms*2048cycles=4369.067ms$
上述可以看到在该设置下的溢出时间大概为 4s, 那么 1s 的计数为 $1s/2.13ms=468.757$ 。	

IWDT 计数周期

IWDT 开始从 100% 计数到 0%，形成一个倒计时。
在允许刷新阶段，通过刷新信号将计数器值恢复到初始值，防止计数器溢出。



Watchdog Timer Operation Example

J-Link 调试器的限制

当使用 J-Link 调试器时，IWDT 计数器不会运行（暂停计数）。
这意味着 IWDT 不会触发设备复位（reset）或生成非屏蔽中断（NMI）。

如果你希望在调试期间仍然让 IWDT 正常计数并能触发复位或中断，你需要添加以下代码。

```
/* (Optional) Enable the IWDT to count and generate NMI or reset when the
 * debugger is connected. */
R_DEBUG->DBGSTOPCR_b.DBGSTOP_IWDT = 0;
```

主程序

```
while(1)
{
    if(rtc_flag)
    {
        uint32_t iwdt_counter = 0U;
        /* Read the current IWDT counter value. */
        err = R_IWDT_CounterGet(&g_wdt0_ctrl, &iwdt_counter);
        assert(FSP_SUCCESS == err);
        printf("iwdt_counter=%d\n", iwdt_counter);
        /* Refresh before the counter underflows to prevent reset or NMI. */
        (void) R_IWDT_Refresh(&g_wdt0_ctrl);
        rtc_flag=0;
    }
}
```

演示效果

设置每过 1s 打印一次当前时间，分别设置喂狗和不喂狗，结果如下。
延迟 1s 的计数为 $1s/2.13ms=468.757$ ，打印为 1579，由于是向下计数， $2048-469=1579$ ，符合计算值。

[16:10:18.819]收←◆wdt_counter=15653
[16:10:19.818]收←◆wdt_counter=15653
[16:10:20.819]收←◆wdt_counter=15653
[16:10:21.818]收←◆wdt_counter=15653
[16:10:22.818]收←◆wdt_counter=15654
[16:10:23.818]收←◆wdt_counter=15653
[16:10:24.818]收←◆wdt_counter=15653
[16:10:25.818]收←◆wdt_counter=15653
[16:10:26.818]收←◆wdt_counter=15653
[16:10:27.817]收←◆wdt_counter=15654
[16:10:28.818]收←◆wdt_counter=15653
[16:10:29.818]收←◆wdt_counter=15653
[16:10:30.818]收←◆wdt_counter=15653
[16:10:31.818]收←◆wdt_counter=15653
[16:10:32.817]收←◆wdt_counter=15654

当不执行喂狗时候，计数值到 0 时会进行复位，2 个复位之间为 4s 左右，符合计算的 4369.067ms。

[22:27:05.090] 收←◆hello!

[22:27:06.093] 收←◆iwdt_counter=1579

[22:27:07.093] 收←◆iwdt_counter=1109

[22:27:08.092] 收←◆iwdt_counter=640

[22:27:09.093] 收←◆iwdt_counter=170

[22:27:09.456] 收←◆hello!

[22:27:10.459] 收←◆iwdt_counter=1579

[22:27:11.459] 收←◆iwdt_counter=1109

[22:27:12.459] 收←◆iwdt_counter=640

[22:27:13.459] 收←◆iwdt_counter=170

[22:27:13.823] 收←◆hello!

[22:27:14.826] 收←◆iwdt_counter=1579

[22:27:15.826] 收←◆iwdt_counter=1109

[22:27:16.826] 收←◆iwdt_counter=640